

Python pentru analiza datelor

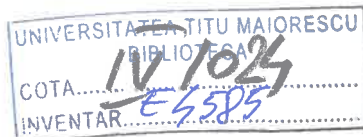
Claudiu VINȚE

Titus Felix FURTUNĂ

Python pentru analiza datelor

Colecția
Informatică

**Editura ASE
București
2020**



Academia de Studii Economice din București

Copyright © 2020, Editura ASE

Toate drepturile asupra acestei ediții sunt rezervate editurii.

Editura ASE

Piața Romană nr. 6, sector 1, București, România

cod 010374

www.ase.ro

www.editura.ase.ro

editura@ase.ro

Descrierea CIP a Bibliotecii Naționale a României

VINȚE, CLAUDIU

Python pentru analiza datelor / Claudiu Vințe, Titus Felix Furtună. -

București : Editura ASE, 2020

Conține bibliografie

ISBN 978-606-34-0361-3

I. Furtună, Titus Felix

004

Editura ASE

Redactor: Luiza Constantinescu

Coperta: Violeta Rogojan

Autorii își asumă întreaga responsabilitate pentru: ideile exprimate, corectitudinea științifică, originalitatea materialului și sursele bibliografice menționate.

Copierea și utilizarea codului Python prezentat în această lucrare este permisă doar însoțită de menționarea sursei.

Celor pentru care aparentul nu este suficient.

Autorii

Cuvânt-înainte

Această carte se adresează cu precădere studenților Facultății de Cibernetică, Statistică și Informatică Economică, ciclul licență, din cadrul Academiei de Studii Economice din București. Poate fi utilă, de asemenea, cititorilor interesați de metodele multivariate pentru analiza datelor și de implementarea acestora într-un limbaj de programare. În acest sens, poate servi și ca lucrare de inițiere în *data science* prin intermediul limbajului de programare Python.

Am dorit să susținem și să completăm în mod particular componenta aplicativă a actului de învățare și să oferim prin această lucrare un suport concret și eficace pentru activitatea desfășurată în cadrul laboratoarelor, dar și pentru studiul individual al disciplinei Analiza datelor.

Lucrarea este structurată în șase capitole. Primele cinci reprezintă o succintă introducere în limbajul de programare Python, reținându-se cu prioritate aspectele ce privesc tipurile de date și tehnicile de utilizare a acestora, pachetele și modulele cu aplicativitate specifică în domeniul analizei datelor. Astfel, în primul capitol oferim o scurtă introducere în mediul de programare Python. Capitolul al doilea tratează elementele de bază ale limbajului, cele ce privesc sintaxa generală, tipurile de date, operatori, funcții ș.a.m.d. Capitolele trei și patru prezintă fundamentele pachetelor *numpy* și *pandas*, pentru lucrul cu masive multidimensionale, respectiv pentru lucrul cu date organizate preponderent în format tabelar. În cadrul capitolului cinci prezentăm exemple practice cu privire la realizarea de grafice în Python, cu scopul de a oferi o perspectivă vizuală asupra datelor în procesul de analiză și interpretare a rezultatelor. Capitolul al șaselea este dedicat prezentării unei suite de metode fundamentale de analiză multivariată a datelor, împreună cu implementarea acestora în limbajul Python. Pentru fiecare metodă abordată oferim un preambul teoretic, ce are rolul de a prezenta rezultatele de bază din domeniul analizei multivariate și de a formaliza conceptele introduse. De asemenea, în cadrul fiecărei metode de analiză, prezentăm elementele metodologice necesare implementării acesteia în limbajul Python. Prezentarea fiecărei metode este acompaniată de un studiu de caz, ce reprezintă un exemplu practic de implementare în Python și procesarea, prin intermediul soluției informatice propuse, a unui sau mai multor seturi de date de intrare. În cadrul fiecărui studiu de caz sunt ulterior discutate și interpretate rezultatele obținute.

Intenția noastră este aceea de a le oferi celor interesați de metodele de analiză a datelor și de implementare a acestora în unul dintre cele mai populare limbaje de programare actuale, un material aplicativ cu utilitate imediată, care să deschidă calea către noi aplicații sau activități de cercetare din domenii adiacente, precum: recunoașterea formelor, *data mining*, *machine learning* ș.a.

Concluzionând, conținutul acestei lucrări pune accentul pe ideea de a sprijini și încuraja o manieră de învățare care să stimuleze construirea conștientă și coerentă de aplicații concrete, scrise în limbajul Python, pentru prelucrarea seturilor de date multivariate. În acest context, bibliografia pe care o includem la finalul cărții are mai curând rolul de indica cititorilor diverse surse utile pentru aprofundarea cunoștințelor introduse prin exemplele practice, prezentate în această lucrare.

Am dori să mulțumim referenților manuscrisului care, prin recomandările făcute, ne-au ajutat să obținem o mai bună organizare a conținutului acestei lucrări. De asemenea, dorim să mulțumim editurii ASE, echipei redacționale și colectivului tipografic, care, prin profesionalism și dăruire, au făcut posibilă apariția acestei cărți.

Autorii

Cuprins

1	Scurtă introducere în mediul de programare Python	13
1.1	Instalare și configurare	13
1.2	Configurare PATH	13
1.3	Variabile de mediu Python	14
1.4	Moduri de rulare a interpretorului Python	15
1.4.1	Interpretor interactiv	15
1.4.2	Lansare script din linia de comandă	16
1.4.3	Utilizarea unui mediu de dezvoltare integrat	16
2	Elemente de bază ale limbajului Python	17
2.1	Sintaxa generală a limbajului Python	18
2.1.1	Identificatori Python	18
2.1.2	Cuvinte rezervate	19
2.1.3	Linii de cod și indentare	19
2.1.4	Declararea unei linii logice pe mai multe linii fizice	20
2.1.5	Utilizarea citărilor în Python	20
2.1.6	Comentarii în Python	21
2.1.7	Utilizarea liniilor libere	21
2.1.8	Declarații multiple pe o singură linie	22
2.1.9	Gruparea declarațiilor în blocuri	22
2.2	Tipuri de variabile	22
2.2.1	Atribuirea de valori variabilelor	22
2.2.2	Atribuire multiplă	23
2.2.3	Tipuri de date standard	23
2.2.4	Testarea valorii de adevăr	24
2.2.5	Comparații	24
2.2.6	Tipuri de date numerice	25
2.2.7	Șiruri de caractere	27
2.2.7.1	Accesarea valorilor în șiruri	27
2.2.7.2	Caractere de control al tipării	28
2.2.7.3	Citarea triplă	29
2.2.7.4	Modificarea șirurilor de caractere	30
2.2.7.5	Operatori speciali pentru șiruri de caractere	30
2.2.7.6	Operatori de formatare a șirurilor de caractere	31
2.2.7.7	Șiruri Unicode	32
2.3	Operatori fundamentali	32
2.3.1	Operatori aritmetici	32
2.3.2	Operatori de comparație	33
2.3.3	Operatori de atribuire	33
2.3.4	Operatori logici pe biți (bitwise)	34
2.4	Structuri de control	35
2.4.1	Structura alternativă	35
2.4.2	Structuri repetitive	36
2.4.2.1	Structura while	36
2.4.2.2	Structura for	36
2.5	Tratarea excepțiilor	36

2.6	Utilizarea funcțiilor în Python	37
2.6.1	Definirea unei funcții	37
2.6.2	Pasarea parametrilor în funcții	38
2.6.2.1	Argumente necesare	40
2.6.2.2	Argumente cu cuvinte-cheie	40
2.6.2.3	Argumente implicite	41
2.6.2.4	Argumente de lungime variabilă	41
2.6.2.5	Funcțiile anonime (lambda)	42
2.6.2.6	Instrucțiunea return	43
2.6.3	Decoratori	43
2.7	Tipuri de date secvențiale	44
2.7.1	Conceptul de listă în Python	44
2.7.1.1	Accesarea valorilor unei liste	45
2.7.1.2	Actualizarea listelor	45
2.7.1.3	Ștergerea de elemente din listă	46
2.7.1.4	Operații de bază cu liste	46
2.7.1.5	Indexare și partiționare	46
2.7.1.6	Funcții și metode implicite pentru liste	47
2.7.2	Conceptul de tuplu	48
2.7.2.1	Modificarea tuplurilor	49
2.7.2.2	Ștergerea elementelor unui tuplu	49
2.7.2.3	Funcții implicite pentru tuplu	50
2.7.3	Tipul interval (range)	51
2.8	Python ca limbaj de programare orientat obiect	52
2.8.1	Crearea obiectelor	53
2.8.2	Metode speciale	53
2.8.3	Crearea atributelor	54
2.8.4	Proprietăți	54
2.9	Structuri de date secvențiale modelate cu tipurile implicite Python	55
2.9.1	Vector (array)	55
2.9.1.1	Reprezentare vector (array)	56
2.9.1.2	Operații de bază cu vectori	56
2.9.1.3	Parcurgerea elementelor unui vector	57
2.9.1.4	Accesarea elementelor unui vector	57
2.9.1.5	Operațiunea de inserare în vector	58
2.9.1.6	Operațiunea de ștergere	58
2.9.1.7	Operațiunea de căutare	59
2.9.1.8	Operațiunea de actualizare	59
2.9.2	Stivă (stack)	60
2.9.2.1	Adăugarea sau punerea unui element în stivă – PUSH într-o stivă	60
2.9.2.2	Extragerea sau scoaterea unui element din stivă – POP dintr-o stivă	61
2.9.3	Coadă (queue)	61
2.9.3.1	Adăugarea de elemente la o coadă – PUT într-o stivă	62
2.9.3.2	Scoaterea elementelor dintr-o coadă	62
2.10	Conceptul de dicționar (tipul map)	63
2.10.1	Accesarea valorilor în dicționar	63
2.10.2	Actualizarea dicționarului	64
2.10.3	Ștergerea elementelor unui dicționar	64
2.10.4	Proprietățile cheilor de dicționar	65
2.10.5	Funcții și metode implicite pentru tipul dicționar	65



2.11	Lucrul cu fișiere	66
2.11.1	Funcții de bază pentru citirea datelor din fișiere	66
2.11.2	Citirea datelor utilizând pachetul pandas	68
3	Lucru cu masive multidimensionale din pachetul numpy	69
3.1	Masive multidimensionale ndarray	69
3.1.1	Funcții de creare	70
3.1.2	Atribute ale masivelor ndarray	72
3.1.3	Indexarea masivelor numpy.ndarray	73
3.1.4	Specificarea dimensiunilor	74
3.2	Operații aritmetice și logice	75
3.3	Funcții universale	76
3.4	Funcții statistice	77
3.5	Funcții pentru inversare, sortare, căutare, selecție	79
3.6	Funcții matematice și de algebră liniară	80
4	Tipuri de date ale pachetului pandas	81
4.1	Serii	81
4.2	Tabele de date (DataFrame)	82
4.3	Modificare, salvare, sortare, agregare și joncțiune în DataFrame	84
4.3.1	Modificarea datelor din DataFrame	84
4.3.2	Salvarea în fișier CSV	85
4.3.3	Sortarea	85
4.3.4	Agregarea datelor	85
4.3.5	Joncțiuni	87
4.4	Date de tip categorial	88
4.5	Clase utile	89
5	Elemente de reprezentare grafică în Python	91
5.1	Adăugarea de subgrafice	92
5.2	Tipuri de grafice în matplotlib	92
5.2.1	Grafice linie – matplotlib.plot	92
5.2.2	Grafice puncte – matplotlib.scatter	94
5.2.3	Grafice puncte seaborn.scatterplot	95
5.2.4	Grafice corelogramă – seaborn.heatmap	96
5.2.5	Histograme matplotlib	97
5.2.6	Histograme pandas	98
5.2.7	Reprezentări grafice utilizate frecvent în analiza datelor	99
5.2.7.1	Cercul corelațiilor	100
5.2.7.2	Graficul varianței explicate de componentele principale	103
5.2.7.3	Graficul corelogramă	104
5.2.7.4	Graficul histogramă cu bare multiple	106
6	Implementarea unor metode de analiză a datelor în Python	108
6.1	Analiza în componente principale (ACP)	108
6.1.1	Datele de prelucrat și etapele analizei	108
6.1.2	ACP – studiu de caz	110
6.1.2.1	Setul de date	110
6.1.2.2	Abordarea ACP în Python	112
6.1.2.3	Interpretarea rezultatelor	117
6.2	Analiza exploratorie a factorilor (AEF)	126
6.2.1	Datele de prelucrat	126
6.2.2	Ipoteze modelului	127

6.2.3 Estimarea existenței factorilor	128
6.2.4 Estimarea parametrilor modelului. Extragerea factorilor	129
6.2.5 Metoda probabilității maxime	130
6.2.6 Metoda analizei componentelor principale	130
6.2.7 Estimarea numărului de factori. Testul Bartlett	131
6.2.8 AEF – studiu de caz	133
6.3 Analiza corelațiilor canonice (ACC)	141
6.3.1 Datele de prelucrat	142
6.3.2 Etapele analizei	142
6.3.3 Factorii canonici	145
6.3.4 Legăturile dintre factori	146
6.3.5 Varianța explicată și redundanța informațională	147
6.3.6 Standardizarea factorilor canonici	148
6.3.7 Relevanța rădăcinilor canonice. Testul de relevanță Bartlett χ^2	148
6.3.8 ACC – studiu de caz	149
6.4 Analiza discriminantă (AD)	158
6.4.1 Organizarea datelor și notații	158
6.4.2 Indicatori de variabilitate și împrăștiere	159
6.4.3 Semnificația modelului. Teste statistice	161
6.4.4 Analiza discriminantă liniară (Linear Discriminant Analysis) – funcțiile Fisher	163
6.4.5 Analiza discriminantă: caz particular al analizei canonice	167
6.4.6 Funcții de clasificare	168
6.4.7 Discriminarea bayesiană	170
6.4.7.1 Probabilitate totală și probabilitate bayesiană	170
6.4.7.2 Clasificatorul bayesian	171
6.4.7.3 Estimarea neparametrică a probabilităților condiționate. Metoda histogramelor	172
6.4.7.4 Metode parametrice	173
6.4.8 Analiza discriminantă – studiu de caz	174
6.4.8.1 Setul de date utilizat	174
6.4.8.2 Investigarea eșantionului de bază – construirea modelului	175
6.4.8.3 Implementarea Python a analizei discriminate	175
6.4.8.4 Prezentarea și interpretarea rezultatelor	178
6.4.8.5 Clasificarea instanțelor din eșantionul de test – aplicarea modelului	182
6.5 Analiza de cluster (AC)	183
6.5.1 Algoritmi ierarhici	184
6.5.2 Metode de grupare ierarhică	186
6.5.3 Distanțe utilizate pentru calculul proximității	187
6.5.4 Analiza de cluster – studiu de caz	188
6.5.4.1 Implementarea analizei de cluster în Python	189
6.5.4.2 Rezultatele obținute și interpretarea acestora	191
6.5.4.3 Clasificarea variabilelor	194
Bibliografie	201
Anexe	
Anexa 1	205
Anexa 2	213

1 Scurtă introducere în mediul de programare Python

1.1 Instalare și configurare

Deoarece limbajul de programare Python este folosit pentru dezvoltarea de diverse aplicații, nu există o singură soluție pentru configurarea Python și a pachetele completare necesare. Python este disponibil pe o mare varietate de platforme, inclusiv Windows, Linux și Mac OS X.

Mulți cititori nu vor avea un mediu complet de dezvoltare Python adecvat pentru a urma împreună secvențele de cod și exemplele prezentate în această carte, de aceea vom furniza în continuare instrucțiuni detaliate pentru configurarea mediului Python pe fiecare sistem de operare. Această carte folosește Python 3.7 și vă recomandăm să utilizați versiunea Python 3.7 sau una superioară. Vom vedea în continuare cum poate fi configurat un mediu de dezvoltare în Python.

Configurarea mediului local – deschideți o fereastră de terminal și tastați "python - V" pentru a afla dacă este deja instalat interpretorul de Python și ce versiune este instalată.

Obținerea Python – codul sursă cel mai recent actualizat, cele binare, documentație, știri etc. sunt disponibile pe site-ul oficial al Python <https://www.python.org/>. Puteți descărca documentația Python de pe <https://www.python.org/doc/>. Documentația este disponibilă în format HTML, PDF și PostScript.

Instalarea Python – distribuția Python este disponibilă pentru o mare varietate de platforme. Trebuie doar să descărcați codul binar corespunzător sistemului de operare utilizat și să instalați Python.

Dacă codul binar pentru sistemul de operare pe care îl utilizați nu este disponibil, atunci aveți nevoie de un compilator C pentru platforma respectivă pentru a compila codul sursă manual. Compilarea codului sursă oferă mai multă flexibilitate în ceea ce privește alegerea funcțiilor de care le aveți nevoie în instalarea pe care doriți să o realizați.

1.2 Configurare PATH

Programele și alte fișiere executabile se pot regăsi în multiple directoare, de aceea sistemele de operare oferă o cale de căutare care listează directoarele în care sistemul de operare caută fișiere executabile.

Calea este stocată într-o variabilă de mediu, ce este un șir de caractere căruia îi este asociat un nume și care este menținut de sistemul de operare. Această variabilă conține informații disponibile pentru *shell*-ul de comandă și alte programe. Variabila de cale este denumită PATH

în Unix sau Path în Windows (Unix diferențiază între literele mari și mici; Windows nu face această diferențiere).

În Mac OS, programul de instalare delegă gestionarea detaliilor căii către interpretorul Python.

Pentru a invoca interpretul Python din orice director anume, trebuie să adăugați directorul Python la calea dvs.

Setarea căii în Unix / Linux

Pentru a adăuga directorul Python la calea pentru o anumită sesiune în Unix:

În *csh* shell – tastați `setenv PATH "$PATH:/usr/local/bin/python"` și apăsați Enter.

În shell-ul *bash* (Linux) – tastați `export PATH = "$PATH:/usr/local/bin/ python"` și apăsați tasta Enter.

În shell-ul *sh* sau *ksh* – tastați `PATH = "$PATH:/usr/local/bin/python"` și apăsați Enter.

Notă - `/usr/local/bin/python` este calea implicită a directorului Python

Setarea căii în Windows

Pentru a adăuga directorul Python la calea pentru o anumită sesiune în Windows:

La promptul din linia de comandă – tastați `path %path%;C:\Python` și apăsați Enter.

Notă - `C:\Python` este calea directorului Python.

1.3 Variabile de mediu Python

Prezentăm în continuare câteva variabile de mediu importante, care pot fi recunoscute de interpretorul Python.

PYTHONPATH

Are un rol similar cu PATH. Această variabilă spune interpretului Python unde să localizeze fișierele importate ale modulului într-un program. Ar trebui să includă directorul bibliotecii sursă Python și directoarele care conțin codul sursă Python. PYTHONPATH este uneori presetat de programul de instalare a interpretorului Python.

PYTHONSTARTUP

Conține calea unui fișier de inițializare către codul sursă Python. Se execută de fiecare dată când porniți interpretul. Este numit `.pythonrc.py` în Unix și conține comenzi care încarcă utilitățile sau modifică PYTHONPATH.

PYTHONCASEOK

Se folosește în Windows pentru a instrui Python să găsească prima potrivire nesemnificativă între majuscule și caractere mici dintr-o declarație de import. Setati această variabilă la orice valoare pentru a o activa.

PYTHONHOME

Este o cale alternativă de căutare a modulului. De obicei, este încorporat în directoarele PYTHONSTARTUP sau PYTHONPATH pentru a ușura bibliotecile modulelor de comutare.

1.4 Moduri de rulare a interpretorului Python

Există trei moduri diferite de a rula Python.

1.4.1 Interpretor interactiv

Puteți lansa interpretorul Python din Unix, DOS sau orice alt sistem care vă oferă un interpretor în linie de comandă sau o fereastră de shell.

Tastați linia de comandă *python*. Începeți codarea imediat în interpretul interactiv.

`$python # Unix/Linux`

sau

`$python3 # Unix/Linux`

sau

`python% # Unix/Linux`

sau

`C:\> python # Windows/DOS`

Furnizăm în continuare lista opțiunilor disponibile pentru linia de comandă:

- **d** oferă informații utile pentru depanare.
- **o** acesta opțiune generează *bytecode* optimizat (rezultând fișiere `.pyo`).
- **s** nu rulați site-ul de import pentru a căuta cai Python la pornire.
- **v** ieșire verbose (oferă posibilitate de a urmări într-o manieră detaliată declarațiile de import).
- **x** dezactivați excepțiile încorporate în clasă (folosiți doar șiruri); ieșiți din uz începând cu versiunea 1.6.
- **c cmd** Rulați scriptul Python trimis în șirul cmd.
- **fișier** Executați scriptul Python din fișierul furnizat ca parametru.

1.4.2 Lansare script din linia de comandă

Un script Python poate fi executat din linia de comandă invocând interpretorul și fișierul ce conține codul, ca în exemplele următoare:

```
$python script.py # Unix/Linux
```

sau

```
python% script.py # Unix/Linux
```

sau

```
C:\>python script.py # Windows/DOS
```

Notă - Asigurați-vă că aveți permisiunile necesare asupra fișierului pentru a-l putea lansa în execuție.

1.4.3 Utilizarea unui mediu de dezvoltare integrat

Putem rula Python și dintr-o (*Graphical User Interface* – GUI), dacă aveți o aplicație GUI pe sistemul dvs. care integrează interpretorul Python.

Unix – IDLE este chiar primul Unix IDE pentru Python.

Windows – PythonWin este prima interfață grafică pentru utilizatorii Windows pentru Python și reprezintă un mediu de dezvoltare integrat (*Integrated Development Environment* – IDE).

Macintosh – Versiunea Macintosh a Python împreună cu IDLE IDE este disponibilă pe site-ul principal, descărcându-se fie în fișiere MacBinary, fie BinHex.

Unul dintre cele mai populare IDE existente la momentul conceperii acestei lucrări este PyCharm, care disponibil pentru toate platformele de dezvoltare. Poate fi descărcat și utilizat în mod gratuit în varianta *community* de pe site-ul: <https://www.jetbrains.com/pycharm/>

Notă - Toate exemplele date în capitolele următoare sunt executate cu versiunea Python 3.7.6 și dezvoltate cu ajutorul IDE PyCharm community 2020.3.

2 Elemente de bază ale limbajului Python

Limbajul Python are multe asemănări cu Perl, C și Java. Cu toate acestea, există unele diferențe clare între aceste limbaje de programare. Vom prezenta în continuare cum putem să executăm un program Python.

Rularea de cod Python în mod interactiv

Invocarea interpretului fără a trece un fișier script ca parametru afișează următorul prompter. Acest cursor `>>>` indică faptul că suntem în *shell-ul* Python și instrucțiunile pe care le vom scrie de acum încolo vor fi interpretate în mod interactiv de interpretorul Python.

```
Command Prompt - python
Microsoft Windows [Version 10.0.19041.508]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\claudiu>python
Python 3.7.6 (tags/v3.7.6:43364a7ae0, Dec 19 2019, 00:42:30) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Tastați următorul text în dreptul cursorului Python și apăsați tasta Enter.

```
>>> print('Buna Python!')
```

Veți obține rezultatul afișat în cadrul shell-ul Python rezultatul de mai jos.

```
Command Prompt - python
Microsoft Windows [Version 10.0.19041.508]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\claudiu>python
Python 3.7.6 (tags/v3.7.6:43364a7ae0, Dec 19 2019, 00:42:30) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print('Buna Python!')
Buna Python!
>>>
```

Închiderea sesiunii interactive a interpretorul Python și ieșirea din shell-ul acestuia se realizează prin apelul funcției `>>> exit()`

```
Command Prompt
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\claudiu>python
Python 3.7.6 (tags/v3.7.6:43364a7ae0, Dec 19 2019, 00:42:30) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print('Buna Python!')
Buna Python!
>>> exit()

C:\Users\claudiu>
```

Rularea de cod Python în modul script

Dacă atunci când invocăm interpretorul Python furnizăm și un script (fișier ce conține cod Python) ca parametru în linia de comandă, atunci interpretorul va începe să execute scriptul și va continua până când scriptul este parcurs integral. Când scriptul este terminat, interpretul nu mai este activ.

Să scriem în continuare un simplu program Python într-un fișier text. Fișierele Python au extensia `.py`. Tastați următorul cod sursă într-un fișier `test.py` și salvați fișierul.

```
print('Buna, Python!')
```

Presupunând că aveți instalat un interpretor Python setat în variabila de mediu PATH, încercați să rulați acest program după cum urmează.

```
$python test.py # Unix/Linux
```

sau

```
python% test.py # Unix/Linux
```

sau

```
C:\>python test.py # Windows/DOS
```

Aceasta produce următorul rezultat:

```
Bună, Python!
```

2.1 Sintaxa generală a limbajului Python

2.1.1 Identificatori Python

Un identificator Python este un nume folosit pentru a identifica o variabilă, o funcție, o clasă, un modul sau un alt obiect. Un identificator începe cu o literă de la A la Z sau de la a la z sau cu o linie de subliniere (`_`) urmată de zero sau mai multe litere, alte linii de subliniere și cifre (de la 0 la 9).

Python nu permite caracterele precum `@`, `$` și `%` să facă parte din identificatori. Python este un limbaj de programare sensibil la majuscule. Astfel, `ClasaMea` și `clasamea` sunt doi identificatori diferiți în Python.

Prezentăm succint în continuare câteva dintre convențiile de denumire pentru identificatorii Python:

- Numele claselor încep cu o literă mare. Toți ceilalți identificatori încep cu litere mici.
- Prefixarea sau denumirea unui identificator cu o singură linie de subliniere indică faptul că identificatorul este privat.
- Prefixarea denumirii unui identificator cu două linii de subliniere semnifică faptul că identificatorul puternic este privat.

- Dacă identificatorul se termină de asemenea și cu două linii de subliniere, atunci identificatorul este un nume special definit în cadrul limbajului (de exemplu, identificatorul constructorului unei clase).

2.1.2 Cuvinte rezervate

Următorul tabel prezintă cuvintele-cheie din limbajul Python. Acestea sunt cuvinte rezervate și nu pot fi folosite drept constante sau variabile ori ca orice alt nume de identificator. Toate cuvintele-cheie Python conțin numai litere mici (tabelul 2.1).

Tabelul 2.1 Cuvintele-cheie Python

<code>and</code>	<code>exec</code>	<code>not</code>
<code>as</code>	<code>finally</code>	<code>or</code>
<code>assert</code>	<code>for</code>	<code>pass</code>
<code>break</code>	<code>from</code>	<code>print</code>
<code>class</code>	<code>global</code>	<code>raise</code>
<code>continue</code>	<code>if</code>	<code>return</code>
<code>def</code>	<code>import</code>	<code>try</code>
<code>del</code>	<code>in</code>	<code>while</code>
<code>elif</code>	<code>is</code>	<code>with</code>
<code>else</code>	<code>lambda</code>	<code>yield</code>
<code>except</code>		

2.1.3 Linii de cod și indentare

Python nu utilizează acolade pentru a indica blocuri de cod pentru definițiile claselor și funcțiilor sau pentru controlul fluxului. Blocurile de cod sunt denotate prin indentarea de linie, care trebuie aplicată într-o manieră consistentă în cadrul codului.

Numărul de spații din indentare este variabil, dar toate instrucțiunile dintr-un bloc trebuie să fie indentate cu același număr de spații libere.

De exemplu, codul de mai jos

```
if True:
    print("True")
else:
    print("False")
```

utilizează o indentare corectă pentru codului blocurilor din cadrul structurii alternative.

Cu toate acestea, următoarea secvență de cod generează o eroare, în condițiile în care interpretorul Python aștepta utilizarea indentării pentru semnalizarea blocurilor aferente structurii alternative.

```
if True:
print("Prima varianta")
```

```
print("True")
else:
print("A doua varianta")
print("False")
```

Concluzionând, într-o secvență de cod Python toate liniile successive care sunt indentate cu același număr de spații fac parte din același bloc.

2.1.4 Declararea unei linii logice pe mai multe linii fizice

Instrucțiunile din Python se termină de obicei atunci când este întâlnit caracterul sau secvența de caractere care semnifică trecerea pe o nouă linie. În funcție de sistemul de operare, marcarea sfârșitului de linie poate fi implementată în diferite moduri.

- Sisteme Unix și de tip Unix (Linux, macOS, FreeBSD, AIX etc.) – caracterul *line feed* <LF>, respectiv prin utilizare secvenței de escape (\n)
- Microsoft Windows, DOS (MS-DOS, PC DOS etc.), OS/2, Symbian OS, Palm OS și majoritatea celorlalte sisteme de operare timpurii non-Unix și non-IBM – utilizează secvența de caractere *carriage return* și *line feed*, <CR><LF>, respectiv în termeni de secvență de escape (\r\n)

Python permite totuși continuarea unei linii logice pe mai multe linii fizice prin utilizarea caracterului (\) pentru a indica faptul că linia ar trebui să continue. De exemplu, asumând că identificatorii utilizați în secvența de cod următoare sunt variabile care au fost în prealabil inițializate, putem avea:

```
suma = unu + \
      doi + \
      trei + \
      patru + \
      cinci
```

Instrucțiunile conținute între paranteze [], {} sau () nu trebuie să utilizeze caracterul de continuare a liniei. În exemplul următor, inițializarea elementelor unei liste Python se poate realiza pe mai multe linii fizice de cod.

```
cos_cumparaturi = ['lapte', 'paine', 'oua', 'rosii',
                  'unt', 'apa minerala']
```

2.1.5 Utilizarea citărilor în Python

Python acceptă apostroful ('), ghilimele (") și combinații duble (' ' sau " ") sau triple (" " " sau ' ' ') ale acestora pentru a indica literalele unui șir, atât timp cât același tip de ghilimele (apostrof) începe și termină șirul de caractere.

Ghilimelele triple sunt folosite pentru a continua șirul pe mai multe linii sau pentru a include în cadrul șirului alte ghilimele sau apostrofuri. De exemplu, toate modalități următoare de utilizare a citărilor sunt corecte.

```
cuvant = 'cuvant'
cuvant_citat = '"cuvant"'
propozitie = "Propozitia contine mai multe cuvinte."
fraza = """O fraza poate contine mai multe propozitii si se poate intinde
        pe mai multe linii."""
citare = '''Pot fi incluse citate din limba engleza: "Please, don't do that!"'''
```

2.1.6 Comentarii în Python

Un caracter diez (#) care nu se află într-un șir literal semnifică începutul un comentariu. Toate caracterele de după # și până la sfârșitul liniei fizice fac parte din comentariu, iar interpretul Python le ignoră. Secvența următoare de cod:

```
# Un comentariu
print("Buna, Python!") # Un alt comentariu
```

va genera la rulare următorul rezultat:

```
Buna, Python!
```

Un comentariu poate fi introdus pe aceeași linie după o declarație sau o expresie

```
culoare = "albastru" # Acesta este un comentariu ce poate fi cu privire la culoare
```

sau pot fi create mai multe linii de comentariu:

```
# Avem in continuare
# mai multe linii de comentariu
# care exemplifica modul in care
# pot fi create comentariile.
```

Următoarele șiruri de caractere citate triplu, cu apostrofuri sau cu ghilimele, sunt, de asemenea, ignorate de interpretul Python și pot fi folosite drept comentarii ce se întind pe mai multe linii.

```
'''
Acesta este
un comentariu care
se intinde pe mai multe linii.
'''
```

```
"""
Si acesta este un comentariu
care se intinde
pe mai multe linii.
"""
```

2.1.7 Utilizarea liniilor libere

O linie care conține doar spații albe, eventual cu un comentariu, este cunoscută sub numele de linie goală și interpretorul Python o ignoră total. Cu toate acestea, într-o sesiune de utilizare a interpretorului în mod interactiv este necesar a fi introdusă o linie fizică goală pentru a termina o instrucțiune ce se întinde pe mai multe linii.

2.1.8 Declarații multiple pe o singură linie

Punctul și virgula (;) permite scrierea mai multor instrucțiuni pe o singură linie, dat fiind că niciuna dintre instrucțiuni nu marchează începutul unui nou bloc de cod. De exemplu, linia următoare de cod:

```
nume = 'Vinte'; prenume = 'Claudiu'; print('Nume si prenume: ', nume, prenume)
```

este echivalentă cu secvența:

```
nume = 'Vinte'
prenume = 'Claudiu'
print('Nume si prenume: ', nume, prenume)
```

2.1.9 Gruparea declarațiilor în blocuri

Un grup de declarații individuale, care constituie un singur bloc de cod, se numește în Python suită. Instrucțiunile compuse sau complexe, cum ar fi `if`, `while`, `def` și `class` necesită o linie de antet și o suită.

Linia care constituie antetul marchează începutul declarației (utilizează un cuvânt-cheie) și se termină cu un caracterul două puncte (:). Această linie de antet este apoi urmată de una sau mai multe linii care alcătuiesc suita sau blocul de cod. De exemplu:

```
if nume :                               # antet
    print('Nume: ', nume)                # suita
    existaNume = True
elif prenume :                           # antet
    print('Prenume: ', prenume)          # suita
    existaPrenume = True
    existaNume = False
else :                                   # antet
    print('Nume si prenume indisponibile') # suita
    existaNume = False
    existaPrenume = False
```

2.2 Tipuri de variabile

Variabilele nu sunt altceva decât locații de memorie rezervate pentru a stoca valori. Aceasta înseamnă că, atunci când este creată o variabilă, este rezervat spațiu în memorie. În funcție de tipului de dată al unei variabile, interpretul alocă memorie și decide ce poate fi stocat în memoria rezervată. Prin urmare, prin atribuirea diferitelor tipuri de date variabilelor pot fi stocate în memorie numere întregi, numere în virgulă mobilă sau caractere.

2.2.1 Atribuirea de valori variabilelor

Variabilele Python nu au nevoie de declarații explicite cu privire la tip pentru a rezerva spațiul de memorie necesar. Declarația de tip este realizată în mod automat de către interpretor atunci când este atribuită o valoare unei variabile. Semnul egal (=) este utilizat pentru a atribui valori variabilelor.

Operandul din stânga operatorului = reprezintă numele variabilei, iar operandul din dreapta operatorului = este valoarea stocată în variabilă. De exemplu, secvența următoare de cod:

```
inaltime = 170           # un intreg ce reprezinta inaltimea in cm
greutate = 69.7          # un numar in virgula mobila
nume = "Apavaloaiei Maria" # un sir de caractere
```

```
print(nume)
print(inaltime)
print(greutate)
```

va genera rezultatul:

```
Apavaloaiei Maria
170
69.7
```

2.2.2 Atribuire multiplă

Python permite atribuirea unei singure valori mai multor variabile simultan. De exemplu:

```
a = b = c = 1
```

În acest caz, este creat un obiect de tip întreg cu valoarea 1 și toate cele trei variabile vor face referire la aceeași locație de memorie. De asemenea, pot fi atribuite mai multe obiecte mai multor variabile. De exemplu:

```
inaltime, greutate, nume = 170, 69.7, "Apavaloaiei Maria"
```

În această situație, în ordinea enumerării operanzilor din stânga și, corespunzător, a celor din dreapta operatorului =, un obiect de tip întreg este atribuit variabilei ce stochează înălțimea, un obiect de tip număr în virgulă mobilă este atribuit variabilei ce stochează greutatea și un obiect de tip șir de caractere este atribuit variabilei ce stochează numele.

2.2.3 Tipuri de date standard

Datele stocate în memorie pot fi de mai multe tipuri. De exemplu, greutatea unei persoane este stocată ca valoare numerică, iar adresa de reședință a acesteia este stocată ca șir de caractere alfanumerice. Python are diferite tipuri de date standard, care sunt utilizate pentru a defini operații ce pot fi realizate cu acestea și metoda de stocare pentru fiecare dintre ele.

Python are cinci tipuri de date standard:

- Numere
- Șir de caractere
- Listă
- Tuplu
- Dicționar

2.2.4 Testarea valorii de adevăr

În Python, orice obiect poate fi testat din perspectiva valorii de adevăr, situație ce poate apărea în contextul unei condiții de tip `if` sau `while` ori ca operand al operațiilor booleene de mai jos pe care le vom prezenta în continuare.

În mod implicit, un obiect este considerat adevărat, cu excepția cazului în care clasa sa definește fie o metodă `__bool__()` care returnează `False`, fie o metodă `__len__()` care returnează zero, atunci când este apelată cu obiectul ca parametru.

Prezentăm în continuare exemple de obiecte Python implicite, considerate a fi `False`:

- constante definite ca având cu valoarea de adevăr fals: `None` și `False`
- zero de orice tip numeric: `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0, 1)`
- secvențe și colecții goale: `"`, `()`, `[]`, `{}`, `set()`, `range(0)`

Python pune la dispoziția programatorului următorii operatori booleani: `and`, `or`, `not`. Operațiile și funcțiile implicite (de bază sau încorporate) care au un rezultat boolean, returnează întotdeauna `0` sau `False` pentru fals și `1` sau `True` pentru adevărat, cu excepția cazului în care se specifică altfel. Excepție importantă: operațiile booleene `or` și `and` returnează întotdeauna unul dintre operandii lor.

Prezentăm în continuare operațiile booleene, în ordinea crescătoare a priorității acestora (tabelul 2.2).

Tabelul 2.2 Operatorii booleani, în ordinea crescătoare a priorității

Operație	Rezultat	Comentariu
<code>x or y</code>	dacă <code>x</code> este fals, atunci <code>y</code> , altfel <code>x</code>	Acesta este un operator de scurtcircuit, în consecință evaluează cel de al doilea argument numai dacă primul este fals.
<code>x and y</code>	dacă <code>x</code> este fals, atunci <code>x</code> , altfel <code>y</code>	Acesta este un operator de scurtcircuit, deci evaluează al doilea argument numai dacă primul este adevărat.
<code>not x</code>	dacă <code>x</code> este fals, atunci adevărat, altfel fals	<code>not</code> are o prioritate mai mică decât operatorii non-booleeni, deci <code>not a == b</code> este interpretat ca <code>not (a == b)</code> , iar <code>a == not b</code> este o eroare de sintaxă.

2.2.5 Comparatii

Există opt operații de comparație în Python. Toate au aceeași prioritate, ce este mai mare decât cea a operațiilor booleene. Comparațiile pot fi înlănțuite în mod arbitrar; de exemplu, `x < y <= z` este echivalent cu `x < y and y <= z`, cu excepția faptului că `y` este evaluat o singură dată, însă în ambele cazuri `z` nu este evaluat deloc atunci când se constată că `x < y` este fals. Tabelul 2.3 sintetizează operațiile de comparare.

Tabelul 2.3 Operatorii de comparare

Operator	Semnificație
<code><</code>	strict mai mic decât
<code><=</code>	mai mic sau egal
<code>></code>	strict mai mare decât
<code>>=</code>	mai mare sau egal
<code>==</code>	egal
<code>!=</code>	nu este egal (diferit)
<code>is</code>	identitatea obiectului
<code>is not</code>	negare identitate obiect

Obiectele de diferite tipuri, cu excepția tipuri numerice, nu se compară niciodată cu operatorul de egalitate. Operatorul `==` este întotdeauna definit, dar pentru unele tipuri de obiecte, de exemplu obiecte reprezentând instanțe a unor clase, acesta este echivalent cu `is` (operatorul de verificare a identității). Operatorii `<`, `<=`, `>` și `>=` sunt definiți doar acolo unde au sens. Spre exemplu, ei aruncă o excepție de tip `TypeError` atunci când unul dintre argumente este un număr complex.

Instanțele unei clase care nu sunt neidentice se compară în mod normal prin operatorul `!=` (nu este egal), cu excepția cazului în care clasa definește metoda `__eq__()`.

Instanțele unei clase nu pot fi comparate și, în consecință, ordonate în raport cu alte instanțe din aceeași clasă sau cu alte tipuri de obiecte, cu excepția cazului în care clasa definește suficient de coerent metode de felul: `__lt__()`, `__le__()`, `__gt__()` și `__ge__()`. De cele mai multe ori, metodele `__lt__()` și `__eq__()` sunt suficiente, dacă se dorește oferirea funcționalității furnizate de operatorii convenționali de comparație.

Comportamentul operatorilor `is` și `is not` nu poate fi personalizat. De asemenea, acești operatori pot fi aplicați oricărui două obiecte și nu aruncă niciodată o excepție.

Operatorii `in` și `not in`, operatori ce au aceeași prioritate sintactică, sunt acceptați de către tipuri de obiecte care sunt iterabile sau a căror clase implementează metoda `__contains__()`.

2.2.6 Tipuri de date numerice

Tipurile de date numerice stochează valori numerice. Obiectele numerice sunt create atunci când este atribuită o valoare numerică unei variabile. De exemplu, putem avea:

```
variabila_1 = 1
variabila_2 = 3.14
```

Python oferă posibilitatea de ștergere referința către un obiect de tip numeric prin utilizarea instrucțiunii `del`. Sintaxa pentru utilizarea declarației `del` este următoarea:

```
del var1 [, var2 [, var3 [... , varN]]]
```

Se observă din sintaxă că prin utilizarea instrucțiunii `del`, poate fi șters unul sau multe obiecte printr-o singură utilizare. De exemplu, rularea următoarei secvențe de cod:

```
variabila_1 = 1
variabila_2 = 3.14
variabila_3 = 100.

del variabila_1
del variabila_2, variabila_3
print(variabila_3)
```

va genera rezultatul:

```
print(variabila_3)
NameError: name 'variabila_3' is not defined
```

Python acceptă 3 tipuri diferite de date numerice:

- `int` (numere întregi cu semn) – acestea sunt adesea numite doar numere întregi și modelează conceptul matematic de numere întregi pozitive sau negative fără punct zecimal; numerele întregi din Python 3 sunt de dimensiuni nelimitate și nu mai există timpul de număr întreg `long` în Python 3.
- `float` (valori reale în virgulă mobilă) – acestea reprezintă numere reale și sunt scrise cu un punct zecimal ce separă partea întreagă de partea fracționară a numărului real; pot fi scrise, de asemenea în notatie științifică, cu `E` sau `e`, indicând puterea lui 10 ($3.14e2 = 3.14 \times 10^2 = 314$).
- `complex` (numere complexe) – acestea sunt de forma `a + bj`, unde `a` și `b` sunt numere de tip `float`, iar `J` (sau `j`) reprezintă radical din -1 (care este un număr imaginar); partea reală a numărului este `a`, iar partea imaginară este `b`; trebuie recunoscut că numerele complexe nu sunt utilizate foarte des în programarea Python.

Un număr întreg poate să fie reprezentat și în formă hexazecimală sau octală. Un număr complex constă dintr-o pereche ordonată de numere reale în virgulă mobilă notate cu `a + bj`, unde `a` este partea reală și `b` este partea imaginară a numărului complex. Tabelul 2.4 prezintă câteva exemple de valori pentru tipurile de date numerice.

Tabelul 2.4 Exemple de valori pentru tipurile de date numerice

int	float	complex
10	0.0	3.14j
100	3.14	15.j
-786	-22.1	7.321e-27j
080	32.1+e11	.256j
-0430	-910.	-.1354+0j
-0xA0E	-22.51e120	23e+7j
0o67	67.23-E12	1.23e-5j

Pentru evaluarea unei expresii ce conține tipuri mixte de date numerice, interpretorul Python convertește intern numerele la un tip numeric comun. Uneori este necesar să convertim în mod explicit un număr de la un tip la altul, pentru a satisface cerințele unui parametru, operator sau funcție. Avem la dispoziție următoarele funcții de conversie:

- `int(x)` pentru a converti `x` într-un întreg;
- `float(x)` pentru a converti `x` într-un număr reprezentat în virgulă mobilă;
- `complex(x)` pentru a converti `x` într-un număr complex cu partea reală `x` și partea imaginară zero;
- `complex(x, y)` pentru a converti `x` și `y` într-un număr complex cu partea reală `x` și partea imaginară `y`; `x` și `y` sunt expresii numerice.

În pachetul `math` sunt definite și implementate funcții de lucru cu date de tip numeric: funcții matematice, funcții pentru lucru cu numere aleatorii, funcții trigonometrice, constante numerice. Pentru mai multe detalii cu privire la funcțiile definite în pachetul `math` poate fi consultată documentația de la link-ul următor: <https://docs.python.org/3/library/math.html>

2.2.7 Șiruri de caractere

Datele de tip text din Python sunt tratate cu obiecte `str` sau șiruri de caractere. În Python, șirurile de caractere sunt secvențe imuabile de semne codificate în Unicode. Șirurile de caractere în Python sunt identificate ca un set contiguu de caractere încadrate de apostrofuri sau de ghilimele. Python permite fie perechi de apostrofuri, fie perechi de ghilimele pentru delimitarea șirurilor de caractere. Crearea șirurilor este la fel de simplă ca atribuirea unei valori unei variabile. Semnul plus (+) este operatorul de concatenare a șirurilor și asteriscul (*) este operatorul de repetare. De exemplu, variabilelor definite mai jos le vor fi atribuite șirurile de caractere specificate în operandul din dreapta operatorului de asignare:

```
sir1 = 'Acesta est eun sir de caractere Python'
sir2 = "Si acesta este un sir de caractere Python"
```

2.2.7.1 Accesarea valorilor în șiruri

Python nu are un tip de date care să modeleze un singur caracter. Tipul caracter este modelat ca un șir de caractere de lungime 1.

Pentru a accesa caractere individuale sau subșiruri dintr-un șir de caractere sunt utilizate parantezele pătrate. Subseturile de șiruri pot fi preluate dintr-un șir de caractere folosind operatorul de partiționare (`[]` și `[:]`) cu indexuri care încep de la 0 pentru începutul șirului și merg până la sfârșitul șirului (lungimea șirului -1). Rulând secvența de cod următoare:

```
sir1 = 'Acesta est eun sir de caractere Python'
sir2 = "Si acesta este un sir de caractere Python"

print("sir1[0]: ", sir1[0])
print("sir2[4:9]: ", sir2[3:9])
print("Tot sirul sir1: ", sir1[:])
print("Ultimul caracter al sirului sir2: ", sir2[-1])
```

vom obține rezultatele de mai jos:

```
sir1[0]: A
sir2[4:9]: acesta
Tot sirul sir1: Acesta est eun sir de caractere Python
Ultimul caracter al sirului sir2: n
```

Este de remarcat faptul că, în Python, vom discuta acesta aspect și în cazul listelor și a dicționarelor, indexul poate să ia și valori negative. În exemplul anterior, `sir2[-1]` are semnificația de index pentru ultimul caracter al șirului. În acest context, putem accesa caracterele unui șir de la dreapta la stânga utilizând indecși negativi.

Șirurile de caractere Python pot fi percepute ca un caz particular de listă Python, situație în care elementele listei sunt constituite din caractere. În acest context, metoda `len()` returnează numărul efectiv de caractere din cadrul șirului.

2.2.7.2 Caractere de control al tipăririi

Prezentăm o listă de caractere de control al tipăririi în tabelul 2.5, caractere ce nu pot fi imprimare și care sunt prefixate cu notația `\` (backslash). Un caracter de control al tipăririi este interpretat ca un caracter de evadare (*escape*) în interiorul unui șir de caractere citat fie între apostrofuri, fie între ghilimele.

Tabelul 2.5 Caractere de control al tipăririi

Notăție cu \ (backslash)	Caracter hexadecimal	Descriere
<code>\a</code>	0x07	Clopoțel sau alertă
<code>\b</code>	0x08	<i>Backspace</i>
<code>\cx</code>		Control-x
<code>\C-x</code>		Control-x
<code>\e</code>	0x1b	Evadare (<i>Escape</i>)
<code>\f</code>	0x0c	<i>Formfeed</i>
<code>\M-\C-x</code>		Meta-Control-x
<code>\n</code>	0x0a	Linie nouă
<code>\nnn</code>		Notare octală, unde n este în intervalul [0, 7]
<code>\r</code>	0x0d	Retur cursor
<code>\s</code>	0x20	Spațiu
<code>\t</code>	0x09	Tab
<code>\v</code>	0x0b	Tab verticală
<code>\x</code>		Caracterul x
<code>\xnn</code>		Notare hexadecimală, unde n este în intervalul [0, 9], [a, f] sau [A, F]

2.2.7.3 Citarea triplă

Citarea triplă în Python este foarte utilă atunci când avem nevoie ca șirul de caractere pe care vrem să-l cităm să conțină chiar caracterele apostrof și/sau ghilimele.

De asemenea, oferă posibilitatea de a cita un șir de caractere care se poate întinde pe mai multe linii sau care conține caractere de control al tipăririi și orice alte caractere speciale.

Sintaxa pentru citare triplă constă într-o succesiune de trei apostrofuri sau ghilimele consecutive. De exemplu, șirul declarat în continuare:

```
sirLung = """Acesta este un sir ce
se intinde pe mai multe linii si contine urmatorul citat
in limba engleza: \n
\t>Please don't rush it that much", the couch told him.\n
Ati observat si caracterele de control al tiparirii, pentru
trecerea explicita la o noua linie si inserarea unui Tab."""
print (sirLung)
```

va produce, în urma tipăririi la consolă, următorul rezultat:

```
Acesta este un sir ce
se intinde pe mai multe linii si contine urmatorul citat
in limba engleza:
```

```
"Please don't rush it that much", the couch told him.
```

```
Ati observat si caracterele de control al tiparirii, pentru
trecerea explicita la o noua linie si inserarea unui Tab.
```

Trebuie remarcat faptul că șirurile marcate ca șirurile brute (*raw*) nu tratează *backslash-ul* (`\`) ca pe un caracter special. Fiecare caracter care este inclus într-un șir care este marcat ca fiind un șir brut, rămâne așa cum este scris. Un șir brut este precedat de caracterul `"r"`.

De exemplu, dacă dorim să specificăm următoare cale către un director citat între ghilimele:

```
print ('Directorul de lucru este F:\De lucru\pentru scoala')
```

am obține următorul rezultat:

```
Directorul de lucru este F:"De lucru"pentru scoala"
```

Prin specificarea faptului că dorim un șir de caractere brut, prefixând șirul cu `"r"`, putem prezerva conținutul șirului și preveni interpretarea *backslash-ul* (`\`) drept caracter de control al tipăririi.

```
print (r'Directorul de lucru este F:\De lucru\pentru scoala')
```

Executând tipărirea de mai sus obținem:

```
Directorul de lucru este F:\De lucru\pentru scoala"
```

2.2.7.4 Modificarea șirurilor de caractere

Un șir de caractere poate fi „actualizat” reatribuind variabilei ce stochează șirul, un alt șir de caractere. Noua valoare a variabilei poate să aibă vreo legătură cu valoarea sa anterioară sau poate fi un șir complet diferit. Din această perspectivă șirurile de caractere din Python sunt foarte apropiate de obiectele de tip `String` din limbajul Java, de exemplu, care sunt instanțe imutabile. Spre exemplu, secvența următoare de cod:

```
sir = 'Sirul initial'
sir = 'Sirul "modificat"'
print('Valoare stocata in variabila sir: ', sir)
```

va genera următorul rezultat la consolă:

Valoare stocata in variabila sir: Sirul "modificat"

Șirurile de caractere pot conține caractere de *escape*, care au aceeași semnificație moștenită din limbajul de programare C. Acestea sunt caractere de control al tipăririi, dar care nu pot fi tipărite și sunt reprezentate cu notația de tip *backslash*. De exemplu, secvența din șirul `'\r\n'` generează <CR><LF>, respectiv mutarea cursorului tipăririi la începutul liniei următoare.

2.2.7.5 Operatori speciali pentru șiruri de caractere

Pentru exemplificarea utilizării operatorilor Python pentru șiruri de caractere, să presupunem că variabila șir `x` conține „Salut”, iar variabila șir `y` conține „Python”. Tabelul 2.6 prezintă identificatorul operatorului, semnificația operatorului atunci când este folosit cu operanzi șiruri de caractere și rezultatul utilizării operatorului împreună cu operanzii `x` și `y`.

Tabelul 2.6 Operatori pentru șiruri de caractere

Operator	Descriere	Exemplificare utilizare
<code>+</code>	Concatenare – adaugă valori de ambele părți ale operatorului	<code>x + y</code> va da SalutPython
<code>*</code>	Repetare – creează șiruri noi, concatenând mai multe copii ale aceluiași șir	<code>x * 2</code> va da SalutSalut
<code>[]</code>	Index – furnizează caracterul de la indexul dat	<code>x[0]</code> va da S
<code>[:]</code>	Partiționare – furnizează subșirul de caractere din intervalul dat	<code>x[2:4]</code> va da lut
<code>in</code>	Apartenență – returnează adevărat dacă există un caracter în șirul dat	<code>S in x</code> va da 1
<code>not in</code>	Neapartenență – returnează adevărat dacă un caracter nu există în șirul dat	<code>X not in x</code> va da 1

Operator	Descriere	Exemplificare utilizare
<code>r/R</code>	Șir brut – suprimă semnificația reală a caracterelor de <i>escape</i> . Sintaxa pentru șirurile brute este exact aceeași ca și pentru șirurile normale, cu excepția operatorului șirului brut, litera „r”, care precedă ghilimelele. „R” poate fi cu litere mici (r) sau cu majuscule (R) și trebuie plasat imediat înaintea primului caracter apostrof sau ghilimele.	<code>print(r'\n')</code> va tipări \n <code>print(R"\t")</code> va tipări \t
<code>%</code>	Format – efectuează formatarea șirului	A se vedea secțiunea următoare

2.2.7.6 Operatori de formatare a șirurilor de caractere

Una dintre caracteristicile ce trădează puternica legătură dintre limbajul Python și limbajul C este operatorul `%` utilizat pentru de formatarea șirurilor de caractere. Acest operator este unic pentru șiruri și are o utilizare similară cu cea întâlnită în funcțiile de tipărire din familia `printf()` din limbajul C. În exemplul următor:

```
print ("Sunt student la %s si am %d de ani" % ("CSIE - ASE", 20))
```

atunci când codul este executat, obținem rezultatul:

Sunt student la CSIE - ASE si am 20 de ani

Lista de mai jos conține setului complet de simboluri care pot fi utilizate împreună cu operatorul `%`:

- `%c` caracter
- `%s` conversia șirului prin apelul funcției `str()` înainte de formatare
- `%i` număr întreg cu sem
- `%d` număr zecimal cu sem
- `%u` număr zecimal fără sem
- `%o` întreg octal
- `%x` întreg hexazecimal (litere mici)
- `%X` întreg hexazecimal (litere MAJUSCULE 'E')
- `%e` număr real în notație științifică cu exponent (cu litere mici „e”)
- `%E` număr real în notație științifică cu exponent (cu litere MAJUSCULE 'E')
- `%f` număr real în virgulă mobilă
- `%g` cea mai scurtă formă dintre `%f` și `%e`
- `%G` cea mai scurtă formă dintre `%f` și `%E`

2.2.7.7 Șiruri Unicode

În Python 3, toate șirurile sunt reprezentate în Unicode. În Python 2 șirurile de caractere erau stocate intern pe 8 octeți folosindu-se codul ca ASCII. În aceste condiții, era necesar să atașăm la început caracterul „u” pentru a face șirurile Unicode. În Python 2 acest lucru nu mai este necesar.

Python include o multitudine de metode încorporate pentru a lucra cu șiruri de caractere Unicode. Pentru detalii privind denumirea acestor metode și descrierea funcționalității acestora, se poate consulta pagina de web: <https://docs.python.org/3/library/stdtypes.html#string-methods>

2.3 Operatori fundamentali

Operatorii sunt entități care pot manipula valoarea operanzilor. Limbajul Python acceptă următoarele tipuri de operatori:

- operatori aritmetici
- operatori de comparație (relaționali)
- operatori de asignare
- operatori logici
- operatori logici pe biți (*bitwise*)
- operatori de testare a apartenenței
- operatori de verificare a identității

În continuare vom prezenta succint fiecare din aceste tipuri de operatori.

2.3.1 Operatori aritmetici

Să presupunem că variabila *a* deține valoarea 11 și variabila *b* deține valoarea 7, atunci operatorii aritmetici, aplicați asupra acestor doi operanzi, au semnificația și produc rezultatele prezentate în tabelul 2.7.

Tabelul 2.7 Operatori aritmetici

Operator	Semnificație	Exemplu de utilizare
+ Adunare	Adaugă valori de ambele părți ale operatorului.	$a + b = 18$
- Scădere	Scade operandul din partea dreaptă din operandul din partea stângă.	$a - b = 4$
* Înmulțire	Înmulțește valorile de ambele părți ale operatorului	$a * b = 77$
/ Împărțire	Împarte operandul din partea stângă la operandul din partea dreaptă	$a / b = 1.57$

Operator	Semnificație	Exemplu de utilizare
% Modul	Împarte operandul din partea stângă la operandul din partea dreaptă și returnează restul	$a \% b = 4$
** Exponent	Efectuează calculul de ridicare la putere a valorii furnizate în partea stângă, cu din partea dreaptă a operatorului.	$a ** b = 11$ la puterea 7
//	Câtul împărțirii operandului din partea stângă la operandul din partea dreaptă. Dacă însă unul dintre operanzi este negativ, atunci rezultatul este rotunjit către următorul număr negativ mai mic decât câtul (<i>floor division</i>).	$a // b = 1$ și $(-a) // b = -2$

2.3.2 Operatori de comparație

Acești operatori compară valorile aflate de ambele părți ale acestora și decid relația dintre aceste valori. Se mai numesc și operatori relaționali. Să presupunem că variabila *a* are valoarea 1 și variabila *b* are valoarea 2, atunci operatorii de comparație, aplicați asupra acestor doi operanzi, au semnificația și produc rezultatele prezentate în tabelul 2.8.

Tabelul 2.8 Operatori de comparație

Operator	Semnificație	Exemplu de utilizare
==	Dacă valorile celor doi operanzi sunt egale, atunci condiția este adevărată.	$(a == b)$ nu este adevărat
!=	Dacă valorile celor doi operanzi nu sunt egale, atunci condiția este adevărată.	$(a != b)$ este adevărat
>	Dacă valoarea operandului stâng este mai mare decât valoarea operandului drept, atunci condiția este adevărată.	$(a > b)$ nu este adevărat
<	Dacă valoarea operandului stâng este mai mică decât valoarea operandului drept, atunci condiția este adevărată.	$(a < b)$ este adevărat
>=	Dacă valoarea operandului din stânga este mai mare sau egală cu valoarea operandului din dreapta, atunci condiția este adevărată.	$(a >= b)$ nu este adevărat
<=	Dacă valoarea operandului stâng este mai mică sau egală cu valoarea operandului stâng, atunci condiția este adevărată.	$(a <= b)$ este adevărat

2.3.3 Operatori de atribuire

Să presupunem variabilele *a*, *b*, și *c*. Operatorii de atribuire, utilizați împreună cu acești doi operanzi, au semnificația și produc rezultatele prezentate în tabelul 2.9:

Tabelul 2.9 Operatori de atribuire

Operator	Semnificație	Exemplu de utilizare
=	Atribuire valorile operandilor din partea dreaptă la operandul din partea stângă.	$c = a + b$ atribuie valoarea lui $a + b$ în c
+= Adunare și atribuire	Adună operandul drept la operandul stâng și atribuie rezultatul operandului stâng.	$c += a$ este echivalent cu $c = c + a$
-- Scădere și atribuire	Scade operandul drept din operandul stâng și atribuie rezultatul operandului stâng.	$c -= a$ este echivalent cu $c = c - a$
*= Înmulțire și atribuire	Înmulțește operandul drept cu operandul stâng și atribuie rezultatul operandului stâng.	$c *= a$ este echivalent cu $c = c * a$
/= Împărțire și atribuire	Împarte operandul stâng cu operandul drept și atribuie rezultatul operandului stâng.	$c /= a$ este echivalent cu $c = c / a$
%= Modul și atribuire	Împarte operandul stâng la operandul drept și atribuie restul operandului stâng.	$c \% = a$ este echivalent cu $c = c \% a$
**= Exponent și atribuire	Efectuează calculul de ridicare la putere a operandului stâng cu valoarea din partea dreaptă a operatorului și atribuie rezultatul operandului stâng.	$c ** = a$ este echivalent cu $c = c ** a$
//=	Calculează câtul împărțirii operandului din partea stângă la operandul din partea dreaptă și atribuie rezultatul operandului stâng. Dacă unul dintre operanzi este negativ, atunci rezultatul este rotunjit către următorul număr negativ mai mic decât câtul (<i>floor division</i>) și rezultatul este atribuit operandului stâng.	$c //= a$ este echivalent cu $c = c // a$

2.3.4 Operatori logici pe biți (bitwise)

Operatorii *bitwise* lucrează pe biți și efectuează operații bit-cu-bit. Funcția implicită Python `bin()` poate fi utilizată pentru a obține reprezentarea binară a unui număr întreg.

Să presupunem că avem variabilele $a = 7$ și $b = 4$.

```
a = 7
b = 4
print('a =', bin(7))
print('b =', bin(4))
```

Secvența de cod de mai sus va tipări la consolă următorul rezultat.

```
a = 0b111
b = 0b100
```

Operatorii logici pe biți, utilizați împreună cu operandii de mai sus, au semnificația și produc rezultatele prezentate în tabelul 2.10:

Tabelul 2.10 Operatori logici pe biți

Operator	Semnificație	Exemplu de utilizare
& ȘI pe biți	Operatorul copiază bitul 1 în rezultat, dacă biții de pe aceeași poziție sunt 1 în ambii operanzi.	$(a \& b) = 4$ (înseamnă 0100)
 SAU pe biți	Copiază bitul setat pe 1 dacă există în oricare operand.	$(a b) = 7$ (înseamnă 0111)
^ SAU exclusiv pe biți	Copiază bitul dacă este setat pe 1 într-un singur operand, dar nu în ambele.	$(a \wedge b) = 3$ (înseamnă 0011)
~ Complement pe biți	Este un operator unar și are ca efect inversarea biților: 1 devine 0 și 0 devine 1.	$(\sim a) = -8$ (înseamnă 1000 în forma de complement 2 datorită unui număr binar semnat.
<< mutare pe biți la stanga	Valoarea operandului din stânga este mutată la stânga de numărul de biți specificat de operandul din dreapta.	$a << 2 = 28$ (înseamnă 0001 1100)
>> mutare pe biți la dreapta	Valoarea operandului din stânga este mutată la dreapta de numărul de biți specificat de operandul din dreapta.	$a >> 2 = 1$ (înseamnă 001)

2.4 Structuri de control

2.4.1 Structura alternativă

Structura alternativă are următoarea sintaxă:

```
if boolean_expression1:
    blocul 1
elif boolean_expression2:
    blocul 2
...
elif boolean_expressionN:
    blocul N
else:
    blocul else
```

unde *blocurile* sunt secvențe de cod executate ca rezultat al evaluării expresiilor booleene.

În unele situații structura alternativă poate fi înlocuită cu expresia condițională:

```
expression1 if boolean_expression else expression2
```


2.4.2 Structuri repetitive

2.4.2.1 Structura while

```
while boolean_expression:
    blocul while
else:
    blocul else
```

Ramura `else` este opțională.

2.4.2.2 Structura for

```
for expression in iterable:
    blocul for
else:
    blocul else
```

Ramura `else` este opțională. Termenul *iterable* indică faptul că trebuie să fie furnizat un obiect ce conține o colecție iterabilă de valori.

2.5 Tratarea excepțiilor

În Python tratarea excepțiilor este opțională și se realizează prin instrucțiunea `try`:

```
try:
    bloc try
except exception_group1 as variable1:
    bloc excepție group1
...
except exception_groupN as variableN:
    bloc excepție groupN
else:
    bloc else
finally:
    bloc finally
```

Obiectele excepție sunt captate în variabilele *variable1*, ..., *variableN*. Blocul *finally* este opțional și se execută la sfârșit, structurii, indiferent de captarea sau nu a vreunei excepții.

2.6 Utilizarea funcțiilor în Python

O funcție este un bloc de cod organizat, reutilizabil, care este folosit pentru a efectua o singură acțiune asociată. Funcțiile oferă o mai bună modularitate în organizarea aplicațiilor și un grad ridicat de reutilizare a codului.

Limbajul de programare Python oferă multe funcții încorporate (implicite), cum ar fi de exemplu `print()` etc., dar putem să ne definim propriile noastre funcții. Aceste funcții se numesc funcții definite de utilizator.

2.6.1 Definirea unei funcții

Putem defini funcții pentru a furniza funcționalitatea și modularitatea necesare programului avut în vedere ca soluție informatică. În continuare, vom trece succint în revistă câteva reguli simple pentru a defini o funcție în Python.

Blocurile funcționale încep cu cuvântul-cheie `def`, urmat de numele funcției și paranteze rotunde `()`. Orice parametri de intrare sau argumente trebuie plasate între aceste paranteze. De asemenea, pot fi definiți parametri în interiorul acestor paranteze.

Prima instrucțiune a unei funcții poate fi o instrucțiune opțională – șirul de documentare al funcției sau *docstring*.

Blocul de cod din fiecare funcție începe cu două puncte `:` și este utilizată indentația.

Instrucțiune `return [expresie]` este cea care încheie corpul unei funcții și care, în mod opțional, transmite valoarea expresiei către programul apelant. O declarație `return` fără argumente este aceeași cu `return None`.

Sintaxa generală este următoarea:

```
def nume_functie(parametri):
    "șirul de documentare"

    corpul funcției

    return [expresie]
```

În mod implicit, parametrii au un comportament pozițional și trebuie să fie pasați la momentul apelului în aceeași ordine în care au fost definiți.

De exemplu, următoarea funcție ia un număr ca parametru de intrare și îl imprimă la dispozitivul standard de ieșire.

```
def tiparireNumar(numar):
    "Aceasta functie trimite un numar la dispozitivul standard de iesire"
    print (numar)
    return None
```

Apelul ei, într-o altă funcție sau direct în promptul Python:

```
tiparireNumar(11)
```

va genera următorul rezultat la consolă:

2.6.2 Pasarea parametrilor în funcții

Toți parametrii (argumentele) în limbajul Python sunt pasați prin referință sau mai precis ca referință la obiecte. Aceasta implică faptul că, dacă modificăm valoarea la care face referință un parametru în cadrul unei funcții, schimbarea se reflectă și înapoi, în funcția apelatoare. De exemplu, secvența următoare de cod:

```
def interschimb(lista):
    "Aceasta functie schimba valori la referinta primita ca parametru"
    print('in functia interschimb(lista):')
    print('identificator lista=', id(lista))
    print('identificator lista[0]=', id(lista[0]))
    print('identificator lista[1]=', id(lista[1]))

    local = lista[0]
    lista[0] = lista[1]
    lista[1] = local

    print('identificator lista[0]=', id(lista[0]))
    print('identificator lista[1]=', id(lista[1]))

    return None

lista_1 = [1, 2]
print('lista=', lista_1)
print('identificator lista=', id(lista_1))
print('identificator lista[0]=', id(lista_1[0]))
print('identificator lista[1]=', id(lista_1[1]))

interschimb(lista_1)
print('dupa interschimb(lista):')
print('lista=', lista_1)
print('identificator lista=', id(lista_1))
print('identificator lista[0]=', id(lista_1[0]))
print('identificator lista[1]=', id(lista_1[1]))
```

va genera, în urma rulării, următorul rezultat:

```
lista= [1, 2]
identificator lista= 2963393157960
identificator lista[0]= 140734694650128
identificator lista[1]= 140734694650160
in functia interschimb(lista):
identificator lista= 2963393157960
identificator lista[0]= 140734694650128
identificator lista[1]= 140734694650160
identificator lista[0]= 140734694650160
identificator lista[1]= 140734694650128
dupa interschimb(lista):
lista= [2, 1]
identificator lista= 2963393157960
identificator lista[0]= 140734694650160
identificator lista[1]= 140734694650128
```

Se poate observa că în interiorul funcției `interschimb(lista)` au modificate valori ale obiectului `lista_1` primit ca referință din programul apelator. Urmărind identificatorul fiecărui obiect, furnizat ca valoare unică de funcția implicită Python `id(obiect)`, putem confirma că referința la parametrul pasat în funcție nu s-a modificat, în schimb au fost modificate valorile conținute în obiectul listă la care s-a făcut referire.

Următorul exemplu este o funcție ce primește ca parametri referințele către două obiecte și rezultatul interschimbului de referințe nu se reflectă în programul apelator:

```
def interschimb(a, b):
    "Aceasta functie schimba referinte"
    print('in functia interschimb:')
    print('identificator a=', id(a))
    print('identificator b=', id(b))

    local = a
    a = b
    b = local

    print('identificator a=', id(a))
    print('identificator b=', id(b))

    return None

x = 1
y = 2
print('x=', x, 'y=', y)
print('identificator x=', id(x))
print('identificator y=', id(y))

interschimb(x, y)

print('dupa interschimb:')
print('x=', x, 'y=', y)
print('identificator x=', id(x))
print('identificator y=', id(y))
```

Rezultatul rulării acestei secvențe de cod este următorul:

```
x= 1 y= 2
identificator x= 140734694650128
identificator y= 140734694650160
in functia interschimb:
identificator a= 140734694650128
identificator b= 140734694650160
identificator a= 140734694650160
identificator b= 140734694650128
dupa interschimb:
x= 1 y= 2
identificator x= 140734694650128
identificator y= 140734694650160
```

și confirmă nerealizarea interschimbului de valori pentru obiectele ale căror referință au fost primite ca parametri pentru că nu au fost modificate valori de la acele referințe în interiorul funcției `interschimb(a, b)`.

În Python, o funcție poate fi apelată utilizând următoarele tipuri de argumente formale:

- argumente necesare
- argumente cu cuvinte-cheie
- argumente implicite
- argumente de lungime variabilă

2.6.2.1 Argumente necesare

Argumentele necesare sunt argumentele transmise unei funcții în ordinea pozițională corectă. În aceste condiții, numărul de argumente din apelul funcției trebuie să se potrivească exact cu definiția funcției. De exemplu, apelul fără parametru al funcției următoare:

```
def interschimb(lista):
    "Aceasta functie schimba valori la referinta primita ca parametru"
    local = lista[0]
    lista[0] = lista[1]
    lista[1] = local
    return None
```

```
interschimb()
```

va genera în urma rulării următoarea eroare:

```
TypeError: interschimb() missing 1 required positional argument: 'lista'
```

2.6.2.2 Argumente cu cuvinte-cheie

Argumente cu cuvinte-cheie privesc, în primul rând, apelurile de funcții. Când utilizați argumente cu cuvinte-cheie într-un apel de funcție, apelantul identifică argumentele după numele parametrului. Acest lucru ne permite să ignorăm anumite argumente din lista de argumente a funcției sau să plasăm argumentele la apel în altă ordine față de cea în care au fost furnizate la definirea funcției. Interpretul Python poate folosi cuvintele-cheie furnizate la apel pentru a face corespondența valorilor cu parametri.

```
def identitateStudent(nume, prenume):
    print('Nume:', nume)
    print('Prenume:', prenume)
    return
```

```
identitateStudent(nume='Vinte', prenume='Claudiu')
```

La rularea secvenței de cod anterioare vom obține următorul rezultat tipărit la consolă:

```
Nume: Vinte
Prenume: Claudiu
```

2.6.2.3 Argumente implicite

Un argument implicit este un argument care își asumă o valoare implicită, dacă o valoare nu este furnizată în apelul de funcție pentru acel argument. Următorul exemplu oferă o idee despre argumentele implicite, imprimă denumirea programului de studii universitare, dacă acesta nu este furnizat în mod explicit la apelul funcției:

```
def identitateStudent(nume, prenume, program='Informatica Economica'):
    print('Nume:', nume)
    print('Prenume:', prenume)
    print('Denumire program studii:', program)
    return None
```

```
identitateStudent(nume='Vinte', prenume='Claudiu')
```

La consola va fi tipărit următorul rezultat:

```
Nume: Vinte
Prenume: Claudiu
Denumire program studii: Informatica Economica
```

2.6.2.4 Argumente de lungime variabilă

Sunt situații când este necesar apelăm o funcție cu mai multe argumente decât am specificat la momentul definirii acesteia. Aceste argumente se numesc argumente de lungime variabilă și nu sunt denumite în definiția funcției, spre deosebire de argumentele obligatorii și implicite. Sintaxa pentru o funcție cu argumente de lungime variabilă este următoarea:

```
def nume_functie(*var_args_tuple [,formal_args]):
    "șirul de documentare"

    corpul funcției

    return [expresie]
```

Un asterisc (*) este plasat înaintea numelui variabilei care deține valorile tuturor argumentelor ce nu au în mod individual o denumire explicit furnizată. Acest tuplu rămâne gol dacă nu sunt specificate argumente suplimentare în timpul apelului de funcție. Următorul este un exemplu prezintă un apel pentru un număr diferite de note pentru un student:

```
def noteStudent(*note, nume='Popescu', prenume='Andrei'):
    "Funcția tipărește un număr variabil de argumente"
    print(nume, prenume)
    print('Note obtinute:')
    for nota in note:
        print(nota)
    return None
```

```
noteStudent()
noteStudent(9, 10, 8, 10)
```

Rularea secvenței de cod anterioare va genera la consolă următorul rezultat:

```
Popescu Andrei
Note obtinute:
Popescu Andrei
Note obtinute:
9
10
8
10
```

2.6.2.5 Funcțiile anonime (lambda)

Aceste funcții sunt denumite anonime, deoarece nu sunt declarate în mod standard prin utilizarea cuvântului-cheie `def`. Poate fi utilizat cuvântul-cheie **lambda** pentru a crea mici funcții anonime. Formele lambda pot lua orice număr de argumente, dar returnează doar o valoare sub forma unei expresii. Nu pot conține comenzi sau expresii multiple.

O funcție anonimă nu poate fi un apel direct la imprimare, deoarece lambda necesită o expresie.

Funcțiile lambda au propriul spațiu de nume local și nu pot accesa alte variabile decât cele din lista lor de parametri și cele din spațiul de nume global.

Deși poate părea că funcțiile lambda sunt o versiune cu o singură linie a unei funcții, ele nu sunt echivalente cu instrucțiunile `inline` în C sau C++, al căror scop este de nu mai încărca pe stivă o funcție la momentul invocării acesteia din motive de performanță.

Sintaxa funcțiilor lambda conține doar o singură declarație, care este după cum urmează:

```
lambda [arg1 [,arg2,...argn]]:expresie
```

Următoare secvență de cod arată cum funcționează o funcție lambda.

```
# Expresie lambda pentru calculul mediei a doua numere
medie = lambda arg1, arg2: (arg1 + arg2) / 2

# Tiparire la consola a valorii medii
print ("Media=", medie(1, 2))
print ("Media=", medie(2, 3))
print ("Media=", medie(3, 4))
```

La consolă va tipărit următorul rezultat în urmă rulării:

```
Media= 1.5
Media= 2.5
Media= 3.5
```

2.6.2.6 Instrucțiunea return

Instrucțiunea `return [expresie]` semnifică ieșirea dintr-o funcție, opțional trimițând înapoi o expresie către programul apelant. O declarație `return` fără argumente este aceeași cu `return None`.

În toate exemplele anterioare nu am returnat nicio valoare dintr-o funcție. În exemplul următor funcția va returna valoarea medie a notelor studentului.

```
def medieStudent(*note, nume='Popescu', prenume='Andrei'):
    "Funcția calculează media pentru un numar variabil de note"
    print(nume, prenume)
    media = 0
    for nota in note:
        media += nota
    return media / len(note)
```

```
print(medieStudent(9, 10, 8, 10))
```

Rezultatul tipărit la consolă în urma execuției este următorul:

```
Popescu Andrei
9.25
```

2.6.3 Decoratori

Decoratorii sunt funcții care extind funcționalitatea unor alte funcții. Funcția decorator primește ca argument referința la funcția a cărei funcționalitate este extinsă și definește o funcție nouă, funcție internă, prin care este extinsă funcționalitatea funcției primite. Decoratorul întoarce referința la funcția nou definită.

Decoratorii sunt marcați sintactic printr-o adnotare de forma:

```
@nume_decorator
```

unde `nume_decorator` este numele funcției decorator. Adnotarea se face în fața funcției decorate (a cărei funcționalitate este extinsă).

În exemplul următor vom defini un decorator pentru o funcție care afișează un mesaj. În modulul `decorators.py` este creat decoratorul și sunt definite două funcții de afișare, dintre care una va fi adnotată cu decoratorul creat.

```
def afisare(mesaj):
    print(mesaj)

def decorator_afisare(funcție):
    def afisare_wrapper(mesaj):
        print("Mesaj:", end="")
        funcție(mesaj)
    return afisare_wrapper
```

```
@decorator_afisare
```

```
def afisare1(mesaj):
    print(mesaj)
```

În modulul *main.py* este testat decoratorul.

```
import decoratori as decor
```

```
decorator.afisare("Salut Python!")
```

```
# Testare decorator fara adnotare
functie_decorator = decor.decorator_afisare(decorator.afisare)
functie_decorator("Salut Python!")
```

```
#Test decorator
decor.afisare1("La revedere Python!")
```

Programul va afișa în urma rulării:

```
Salut Python!
Mesaj:Salut Python!
Mesaj:La revedere Python!
```

2.7 Tipuri de date secvențiale

Structura de date fundamentală în Python este secvența. Fiecărui element al unei secvențe i se atribuie un număr ce reprezintă poziția sau indexul său. Primul index este zero, al doilea index este unul și așa mai departe.

Python are șase tipuri de date secvențiale implicite, dar cele mai frecvente sunt listele, tuplurile și obiectele de tip interval de valori. Există elemente comportamentale și facilități de utilizare care se întâlnesc la toate datele de tip secvențial. Aceste operațiuni includ indexarea, partiționarea, adăugarea, multiplicarea și verificarea calității de membru. În plus, Python are funcții implicite pentru determinarea lungimii unei secvențe și pentru găsirea elementelor sale cele mai mari și cele mai mici.

2.7.1 Conceptul de listă în Python

Fiecare element de date conține un link către un alt element împreună cu datele prezente în acesta. Lista este similar cu masivul, cu excepția faptului că elementele acesteia pot fi de tipuri de date diferite. Într-o listă Python putem avea atât date numerice, cât și date de tip șir de caractere sau obiecte complexe.

Lista înlănțuită este cea mai versatilă structură de date disponibilă în Python. Poate fi creată ca o listă de valori (elemente) separate de virgulă între paranteze pătrate. O caracteristică foarte importantă a implementării de listă înlănțuită în Python este aceea că articolele sau elementele listei nu trebuie să fie de același tip. Lista Python poate conține tipuri eterogene de date.

Crearea unei liste este la fel de simplă ca înșiruirea de valori diferite, separate prin virgulă, între paranteze pătrate. De exemplu:

```
list1 = ['luni', 'marti', 1989, 2000]
list2 = [1, 2, 3, 4, 5]
list3 = ["a", "b", "c", "d"]
```

Similar cu indicii masivelor (vectorilor), indicii de listă încep de la 0, iar listele pot fi împărțite în sub-liste, concatenate și așa mai departe.

2.7.1.1 Accesarea valorilor unei liste

Pentru a accesa valorile unei liste, utilizăm parantezele pătrate împreună cu indexul sau indecșii pentru a obține valoarea disponibilă la acel index, respectiv sub-lista de valori conținută între doi indecși furnizați. De exemplu:

```
list1 = ['luni', 'marti', 1989, 2000]
list2 = [1, 2, 3, 4, 5]
```

```
print("list1[0]: ", list1[0])
print("list2[1:5]: ", list2[1:5])
```

Codul de mai sus produce în urmă executării următorul rezultat:

```
list1[0]: luni
list2[1:5]: [2, 3, 4, 5]
```

2.7.1.2 Actualizarea listelor

Putem actualiza elemente unice sau multiple ale listelor prin furnizarea unui interval de indecși pentru partea stângă a operatorului de atribuire. Putem, de asemenea, adăuga noi elemente la o listă prin intermediul metodei `append()`, metodă pe care o vom discuta în secțiunile următoare. De exemplu:

```
list1 = ['luni', 'marti', 1989, 2000]

# modificarea (actualizarea) unui element al listei
print("Valoare de la index 2 este ")
print(list1[2])
list1[2] = 2001
print("Noua valoare de la index 2 este ")
print(list1[2])
```

Execută codul de mai sus obținem următorul rezultat:

```
Valoare de la index 2 este
1989
Noua valoare de la index 2 este
2001
```

2.7.1.3 Ștergerea de elemente din listă

Pentru a elimina un element de listă, putem utiliza fie instrucțiunea `del` dacă știm exact indexul elementului pe care dorim să îl ștergem, fie metoda `remove()` dacă nu cunoaștem acest index. Metoda `remove()` o vom discuta în secțiunile următoare. De exemplu:

```
list1 = ['luni', 'marti', 1989, 2000]
print(list1)

# ștergerea unui element al listei pe baza indexului acestuia
del list1[2]
print("Dupa ștergerea valorii de la index 2 : ")
print(list1)
```

Codul de mai sus va genera la execuție următorul rezultat:

```
['luni', 'marti', 2001, 2000]
Dupa ștergerea valorii de la index 2 :
['luni', 'marti', 2000]
```

2.7.1.4 Operații de bază cu liste

Listele răspund la operatorii `+` și `*` la fel ca șirurile de caractere (`string`). Aceasta înseamnă că listele pot fi concatenate și putem realiza repetare de secvență de listă, cu excepția faptului că rezultatul este o listă nouă, nu un șir.

De fapt, listele răspund la toate operațiunile secvențiale generale pe care le-am folosit pe șiruri atunci când am discutat despre vectori. Tabelul 2.11 oferă o scurtă sinteză în acest sens.

Tabelul 2.11 Operații de bază cu liste Python

Expresie Python	Rezultat	Semnificație
<code>len([1, 2, 3])</code>	3	Lungime
<code>[1, 2, 3] + [4, 5, 6]</code>	[1, 2, 3, 4, 5, 6]	Concatenare
<code>['Salut!'] * 4</code>	['Salut!', 'Salut!', 'Salut!', 'Salut!']	Repetare
<code>3 in [1, 2, 3]</code>	True	Apartenență
<code>for x in [1, 2, 3]:</code>	1 2 3	Iterare

2.7.1.5 Indexare și partiționare

Deoarece listele sunt secvențe, indexarea și partiționarea funcționează în același mod pentru liste ca și pentru șiruri de caractere. De exemplu, secvența următoare de cod:

```
lista = ['universitate', 'facultate', 'specializare']
print(lista[1])
print(lista[-1])
print(lista[0:-1])
```

va tipări la consolă în urma execuției rezultatul de mai jos.

```
facultate
specializare
['universitate', 'facultate']
```

Pentru partiționare, pe lângă indexul de început și indexul final, poate fi furnizat și pasul în sintaxa următoare:

```
listaNumere = [1, 2, 3, 4, 5, 6, 7, 8, 9]
# tiparire numere cu index impar (valorile intregi pare din lista)
print(listaNumere[1::2])
```

Rezultatul tipărit la consolă în urma execuției este următorul.

```
[2, 4, 6, 8]
```

2.7.1.6 Funcții și metode implicite pentru liste

Python include următoarele funcții implicite pentru tipul de listă, prezentate succint în tabelul 2.12.

Tabelul 2.12 Funcții implicite pentru tipul list

Nr. crt.	Denumirea funcției și descrierea acesteia
1	<code>len(list)</code> Oferă lungimea totală a listei.
2	<code>max(list)</code> Returnează elementul din listă cu valoarea maximă.
3	<code>min(list)</code> Returnează elementul din listă cu valoarea minimă.
4	<code>list(secvență)</code> Convertește un tuplu la o listă.

Pe lângă funcțiile implicite disponibile la nivelul limbajului, pentru clasa Python `list` include metodele descrise în tabelul 2.13.

Tabelul 2.13 Metode aparținând clasei list

Nr. crt.	Denumirea metodei și descrierea acesteia
1	<code>list.append(obiect)</code> Adaugă obiectul <code>obiect</code> la listă.
2	<code>list.count(obiect)</code> Returnează numărul de câte ori apare obiectul <code>obiect</code> în listă.

Nr. crt.	Denumirea metodei și descrierea acesteia
3	<code>list.extend(secvență)</code> <i>Se adaugă conținutul secvenței la listă</i>
4	<code>list.index(obiect)</code> <i>Returnează cel mai mic index din listă la care apare obiectul obiect</i>
5	<code>list.insert(index, obiect)</code> <i>Inserează obiectul obiect în listă la indexul specificat</i>
6	<code>list.pop(obiect = list [-1])</code> <i>Elimină și returnează ultimul obiect sau obiect din listă</i>
7	<code>list.remove(obiect)</code> <i>Elimină obiectul obiect din listă</i>
8	<code>list.reverse()</code> <i>Inversează ordinea obiectelor listei</i>
9	<code>list.sort([func])</code> <i>Sortează obiectele listei, utilizând funcția de comparare func, dacă aceasta este furnizată</i>

2.7.2 Conceptul de tuplu

Un tuplu în Python este o colecție de obiecte ordonate și imuabile. Tuplurile sunt secvențe, la fel ca listele. Principala diferență între tupluri și liste este aceea că, spre deosebire de liste, tuplurile nu pot fi modificate. Tuplurile folosesc paranteze rotunde, în timp ce listele folosesc paranteze pătrate. Crearea unui tuplu este la fel de simplă precum crearea unei liste, respectiv prin enumerarea diferitelor valori separate prin virgulă. Aceste valori înșiruite și separate prin virgulă pot fi incluse opțional între paranteze rotunde, pentru delimitarea logică a structurii de date. De exemplu, secvența următoare de cod:

```
tuplu = 'universitate', 'facultate', 'specializare'
print(tuplu[1])
print(tuplu[-1])
print(tuplu[0:-1])
```

reprezintă declarația unui tuplu și va tipări la consolă următorul rezultat:

```
facultate
specializare
('universitate', 'facultate')
```

Un tuplu gol se va declara totuși utilizând paranteze rotunde, între care nu se specifică nimic.

```
tupluGol = () # declararea unui tuplu ce nu conține nici un obiect
```

Pentru a crea un tuplu ce conține o singură valoare, trebuie ca din perspectivă sintactică să includem o virgulă în secvență, chiar dacă există o singură valoare.

```
tupluGol = (,)
print(type(tupluGol), tupluGol)
```

```
tupluDA = (1,) # declararea unui tuplu continand o singura valoare
# tupluDA = 1, # parantezele pot lipsi, virgula indica faptul ca este un tuplu
tupluNU = (1) # este de fapt initializare unei variabile intregi cu valoarea 1
# intre paranteze

print(tupluDA)
print(tupluNU)
```

Rezultatul execuției secvenței de cod de mai sus este următorul.

```
<class 'tuple'> ()
(1,)
1
```

Notă – Este de reținut faptul că, în mod similar cu indicii șirurilor de caractere și indicii listelor, indicii unui tuplului încep de la 0 și pot fi partiționați, concatenați și așa mai departe.

2.7.2.1 Modificarea tuplurilor

Tuplurile Python sunt imuabile, ceea ce înseamnă că nu putem actualiza sau modifica valorile elementelor tuplului. Putem lua porțiuni din tuplurile existente pentru a crea noi tupluri, așa cum demonstrează următorul exemplu:

```
tuplu1 = (1, 2, 3)
tuplu2 = 1, 2, 3
tuplu3 = 4, 5, 6
tuplu4 = tuplu1 + tuplu2 + tuplu3

print('Identificator tuplu1: ', id(tuplu1))
print('Identificator tuplu2: ', id(tuplu2))
print('Identificator tuplu3: ', id(tuplu3))
print('Identificator tuplu4: ', id(tuplu4))
print('Continut tuplu4: ', tuplu4)
```

Rezultatul execuției codului de mai sus este următorul:

```
Identificator tuplu1: 2353391753160
Identificator tuplu2: 2353391753160
Identificator tuplu3: 2353391621688
Identificator tuplu4: 2353390382152
Continut tuplu4: (1, 2, 3, 1, 2, 3, 4, 5, 6)
```

Funcția implicită Python `id(obiect)` returnează identificatorul unic al unui obiect în cadrul interpretorului Python. Se poate observa faptul că `tuplu1` și `tuplu2` au același identificator pentru că fac referire la aceeași înșiruire imutabilă de obiecte. De asemenea, este confirmat faptul că `tuplu4`, obținut prin concatenarea a 3 tupluri reprezintă de fapt un nou tuplu, o nouă entitate în memoria interpretorului Python, ce are asociat un identificator distinct.

2.7.2.2 Ștergerea elementelor unui tuplu

Eliminarea elementelor individuale ale tuplului nu este posibilă, fiind un tip de dată imuabil. Bineînțeles, poate fi oricând construit un tuplu nou ce conține elementele unui tuplu existent din care am eliminat elementele nedorite.

Pentru a elimina în mod explicit un întreg tuplu, avem la dispoziție doar să folosiți instrucțiunea `del`. De exemplu, secvența următoare de cod:

```
tuplu1 = ('Python', 'Java', 'C++', 'C')
print(tuplu1)
tuplu2 = ('Python', 'Java', 'C++')
print(tuplu2)
del tuplu1
print(tuplu1)
```

va scrie la consolă, în urma execuției, următorul rezultat:

```
('Python', 'Java', 'C++', 'C')
('Python', 'Java', 'C++')
NameError: name 'tuplu1' is not defined
```

De remarcat faptul că eroare generată se datorează faptului că, după `del tuplu1`, variabila `tuplu1` nu mai există.

Notă – Așa cum am văzut în exemplele anterioare, orice secvență de valori, obiecte multiple separate prin virgulă, scrise fără simboluri de identificare (delimitare a secvenței), respectiv fără paranteze pătrate pentru liste, fără paranteze rotunde pentru tupluri etc., indică implicit faptul că înșiruirea descrie un tuplu.

2.7.2.3 Funcții implicite pentru tuplu

Python include următoarele funcții implicite pentru tuplu, prezentate sintetic în tabelul 2.14:

Tabelul 2.14 Funcții implicite pentru tipul tuplu

Nr. crt.	Denumirea funcției și descrierea acesteia
1	<code>cmp(tuplu1, tuplu2)</code> Compară elementele celor două tupluri.
2	<code>len(tuplu)</code> Oferă lungimea totală a tuplului.
3	<code>max(tuplu)</code> Returnează elementul din tuplu cu valoarea maximă.
4	<code>min(tuplu)</code> Returnează elementul din tuplu cu valoarea minimă.
5	<code>tuple(secvență)</code> Converteste o listă într-un tuplu.

2.7.3 Tipul interval (range)

Tipul interval în Python reprezintă o secvență imuabilă de numere și este utilizat în mod curent pentru repetarea de un anumit număr de ori a unui bloc din cadrul unei structuri repetitive. Argumentele pentru constructorul clasei `range` trebuie să fie întregi (fie tipul numeric implicit `int`, fie orice obiect care implementează metoda specială `__index__`).

```
class range(stop)
class range(start, stop[, pas])
```

Parametrii furnizați constructorilor au următoarea semnificație:

`start` – valoarea parametrului de pornire sau 0 dacă parametrul nu a fost furnizat;
`stop` – valoarea parametrului de oprire;
`pas` – valoarea pasului sau 1 dacă parametrul nu a fost furnizat.

Dacă argumentul `pas` este omis, acesta este implicit la egal cu 1. Dacă argumentul inițial este omis, acesta este implicit egal cu 0. Dacă însă este furnizat un `pas` egal cu zero, este aruncată o excepție de tip `ValueError`.

Pentru un `pas` pozitiv, conținutul unui interval `r` este determinat prin formula:

$r[i] = start + pas * i$, unde $i \geq 0$ și $r[i] < stop$.

Pentru un `pas` negativ, conținutul intervalului este încă determinat prin aceeași formulă:

$r[i] = start + pas * i$, dar constrângerile sunt $i \geq 0$ și $r[i] > stop$, respectiv $(start + pas * i) > stop$. Cu alte cuvinte, valoarea de start trebuie să fie mai mare decât valoarea de început, atunci când pasul este negativ.

Tipul `range` acceptă indici negativi, dar aceștia sunt interpretați ca indexare de la sfârșitul secvenței determinate de indicii pozitivi. În urma rulării exemplului următor:

```
tuplu1 = tuple(range(3, 5, 1))
print(tuplu1)
tuplu2 = tuple(range(-3, -5, -1))
print(tuplu2)
```

obținem următorul rezultat.

```
(3, 4)
(-3, -4)
```

Un obiect de tip interval (`range`) va fi gol dacă `r[0]` nu îndeplinește constrângerea de valoare. De exemplu, secvența următoare de cod:

```
print(tuple(range(-6, -5, -1)))
print(tuple(range(0)))
```

va tipări la consolă următorul rezultat.

```
()
()
```

Intervalele care conțin valori absolute mai mari decât `sys.maxsize` sunt permise, dar unele caracteristici, cum ar fi `len()`, pot arunca într-un astfel de context excepția `OverflowError`.

Tipul *range* implementează toate operațiile secvenței obișnuite, cu excepția concatenării și repetării (datorită faptului că obiectele intervalului pot reprezenta doar secvențe care urmează un model strict, iar repetarea și concatenarea vor încălca, de obicei, acel model).

Avantajul tipului de gamă față de o listă obișnuită sau un tuplu este că un obiect de gamă va ocupa întotdeauna aceeași cantitate, de obicei redusă, de memorie, indiferent de dimensiunea intervalului pe care îl reprezintă. Bineînțeles, aceasta deoarece sunt stocate doar valorile de pornire, oprire și pasul, calculând elemente individuale și subintervale după cum este necesar.

Obiectele de tip *range* implementează `collections.abc.Sequence` ABC și oferă caracteristici precum testarea existenței unui element într-un anumit interval, căutare după index, partiționare pe baza indecșilor. De asemenea, similar cu celelalte tipuri de date în secvență, respectiv *list* și *tuple*, tipul *range* permite utilizarea de indici negativi.

Testarea obiectelor de tip *range* pentru egalitate cu operatorii `==` și `!=` implică compararea lor ca secvențe. Cu alte cuvinte, două obiecte de tip interval sunt considerate egale dacă reprezintă aceeași succesiune de valori. De remarcat faptul că două obiecte de tip interval, care sunt egale în urma comparării, ar putea avea atribute diferite de pornire, oprire și pas. Spre exemplu, `range(0) == range(2, 1, 2)` și `range(-1, 1, 2) == range(-1, 0)`.

2.8 Python ca limbaj de programare orientat obiect

Clasele Python includ date, numite și **atribute** care sunt alte obiecte Python, **metode** și **metode speciale**. Atributele sunt implementate ca variabile de instanță, adică variabile care sunt unice pentru un anumit obiect. Atributele sunt accesate prin numele de instanță plus operatorul punct. Atributele care sunt accesate prin metode ale clasei sunt numite **proprietăți**. Metodele speciale sunt metode cu sarcini (roluri) standard, aceleași pentru toate clasele. Exemplu de metode speciale: `__add__()` și `__sub__()` pentru implementarea operatorilor `+` și `-`.

Metoda este o funcție al cărei prim argument este instanța pe care va opera metoda. Metodele sunt definite astfel:

```
def nume_metoda(self, ...):
    corp_metoda
```

În Python folosim termenul de *clasă de bază* sau *superclasă* pentru a face referire la o clasă care este moștenită de alte clase. Folosim termenul de subclasă pentru clasele derivate. Orice metodă poate fi suprascrisă, adică reimplementată într-o subclasă. Dacă avem un obiect din clasa *MyDict* (o clasă de utilizator care moștenește *dict*) și suprascrim o metodă din *dict*, Python va apela corect metoda din *MyDict*. Prin urmare, toate metodele în Python sunt metode virtuale. Dacă trebuie să apelăm metoda din superclasă, o putem face folosind funcția `super()` din modulul *builtins*.

Specific limbajului Python, ca și altor limbaje dinamice, este faptul că nu verifică tipul unui obiect atunci când este invocată o metodă ci este căutată și invocată direct metoda (*Duck Typing*).

O altă particularitate a limbajului Python este că nu are modificatori de acces (*private*, *protected*). Atributele și metodele sunt publice. Cu toate acestea, dacă creăm atribute sau metode

care încep cu două caractere *underscore* (liniuța de subliniere), va fi împiedicat accesul la acestea, astfel încât să poată fi considerate private.

Spre deosebire de alte limbaje orientate obiect, Python nu suportă supraîncărcarea metodelor (*overloading*).

Sintaxa pentru declarația unei clase este:

```
class className():
    descriere

class className(base_classes):
    descriere
```

Prima sintaxă semnifică faptul: clasa are ca superclasă clasa *object*, rădăcina tuturor claselor.

Metodele Python au ca prin argument referința la obiectul curent, care, prin convenție, este denumită *self*. Orice metodă și atribut vor fi referite prin *self*.

2.8.1 Crearea obiectelor

Pentru crearea unui obiect, mai întâi este invocată metoda specială `__new__()` moștenită din *object*, apoi metoda `__init__()` care funcționează ca un constructor.

```
__init__(self[, ...])
```

Constructorul superclasei este apelat folosind funcția `super()` din *builtins*:

```
super().__init__(...)
```

2.8.2 Metode speciale

Metodele care încep și se termină cu două caractere de subliniere (*underscore*) `"_"` sunt metode speciale. Cele mai multe dintre acestea sunt asociate cu operațiunile care se execută la utilizarea operatorilor, cu sunt metodele speciale asociate operatorilor de comparare.

Alte metode speciale sunt:

```
__repr__(self)
```

Metodă apelată de funcția *builtins* `repr()` pentru a determina reprezentarea „oficială” de tip șir a unui obiect. Această reprezentare se obișnuiește a fi o expresie Python validă. Valoarea returnată trebuie să fie un obiect șir.

```
__str__(self)
```

Metodă apelată de funcțiile *builtins* `str()`, `format()` și `print()` pentru a determina reprezentarea informală de tip șir. Valoarea returnată trebuie să fie un obiect șir.

2.8.3 Crearea atributelor

Atributele sunt create prin atribuiri la nivelul clasei de forma:

```
variabila = valoare
```

Pot fi create atribute și în interiorul metodelor prin atribuiri de forma:

```
self.variabila = valoare
```

Referirea se face prin numele obiectului:

```
nume_obiect.variabila
```

Exemplu de mai jos prezintă definirea și instanțierea unei clase în Python.

```
class materiale():
    simbol_cont='3011'
    def __init__(self, denumire, valoare):
        self.denumire=denumire
        self.valoare = valoare
    def __str__(self):
        return self.simbol_cont+","+self.denumire+","+str(self.valoare)

material = materiale(denumire="Ciment",valoare=100000)
print(material.simbol_cont,material.valoare)
```

2.8.4 Proprietăți

Sunt atribute care sunt accesate prin mecanismul de tip *getter* și *setter* folosind decoratorii de tip `@property` și `@nume_variabila.setter` definiți în *builtins*.

Spre exemplu, să presupunem că avem două clase: *publicatie* (clasă părinte) și *carte* (clasă derivată). La nivelul clasei *publicatie* avem definit un câmp pentru denumire publicație de tip șir de caractere, iar la nivelul clasei *carte* avem câmpuri pentru numele autorilor (șir) și număr de volume (întreg). Câmpurile sunt implementate astfel:

denumire – atribut public de tip *str*

__autori – atribut privat de tip *str*

numar_volume – proprietate de tip *str*

În următoare secvență de cod sunt definite cele două clase:

```
class publicatie():
    def __init__(self,denumire):
        self.denumire = denumire
    def __str__(self):
        return self.denumire

class carte(publicatie):
    def __init__(self,denumire,autori,numar_volume):
        super().__init__(denumire)
```

```
self.__autori=autori
self.__numar_volume=numar_volume

def __str__(self):
    return self.denumire+","+self.__autori+","+str(self.__numar_volume)

def get_autori(self):
    return self.__autori

def set_autori(self,autori):
    self.__autori = autori

@property
def numar_volume(self):
    return self.__numar_volume

@numar_volume.setter
def numar_volume(self,numar_volume):
    self.__numar_volume=numar_volume
```

Instanțierea unui obiect carte și modificarea câmpurilor se poate face astfel:

```
c1 = biblioteca.carte("Istoria Romana","Theodor Mommsen",4)
print(c1)
c1.denumire="Istoria declinului si prabusirii Imperiului Roman"
c1.numar_volume=7
c1.set_autori("Edward Gibbon")
print(c1)
```

Ca rezultat al execuției programului sunt tipărite la dispozitivul standard de ieșire următoarele:

```
Istoria Romana,Theodor Mommsen,4
Istoria declinului si prabusirii Imperiului Roman,Edward Gibbon,7
```

2.9 Structuri de date secvențiale modelate cu tipurile implicite Python

Structurile de date sunt concepte fundamentale ale informaticii care ajută la scrierea de programe eficiente în orice limbaj de programare. Python este un limbaj de programare de nivel înalt, interpretat, interactiv și orientat pe obiecte, prin intermediul căruia putem studia structurile de date fundamentale într-un mod mai sugestiv și intuitiv în comparație cu alte limbaje de programare.

În acest capitol vom prezenta o succintă perspectivă de ansamblu asupra unor structuri de organizare utilizate în general în lucru cu date, precum și modul în care acestea sunt legate de unele tipuri de date specifice limbajul de programare Python.

2.9.1 Vector (array)

Este un aranjament secvențial de elemente împerecheate cu indexul elementului de date. Vectorul (*array*) este un container care poate reține un număr fix de articole și aceste articole ar trebui să fie de același tip. Majoritatea structurilor de date utilizează tablouri pentru a

implementa algoritmi lor. Următoarea terminologie este importantă pentru a înțelege conceptul de vector, tablou sau `array`:

- Element – fiecare element stocat într-un tablou se numește element.
- Index – fiecare locație a unui element dintr-un tablou are un indice numeric, care este utilizat pentru a identifica elementul.

2.9.1.1 Reprezentare vector (`array`)

Vectorii pot fi declarați în diverse moduri în diferite limbaje de programare. În Python sunt câteva aspecte importante care trebuie luate în considerare:

```
vector = array(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

- Indicele începe cu 0.
- Lungimea vectorului este de 10, ceea ce înseamnă că poate stoca 10 elemente.
- Fiecare element poate fi accesat prin intermediul indexului său. De exemplu, putem obține un elementul al 6-lea de la indexul 5 ca având valoarea 6.

2.9.1.2 Operații de bază cu vectori

Vom discuta în continuare operațiunile de bază acceptate de un tablou:

- Traversare – imprimăm toate elementele tabloului unul câte unul.
- Inserție – adaugă un element la indexul dat.
- Ștergere – șterge un element din indexul dat.
- Căutare – caută un element folosind indicele dat sau după valoare.
- Actualizare – actualizează un element la indexul dat.

Structura de date de tip vector (`array`) este disponibilă în Python prin importarea modulului `array` în programul Python. Codul din exemplul următor declară și inițializează un vector cu 5 elemente, după care îi tipărește conținutul la consolă.

```
from array import *

# NumeArray = array(typecode, [Initializers])
vector = array('i', [1, 2, 3, 4, 5])
print(vector)
```

Rezultatul rulării este următorul:

```
array('i', [1, 2, 3, 4, 5])
```

Codul tipului de dată reprezintă codul care este utilizat pentru a defini tipul de valoare pe care o va deține tabloul. Includem în tabelul 2.15 codurile utilizate cel mai frecvent pentru tipul de date stocate într-un `array` Python.

Tabelul 2.15 Codurile utilizate cel mai frecvent pentru tipul de date stocate într-un `array`

Codul tipului	Valoare
b	Reprezintă un număr întreg cu semn, având dimensiunea de 1 byte (octet)
B	Reprezintă un număr întreg fără semn, având dimensiunea de 1 byte
c	Reprezintă un caracter cu dimensiunea de 1 byte
i	Reprezintă un număr întreg cu semn, având dimensiunea de 2 octeți
I	Reprezintă un număr întreg fără semn, având dimensiunea de 2 octeți
f	Reprezintă un număr în virgulă mobilă având dimensiunea de 4 octeți
d	Reprezintă un număr în virgulă mobilă având dimensiunea de 8 octeți

2.9.1.3 Parcurgerea elementelor unui vector

Înainte de a investiga diverse operații masive, vom începe prin a crea și tipări la consolă un masiv în Python. Secvența de cod de mai jos creează un masiv numit `vector1`.

```
from array import *

# tiparirea valorilor unui masiv, element cu element
vector1 = array('i', [100, 200, 300, 400, 500])

for x in vector1:
    print(x)
```

Interpretat și executat, programul Python de mai sus produce următorul rezultat:

```
100
200
300
400
500
```

2.9.1.4 Accesarea elementelor unui vector

Putem accesa în mod direct fiecare element al unui masiv folosind indexul elementului. Codul de mai jos arată cum putem realiza aceasta:

```
from array import *

vector1 = array('i', [100, 200, 300, 400, 500])

# acces direct la elementele unui vector
print(vector1[0])
print(vector1[2])
```

Când executăm programul de mai sus, acesta produce următorul rezultat ce arată care sunt elementele vectorului vector1 de la indexii 0 și 2:

```
100
300
```

2.9.1.5 Operațiunea de inserare în vector

Operațiunea de inserare este acțiunea prin care se introduc unul sau mai multe elemente de date într-un vector sau masiv. În funcție de nevoia concretă, un element nou poate fi inserat la început, adăugat la sfârșit sau la orice index specificat în interiorul masivului.

În codul de mai jos exemplificăm inserarea unui element de date în interiorul masivului folosind metoda Python `insert()`:

```
from array import *

vector1 = array('i', [100, 200, 300, 400, 500])

# inserare in interiorul masivului
vector1.insert(1,600)

for x in vector1:
    print(x)
```

Atunci când este interpretat și executat programul de mai sus, acesta produce următorul rezultat, care arată că elementul cu valoarea 600 este inserat la poziția indexului egală cu 1.

```
100
600
200
300
400
500
```

2.9.1.6 Operațiunea de ștergere

Ștergerea se referă la eliminarea unui element existent din masiv și reorganizarea tuturor elementelor rămase în masiv după această operațiune.

Prin secvența de cod următoare vom elimina un element de date din interiorul vectorului folosind metoda Python `remove()`.

```
from array import *

vector1 = array('i', [100, 200, 300, 300, 400, 500])

# ștergerea primului element care are valoarea egală cu parametrul
# furnizat funcției remove()
vector1.remove(300)
for x in vector1:
    print(x)
```

Când executăm programul de mai sus, acesta produce următorul rezultat, care arată că primul element egal cu valoarea 300 este eliminat din vector.

```
100
200
300
400
500
```

2.9.1.7 Operațiunea de căutare

Putem efectua o căutare pentru un element în vector pe baza valorii sau a indexului acestui element. Prin secvența de cod următoare căutăm un element de date folosind metoda Python `index()`:

```
from array import *

vector1 = array('i', [100, 200, 300, 400, 500])

# regasire index element dup valoare
print(vector1.index(400))
```

Când executăm programul de mai sus, acesta produce următorul rezultat, care arată indexul elementului:

```
3
```

Dacă valoarea nu este prezentă în tablou, atunci programul returnează o eroare.

2.9.1.8 Operațiunea de actualizare

Operațiunea de actualizare se referă la modificarea unui element existent în vector la un indice furnizat ca parametru. În acest caz, pur și simplu, reassignăm o nouă valoare indexului la care dorim să facem actualizarea:

```
from array import *

vector1 = array('i', [100, 200, 300, 400, 500])

# actualizare valoare vector la index = 2
vector1[2] = 10

for x in vector1:
    print(x)
```

Executând programul de mai sus, acesta produce următorul rezultat, care arată noua valoare a vectorului la poziția index egal cu 2:

```
100
200
10
400
500
```

2.9.2 Stivă (stack)

Stiva este o structură de date care urmează o ordine specifică de funcționare, respectiv ultimul intrat, primul ieșit.

Termenul de stivă înseamnă aranjarea obiectelor unul peste altul. Aceasta este modalitatea în care memoria este alocată în această structură de date. Stiva stochează elementele de date într-o manieră similară modul în care, de exemplu, farfuriile sunt depozitate una peste alta în bucătărie. Deci, structurarea datelor într-o stivă permite operațiuni la un capăt al acesteia, capăt ce poate fi numit partea superioară a stivei. Putem adăuga elemente sau elimina elemente doar pe la acest capăt al stivei (la partea superioară).

Într-o stivă, elementul inserat ultimul în secvență va ieși cel dintâi, deoarece putem elimina doar din partea de sus a stivei. Această caracteristică este cunoscută drept logică *Last In First Out* (LIFO) în limba engleză sau ultimul intrat, primul ieșit. Operațiunile de adăugare și îndepărtare a elementelor sunt cunoscute sub numele de PUSH și POP. În programul următor vom implementa aceste două operațiuni specifice stivelor ca funcții de adăugare și, respectiv, de eliminare sau de extragere dintr-o listă Python. Declarăm o listă goală și folosim metodele `append()` și `pop()` pentru a adăuga și extrage elemente în și dintr-o listă Python.

2.9.2.1 Adăugarea sau punerea unui element în stivă – PUSH într-o stivă

```
class Stack:
```

```
    def __init__(self):
        self.stack = []

    # folosim metoda append a clasei List pentru a insera un element in stiva
    def push(self, data):
        if data not in self.stack:
            self.stack.append(data)
            return True
        else:
            return False

    # implementam metoda peek pentru a accesa elementul de la partea de sus a stivei
    def peek(self):
        return self.stack[-1]
```

```
stiva = Stack()
stiva.push("luni")
stiva.push("marti")
stiva.peek()
print(stiva.peek())
stiva.push("miercuri")
stiva.push("joi")
print(stiva.peek())
```

Rularea codului de mai sus va produce următorul rezultat:

```
marti
joi
```

2.9.2.2 Extragerea sau scoaterea unui element din stivă – POP dintr-o stivă

După cum știm, putem elimina doar cel mult un element din stivă la un moment dat, iar această operațiune se realizează pe la același capăt pe la care au fost inserate elemente în stivă. Funcția de scoatere din stivă returnează elementul cel mai de sus, respectiv elementul cel mai recent inserat în stivă (logică LIFO). Verificăm existența elementul de top calculând mai întâi dimensiunea stivei și apoi folosim metoda `pop()` implementată în clasa `list` din Python pentru a extrage elementul cel mai de sus din stivă.

```
class Stack:
```

```
    def __init__(self):
        self.stack = []

    # folosim metoda append a clasei List pentru a insera un element in stiva
    def push(self, data):
        if data not in self.stack:
            self.stack.append(data)
            return True
        else:
            return False

    # implementam metoda peek pentru a accesa elementul de la partea de sus a stivei
    def peek(self):
        return self.stack[-1]

    # folosim metoda pop a clasei List pentru a extrage un element
    # de la partea de sus stivei
    def pop(self):
        if len(self.stack) <= 0:
            return ("Nu este nici un element in Stack")
        else:
            return self.stack.pop()
```

```
stiva = Stack()
stiva.push("luni")
stiva.push("marti")
stiva.push("miercuri")
stiva.push("joi")
stiva.push("vineri")
print(stiva.pop())
print(stiva.pop())
```

Rulând acest program obținem următorul rezultat:

```
vineri
joi
```

2.9.3 Coadă (queue)

Coadă este o structură de date asemănătoare cu stiva, care urmează însă o ordine specifică de funcționare, respectiv primul element intrat în coadă este și primul care poate fi scos din coadă. Suntem cu toții familiarizați cu noțiunea de coadă din viața noastră de zi cu zi, în timp ce

așteptăm obținerea accesului la un serviciu, de exemplu. Structura de date de tip coadă este guvernată de modul în care elementele de date sunt aranjate în cadrul acesteia.

Unicitatea cozii constă în modul în care se adaugă și se elimină articolele. Este permisă adăugarea de elemente la sfârșit și eliminarea de elemente la capătul care joacă rolul de început al cozii. Așadar, este utilizată o logică de tip primul intrat, primul ieșit sau *First In First Out* (FIFO) în limba engleză.

O coadă poate fi implementată folosind clasa `list` din Python, situație în care vom folosi metodele `insert()` și `pop()` ale clasei `list` pentru a adăuga (metoda `put()`) și scoate (metoda `get()`) elemente din coadă. Nu este vorba de o inserare în interiorul listei, deoarece elementele de date sunt adăugate întotdeauna la sfârșitul cozii.

2.9.3.1 Adăugarea de elemente la o coadă – PUT într-o stivă

În exemplul de mai jos, creăm o clasă pentru structura de date coadă în care implementăm logica FIFO. Folosim metoda de inserție a clasei `list` pentru adăugarea elementelor de date.

```
class Queue:
    def __init__(self):
        self.queue = list()

    # adaugare in coada folosind metoda insert a clasei Python List
    def put(self, data):
        if data not in self.queue:
            self.queue.insert(0, data)
            return True
        return False

    # Lungimea cozii
    def size(self):
        return len(self.queue)
```

```
coada = Queue()
coada.put("ianuarie")
coada.put("februarie")
coada.put("martie")
print(coada.size())
```

Rularea secvenței de cod anterioare va produce următorul rezultat la consolă:

```
3
```

2.9.3.2 Scoaterea elementelor dintr-o coadă

În exemplul de mai jos, creăm o clasă pentru structura de date de tip coadă în care introducem datele prin intermediul metodei `put()` și apoi eliminăm date prin implementarea metodei `get()`. Pentru implementarea metodei `get()` utilizăm metoda `pop()` a clasei Python `list`.

```
class Queue:
    def __init__(self):
        self.queue = list()
```

```
# adaugare in coada folosind metoda insert a clasei Python List
def put(self, data):
    if data not in self.queue:
        self.queue.insert(0, data)
        return True
    return False
```

```
# Lungimea cozii
def size(self):
    return len(self.queue)
```

```
# scoaterea unui element din coada folosind metoda pop a clasei Python List
def get(self):
    if len(self.queue) > 0:
        return self.queue.pop()
    return ("Nu mai este nici un element in Queue!")
```

```
coada = Queue()
coada.put("ianuarie")
coada.put("februarie")
coada.put("martie")
print(coada.size())
print(coada.get())
print(coada.get())
print(coada.size())
```

În urma rulării secvenței de cod anterioare obținem următorul rezultat:

```
3
ianuarie
februarie
1
```

2.10 Conceptul de dicționar (tipul *map*)

Structura de date `Dictionary`, implicită la nivelul limbajului Python, implementează conceptul de *map*: o colecție de perechi (cheie, valoare). Fiecare-cheie este separată de valoarea sa prin caracterul `(:)`, iar elementele sunt separate prin virgule. Întreaga construcție delimitată din perspectivă sintactică de paranteze acolade. Un dicționar gol, fără elemente, este scris cu doar două paranteze acolade, astfel: `{ }`.

Cheile într-un dicționar sunt unice, în timp ce valorile pot să nu fie în mod necesar unice. Valorile unui dicționar pot fi de orice tip, dar cheile trebuie să aibă un tip de date imuabil, cum ar fi șiruri, numere sau tuple.

2.10.1 Accesarea valorilor în dicționar

Pentru a accesa elementele unui dicționar putem utiliza familiarele paranteze pătrate, împreună cu cheia pentru a obține valoarea asociată acesteia. Următorul exemplu prezintă declararea unui dicționar și inițializarea acestuia, împreună cu accesul la valori prin intermediul cheilor:

```
dict1 = {'Col1': [1, 2, 3], 'Col2': [3, 4, 5], 'Col3': [6, 7, 8]}
for j in range(len(dict1)):
    print ('Col' + str(j+1) + ':', dict1['Col'+str(j+1)])
```

În urma rulării secvenței de cod anterioare, obținem următorul rezultat:

```
Col1: [1, 2, 3]
Col2: [3, 4, 5]
Col3: [6, 7, 8]
```

Trebuie remarcat faptul că tentativa de a accesa un element de date cu o cheie care nu face parte din dicționar, va genera o eroare de tip cheie invalidă.

2.10.2 Actualizarea dicționarului

Un dicționar Python poate fi actualizat adăugând o intrare nouă sau o pereche cheie-valoare, modificând o intrare existentă sau ștergând o intrare existentă, așa cum se poate vedea în exemplul de mai jos

```
dict_1 = {'Col1': [1, 2, 3], 'Col2': [3, 4, 5], 'Col3': [6, 7, 8]}

dict_1['Col1'] = ['a', 'b', 'c']    # modificarea valorii de la cheia 'Col1'
dict_1['Col4'] = ['A', 'B']        # adaugarea unei noi perechi cheie:valoare

print(dict_1)
```

Rezultatul rulării codului de mai sus este următorul:

```
{'Col1': ['a', 'b', 'c'], 'Col2': [3, 4, 5], 'Col3': [6, 7, 8], 'Col4': ['A', 'B']}
```

2.10.3 Ștergerea elementelor unui dicționar

Dintr-un dicționar pot fi eliminate elemente individuale sau poate fi șters întregul conținut al unui dicționar. De asemenea, poate fi șters întregul dicționar într-o singură operație. Pentru a elimina în mod explicit un întreg dicționar, trebuie doar să folosim declarația `del`. Următorul este un exemplu ce prezintă aceste operații:

```
dict_2 = {1: 'unu', 2: 'doi', 3: 'trei'}
print(dict_2)

del dict_2[1]    # sterge elementul cu cheia 1
print(dict_2)

dict_2.clear()   # elimina toate intrarile din dict_2
print(dict_2)

del dict_2        # sterge intreg dictionarul dict_2
print(dict_2)
```

Rezultatul tipărit la consolă în urma rulării este:

```
{1: 'unu', 2: 'doi', 3: 'trei'}
{2: 'doi', 3: 'trei'}
{}
print(dict_2)
NameError: name 'dict_2' is not defined
```

Este aruncată o excepție deoarece după `del dict_2`, dicționarul nu mai există.

2.10.4 Proprietățile cheilor de dicționar

Valorile dicționarului nu au restricții. Ele pot fi în mod arbitrar orice obiect Python, fie obiecte standard, fie obiecte definite de utilizator. Cu toate acestea, nu este valabil același lucru și pentru chei. Sunt de reținut două aspecte importante cu privire la cheile unui dicționar, și anume:

- Nu sunt permise mai multe intrări cu o aceeași cheie. Aceasta înseamnă că nu este permisă nicio cheie duplicată. Când se întâlnesc chei duplicate în timpul repartizării, câștigă ultima repartizare. De exemplu, secvența următoare:

```
dict_3 = {'Luni': 'prima zi a saptamanii', 'Luni': 'ziua de dupa duminica'}
print(dict_3)
```

va genera următorul rezultat.

```
{'Luni': 'ziua de dupa duminica'}
```

- Cheile trebuie să fie imuabile. Aceasta înseamnă că putem utiliza șiruri de caractere, numere sau tuple ca chei de dicționar, dar ceva de genul `['cheie']`, ce reprezintă practic o listă, nu este permis.

2.10.5 Funcții și metode implicite pentru tipul dicționar

Python include următoarele funcții implicite pentru tipul dicționar, prezentate sintetic în tabelul 2.16.

Tabelul 2.16 Funcții implicite aplicabile tipului `dict`

Nr. crt.	Denumirea funcției și descrierea acesteia
1	<code>cmp(dict1, dict2)</code> Nu mai este disponibilă în Python 3.
2	<code>len(dict)</code> Returnează lungimea totală a dicționarului. Acesta reprezintă numărul de articole, perechi cheie-valoare din dicționar.
3	<code>str(dict)</code> Produce o reprezentare imprimabilă prin intermediul unui șir de caractere conținutului unei date de tip dicționar. Formatul poate fi variabil, în funcție de conținutul dicționarului.
4	<code>type(variable)</code> Returnează tipul variabilei furnizate ca parametru. Dacă variabila pasată este un dicționar, atunci va returna tipul dicționar.

Limbajul Python oferă, de asemenea, următoarele metode specifice tipului dicționar (tabelul 2.17):

Tabelul 2.17 Metode specifice tipul dict

Nr. crt.	Numele metodei și descrierea acesteia
1	<code>dict.clear()</code> Elimină toate elementele din dicționarul dict.
2	<code>dict.copy()</code> Returnează o copie superficială a dicționarului dict.
3	<code>dict.fromkeys()</code> Creează un dicționar nou ce va conține chei furnizate ca secvență și valori setate dintr-o mulțime de valori. De exemplu: <code>print(dict_1.fromkeys(range(2), ['mama', 'tata']))</code> va tipări: <code>{0: ['mama', 'tata'], 1: ['mama', 'tata']}</code>
4	<code>dict.get(key, default=None)</code> Returnează valoarea pentru cheia key sau valoarea implicită dacă cheia nu este în dicționar.
5	<code>dict.has_key(key)</code> Eliminată, este utilizat în schimb operatorul: <code>in</code> .
6	<code>dict.items()</code> Returnează o listă de perechi de tuple (cheie, valoare).
7	<code>dict.keys()</code> Returnează lista de chei ale dicționarului dict.
8	<code>dict.setdefault(key, default=None)</code> Similară cu <code>get()</code> , dar va seta <code>dict[key]</code> cu valoarea implicită dacă cheia nu este deja în dicționar.
9	<code>dict.update(dict2)</code> Adaugă perechile cheie-valoare dicționar dict2 la dict.
10	<code>dict.values()</code> Returnează lista valorilor dicționarului dict.

2.11 Lucrul cu fișiere

2.11.1 Funcții de bază pentru citirea datelor din fișiere

În Python obiectele fișier sunt create prin funcția `open`:

```
open(file, mode='r', ...)
```

Dacă nu se poate crea fișierul, este aruncată o excepție `OSError`. Parametrul `file` conține calea și numele fișierului. Parametrul `mode` specifică modul în care va fi deschis fișierul și poate avea valorile prezentate în tabelul 2.18:

Tabelul 2.18 Valori pentru controlul modului de deschidere a unui fișier

Valoare	Semnificație
'r'	citire (implicit)
'w'	scriere
'x'	creare exclusivă cu eșuare dacă fișierul există
'a'	deschidere pentru adăugare
'b'	deschidere în mod binar
't'	deschidere în mod text (implicit)

Spre exemplu, în secvența următoare de cod sunt citite și afișate liniile dintr-un fișier text.

```
f_input = open("f1.txt", 'r')
print(type(f_input))
for linie in f_input:
    print(linie, end="")

f_input.close()
```

Funcția `open` întoarce un obiect din clasa `io.TextIOBase`, în mod specific pentru un fișier text din subclasa `io.TextIOWrapper` sau din clase precum `io.BufferedReader` etc. pentru fișiere binare. Aceste clase permit tratarea fișierelor ca fluxuri de intrare/ieșire (stream).

Pentru citirea liniilor din flux pentru un fișier text se folosesc metodele `readline()` și `readlines()`. Prima metodă întoarce șir vid dacă este întâlnit sfârșitul de fișier. Închiderea fluxului se realizează prin metoda `close()`. Sevența de cod următoare exemplifică utilizarea acestor metode pentru deschiderea și citirea liniilor unui fișier text.

```
f_input.close()
f_input = open("f1.txt", 'r')
linii = f_input.readlines()
for linie in linii:
    print(linie, end="")
f_input.close()
```

```
f_input = open("f1.txt", 'r')
linie = f_input.readline()
while linie:
    print(linie, end="")
    linie=f_input.readline()
f_input.close()
```

2.11.2 Citirea datelor utilizând pachetul pandas

Pentru citire din fișiere text se folosesc funcțiile `read_csv()` și `read_table()`. Principalii parametri ai acestor funcții sunt prezentați în tabelul 2.19.

Tabelul 2.19 Parametrii funcțiilor pandas `read_csv()` și `read_table()`

Parametru	Tip valori	Valoare implicită	Semnificație
<code>filepath_or_buffer</code>			Cale fișier text
<code>sep</code>	str	',' sau '\t'	Separator coloane
<code>header</code>	întreg ori listă de întregi	'infer'	Stabilește cum se citesc numele de coloane. <i>infer</i> sau 0 – se citesc din prima linie; None – vor fi etichetate de la 0 la N; un întreg nenul indică linia din care se citesc numele de coloane
<code>names</code>	listă	None	Lista cu numele asociate coloanelor
<code>index_col</code>	întreg		Coloana care furnizează numele instanțelor

3 Lucru cu masive multidimensionale din pachetul numpy

NumPy este o bibliotecă Python utilizată pentru lucrul cu masive multidimensionale. Pachetul are, de asemenea, funcții pentru lucrul în domeniul algebrei liniare, al transformatei de Fourier și al matricelor. NumPy a fost creat în 2005 de Travis Oliphant. Este un proiect *open source* și poate folosit în mod gratuit pentru integrarea în aplicații complexe de calcul. Numele bibliotecii NumPy este o prescurtare de la *Numerical Python*. Codul sursă pentru NumPy se poate descărca din *github* de la acest link: <https://github.com/numpy/numpy>

Așa cum am văzut în capitolul dedicat elementelor de bază ale limbajului, în Python avem la dispoziție liste ce pot fi utilizate pentru modelarea matricelor și a masivelor cu mai multe dimensiuni, însă acestea sunt procesare lent.

NumPy își propune să furnizeze un obiect matrice care este cu până la 50 de ori mai rapid decât listele tradiționale Python. Obiectul matrice din NumPy se numește `ndarray` și biblioteca oferă, de asemenea, o multitudine de funcții care facilitează lucrul cu `ndarray`.

Tablourile sunt foarte frecvent utilizate în analiza datelor, unde viteza și modalitatea de gestiune a resurselor sunt foarte importante atunci când avem de prelucrat seturi mari de date.

Tablourile NumPy sunt stocate într-un singur loc contiguu în memorie (este implementat conceptul de referințe localizate), spre deosebire de liste, astfel încât procesele care le utilizează să le poată accesa și manipula foarte eficient. Acesta este principalul motiv pentru care NumPy este mai rapid decât listele. De asemenea, biblioteca este optimizată și continuu actualizată pentru a putea beneficia de performanțele celor mai recente arhitecturi CPU. NumPy este o bibliotecă Python și este scrisă parțial în Python, dar majoritatea părților care necesită calcul rapid sunt scrise în C sau C ++.

3.1 Masive multidimensionale ndarray

Masivele de tip `ndarray` sunt folosite de structuri de date Python ca suport de tip container (de exemplu `DataFrame` din *pandas*). Obiectele `ndarray` pot fi create utilizând constructorul sau prin diverse funcții precum `array`, `zeros`, `empty`, `full` etc. Forma generală a constructorului.

```
class numpy.ndarray(shape, dtype=float, buffer=None, offset=0, strides=None, order=None)
```

De exemplu, crearea unei instanțe de `ndarray` se realizează după cum urmează:

```
import numpy as np

x = np.ndarray(shape=(2,1),dtype=int,buffer=bytes([0,0,0,0,0,0,0,0,0,0]),offset=2)
print(x)
```


generează următorul posibil rezultat, în funcție de valorile generate aleatoriu:

```
x:
[[0.4990161  0.98840337 0.31075348]
 [0.29552643 0.873924  0.44115962]
 [0.23291045 0.49319993 0.69960656]]
y:
[[-0.6274206  -0.51708741  1.0099042 ]
 [-0.16381457 -1.88115724 -0.78761556]
 [-0.95566761 -0.44526572 -0.72777706]]
```

```
numpy.diag(v, k=0)
```

Furnizează o matrice diagonală cu elementele din *v* dacă *v* este unidimensional sau diagonală lui *v* dacă *v* este matrice.

```
numpy.trace(a, offset=0, axis1=0, axis2=1, dtype=None, out=None)
```

Furnizează urma matricei *a*.

```
numpy.tril(m, k=0)
```

Furnizează matricea inferior diagonală a lui *m*.

```
numpy.triu(m, k=0)
```

Furnizează matricea superior diagonală

```
numpy.tri(N, M=None, k=0, dtype=<class 'float'>)
```

Furnizează o matrice cu valori 1 sub diagonală și pe diagonală, restul fiind 0. *N* este numărul de linii, iar *M* este numărul de coloane. Dacă nu este specificat *M*, acesta va fi *N*.

3.1.2 Atribute ale masivelor *ndarray*

Constructorul clasei *ndarray* poate avea următoarele atribute:

shape – tuplu cu numărul de elemente pe fiecare dimensiune. Modificarea atributului *shape* se poate face doar cu menținerea numărului de elemente.

ndim – număr de dimensiuni

size – număr de elemente

real, *imag* – parte reală/imaginară

T – transpusă

nbytes – numărul total de octeți

itemsize – numărul de octeți ai unui element

3.1.3 Indexarea masivelor *numpy.ndarray*

Referirea unui element se face prin indecși de linie, coloană șamd. Indecșii sunt trecuți între paranteze pătrate sub forma:

```
[index_linie, index_coloana] sau [index_linie][ index_coloana]
```

Referirea unor părți întregi din masiv (feliere, partiționare – *slice*) se face prin expresii de forma:

```
start:stop:step
```

unde *start* este indexul de început, *stop* este indexul de sfârșit, iar *step* este pasul.

În acest interval, indexul va avea valorile în intervalul deschis la dreapta

```
[start, stop-step)
```

În cazul partiționării folosind indexarea prin paranteze duble `[] []` expresiile de partiționare au sens doar pentru ultima dimensiune.

Prin partiționare se obține o perspectivă a masivului original. Datele nu sunt copiate în alt masiv. Modificările făcute afectează masivul original.

Secvența următoare de cod:

```
import numpy as np
```

```
x = np.array(np.arange(start=1, stop=4*5+1)).reshape(4,5)
print("x:", x)
print("Elementul de pe linia 3 si coloana 4:", x[2,3])
print("Elementul de pe linia 3 si coloana 4:", x[2][3])
print("Coloana 3:", x[:,2])
print("Linia 3:", x[2,:])
print("Linia 3:", x[2][:])
print("Submatrice:", x[1:3,1:3])
y = x[1:3,1:3]
y[:,:] = 0
print('z:', x)
```

va genera, în urma execuției, următorul rezultat:

```
x: [[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]]
Elementul de pe linia 3 si coloana 4: 14
Elementul de pe linia 3 si coloana 4: 14
Coloana 3: [ 3  8 13 18]
Linia 3: [11 12 13 14 15]
Linia 3: [11 12 13 14 15]
Submatrice: [[ 7  8]
 [12 13]]
z: [[ 1  2  3  4  5]
 [ 6  0  0  9 10]
 [11  0  0 14 15]
 [16 17 18 19 20]]
```


Numpy permite indexarea booleană. Indexarea masivelor se poate face folosind ca index un alt masiv de tip boolean. Masivul-index trebuie să aibă aceleași dimensiuni cu masivul indexat. Prin indexarea booleană sunt selectate valorile care corespund elementelor *True* din masivul-index. De exemplu:

```
import numpy as np
```

```
y = np.array(np.arange(1,16)).reshape((3,5))
print(y[y>7])
```

va produce:

```
[8 9 10 11 12 13 14 15]
```

ceea ce înseamnă că au fost selectate valorile mai mari decât 7 din *y*.

O altă modalitate flexibilă de indexare este prin masive *numpy* și prin tuple. Dacă masivul-index este vector, elementele vectorului vor specifica elementele primei dimensiuni din masivul indexat. Dacă masivul-index este o matrice, prin indexare se va selecta un masiv tridimensional cu un număr de matrice egal cu numărul de linii al matricei-index, fiecare matrice fiind construită de elementele din liniile matricei-index. La indexarea prin tuple, numărul de elemente din tuple nu trebuie să depășească numărul de dimensiuni.

3.1.4 Specificarea dimensiunilor

La indexarea unui *ndarray* dimensiunea este indicată de paranteze:

```
[dimensiune1, dimensiune2, ...]
```

sau

```
[dimensiune1][dimensiune2]...
```

În funcții care implică efectuarea unor calcule la nivelul masivului, dimensiunile sunt specificate prin axe care specifică sensul calculului, așa cum este indicat în figura 3.1.

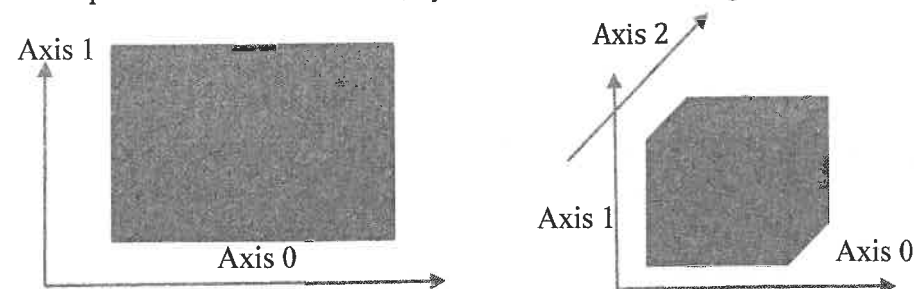


Figura 3.1 Specificarea dimensiunilor unui masiv *ndarray* prin axe

Redimensionarea masivelor se face prin funcțiile:

```
ndarray.reshape(shape, order='C')
```

```
numpy.reshape(a, newshape, order='C')
```

3.2 Operații aritmetice și logice

Pot fi efectuate operații aritmetice la nivel de masiv, fără repetiții prin vectorizare. Acestea pot fi realizate utilizând operatorii clasici:

*	- înmulțire la nivel de element
/	- împărțire
+, -	- adunare, scădere
@	- înmulțire matriceală
**	- ridicare la putere
>, <, >=, <=	- comparații

Spre exemplu, codul următor:

```
import numpy as np
```

```
x = np.array([[1., 2., 3.], [4., 5., 6.]])
y = np.arange(9).reshape((3,3))
print("x:", x, "\ny:", y)
print("x+x:", x+x); print("x-x:", x-x); print("x*2:", x*2); print("x/2:", x/2)
print("x**0.5:", x**0.5); print("x@y:", x@y)
print("x<=3:", x<=3)
```

va produce la rulare următorul rezultat:

```
x: [[1. 2. 3.]
     [4. 5. 6.]]
y: [[0 1 2]
     [3 4 5]
     [6 7 8]]
x+x: [[ 2.  4.  6.]
      [ 8. 10. 12.]]
x-x: [[0. 0. 0.]
      [0. 0. 0.]]
x*2: [[ 2.  4.  6.]
      [ 8. 10. 12.]]
x/2: [[0.5 1.  1.5]
      [2.  2.5 3.  ]]
x**0.5: [[1.          1.41421356  1.73205081]
         [2.          2.23606798  2.44948974]]
x@y: [[24. 30. 36.]
      [51. 66. 81.]]
x<=3: [[ True  True  True]
      [False False False]]
```


3.3 Funcții universale

Sunt funcții generale care se pot executa și la nivel de simpli scalari. În numpy acestea sunt executate prin vectorizare. Tabelul 3.1 prezintă sintetic aceste funcții:

Tabelul 3.1 Funcții universal puse la dispoziție de biblioteca numpy

Funcție	Descriere
abs fabs	Valoare absolută
sqrt	Radical
square	Pătrat
exp	Putere naturală
log log10 log2 log1p	Logaritmi: $e, 10, 2 \log_e(1 + x)$
sign	Calculează semnul: 1 (pozitiv) 0 (zero) -1 (negativ)
ceil	Cel mai mic întreg mai mare sau egal cu argumentul
floor	Cel mai mare întreg mai mic sau egal cu argumentul
rint	Rotunjire întreagă
modf	Întoarce partea întreagă și fracționară în masive separate
isnan	Întoarce masiv cu indicații booleene asupra valorii NaN
isfinite isinf	Întoarce masiv cu indicații booleene asupra valorilor finite/infinite
cos cosh sin sinh tan tanh	Funcții trigonometrice normale și hiperbolice
arccos arccosh arcsin arcsinh arctan arctanh	Funcții trigonometrice inverse
add	Adunare scalară
subtract	Scădere
multiply	Înmulțire
divide floor_divide	Împărțire
power	Putere
maximum fmax	Maxim
minimum fmin	Minim
mod	Modul
greater greater_equal less less_equal equal not_equal	Comparații
logical_and logical_or logical_xor	Operații logice

Tipurile de date ce pot fi furnizate unui masiv numpy.ndarray sunt prezentate sintetic în tabelul 3.2.

Tabelul 3.2 Tipurile de date ce pot fi furnizate unui masiv numpy.ndarray

Tip	Descriere
int8 uint8	Întreg cu/fără semn pe 1 byte
int16 uint16	Întreg cu/fără semn pe 2 bytes
int32 uint32	Întreg cu/fără semn pe 32 biți
int64 uint64	Întreg cu/fără semn pe 64 biți
float16	Real virgulă mobilă pe 16 biți
float32	Real virgulă mobilă pe 32 biți; compatibil cu float C
float64	Real virgulă mobilă pe 64 biți compatibil cu float C și standard float Python
float128	Real virgulă mobilă pe 64 biți
complex64 complex128 complex256	Complex pe 32, 64 sau 128 biți
bool	Boolean
object	Tip <i>object</i> Python – o valoare care poate fi orice obiect Python
string_	ASCII cod lungime fixă – 1 byte per caracter
unicode_	Unicode

3.4 Funcții statistice

Funcțiile statistice sunt implementate în manieră vectorizată la nivel de masiv/tabel de date sau de linie/coloană.

```
numpy.mean(a, axis=None, dtype=None, out=None, keepdims=<no value>)
```

Calcul mediei. Prin parametrul *axis*, se indică la ce nivel (dimensiune) se va face calculul: *axis*=0 înseamnă prima dimensiune. În cazul unei matrice, aceasta înseamnă că se vor calcula mediile pe coloane.

Observație:

Este important de remarcat faptul că, în Python, dimensiunile masivelor sunt văzute ca un sistem de axe. Axa 0 marchează coloanele, iar axa 1 marchează liniile.

```
numpy.std(a, axis=None, ddof=0, ...)
```

Abaterea standard. Parametrul *ddof* este utilizat în calculul gradelor de libertate: *ddof*=0 înseamnă N grade de libertate, unde N este numărul elementelor.

```
numpy.var(a, axis=None, ddof=0, ...)
```

Varianța – funcția pentru calculul varianței primește parametri similari cu cei necesari calculului abaterii standard.

```
numpy.average(a, axis=None, weights=None, returned=False)
```

Calcul medie ponderată. Ponderile sunt furnizate prin parametrul `weights`.

```
numpy.corrcoef(x, y=None, rowvar=True, bias=<no value>, ddof=<no value>)
```

Calcul matricei coeficienților de corelație Pearson. Parametrul `rowvar` indică dispunerea variabilelor pe linii (`True`) sau pe coloane (`False`). Dacă `x` și `y` sunt matrice, se va calcula un tabel partiționat cu patru matrice de corelație:

$$\begin{bmatrix} R_{xx} & R_{xy} \\ R_{yx} & R_{yy} \end{bmatrix},$$

unde R_{yy} și R_{xx} sunt matricele de corelație ale lui `x` și `y`, R_{xy} și R_{yx} sunt matricele de corelație între coloanele/liniile matricei `x` și cele ale matricei `y`.

```
numpy.cov(m, y=None, rowvar=True, bias=False, ddof=None, fweights=None, aweights=None)
```

Calcul matrice covarianță. Parametrii `bias` și `ddof` controlează gradele de libertate: `bias=False`, `ddof=1` înseamnă $N-1$ grade de libertate, `bias=True`, `ddof=0` înseamnă N grade de libertate. Parametrii `fweights` și `aweights` furnizează frecvențe și ponderi folosite în calcul. Parametrul `rowvar` indică așezarea seriilor de date pe linie (`True`) sau coloană (`False`).

Codul de mai jos exemplifică standardizarea valorilor unei matrice:

```
import numpy as np
```

```
x = np.asarray([[3,4,5],[1,1,1],[2,2,2]])
print("x:",x)
stdx = np.std(x,axis=0)
print("Abateri standard pe coloane:",stdx)
meanx = np.mean(x,axis=0)
print("Medii pe coloane:",meanx)
z = (x-meanx)/stdx
print("x standardizat:",z)
```

Rezultatul rulării este următorul:

```
x: [[3 4 5]
     [1 1 1]
     [2 2 2]]
Abateri standard pe coloane: [0.81649658 1.24721913 1.69967317]
Medii pe coloane: [2.          2.33333333 2.66666667]
x standardizat: [[ 1.22474487  1.33630621  1.37281295]
                 [-1.22474487 -1.06904497 -0.98058068]
                 [ 0.          -0.26726124 -0.39223227]]
```

```
numpy.histogram(a, bins=10, range=None, normed=None, weights=None, density=None)
```

Calcul histograme. Funcția întoarce frecvențele și intervalele, iar parametrii funcției au semnificațiile următoare:

`a` - masiv *numpy* sau orice obiect compatibil (listă, serie Pandas etc); masivele multidimensionale sunt liniarizate.

`bins` - număr de grupe sau limite de grupe sau numele strategiei de grupare (ex. 'sturges')

`range` - limitele. Implicit: `(a.min(), a.max())`

`weights` - ponderi asociate valorilor

`density` - calcul și returnare frecvențe sub formă de probabilități, astfel:

$$p_i = n_i / (n * latime_interval_i),$$

unde p_i este probabilitatea calculată pentru grupa i , n_i este frecvența grupei i , n este numărul total de elemente $latime_interval_i$ este lățimea intervalului pentru grupa i (limita superioară - limita inferioară).

3.5 Funcții pentru inversare, sortare, căutare, selecție

Introducem succint următoarele funcții pentru inversare, sortare, căutare și selecție.

```
numpy.sort(a, axis=-1, kind='quicksort', order=None)
```

Întoarce o copie sortată a obiectului (convertibil la masiv). Parametrul `axis` poate fi `None`, caz în care este sortat tot masivul liniarizat, `-1` pentru sortare după ultima axă sau poate specifica o axă de sortare. Parametrul `kind` specifică algoritmul și poate fi: 'quicksort', 'mergesort', 'heapsort'. Parametrul `order` specifică în ce ordine sunt utilizate câmpurile în sortare, în cazul când un element din masiv are mai multe câmpuri.

```
numpy.argsort(a, axis=-1, kind='quicksort', order=None)
```

Întoarce indicii elementelor sortate.

```
numpy.flipud(m), numpy.fliplr(m)
```

Inversează elementele unui masiv pe linii/coloane. `ud` -> up/down (pe coloane), `lr` -> left/right (pe linii). Prima funcție este aplicată și pentru masive unidimensionale și produce inversarea elementelor.

```
numpy.where(condition[, x, y])
```

Întoarce indecșii elementelor care îndeplinesc condițiile sub formă de tuplu.

```
numpy.take(a, indices, axis=None, out=None, mode='raise')
```

Întoarce valorile din `a` corespunzătoare indicilor.

3.6 Funcții matematice și de algebră liniară

Funcțiile matematice implementează operații matematice de bază precum sume, produse, funcții trigonometrice, logaritmice, exponențiale etc.

```
numpy.sum(a, axis=None, ..., initial=<no value>)
```

Calculează suma la nivel de masiv sau pe axe (coloane sau linii pentru un masiv bidimensional). Parametrul *inițial* este valoarea inițială a sumei.

```
numpy.prod(a, axis=None, ..., initial=<no value>)
```

Idem produsul.

```
numpy.cumsum(a, axis=None, ...)
```

Calculează sume cumulative (progresive) la nivel de masiv sau pe dimensiuni.

```
numpy.cumprod(a, axis=None, ...)
```

Idem produse.

```
numpy.transpose(a, axis=None)
```

Calcul transpusă pentru masivul *a*. *axes* este o listă de întregi reprezentând axele transpuse.

```
numpy.linalg.inv(a)
```

Calculează inversa matricei *a*.

```
numpy.matmul(a, b, out=None)
```

Produsul a două matrice.

```
numpy.linalg.eig(a)
```

Calculează valorile proprii și vectorii proprii ai matricei *a*.

```
numpy.linalg.svd(a, full_matrices=True, compute_uv=True)
```

Efectuează descompunerea în valori singulare pentru matricea *a*. Parametrul *compute_uv* indică returnarea matricelor ortogonale. Parametrul *full_matrices* indică determinarea matricelor ortogonale în formă completă.

4

Tipuri de date ale pachetului pandas

4.1 Serii

Seriile sunt masive unidimensionale etichetate care operează cu date de orice tip (întregi, șiruri, numere reale, obiecte Python etc.). Etichetele axelor sunt denumite indici.

```
class pandas.Series(data=None, index=None, dtype=None, name=None, copy=False, fastpath=False)
```

data – structura de date care va furniza conținutul. Poate fi orice este reductibil la masiv unidimensional sau *dict*.

index – furnizează etichetele și poate fi o structură reductibilă la masiv unidimensional. Trebuie să aibă aceeași dimensiune cu *data*.

dtype – tipul de dată al elementelor din *data*. Poate fi *str* sau un tip *numpy*.

copy – indică dacă se creează structura *data* prin copierea sursei.

Spre exemplu, secvența următoare de cod:

```
import pandas as pd

a = np.array([1,2,3])
s1 = pd.Series(data=a)
s2 = pd.Series(data=a, copy=True)
a[2] = 1000
print(s1,s2,sep="\n")
```

va genera următorul rezultat:

```
0      1
1      2
2    1000
dtype: int32
0      1
1      2
2      3
dtype: int32
```

Este o structură compatibilă cu *ndarray* din *numpy* și *dict*, deci poate fi operată în același mod. În operațiunile cu două serii vor fi identificate și operate elementele care au același index. În exemplul de mai jos se va face media pe discipline pentru două serii care reprezintă note identificate prin numele disciplinelor. Depinde de textul enunțat.

4.2 Tabele de date (DataFrame)

Un tabel de date reprezintă o structură de date bidimensională ce constă într-o colecție ordonată de coloane, fiecare dintre acestea putând avea un tip de valoare diferit (numeric, șir, boolean, etc.). Un *DataFrame* are atât indici de rând, cât și de coloană. Poate fi considerat ca un dicționar de serii.

```
class pandas.DataFrame(data=None, index=None, columns=None, dtype=None,
copy=False)
```

data – este o structură de tip *dict*, NumPy *ndarray* sau *DataFrame*.

index – numele de rânduri

columns – numele de coloane

dtype – tip de date aplicat întregului tabel. În caz de incompatibilitate este asumat tipul *object*.

copy – indică copierea structurii de date. Parametrul se aplică doar în cazul masivelor *numpy* bidimensionale.

Accesul la numele liniilor și coloanelor se face prin proprietățile *index* și *columns*. Aceste proprietăți sunt din clasa *pandas.Index*. Elementele pot fi accesate prin funcția *get_values()* a clasei *Index*. Indexul numeric al unei valori se poate obține prin metoda *get_loc(valoare)*, unde *valoare* este numele unei linii sau coloane.

Spre exemplu, codul următor:

```
import pandas as pd
```

```
note = pd.DataFrame({"Structuri de date": [5, 4, 6],
                    "Analiza datelor": [10, 6, 7],
                    "Algebra": [7, 8, 7]},
                    index=["Ionescu Dan", "Popescu Diana", "Georgescu Radu"])

print(note)
print(note.index, note.columns, sep='\n')
print(note.index[0], note.columns[1])
```

va genera rezultat de mai jos:

	Structuri de date	Analiza datelor	Algebra
Ionescu Dan	5	10	7
Popescu Diana	4	6	8
Georgescu Radu	6	7	7

```
Index(['Ionescu Dan', 'Popescu Diana', 'Georgescu Radu'], dtype='object')
Index(['Structuri de date', 'Analiza datelor', 'Algebra'], dtype='object')
Ionescu Dan Analiza datelor
```

Numele de coloane este preluat în exemplul precedent din cheile dicționarului. Selecția unor coloane se face specificând numele coloanelor între paranteze pătrate:

```
nume_frame[['coloana1', ..., 'coloanaK']]
```

Selecția liniilor se poate realiza prin expresii de tip partiționare:

```
index_initial:index_final
```

unde indecșii sunt opționali (dacă lipsesc se selectează toate liniile).

Selecția se face de la *index_initial* la *index_final-1*.

Selecția și partiționarea se pot face prin atributele *loc* și *iloc*, folosind numele de item (pentru *loc*) sau indexul său (pentru *iloc*).

De exemplu, pentru tabelul *DataFrame* creat în exemplul precedent, putem să continuăm scrierea următoarei secvențe de cod:

```
print(note.loc["Popescu Diana", "Algebra"])
print(note.loc["Popescu Diana"]["Algebra"])
print(note.loc["Popescu Diana", : "Algebra"])
print(note.loc["Popescu Diana"][: "Algebra"])
print(note.loc["Popescu Diana", : "Algebra"])
print(note.loc["Popescu Diana"][: "Algebra"])
print(note.iloc[1, 2])
print(note.iloc[1][2])
print(note.iloc[[1, 2], 2])
#print(note.iloc[[1, 2]][2])#Gresit
print(note.iloc[1:, 2])
#print(note.iloc[1:][2])#Gresit
print(note.iloc[[1, 0], [1, 2]])
print(note.iloc[1][[1, 2]])
print(note.iloc[1, 1:])
print(note.iloc[1][1:])
print(note.iloc[1:, 1:])
```

Rezultatul tipărit la consolă va fi următorul:

```
8
8
Structuri de date      4
Analiza datelor       6
Algebra               8
Name: Popescu Diana, dtype: int64
Structuri de date      4
Analiza datelor       6
Algebra               8
Name: Popescu Diana, dtype: int64
      Structuri de date  Analiza datelor  Algebra
Popescu Diana           4              6        8
Georgescu Radu          6              7        7
      Structuri de date  Analiza datelor  Algebra
Popescu Diana           4              6        8
Georgescu Radu          6              7        7
8
8
Popescu Diana          8
Georgescu Radu         7
```

```
Name: Algebra, dtype: int64
Popescu Diana      8
Georgescu Radu     7
Name: Algebra, dtype: int64
      Analiza datelor  Algebra
Popescu Diana          6      8
Ionescu Dan           10      7
Analiza datelor      6
Algebra              8
Name: Popescu Diana, dtype: int64
Analiza datelor      6
Algebra              8
Name: Popescu Diana, dtype: int64
Analiza datelor      6
Algebra              8
Name: Popescu Diana, dtype: int64
      Analiza datelor  Algebra
Popescu Diana          6      8
Georgescu Radu         7      7
```

4.3 Modificare, salvare, sortare, agregare și joncțiune în DataFrame

4.3.1 Modificarea datelor din DataFrame

Modificarea valorilor dintr-o coloană se poate face prin atribuire directă:

```
nume_frame['nume_coloana']=valoare
```

Când atribuirea se face cu o listă sau un masiv, acestea trebuie să aibă un număr de elemente egal cu numărul de itemi din *DataFrame*.

Ștergerea unei coloane se face prin comanda `del`:

```
del nume_frame['nume_coloana']
```

Inserarea unei coloane se face prin metoda *insert*:

```
DataFrame.insert(loc, column, value, allow_duplicates=False)
```

unde *loc* este indexul inserției, *column* este numele coloanei, iar *value* este valoarea asociată itemilor pentru coloana inserată.

O altă posibilitate de a șterge atât coloane, cât și linii este prin metoda *drop*:

```
DataFrame.drop(self, labels=None, axis=0, index=None, columns=None, level=None,
inplace=False, errors='raise');
```

unde:

labels – reprezintă o listă cu nume de linii sau coloane;

axis – este 0 sau 1 după cum *labels* este interpretat ca nume de linii sau coloane;

columns – este listă de coloane și este o alternativă la *labels* și *axis=1*;

inplace – este un boolean care indică dacă modificarea se face cu înlocuirea obiectului curent (*True*) sau se realizează și se returnează o copie modificată și nu se modifică obiectul curent.

4.3.2 Salvarea în fișier CSV

```
DataFrame.to_csv(path_or_buf=None, sep=',', na_rep='', float_format=None,
columns=None, header=True, index=True, index_label=None, ...)
```

path_or_buf – este numele fișierului. Dacă nu este furnizat, metoda întoarce string.

4.3.3 Sortarea

Sortarea tabelelor se poate face prin metoda *sort_values*:

```
DataFrame.sort_values(by, axis=0, ascending=True, inplace=False,
kind='quicksort', na_position='last')
```

by – reprezintă criteriul. Poate și un șir de caractere reprezentând numele coloanei sau o listă de șiruri de caractere pentru o sortare multicriterială – ["nume1", "nume2", ...].

axis – reprezintă direcția de sortare – 0 sau 'index', 1 sau 'columns'. Dacă se indică 1, atunci și criteriile din *by* trebuie să fie nume de linii.

ascending – sensul sortării. Se furnizează precum criteriul, valoare logică sau listă de valori logice.

inplace – indică sortarea în același tabel.

kind – metoda de sortare. Variante: {'quicksort', 'mergesort', 'heapsort'}.

na_position – locul în care sunt puse liniile/coloanele care au NaN pentru criteriile de sortare. Variante: {'first', 'last'}.

4.3.4 Agregarea datelor

Obiectele de tip *DataFrame* pot fi supuse unor operațiuni de agregare, transformare, filtrare. Prima operațiune care se efectuează înainte de agregare, transformare sau filtrare este împărțirea datelor în grupe după diverse criterii (*split*). Metoda *groupby* realizează acest lucru:

```
DataFrame.groupby(by=None, axis=0, level=None, as_index=True, sort=True,
group_keys=True, squeeze=False, observed=False, **kwargs)
```

by – criteriul de grupare. Poate fi numele unei coloane sau o serie sau un vector sau o funcție care să identifice grupele.

axis – axa de grupare (0 – grupare pe coloane).

as_index – indică folosirea cheilor de grupare ca index în rezultatul prelucrării, în cazul care prelucrarea este o agregare

sort – sortarea automată după cheile criteriului de grupare

Rezultatul prelucrării îl constituie un obiect *pandas.core.groupby.DataFrameGroupBy*. Metodele acestei clase permit operațiuni de agregare, transformare, filtrare.

Metode *DataFrameGroupBy* pentru sume, medii, numărare, varianță, abatere standard și produs:

```
GroupBy.sum(**kwargs)
```

```
GroupBy.mean(*args, **kwargs)
```

```
GroupBy.count()
```

```
GroupBy.var(ddof=1, *args, **kwargs)
```

```
GroupBy.std(ddof=1, *args, **kwargs)
```

```
GroupBy.prod(**kwargs)
```

Transformarea datelor se poate realiza prin funcția *transform*:

```
GroupBy.transform(func, *args, **kwargs)
```

func – este funcția aplicată. Se poate folosi operatorul *lambda* pentru transmiterea datelor grupului pentru transformare.

Funcțiile care permit agregarea particularizată a datelor sunt *agg* și *apply*:

```
GroupBy.apply(func, *args, **kwargs)
```

```
GroupBy.agg(arg, *args, **kwargs)
```

Funcțiile *func*, *arg* sunt funcțiile aplicate grupurilor.

În exemplul din codul următor sunt realizate agregare cu sumarizare, transformare și o agregare particularizată pentru un tabel de date:

```
import pandas as pd
import pandas.core.groupby as gby
```

```
# Calculul ponderilor pe coloane
```

```
def ponderi(x):
```

```
    return x / x.sum()
```

```
# Calculul sumelor patratelor ponderilor
```

```
def sume2(x):
```

```
    p = x / x.sum()
```

```
    p2 = p * p
```

```
    return p2.sum()
```

```
t1 = pd.DataFrame({
```

```
    'c1': [1, 1, 2, 1, 1, 2, 2, 2, 1],
```

```
    'c2': [10, 20, 30, 10, 40, 110, 140, 125, 100],
```

```
    'c3': [2.3, 2.6, 3, 0, 14, 10, 5.5, 11, 11.5]})
```

```
    index=['i1', 'i2', 'i3', 'i4', 'i5', 'i6', 'i7', 'i8', 'i9'])
g = t1.groupby('c1')
assert isinstance(g, gby.DataFrameGroupBy)
print("Medii pe grupe:", g.mean(), sep="\n")
print("Ponderi pe grupe:", g.transform(func=lambda x: ponderi(x)), sep="\n")
print("Sume de patrate (apply):", g.apply(func=sume2), sep="\n")
print("Sume de patrate (agg):", g.agg(arg=sume2), sep="\n")
```

Execuția codului va produce următorul rezultat:

```
Medii pe grupe:
      c2      c3
c1
a   20.0  5.433333
b   87.5  7.266667
Ponderi pe grupe:
      c2      c3
i1  0.166667  0.141104
i2  0.038095  0.059633
i3  0.057143  0.068807
i4  0.166667  0.000000
i5  0.666667  0.858896
i6  0.209524  0.229358
i7  0.266667  0.126147
i8  0.238095  0.252294
i9  0.190476  0.263761
Sume de patrate (apply):
      c1      c2      c3
c1
1   0.20  0.376543  0.368226
2   0.25  0.294010  0.299052
Sume de patrate (agg):
      c2      c3
c1
1   0.376543  0.368226
2   0.294010  0.299052
```

Se poate observa că, în cazul funcției *apply*, spre deosebire de *agg*, sumarizarea s-a aplicat inclusiv coloanei care identifică grupurile (*c1*).

4.3.5 Joncțiuni

Joncțiunea tabelelor se poate realiza prin metoda *merge*:

```
DataFrame.merge(right, how='inner', on=None, left_on=None, right_on=None,
left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'),
copy=True, indicator=False, validate=None)
```

right – celălalt tabel.

how – metoda de joncționare. Implicit *inner* – pe chei comune.

on – numele coloanei după care se face joncțiunea. Trebuie să aparțină ambelor tabele.

Printre metodele specifice clasei *Counter* se găsesc:

most_common([n]) - furnizează sub formă de listă de tuple cele mai frecvente *n* perechi (*cheie:valoare*).

subtract([iterable-or-mapping]) - decrementează pentru cheile specificate valorile, cu valorile specificate.

În exemplu pentru obiectul *counter* din codul anterior:

```
print(counter.most_common(2))
counter.subtract({"a": 2})
print(counter)
```

va genera următorul rezultat:

```
[('a', 3), ('c', 3)]
Counter({'c': 3, 'a': 1, 'b': 1})
```

5

Elemente de reprezentare grafică în Python

Vizualizarea informației este un proces important în analiza datelor. Una dintre cele mai populare și utilizate biblioteci Python pentru vizualizări statice sau dinamice este *matplotlib*. Pe *matplotlib* se bazează biblioteci adiționale, cum ar fi *seaborn* sau modulul *pandas.plotting* din *pandas*.

Construirea unei ferestre grafice interactive se face prin funcția *figure*.

```
matplotlib.pyplot.figure(num=None, figsize=None, dpi=None, facecolor=None,
edgecolor=None, frameon=True, FigureClass=<class 'matplotlib.figure.Figure'>,
clear=False, **kwargs)
```

num - antetul ferestrei

figsize - dimensiunea figurii, lățime și înălțime în inches.

dpi - rezoluția

facecolor - culoarea fondului

edgecolor - culoarea bordurii

frameon - inhibă desenarea pentru valoarea False

clear - stabilește dacă se șterge fereastra cu același nume

Afișarea ferestrei se realizează prin apelul metodei *show()*.

Spre exemplu, secvența următoare de cod:

```
from matplotlib import pyplot as plt

f = plt.figure("Fereastra test",figsize=[4,2],facecolor=(1,1,0.8))
plt.show()
```

va genera graficul din figura 5.1.

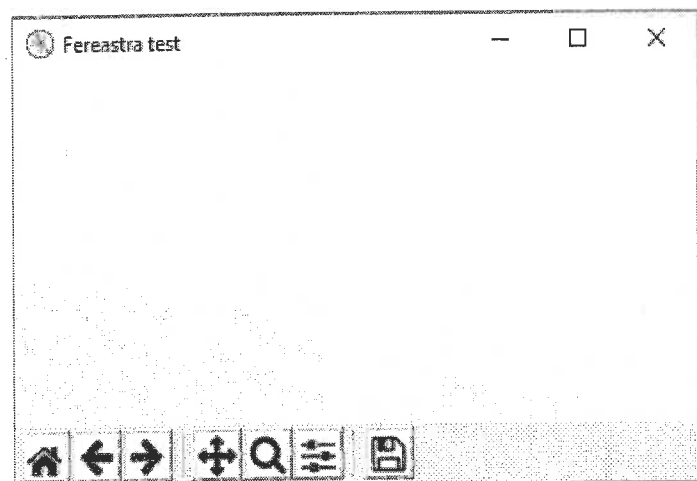


Figura 5.1 Fereastra generică pentru găzduirea graficelor pyplot

5.1 Adăugarea de subgrafice

Adăugarea de subgrafice la figură se realizează prin metoda clasei *figure*:

```
figure.add_subplot(*args, **kwargs)
```

Prin *args* se va specifica structura figurii și poziția subgraficului astfel: *nrows*, *ncols*, *index* – număr linii, număr coloane în figură și indexul subgraficului.

Parametrii *kwargs*: *title* – titlul, *xlabel*, *ylabel* – etichete axe.

Funcția întoarce obiect *pyplot.Axes*.

```
f = plt.figure("Grafic linie",figsize=(10,9))
f1 = f.add_subplot(2,1,1)
f2 = f.add_subplot(2,1,2)
```

5.2 Tipuri de grafice în matplotlib

5.2.1 Grafice linie - *matplotlib.pyplot*

```
matplotlib.pyplot.plot(*args, scalex=True, scaley=True, data=None, **kwargs)
```

args – argument de lungime variabilă format din triplete *[x],y,[fmt]*, unde perechea de paranteze *[]* arată caracterul opțional al parametrului, *x* și *y* sunt vectori sau compatibili cu vectori, iar *fmt* este un șir cu formatul sintetic al afișării. *fmt* este de forma: *'[color][marker][line]'*, parametrii de tip șir din *kwargs*.

Spre exemplu, dacă în codul anterior inserăm:

```
plt.plot(list(range(1990,1995)), [10,11,15,13,10], 'r',
        [1990,1995], [5,10], 'y--',
        [1992,1994,1996], [-11,0,5], 'b-.')
```

rezultatul afișat în fereastra graficului este reprezentat de figura 5.2.

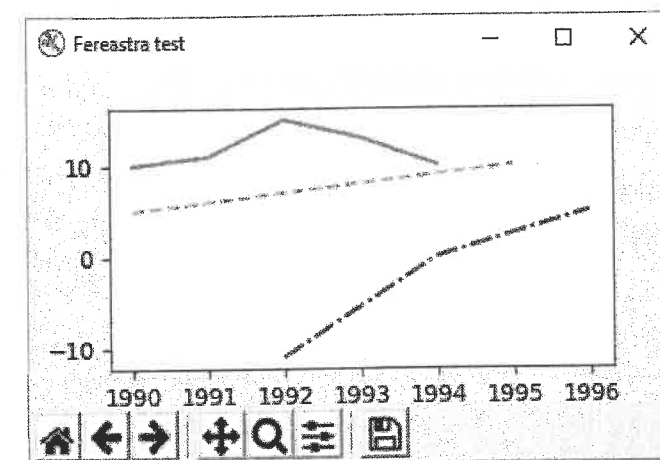


Figura 5.2 Exemple de grafice pyplot de tip linie

scalex, *scaley* – valori logice care indică autoscalarea, adică adaptarea axelor la limitele datelor.

kwargs – parametrii care reprezintă proprietăți ale graficului: culoare, format linie, grosime linie, transparență, hașuri etc.

Parametrii *kwargs* mai des utilizați sunt:

color – culoarea. Există următoarele abrevieri pentru culoare, sintetizate în tabelul 5.1.

Tabelul 5.1 Coduri pentru culoarea liniei

Cod	Culoare
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

linestyle – tipul liniei. Poate avea una din valorile prezentate în tabelul 5.2.

Tabelul 5.2 Valori pentru stilul liniei

Valori stil linie	Semnificație
'-' or 'solid'	linie solidă
'--' or 'dashed'	linie întreruptă
'-.' or 'dashdot'	linie punctată
':' or 'dotted'	linie punctată
'None'	nu desenează nimic
' '	nu desenează nimic
''	nu desenează nimic

marker – tipul de punct. De regulă, apare în grafice *scatter*. Dacă apare în graficele de tip *line*, va determina afișarea punctelor conform marcajului în grafic prezentat în tabelul 5.3.

Tabelul 5.3 Valori pentru tipul de marcaj în matplotlib

Tip punct	Marcaj în grafic
'.'	marcaj punct
'o'	marcaj de cerc
'v'	marcaj triunghi jos
'^'	marcaj triunghi sus
'<'	marcaj triunghi stânga
'>'	marcaj triunghi dreapta
'1'	marcaj triplet jos
'2'	marcaj triplet sus
'3'	marcaj triplet stânga
'4'	marcaj triplet dreapta
's'	marcaj pătrat
'p'	marcaj pentagon
'*'	marcaj stea
'+'	marcaj plus
'x'	marcaj x
'D'	marcaj diamant
'd'	marcaj diamant îngust
' '	marcaj linie verticală
'_'	marcaj linie orizontală

linewidth – lățimea liniei. Este o valoare reală.

label – Eticheta asociată liniei (sau punctelor) în legendă.

5.2.2 Grafice puncte – matplotlib.scatter

```
matplotlib.pyplot.scatter(x, y, s=None, c=None, marker=None, cmap=None,
norm=None, vmin=None, vmax=None, alpha=None, linewidths=None, verts=None,
edgecolors=None, *, data=None, **kwargs)
```

x, y – coordonatele punctelor.

s – mărimea punctelor.

c – culoarea. Poate fi o culoare, n culori pentru cele n puncte sau un vector cu n indecși în schema de culori asociată.

cmap – schemă de culori asociată. O schemă poate fi specificată folosind un nume valid de schemă sau construind una prin constructori de clase derivate din clasa *Colormap* (de exemplu *ListedColormap*).

alpha – transparența punctelor între 0 (transparent) și 1 (opac).

linewidths – lățimea liniei conturului punctelor.

edgecolors – culoarea conturului.

Nume valide de scheme de culori:

```
['Greys', 'Purples', 'Blues', 'Greens', 'Oranges', 'Reds', 'YlOrBr', 'YlOrRd',
'OrRd', 'PuRd', 'RdPu', 'BuPu', 'GnBu', 'PuBu', 'YlGnBu', 'PuBuGn', 'BuGn', 'YlGn']
```

```
['binary', 'gist_yarg', 'gist_gray', 'gray', 'bone', 'pink', 'spring', 'summer',
'autumn', 'winter', 'cool', 'Wistia', 'hot', 'afmhot', 'gist_heat', 'copper']
```

```
['PiYG', 'PRGn', 'BrBG', 'PuOr', 'RdGy', 'RdBu', 'RdYlBu', 'RdYlGn', 'Spectral',
'coolwarm', 'bwr', 'seismic']
```

Clasele *Colormap*, *ListedColormap* se găsesc în modulul *matplotlib.colors*.

```
class matplotlib.colors.ListedColormap(colors, name='from_list', N=None)
```

colors – lista de culori folosită.

N – numărul de culori din schemă. Dacă $N > \text{len}(\text{colors})$, culorile se vor repeta, altfel vor fi reduse la N .

5.2.3 Grafice puncte seaborn.scatterplot

Funcția *scatterplot* din *seaborn* are avantajul, față de aceeași funcție din *matplotlib*, de a facilita trasarea pentru date împărțite în grupe. Punctele care reprezintă instanțele din grupe diferite pot fi afișate în culori și cu stiluri diferite.

```
seaborn.scatterplot(x=None, y=None, hue=None, style=None, size=None, data=None,
palette=None, ..., **kwargs)
```

Datele pot fi furnizate printr-un obiect *DataFrame* (parametrul *data*) sau direct prin vectori folosind parametrii *x* și *y*.

x, y – coordonatele punctelor în cele două axe. Pot fi vectori sau pot fi nume de coloane din tabelul de date transmis prin *data*.

hue – numele coloanei din tabelul de date care furnizează gruparea pentru trasarea în culori diferite.

style – idem, dar pentru trasarea în stiluri diferite (tip marker).

size – idem, dar pentru trasarea cu mărimi diferite.

Codul din exemplul următor:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
```

```
t = pd.DataFrame(data={"grupa": ['a', 'a', 'b', 'b', 'b', 'a', 'a', 'c', 'c', 'c', 'a'],
                        "v1": [10, 20, 23, 45, 12, 78, 45, 56, 24, 12, 66],
                        "v2": [1, 2.6, 2.3, -4.5, 12.7, -7, 5.5, -5.6, 0.24, 12, 6.6]})
```

```

nume_coloane = t.columns
sb.set()

g = sb.scatterplot(x=nume_coloane[1], y=nume_coloane[2],
                  hue=nume_coloane[0], style=nume_coloane[0], data=t)

for i in range(len(t)):
    g.text(t.iloc[i, 1], t.iloc[i, 2], t.index[i])
plt.show()

```

va genera graficul prezentat în figura 5.3. Funcția *set* este folosită pentru a seta parametrii de estetică pe valorile implicite.

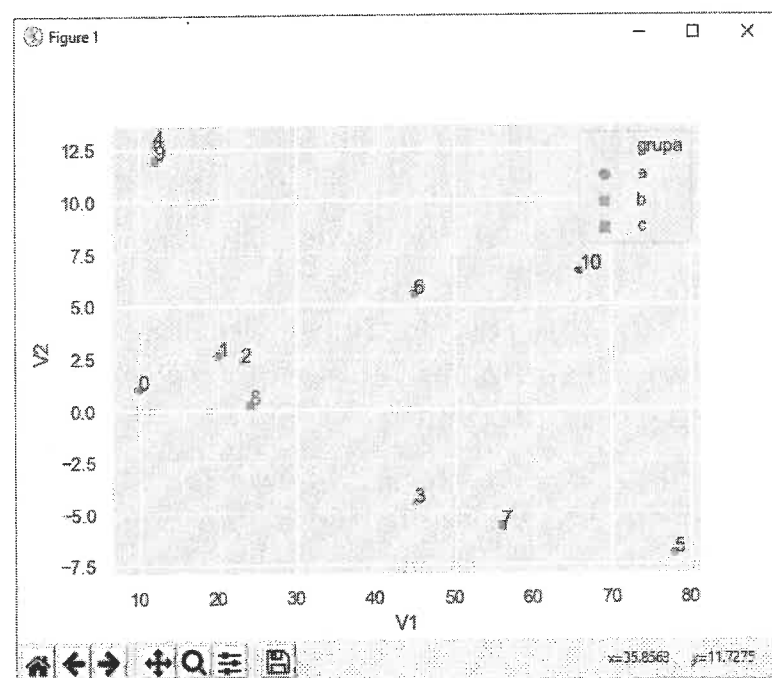


Figura 5.3 Graficul nor de puncte

5.2.4 Grafice corelogramă – *seaborn.heatmap*

Sunt folosite pentru reprezentarea color a unui tabel. De regulă, sunt folosite pentru vizualizarea matricelor de corelații:

```

seaborn.heatmap(data, vmin=None, vmax=None, cmap=None, center=None, robust=False,
annot=None, fmt='.2g', annot_kws=None, linewidths=0, linecolor='white',
cbar=True, cbar_kws=None, cbar_ax=None, square=False, xticklabels='auto',
yticklabels='auto', mask=None, ax=None, **kwargs)

```

data – tabel de date bidimensional cu datele pentru care se face reprezentarea. Poate fi un *DataFrame* sau un *ndarray*. Pentru un *DataFrame* numele de linii și de coloane sunt folosite în reprezentare pentru itemii de linie și de coloană.

vmin, *vmax* – valorile extreme folosite pentru scalarea schemei de culori.

cmap – schema de culori folosită (vezi mai sus *scatter*).

annot – adnotare cu text a celulelor. Poate fi *True*, *False* sau un masiv cu adnotările (trebuie să aibă același *shape* ca *data*).

cbar – indică afișarea schemei de culori.

5.2.5 Histograme *matplotlib*

Histogramele pot fi construite în *matplotlib* utilizând funcția *hist*. Pe lângă trasare, funcția calculează și întoarce informații legate de grupare în conformitate cu funcția *numpy.histogram*. Aceste informații sunt frecvențele și intervalele.

```

matplotlib.pyplot.hist(x, bins=None, range=None, density=None, weights=None,
cumulative=False, bottom=None, histtype='bar', align='mid',
orientation='vertical', rwidth=None, log=False, color=None, label=None,
stacked=False, normed=None, *, data=None, **kwargs)

```

x – masiv unidimensional sau secvență de masive unidimensionale sau compatibile.

bins – poate fi un întreg reprezentând numărul de intervale, o secvență numerică reprezentând limitele de intervale sau un șir reprezentând strategia de grupare (de exemplu 'sturges'). Numărul implicit de grupe este 10 conform cu *numpy.histogram*. Când este furnizată secvență de limite de intervale, primul interval și următoarele sunt închise la stânga iar ultimul este închis la ambele capete.

range – tuplu cu valorile extreme. Implicit este (*x.min()*, *x.max()*).

density – valoare logică ce indică returnarea frecvențelor ca probabilități. Implicit este *False*.

weights – ponderi asociate valorilor din *x*. Este un masiv având același *shape* cu *x*.

cumulative – calcul cumulativ al frecvențelor. Implicit: *False*.

bottom – valoare sau set de valori folosite pentru incrementarea frecvențelor. De exemplu, dacă *bottom* este 10 toate frecvențele pentru toate grupele sunt incrementate cu 10.

histtype – tipul de trasare pentru histogramă. Poate fi {'bar', 'barstacked', 'step', 'stepfilled'}. Implicit este 'bar'.

align – indică felul în care sunt alinate barele. Poate fi {'left', 'mid', 'right'}. Implicit: 'mid'.

orientation – orientarea: {'horizontal', 'vertical'}

rwidth – scalar cuprins între 0 și 1 care indică lățimea barelor ca proporție față de lățimea intervalului. De exemplu pentru 0.9 va genera un spațiu de 10% între bare. Valoare implicită: 1 (fără spațiu).

log – indică utilizarea unei scări logaritmice pentru frecvențe. Implicit *False*.

color – specificare culoare sau set de culori (pentru mai multe secvențe).

label – specifică etichetele pentru secvențe. Sunt folosite în legendă.

stacked – stivuirea barelor în același interval. Implicit: *False*

kwargs – alți parametrii de trasare de tip *matplotlib.patches.Patch* cum ar fi de exemplu *alpha* (pentru transparența barelor).

5.2.6 Histograme pandas

Trasarea histogramelor pe tabele *DataFrame* sau serii de date *Series* din *pandas* se poate face și prin metode *hist* ale acestor clase.

```
DataFrame.hist(column=None, by=None, grid=True, xlabelsize=None, xrot=None,
ylabelsize=None, yrot=None, ax=None, sharex=False, sharey=False, figsize=None,
layout=None, bins=10, **kwargs)
```

column – nume de coloană sau șir de nume de coloană la care se limitează trasarea. Dacă lipsește, va fi trasat câte un grafic pentru fiecare variabilă.

by – nume de coloană după care se poate face grupare și trasare pe grupe. Poate fi și un masiv unidimensional după care să se poată face împărțire pe grupe a instanțelor. Va fi trasat câte un grafic pentru fiecare grupă. Dacă sunt mai multe coloane, trasarea acestora se va face în același grafic pe grupe.

grid – indică trasarea hașurilor.

xlabelsize – mărimea textului pentru etichete pe axa x. Poate fi un int.

xrot – unghiul de rotație al etichetelor pe axa x. Implicit: 0.

ylabelsize, yrot – idem pentru axa y.

ax – obiect *matplotlib.axes.Axes* utilizat pentru partajarea unor elemente (de exemplu axele graficului).

sharex, sharey – indică partajarea axelor.

figsize – stabilește dimensiunea figurii (tuplu).

layout – stabilește dimensiunea graficului histogramă (tuplu).

bins – numărul de histograme. Poate fi furnizat un număr întreg sau o secvență de valori reprezentând limitele de intervale.

kwargs – alți parametrii de configurare din *matplotlib.pyplot.hist* (de exemplu *rwidth* pentru a stabili o distanță între bare)

O cale simplificată de a trasa o histogramă pentru un *DataFrame* este metoda:

```
DataFrame.plot.hist(by=None, bins=10, **kwargs)
```

Aceasta va trasa în același grafic histogramele pentru toate variabilele tabelului. *kwargs* se referă la parametrii adiționali din *pandas.DataFrame.plot()*.

5.2.7 Reprezentări grafice utilizate frecvent în analiza datelor

Pentru realizarea graficelor pe care le propunem în această secțiune avem nevoie de seturi de date specifice: liste, masive multidimensionale sau tablouri *pandas*. Programul prezentat în continuare va genera valori aleatorii pentru popularea unor astfel de structuri de date.

```
'''
mainVisual.py
program pentru exemplificare de functii grafice
pentru analiza datelor
'''

import numpy as np
import pandas as pd

# Funcție adhoc pentru generarea de valori aleatoare
# într-un interval dat [a, b], plecând de la valorile
# generate de funcția np.random.rand în intervalul [0, 1]
def random(a=None, b=None, size=None):
    return a + np.random.rand(size) * (b - a)

# crearea unei liste de 20 de valori
# în intervalul [0.1, 7.1]
list_1 = random(0.1, 7.1, 20)
print(list_1)

# crearea unui numpy.ndarray din valorile listei list_1
nda_1 = np.ndarray(shape=(5, 3), dtype=float, buffer=list_1, order='C')
print(nda_1)

# crearea unui pandas.DataFrame din ndarray nda_1
df_1 = pd.DataFrame(nda_1, index=('C' + str(i + 1) for i in range(nda_1.shape[0])),
                    columns=('R' + str(j + 1) for j in range(nda_1.shape[1])))
print(df_1)

# sortarea descrescătoare a valorilor listei list_1
# pentru emularea valorilor proprii
k_desc = [k for k in reversed(np.argsort(list_1))]
list_1_sort = list_1[k_desc]
print(list_1_sort)
```

În urma rulării, un posibil rezultat ar fi cel de mai jos, ținând cont că generăm prin metoda *random* valori aleatorii în intervalul specificat.

```
[2.63064152 0.20602368 0.67211051 0.43917887 2.08366133 1.13324353
2.48842874 2.42592924 0.7359922 2.04797663 0.43107271 0.56737083
1.0101996 0.14270366 1.26996493 2.67947271 2.53075055 1.96593191
2.68611579 1.60380386]
[[2.63064152 0.20602368 0.67211051]
[0.43917887 2.08366133 1.13324353]
[2.48842874 2.42592924 0.7359922 ]
[2.04797663 0.43107271 0.56737083]
[1.0101996 0.14270366 1.26996493]]
```

```

      R1      R2      R3
C1  2.630642  0.206024  0.672111
C2  0.439179  2.083661  1.133244
C3  2.488429  2.425929  0.735992
C4  2.047977  0.431073  0.567371
C5  1.010200  0.142704  1.269965
[2.68611579 2.67947271 2.63064152 2.53075055 2.48842874 2.42592924
 2.08366133 2.04797663 1.96593191 1.60380386 1.26996493 1.13324353
 1.0101996 0.7359922 0.67211051 0.56737083 0.43917887 0.43107271
 0.20602368 0.14270366]

```

5.2.7.1 Cercul corelațiilor

Cercul corelațiilor reprezintă o perspectivă bidimensională a legăturii dintre două variabile sau distribuției observațiilor (indivizilor) câmpul corelației dintre două variabile. Graficul presupune realizarea unui cerc de rază 1, pentru reprezentarea unor coeficienți de corelație cu valori în intervalul $[-1, 1]$.

Funcția ce realizează construirea cercului utilizează funcția `matplotlib.pyplot` pentru construirea unui cerc din puncte, ale căror coordonate urmează ecuația unui cerc de rază 1, cu valori pentru $x = \cos(t)$ și $y = \sin(t)$, t luând valori în intervalul $[0, 2\pi]$.

```

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def cercCorel(R, k1, k2, xLabel=None, yLabel=None,
              title="Cercul corelatiilor", valMin=-1, valMax=1):
    plt.figure(title, figsize=(8, 8))
    plt.title(title, fontsize=18, color='k', verticalalignment='bottom')
    if xLabel == None or yLabel == None:
        if isinstance(R, pd.DataFrame):
            plt.xlabel(xlabel=R.columns[k1], fontsize=16,
                      color='k', verticalalignment='top')
            plt.ylabel(ylabel=R.columns[k2], fontsize=16,
                      color='k', verticalalignment='bottom')
        if isinstance(R, np.ndarray):
            plt.xlabel(xlabel='Componenta ' + str(k1 + 1), fontsize=16,
                      color='k', verticalalignment='top')
            plt.ylabel(ylabel='Componenta ' + str(k2 + 1), fontsize=16,
                      color='k', verticalalignment='bottom')
    else:
        plt.xlabel(xlabel=xLabel, fontsize=16, color='k', verticalalignment='top')
        plt.ylabel(ylabel=yLabel, fontsize=16, color='k', verticalalignment='bottom')
    T = [t for t in np.arange(0, np.math.pi * 2, 0.01)]
    X = [np.cos(t) for t in T]
    Y = [np.sin(t) for t in T]
    plt.plot(X, Y)
    plt.axhline(0, color='g')
    plt.axvline(0, color='g')

```

```

if isinstance(R, pd.DataFrame):
    plt.scatter(R.iloc[:, k1], R.iloc[:, k2], c='r', vmin=valMin, vmax=valMax)
    for i in range(len(R)):
        plt.text(R.iloc[i, k1], R.iloc[i, k2], R.index[i])
if isinstance(R, np.ndarray):
    plt.scatter(R[:, k1], R[:, k2], c='r', vmin=valMin, vmax=valMax)
    for i in range(len(R)):
        # pozitia textului in grafic (x, y) si textul
        plt.text(R[i, k1], R[i, k2], '(' +
                str(np.round(R[i, k1], 1)) +
                ', ' +
                str(np.round(R[i, k2], 1)) +
                ')')

return None

```

Este de remarcat faptul că parametrul `R` trebuie să aibă valorile unei matrice de corelație și este prelucrat atât ca `pandas.DataFrame`, cât și ca `numpy.ndarray`.

Adăugând următoare secvență de cod la programul `mainVisual.py` de mai sus:

```

import visual as vi

xCor = np.corrcoef(nda_1, rowvar=True)
xCorTab = pd.DataFrame(xCor, index=('C' + str(i + 1) for i in range(xCor.shape[0])),
                      columns=('R' + str(j + 1) for j in range(xCor.shape[1])))
vi.cercCorel(xCorTab, 0, 1, title='Cercul corelatiilor din pandas.DataFrame')
vi.cercCorel(xCor, 0, 1, title='Cercul corelatiilor din numpy.ndarray')
vi.show()

```

Funcția `cercCorel` este importată din fișierul `visual.py`. Rezultatul execuției codului anterior este reprezentat de graficele din figurile 5.4 și 5.5.

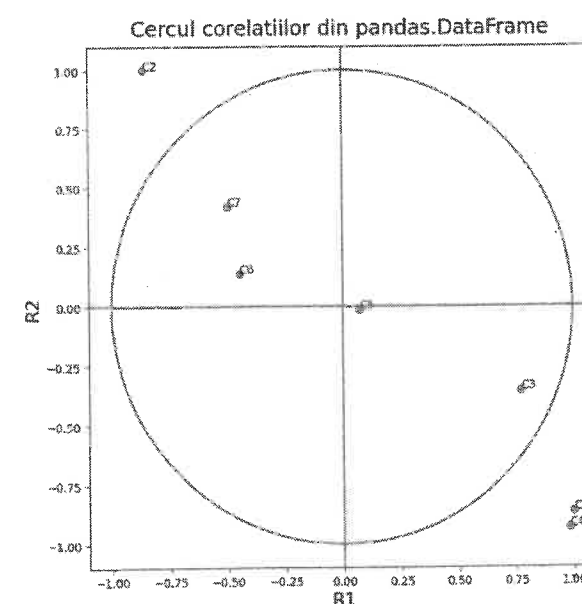


Figura 5.4 Cercul corelațiilor - pandas.DataFrame

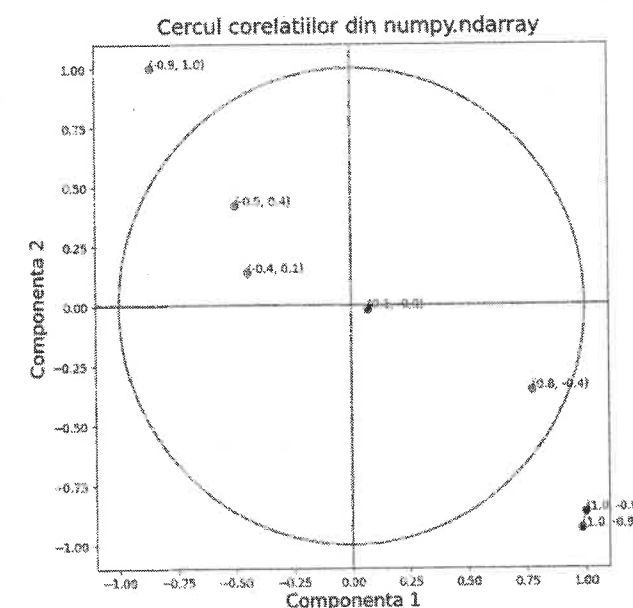


Figura 5.5 Cercul corelațiilor - numpy.ndarray

Putem să extindem puterea de vizualizare utilizând cercul corelațiilor prin reprezentarea cercurilor concentrice asociate cuartilelor de raze: 0.25, 0.50 și respectiv 0.75.

```
# construirea cercului corelatiilor
# avand incluse cercuri concentrice pentru quartile
def cercCorelExt(R, k1, k2, con=False, xLabel=None, yLabel=None,
                 title='Cercul extins al corelatiilor', valMin=-1, valMax=1):
    plt.figure(title, figsize=(8, 8))
    plt.title(title, fontsize=18, color='k', verticalalignment='bottom')
    if xLabel == None or yLabel == None:
        if isinstance(R, pd.DataFrame):
            plt.xlabel(xlabel=R.columns[k1], fontsize=16,
                      color='k', verticalalignment='top')
            plt.ylabel(ylabel=R.columns[k2], fontsize=16,
                      color='k', verticalalignment='bottom')
        if isinstance(R, np.ndarray):
            plt.xlabel(xlabel='Componenta ' + str(k1 + 1), fontsize=16,
                      color='k', verticalalignment='top')
            plt.ylabel(ylabel='Componenta ' + str(k2 + 1), fontsize=16,
                      color='k', verticalalignment='bottom')
    else:
        plt.xlabel(xlabel=xLabel, fontsize=16, color='k', verticalalignment='top')
        plt.ylabel(ylabel=yLabel, fontsize=16, color='k', verticalalignment='bottom')

    # crearea cercului corelatiilor, de raza 1,
    # cu centrul in originea axelor de coordonate
    # avem nevoie de o lista de valori pentru unghi care sa descrie un cerc
    # np.arange(i, f, pas) si furnizeaza valori reale
    T = [t for t in np.arange(0, np.pi * 2, 0.01)]
    X = [np.cos(t) * 1 for t in T] # f(t) = cos(t)
    Y = [np.sin(t) * 1 for t in T] # f(t) = sin(t)
    plt.plot(X, Y)

    if con:
        for k in range(1, 3 + 1):
            X = [np.cos(t) * 0.25 * k for t in T]
            Y = [np.sin(t) * 0.25 * k for t in T]
            plt.plot(X, Y)

    plt.axhline(0, color='g')
    plt.axvline(0, color='g')
    if isinstance(R, pd.DataFrame):
        plt.scatter(R.iloc[:, k1], R.iloc[:, k2], c='r', vmin=valMin, vmax=valMax)
        for i in range(len(R)):
            plt.text(R.iloc[i, k1], R.iloc[i, k2], R.index[i])
    if isinstance(R, np.ndarray):
        plt.scatter(R[:, k1], R[:, k2], c='r', vmin=valMin, vmax=valMax)
        for i in range(len(R)):
            # pozitia textului in grafic (x, y) si continutul textul
            plt.text(R[i, k1], R[i, k2], '(' +
                    str(np.round(R[i, k1], 1)) +
```

```
', ' +
str(np.round(R[i, k2], 1)) +
')')
return None
```

Apelul în programul mainVisual.py este realizat prin secvența de cod următoare.

```
import visual as vi
```

```
xCor = np.corrcoef(nda_1, rowvar=True)
xCorTab = pd.DataFrame(xCor, index=('C' + str(i + 1) for i in range(xCor.shape[0])),
                      columns=('R' + str(j + 1) for j in range(xCor.shape[1])))
vi.cercCorelExt(xCorTab, 0, 1, con=True, title='Cercul extins al corelatiilor din
pandas.DataFrame')
vi.show()
```

Rezultatul execuției codului anterior este reprezentat de figurile 5.6 și 5.7. A se remarca faptul că, pentru trasarea cercurilor concentrice, parametrul *con* al metodei *cercCorelExt* a fost pus pe *True*.

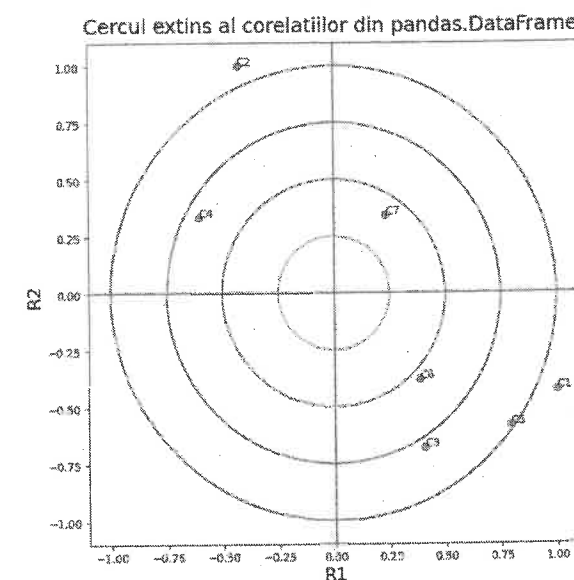


Figura 5.6 Cercul extins al corelațiilor - DataFrame

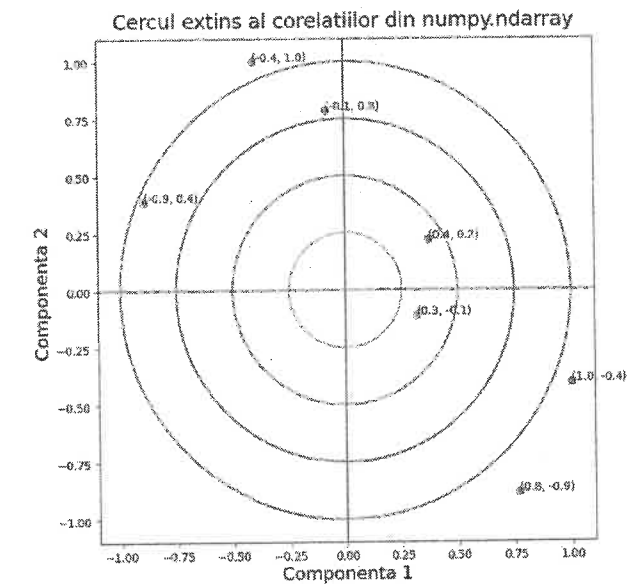


Figura 5.7 Cercul extins al corelațiilor - ndarray

5.2.7.2 Graficul varianței explicate de componentele principale

Graficul linie cu puncte pentru vizualizarea varianței explicate de componentele principale, respectiv a valorilor proprii determinate pentru matricea de varianță-covarianță, este realizat utilizând metoda `matplotlib.pyplot` pentru un masiv de valori proprii sortate descrescător. Funcția `valProp` se găsește, de asemenea, în fișierul `visual.py`.

```
def valProp(alpha):
    plt.figure("Valorile proprii - varianta componentelor principale",
              figsize=(11, 8))
    plt.title("Valorile proprii - varianta componentelor principale",
             fontsize=18, color='k', verticalalignment='bottom')
```



```
plt.xlabel("Componentele principale",
          fontsize=16, color='k', verticalalignment='top')
plt.ylabel("Valorile proprii",
          fontsize=16, color='k', verticalalignment='bottom')
Xindex = ['C' + str(k + 1) for k in range(len(alpha))]
plt.plot(Xindex, alpha, 'bo-')
plt.xticks(Xindex)
plt.axhline(1, color='r')
return None
```

Trebuie remarcată trasarea unei linii orizontale în dreptul valorii 1 a varianței pentru facilitarea aplicării criteriului lui Kaiser cu privire la numărul de componente principale ce merită a fi reținute din modelul.

Pentru apelul funcției `valProp`, în programul apelator `mainVisual.py` vom adăuga următoarea secvență de cod:

```
# sortarea descrescătoare a valorilor listei list1
# pentru emularea valorilor proprii
k_desc = [k for k in reversed(np.argsort(list_1))]
list_1_sort = list_1[k_desc]
print(list_1_sort)
vi.valProp(list_1_sort)
vi.show()
```

Rezultatul execuției este reprezentat de graficul din figura 5.8:

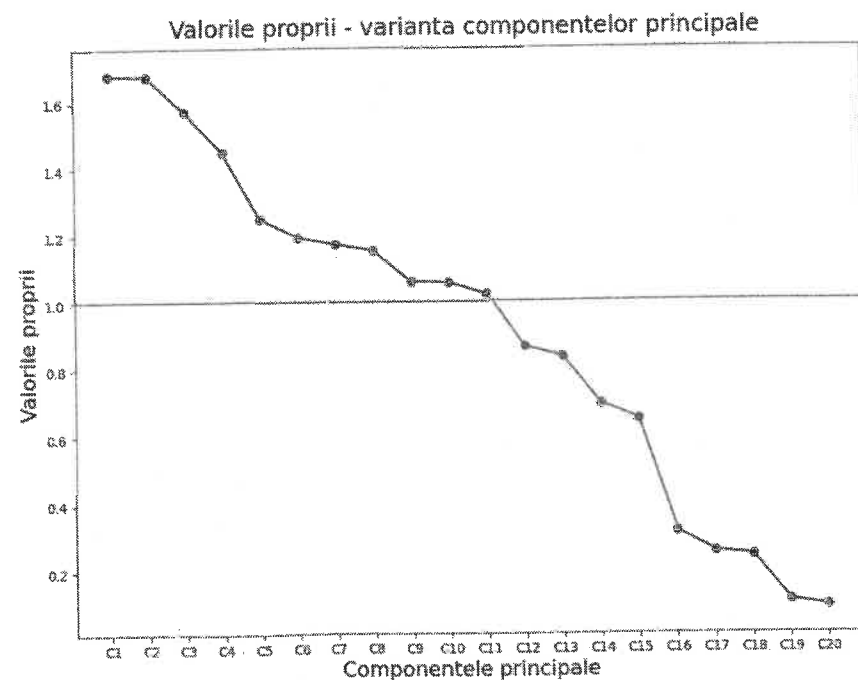


Figura 5.8 Graficul linie al varianței componentelor principale

5.2.7.3 Graficul corelogramă

Plecând de la matricea de corelație, calculată pentru matricea inițială construită cu valori generate aleatoriu, putem construi graficul corelogramă utilizând funcția `seaborn.heatmap`.

```
import seaborn as sb
```

```
# contruirea corelogramei pe baza matricei de corelație
def corelograma(R2, dec=2, title='Corelograma', valmin=-1, valmax=1):
    plt.figure(title, figsize=(15, 11))
    plt.title(title, fontsize=18, color='k', verticalalignment='bottom')
    sb.heatmap(data=np.round(R2, dec), vmin=valmin, vmax=valmax, cmap='bwr', annot=True)
```

Pentru apelul funcției în `mainVisual.py` vom pasa ca parametru un `pandas.DataFrame` pentru a avea posibilitatea să etichetăm rândurile și coloanele cu valorile din `DataFrame.index` respectiv `DataFrame.columns`.

```
import numpy as np
import pandas as pd
import visual as vi
```

```
xCor = np.corrcoef(nda_1, rowvar=True)
xCorTab = pd.DataFrame(xCor, index=('C' + str(i + 1) for i in range(xCor.shape[0])),
                      columns=('R' + str(j + 1) for j in range(xCor.shape[1])))
vi.corelograma(xCorTab)
vi.show()
```

Rezultatul rulării codul anterior este reprezentat de graficul matricei de corelație, figura 5.9, având valorile coeficienților de corelație adnotați pe grafic.

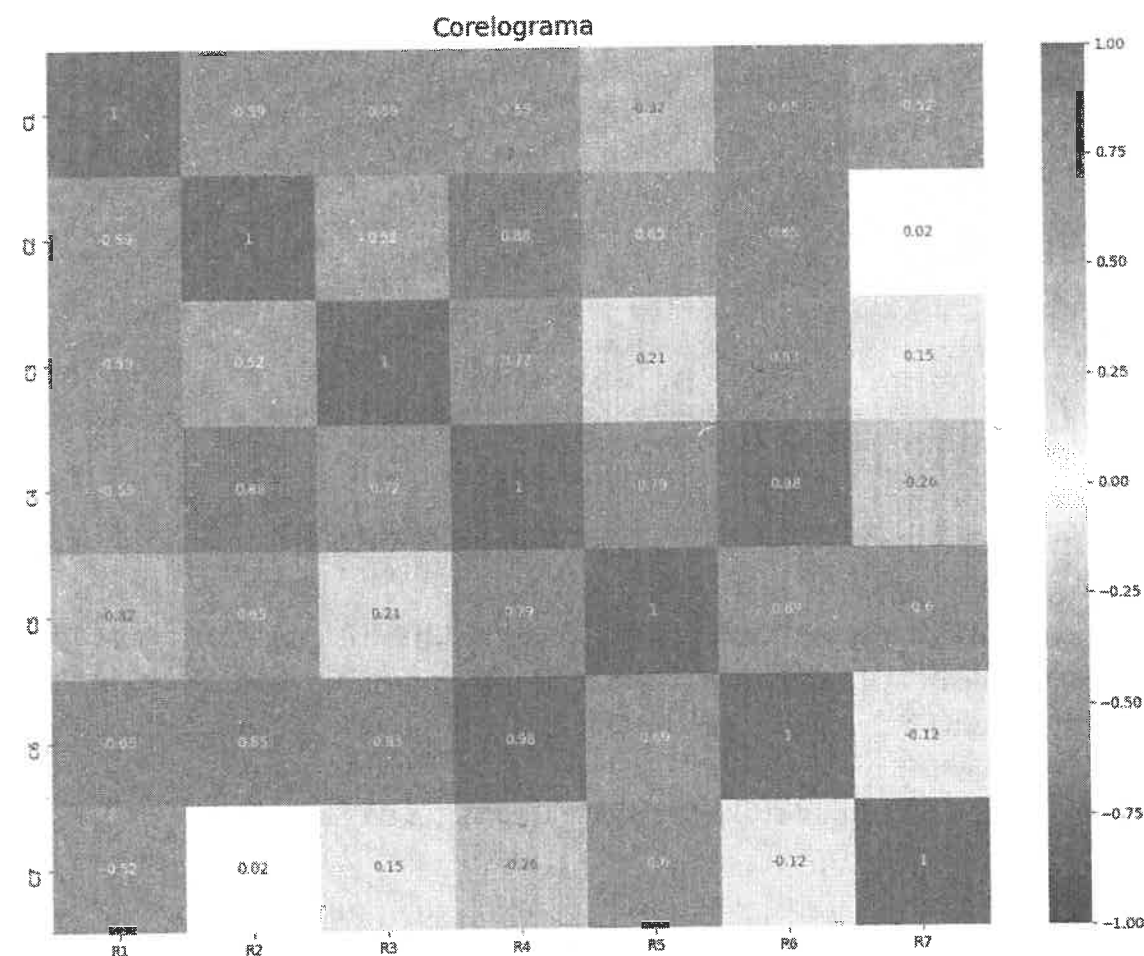


Figura 5.9 Graficul corelogramă, corespunzător matricei de corelație

5.2.7.4 Graficul histogramă cu bare multiple

Graficul histogramă cu bare multiple poate fi utilizat în diverse scopuri, atunci când dorim să vizualizăm valorile unor mănunchiuri de observații în câmpul mai multor variabile. Graficul este utilizarea funcției `matplotlib.pyplot.bar` și implementarea unui algoritm original de calcul a lățimii barelor și distanței dintre mănunchiuri. Codul aferent acestei funcții este prezentat în continuare:

```
import matplotlib.pyplot as plt
import seaborn as sb
import numpy as np
import pandas as pd

def multiHist(X, width=0.8, title='Multi-histograma', xLabel='Observatii',
              yLabel='Valori variabile'):
    plt.figure(title, figsize=(15, 11))
    plt.title(title, fontsize=18, color='k', verticalalignment='bottom')
    plt.xlabel(xlabel=xLabel, fontsize=16, color='k', verticalalignment='top')
    plt.ylabel(ylabel=yLabel, fontsize=16, color='k', verticalalignment='bottom')

    if isinstance(X, pd.DataFrame):
        nrCol = X.values.shape[1]
        nrLin = X.values.shape[0]
        width /= nrCol

        ind = np.arange(nrLin)
        for j in range(nrCol):
            plt.bar(x=ind + (width * j), height=X.iloc[:, j].values,
                    width=width, label=X.columns[j], align='center')
            plt.xticks(ind + width * (nrCol - 1) / 2.0, X.index[:])
    if isinstance(X, np.ndarray):
        nrCol = X.shape[1]
        nrLin = X.shape[0]
        width /= nrCol

        ind = np.arange(nrLin)
        for j in range(nrCol):
            plt.bar(x=ind + (width * j), height=X[:, j],
                    width=width, label='Var' + str(j + 1), align='center')
            plt.xticks(ind + width * (nrCol - 1) / 2.0,
                      ('Set' + str(i + 1) for i in range(nrLin)))

    plt.legend(loc='best')
    return None
```

A se remarca faptul că implementarea este astfel concepută, pentru a lucra cu o matrice de date furnizată ca parametru atât ca `pandas.DataFrame`, cât și ca `numpy.ndarray`. Utilizăm în acest sens introspecția în Python, prin apelul funcției `isinstance`, pentru determinarea clasei obiectului ce urmează a fi prelucrat.

În programul apelator `mainVisual.py` adaugăm următoarea secvență de cod:

```
import numpy as np
import pandas as pd
import visual as vi
```

```
xCor = np.corrcoef(nda_1, rowvar=True)
xCorTab = pd.DataFrame(xCor, index=('C' + str(i + 1) for i in range(xCor.shape[0])),
                       columns=('R' + str(j + 1) for j in range(xCor.shape[1])))
vi.multiHistDF(xCorTab)
vi.show()
```

Rezultatul rulării este reprezentat de următorul grafic histogramă cu bare multiple prezentat în figura 5.10:

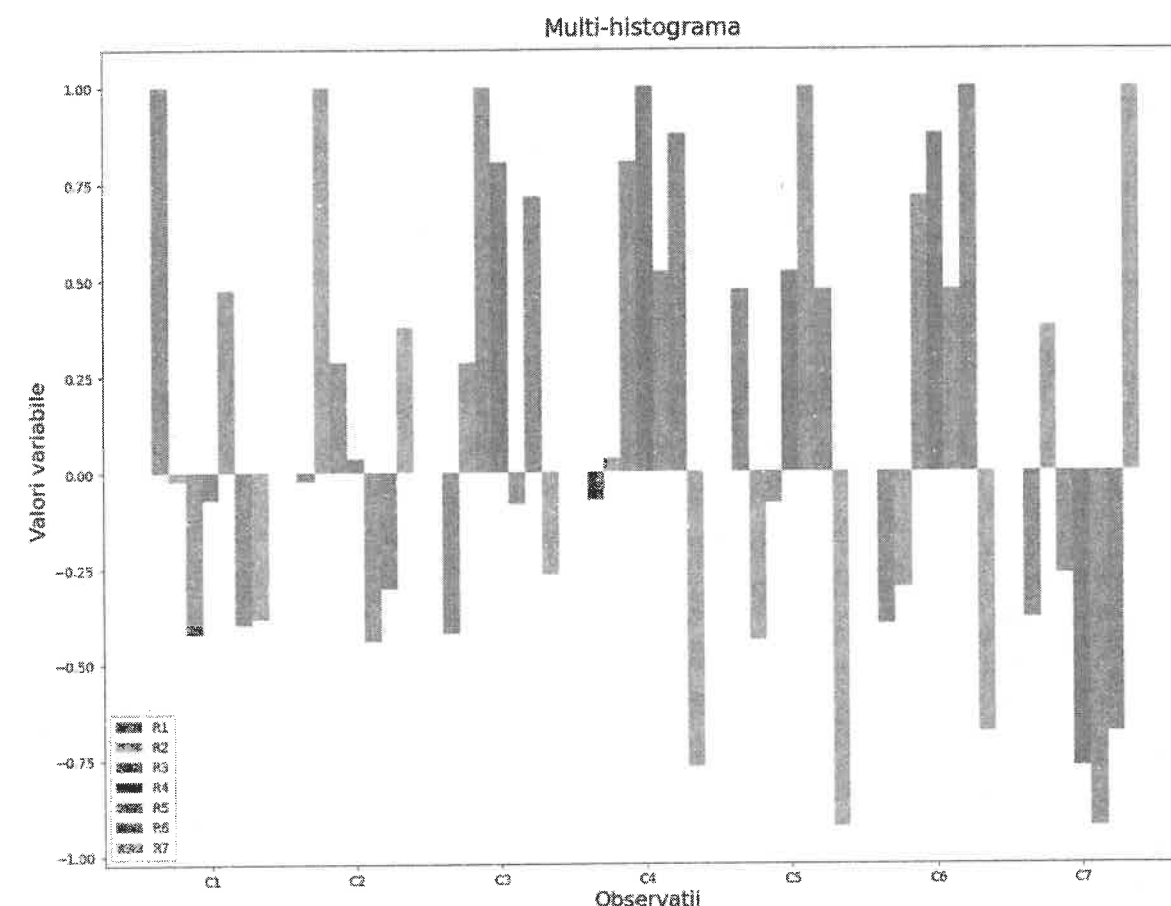


Figura 5.10 Graficul histogramă cu bare multiple, reprezentând valori ale unor mănunchiuri de variabile, pentru un număr dat de observații

6 Implementarea unor metode de analiză a datelor în Python

6.1 Analiza în componente principale (ACP)

Analiza în componente principale (ACP) este cea mai utilizată metodă de analiză a datelor. A fost propusă de Hotteling în 1938, dar necesitând numeroase și laborioase calcule s-a impus în practică abia începând cu anii '70, odată cu apariția calculatoarelor. ACP sintetizează informația conținută în tabelele de date cantitative de mari dimensiuni. Colectivitatea statistică studiată este descrisă, foarte adesea, printr-un număr mare de variabile. Studiul colectivității printr-un număr mare de variabile este dificil de realizat. ACP permite identificarea unui număr mai mic de variabile noi, numite componente principale, care concentrează informația, esențialul, la nivelul colectivității studiate. Componentele principale sunt construite sub formă de combinație liniară de variabile inițiale, care concentrează o cât mai mare parte de informație și sunt caracterizate de o mare variabilitate. Astfel, prima componentă principală preia maximum din varianța variabilelor originale, a doua componentă preia maximum de varianță rămasă după eliminarea primei componente ș.a.m.d.

Ca model, ACP se utilizează nu numai în analiza factorială, ci și în recunoașterea formelor, în scalarea multidimensională din grafică etc. În recunoașterea de forme prin ACP se asigură selectarea caracteristicilor esențiale, semnificative ale formelor analizate, caracteristici care asigură puterea cea mai mare de discriminare. În grafică se asigură reprezentarea în 2D sau 3D a unor obiecte aflate în spații multidimensionale.

6.1.1 Datele de prelucrat și etapele analizei

Analiza componentelor principale (ACP) este o procedură statistică, care utilizează o transformare ortogonală pentru a converti un set de observații ale variabilelor posibil corelate într-un set de valori pentru variabile liniar necorelate numite componente principale. Datele analizate constau într-un tabel de observații, având n rânduri și m coloane.

$$X = \begin{bmatrix} x_{11} & \dots & x_{1m} \\ \dots & & \\ x_{n1} & \dots & x_{nm} \end{bmatrix}$$

unde x_{ij} este valoarea luată de variabila j pentru observația i .

Variabilele descrise în tabloul X sunt cunoscute și sub numele de variabile inițiale, observate sau variabile cauzale. X_j este vectorul coloană care conține valorile variabilei j pentru cele n observații.

Scopul metodei este sintetizarea informației conținute de tabelul X printr-un număr mai mic de variabile noi, necorelate, notate: $C_1, C_2, \dots, C_s$.

Variabila sintetică C_1 , prima componentă principală, este o combinație liniară de variabile X_j și anume:

$$C_1 = a_{11}X_1 + \dots + a_{j1}X_j + \dots + a_{m1}X_m$$

Valoarea luată de componenta principală C_1 pentru o observație oarecare i este:

$$c_{i1} = a_{11}x_{i1} + \dots + a_{j1}x_{ij} + \dots + a_{m1}x_{im}$$

Vom nota cu a_1 vectorul care conține multiplicatorii $a_{j1}, j = \overline{1, m}$.

Cu aceste notații, componenta principală C_k este o combinație liniară de variabile X :

$$C_k = a_{1k}X_1 + \dots + a_{jk}X_j + \dots + a_{mk}X_m$$

unde a_k este vectorul care conține multiplicatorii $a_{jk}, j = \overline{1, m}$.

În consecință, legătura dintre variabilele inițiale, cauzale (X) și componentele principale (C) este dată de ecuația:

$$C_k = X \cdot a_k, k = \overline{1, s}$$

unde s este numărul componentelor principale.

Determinarea componentelor principale implică determinarea matricei multiplicatorilor $a_k, k = \overline{1, s}$. Analiza în componente principale poate fi abordată atât din perspectiva observațiilor cât și din perspectiva variabilelor analizate. Atunci când efectuăm analiza la nivelul observațiilor, spunem că efectuăm o analiză în spațiul observațiilor, iar când analiza se efectuează la nivelul variabilelor, vorbim de analiza în spațiul variabilelor. Cele două direcții de analiză conduc la aceleași rezultate.

Analiza în componente principale realizată în spațiul variabilelor își propune identificarea directă a componentelor principale, astfel încât acestea să fie maxim corelate cu variabilele inițiale și absolut necorelate între ele. Soluția o constituie vectorul propriu al matricei $\frac{1}{n}XX^t$ corespunzător valorii proprii β_k :

$$\frac{1}{n}XX^t \cdot C_k = \beta_k \cdot C_k.$$

În spațiul observațiilor, la etapa k se determină vectorul propriu a_k , care este vectorul unitar al axei k corespunzătoare componentei C_k :

$$\frac{1}{n}X^tX \cdot a_k = \alpha_k \cdot a_k.$$

Înmulțind această relație la stânga cu X obținem:

$$\frac{1}{n}XX^t \cdot X \cdot a_k = X \cdot \alpha_k \cdot a_k \Rightarrow \frac{1}{n}XX^t \cdot C_k = \alpha_k \cdot C_k.$$

Relația obținută $\frac{1}{n}XX^t \cdot C_k = \alpha_k \cdot C_k$, este aceeași cu cea obținută la abordarea problemei în spațiul observațiilor, considerând $\beta_k = \alpha_k$.

Este de remarcat faptul că matricea $\frac{1}{n}X^tX$ reprezintă matricea de varianță-covarianță a matricei X , matrice simetrică față de diagonala principală. Pe diagonala principală a acestei matrice de varianță-covarianță avem covarianța fiecărei variabile cauzale cu ea însăși, respectiv valori egale cu 1.

Numărul maxim de etape în spațiul observațiilor poate fi m (gradul matricei $\frac{1}{n}X^tX$), iar în spațiul variabilelor numărul maxim de etape poate fi n (gradul matricei $\frac{1}{n}XX^t$). Numărul valorilor proprii nenule este $\min(m, n)$.

6.1.2 ACP – studiu de caz

Vom exemplifica în continuarea Analiza în Componente Principale prin prezentarea unui studiu de caz acompaniat de implementarea în limbajul de programare Python a rezolvării problemei și susținerea cu rezultate cantitative a concluziilor.

6.1.2.1 Setul de date

Setul de date pentru acest studiu de caz conține indicatori statistici la nivel județ, ca unitate administrativ-teritorială, iar observațiile sunt reprezentate de valori ale acestor indicatori pentru toate județele României. Datele sunt preluate de la Institutul Național de Statistică și reprezintă valori la finalul anului 2016. Semnificația variabilelor inițiale, sau cauzale, este prezentată în tabelul 6.1. Valorile acestor variabile, pentru fiecare județ al României, sunt furnizate în tabelul 6.2.

Tabelul 6.1 Descrierea variabilelor setului de date

Nr. crt.	Etichetă variabilă	Descriere variabilă
1	RS	Rata șomajului
2	SalN	Salariul nominal brut
3	SupL	Suprafața locuibilă pe locuitor (mp/loc)
4	Apa	Consumul de apă pe locuitor (mc/loc)
5	Pat	Număr de paturi în spitale la 1000 locuitori
6	RataAb	Rata abandonului în învățământul primar și gimnazial
7	AbsUniv	Număr de absolvenți ai învățământului universitar la 100000 locuitori
8	Vol	Număr volume în biblioteci publice la 100000 locuitori
9	Cinema	Număr locuri în cinematografe la 100000 locuitori
10	ProdVeg	Producția vegetală pe locuitor
11	Internet	Traficul de internet pe locuitor
12	RataC	Rata criminalității

Avem un model cu 12 variabile cauzale și 42 de observații, respectiv un tablou bidimensional cu 12 coloane și 42 de rânduri. Ca etichetă pentru observații vom folosi codul județului.

Tabelul 6.2 Setul de date cu privire la dezvoltarea teritorială a României

Județ	RS	SalN	SupL	Apa	Pat	RataAb	AbsUniv	Vol	Cinema	ProdVeg	Internet	RataC
ab	3	1351	15.49	38.02	720	1.384076	944	305	133	3084	20	149
ag	2.7	1442	15.21	38.3	497	2.174435	122	291	186	2093	6	160
ar	2.9	1772	15.7	75.59	1020	1.527908	2626	259	588	1558	5	214
b	3.7	1359	14.33	38.72	643	1.906066	433	271	139	1960	5	224
bc	3	1375	16.05	29.9	514	1.862346	78	310	0	2433	6	226
bh	5.5	1421	16.1	33.04	626	1.391941	21	350	0	3010	8	150
bn	7.1	1506	15.04	76.24	674	1.44152	536	357	92	2531	7	184
br	4.3	1639	15.25	67.59	602	2.733474	2084	184	71	1273	9	142
bt	7.2	1290	15.04	39.51	903	2.16051	208	338	0	3246	13	132
bv	6.5	1322	15.88	39.24	680	1.565174	161	356	0	2145	5	232
bz	4.7	1508	15.07	35.31	731	1.823782	569	343	136	2570	9	185
cj	3.1	1659	16.51	93.76	674	3.533356	1973	303	56	1484	10	178
cl	5.3	1685	13.62	29.48	473	2.849777	359	216	154	1475	6	212
cs	3.6	1390	12.64	17.25	578	2.149424	26	338	111	3057	3	206
ct	5.4	1680	12.26	52.73	816	1.871535	2138	188	433	2034	2	141
cv	4.1	1394	13.89	23.75	478	1.923998	39	228	116	1717	7	236
db	4.3	1452	14.39	23.56	470	1.36764	526	241	0	2229	3	171
dj	10.2	1398	12.9	19.24	564	2.144647	0	289	0	2338	3	265
gj	4.4	1494	13.85	45.53	578	1.852392	206	185	97	3104	2	232
gl	5.7	1479	14.66	31.47	560	2.078297	22	210	83	2458	3	147
gr	3	1736	14.26	70.77	585	3.159107	1166	203	1404	2624	4	155
hd	6.6	1636	13.8	38.11	542	1.190318	863	232	25	2483	2	252
hr	4.4	1491	15.51	50.32	496	2.863246	0	358	168	3409	3	192
if	4.4	1413	15.6	32.27	404	2.928396	39	198	0	2703	3	243
il	4.9	1713	14.87	44.88	604	1.026043	933	224	113	1752	3	116
is	5.1	1408	13.01	26.58	411	3.695645	0	267	0	4262	5	186
mh	5.6	1573	14.65	24.12	412	1.438573	466	218	47	3000	3	146
mm	4.5	1557	14.97	16.52	303	2.398438	0	225	0	3044	3	77
ms	4.9	1444	13.46	34.15	309	2.655276	0	266	502	4470	6	197
nt	3.9	1772	15.33	36.47	542	1.368393	380	196	99	1250	6	136
ot	8.1	1469	14.16	24.12	530	2.2634	76	244	0	2913	5	114
ph	1.3	2126	21.48	23.63	524	1.897165	23	56	0	1231	0	70
sb	1.6	2507	15.85	121.82	1067	2.173256	4726	65	590	24	16	136
sj	8.1	1644	15.22	63.07	632	1.632017	1185	229	125	2560	3	110
sm	7.3	2032	14.51	42.02	599	0.908157	625	249	111	1646	3	158
sv	9.3	1695	16.04	38.58	557	1.856282	478	327	0	2465	5	110
tl	5.3	1608	14.41	29.64	473	1.917748	41	249	0	2809	3	151
tm	4.7	1549	14.23	72.06	557	1.592584	320	353	299	1818	6	125
tr	3.1	1521	17.59	42.48	545	2.4984	1597	271	217	3557	14	290
vl	6	1397	16.21	39.29	639	3.489907	457	311	181	2836	5	133
vn	6.7	1546	15.14	45.59	749	1.418248	286	239	189	1319	8	259
vs	1.6	1767	17.42	57.41	840	2.11628	1846	227	278	2533	12	140

6.1.2.2 Abordarea ACP în Python

Ca abordare pentru tratarea ACP în Python vom alege să definim și să implementăm o clasă care să încapsuleze modelul analizei în componente principale, clasă ce poate fi ulterior instanțiată pentru orice set concret de date. Secvența următoare de cod poate fi inclusă într-un fișier distinct (de exemplu ACP.py) și reprezintă definirea atributelor modelului ACP și implementarea algoritmului de procesare pentru calculul componentelor principale, a varianței explicate și a factorilor asociați fiecărei componente.

```
'''
ACP - clasa ce încapsuleaza un model de prelucrare a datelor
pentru analiza in componente principale (ACP)

X reprezenta matricea datelor de intrare (m variabile pe coloane
si n observatii pe randuri)
matricea X este de asteptat sa fie furnizata ca numpy ndarray
'''

import numpy as np

class ACP:

    # constructorul clasei asuma preluarea
    # unui numpy ndarray ca parametru
    def __init__(self, X):
        self.X = X

    # calculul matricei de corelatie
    # pentru matricea initiala X nestandardizata, plecand
    # de premisa ca variabilele sunt asezate pe coloane
    self.R = np.corrcoef(self.X, rowvar=False)

    '''
    stadardizam matricea initiala X,
    pentru ca ponderile variabilelor cauzale
    sa nu introducă interferenta, in combinatiile liniare
    ce formeaza componentele principale
    '''

    avg = np.mean(self.X, axis=0) # media pe coloane
    std = np.std(self.X, axis=0) # abaterea standard pe coloane
    self.Xstd = (self.X - avg) / std

    # calcularea matricei de varianta-coverianta pentru Xstd,
    # plecand de premisa ca variabilele sunt asezate pe coloane
    self.Cov = np.cov(self.Xstd, rowvar=False)

    # calculul valorilor proprii si a vectorilor proprii
    # pentru matricea de varianta-covarianta Cov = 1/n * (X)t * X
    self.eigenVal, self.eigenVect = np.linalg.eig(self.Cov)

    # sortarea in ordine descrescatoare a valorilor si
    # vectorilor proprii
    k_des = [k for k in reversed(np.argsort(self.eigenVal))]
    self.alpha = self.eigenVal[k_des]
    self.a = self.eigenVect[:, k_des]
```

```
'''
regularizare vectori proprii - valorile vectorilor
proprii sunt relevante din perspectiva variantei explicate;
daca intr-un vector propriu exista valori negative mai mari,
in valoare absoluta, decat cele pozitive, atunci este de
mai mare utilitate in interpretarea rezultatelor
inversarea semnului valorilor vectorului propriu in cauza;
un vector propriu isi mentine calitatea de vector propriu
si dupa inmultirea cu un scalar, respectiv -1

'''

for j in range(len(self.alpha)):
    minim = np.min(self.a[:, j])
    maxim = np.max(self.a[:, j])
    if np.abs(minim) > np.abs(maxim):
        self.a[:, j] = -self.a[:, j]

'''
calculul componentelor pricipale
pentru matricea standardizata X;
in pachetul numpy operatorul @ este supraincarcat pentru
inmultire matriceala, echivalent cu instructiunea urmatoare:
self.C = np.matmul(self.Xstd, self.a)
'''

self.C = self.Xstd @ self.a

# calculul scorurilor, componentele pricipale standardizate
self.Cstd = self.C / np.sqrt(self.alpha)

# calculul factorilor de corelatie
self.Rxc = self.a * np.sqrt(self.alpha)

# self.C2 = np.square(self.C)
C2 = self.C * self.C

# calitatea reprezentarii observatiilor pe axe
# componentelor principale
C2Sum = np.sum(C2, axis=1)
self.CalObs = np.transpose(np.transpose(C2) / C2Sum)

# contributia observatiilor (indivizilor)
# la varianta axelor
self.betha = C2 / (self.alpha * self.X.shape[0])

# comunalitatile componentelor principale
# regasite in variabilele initiale sau cauzale
R2 = self.Rxc * self.Rxc
self.Comun = np.cumsum(R2, axis=1)

# returneaza matricea de corelatie R,
# corespunzatoare matricei initiale X
def getCor(self):
    return self.R

# returneaza matricea initiala X standardizata
def getXstd(self):
    return self.Xstd

# returneaza matricea de varianta-covariant Cov
```

```

# corespunzatore matricei standardizate Xstd
def getCov(self):
    return self.Cov

# returneaza valorile proprii ale matricei de corelatie R
def getValProp(self):
    return self.alpha

# returneaza vectorii proprii ai matricei de corelatie R
def getVectProp(self):
    return self.a

# returneaza matricea factorii de corelatie
def getCorFact(self):
    return self.Rxc

# returneaza componentele principale
def getComp(self):
    return self.C

# returneaza scorurile, componentele pricipale standardizate
def getScor(self):
    return self.Cstd

# returneaza calitatea reprezentarii observatiilor pe axe
def getCalObs(self):
    return self.CalObs

# returneaza contributia observatiilor (indivizilor)
# la varianta axelor
def getContribObs(self):
    return self.betha

# comunalitatile componentelor principale
# regasite in variabilele initiale sau cauzale
def getComun(self):
    return self.Comun

```

Setul de date pe care îl vom procesa este stocat într-un fișier de tip CSV (*comma-separated values*). Conținutul acestui fișier este prezentat în tabloul anterior. Observăm că fișierul conține pe prima linie etichetele variabilele, iar pe prima coloană etichetele observațiilor reprezentate de codul fiecărui județ al României.

Vom citi acest fișier și vom prelua conținutul acestuia utilizând facilitățile oferite de pachetul *pandas* în ceea ce privește instanțierea de obiecte de tip *DataFrame* din date în format tabelar stocate în fișiere CSV. Secvența de cod următoare realizează preluarea conținutului fișierului într-un *pandas DataFrame*. De asemenea, vom extrage din obiectul *DataFrame* obținut numele coloanelor (etichetele variabilelor), numele rândurilor (etichetele observațiilor), precum și matricea valorilor variabilelor cauzale în câmpul observațiilor sub forma unui *numpy ndarray* cu două dimensiuni.

```

import visual.visual as vi
import pandas as pd
import acp.ACP as acp

# citirea setului de date de intrare din fisier csv intr-un pandas.DataFrame
tabel = pd.read_csv("dataIN/Teritorial_2.csv", index_col=0)

```

```

# extragerea matricei de date din pandas.DataFrame ca un numpy ndarray
X = tabel.iloc[:, :].values

```

```

numeObs = tabel.index[:]      # echivalent cu tabel.index
numeVar = tabel.columns[:]    # echivalent cu tabel.columns

```

```

n = X.shape[0]                # numarul de observatii
m = X.shape[1]                # numarul de variabile
print(n, m)
# print(X)

```

Următoarea secvență de cod exemplifică utilizarea clasei ACP, definite și implementate anterior, pe setul de date ce conține ca variabile inițiale (cauzale) indicatorii macroeconomici cu valori la nivel de județ.

```

# instantiere obiect de tip ACP

```

```

acp = acp.ACP(X)
R = acp.getCor()
alpha = acp.getValProp()
a = acp.getVectProp()
Rxc = acp.getCorFact()
C = acp.getComp()
Cstd = acp.getScor()
CalObs = acp.getCalObs()
ContribObs = acp.getContribObs()
Comun = acp.getComun()

```

```

# salveaza in fisier CSV matricea X standardizata
Xstd = pd.DataFrame(data=acp.getXstd(),
                    columns=numeVar, index=numeObs)
Xstd.to_csv(path_or_buf="dataOUT/Xstd.csv")

```

```

# salveaza in fisier CSV matricea de corelatie (R)
# a matricei initiale X
Rtab = pd.DataFrame(data=R,
                    columns=numeVar, index=numeVar)
Rtab.to_csv(path_or_buf="dataOUT/R.csv")
# genereaza graficul corelograma
# pentru matricea de corelatie a variabilelor cauzale
vi.corelograma(Rtab, title='Matricea de corelatie a variabilelor cauzale')

```

```

# salveaza in fisier CSV matricea
# factorilor de corelatie
RxcTab = pd.DataFrame(Rxc, index=numeVar,
                    columns=("C"+str(k+1) for k in range(m)))
RxcTab.to_csv(path_or_buf="dataOUT/Rxc.csv")
# genereaza corelograma factorilor de corelatie
vi.corelograma(RxcTab, dec=2, title="Corelograma factorilor")

```

```

# salveaza in fisier CSV matricea scorurilor,
# componentele principale standardizate
CstdTab = pd.DataFrame(Cstd, index=numeObs,
                    columns=("Comp"+str(k+1) for k in range(m)))
CstdTab.to_csv(path_or_buf="dataOUT/Cstd.csv")
# genereaza corelograma scorurilor
vi.corelograma(CstdTab.iloc[:, :], dec=2, title="Matricea scorurilor")
vi.multiHistDF(CstdTab.iloc[:, :5], width=0.8, title='Scorurile - componentele
principale standardizate')

```



```
# salveaza in fisier CSV matricea ce reflecta calitatea obsevatiilor
# pe axele componentelor principale
CalObsTab = pd.DataFrame(CalObs, index=numeObs,
                        columns=("Comp"+str(k+1) for k in range(m)))
CalObsTab.to_csv(path_or_buf="dataOUT/CalObs.csv")
# genereaza corelograma ce reflecta calitatea obsevatiilor
# # pe axele componentelor principale
vi.corelograma(CalObsTab.iloc[:, :], dec=2,
               title="Calitatea observatiilor pe axele componentelor principale")
vi.multiHistDF(CalObsTab.iloc[:, :5], width=0.8, title='Calitatea observatiilor pe axele
componentelor')
vi.obsNorPuncte(CalObsTab.iloc[:, :5], title='Nor de puncte - calitatea observatiilor pe
axe')
vi.varNorPuncte(CalObsTab.iloc[:, :5], title='Nor de puncte - calitatea observatiilor in
campul componentelor')
```

```
# salveaza in fisier CSV matricea contributiei obsevatiilor
# la varianta axele componentelor principale
ContribObsTab = pd.DataFrame(ContribObs, index=numeObs,
                           columns=("Comp"+str(k+1) for k in range(m)))
ContribObsTab.to_csv(path_or_buf="dataOut/ContribObs.csv")
# genereaza corelograma ce reflecta contributiei obsevatiilor
# # la varianta axele componentelor principale
vi.corelograma(ContribObsTab.iloc[:, :], dec=2,
               title="Contributia observatiilor la axele componentelor principale")
vi.varNorPuncte(ContribObsTab.iloc[:, :], title='Nor de puncte - contributia
observatiilor la axele componentelor principale')
```

```
# genereaza cercul corelatiilor
# variabile initiale in campul componentelor 1 si 2
vi.cercCorel(RxcTab, 0, 1)
vi.cercCorelExt(Rxc, 0, 1, con=True)
# vi.norPuncte(Rxc[:, 0], Rxc[:, 1])
vi.obsNorPuncte(Xstd, title='Nor de puncte multivariat in campul observatiilor')
```

```
# salveaza in fisier CSV matricea comunalitatilor
# componentelor principale in variabilele initiale
ComunTab = pd.DataFrame(Comun, index=numeVar,
                       columns=("C"+str(k+1) for k in range(m)))
ComunTab.to_csv(path_or_buf="dataOUT/Comun.csv")
# genereaza corelograma comunalitatilor
vi.corelograma(ComunTab, dec=2, title="Corelograma comunalitatilor")
```

```
# genereaza graficul valorilor proprii
vi.valProp(alpha)
vi.multiHist(a, width=0.8, title='Vectorii proprii')
```

```
CTab = pd.DataFrame(C, index=numeObs, columns=("Comp"+str(k+1) for k in range(m)))
vi.norPuncte(CTab.iloc[:, 0].values, CTab.iloc[:, 1].values, label=numeObs)
vi.obsNorPuncte(CTab.iloc[:, :5], label=numeObs, title='Nor de puncte - observatiilor pe
axele componentelor principale')
```

```
# afisarea graficelor generate in ferestre distincte
vi.show()
```

6.1.2.3 Interpretarea rezultatelor

Scopul principal al ACP este să evidențieze informațiile semnificative referitoare la ansamblul de date. Prin urmare, prima componentă aglutinează cel mai important tip de informație, deoarece conține varianța maximă. A doua componentă principală explică următorul cel mai important tip de informație, respectiv cea mai mare parte a informației rămase în model după extragerea primei componente principale ș.a.m.d.

Graficul corelogramă de mai jos reprezintă valorile matricei de corelație corespunzătoare matricei inițiale X , având variabilele plasate pe coloane. Matricea de corelație este o matrice pătratică simetrică față de diagonala principală și prezintă corelația dintre fiecare două variabile ale modelului studiat (figura 6.1). Rangul matricei este dat de numărul variabilelor inițiale sau cauzale.

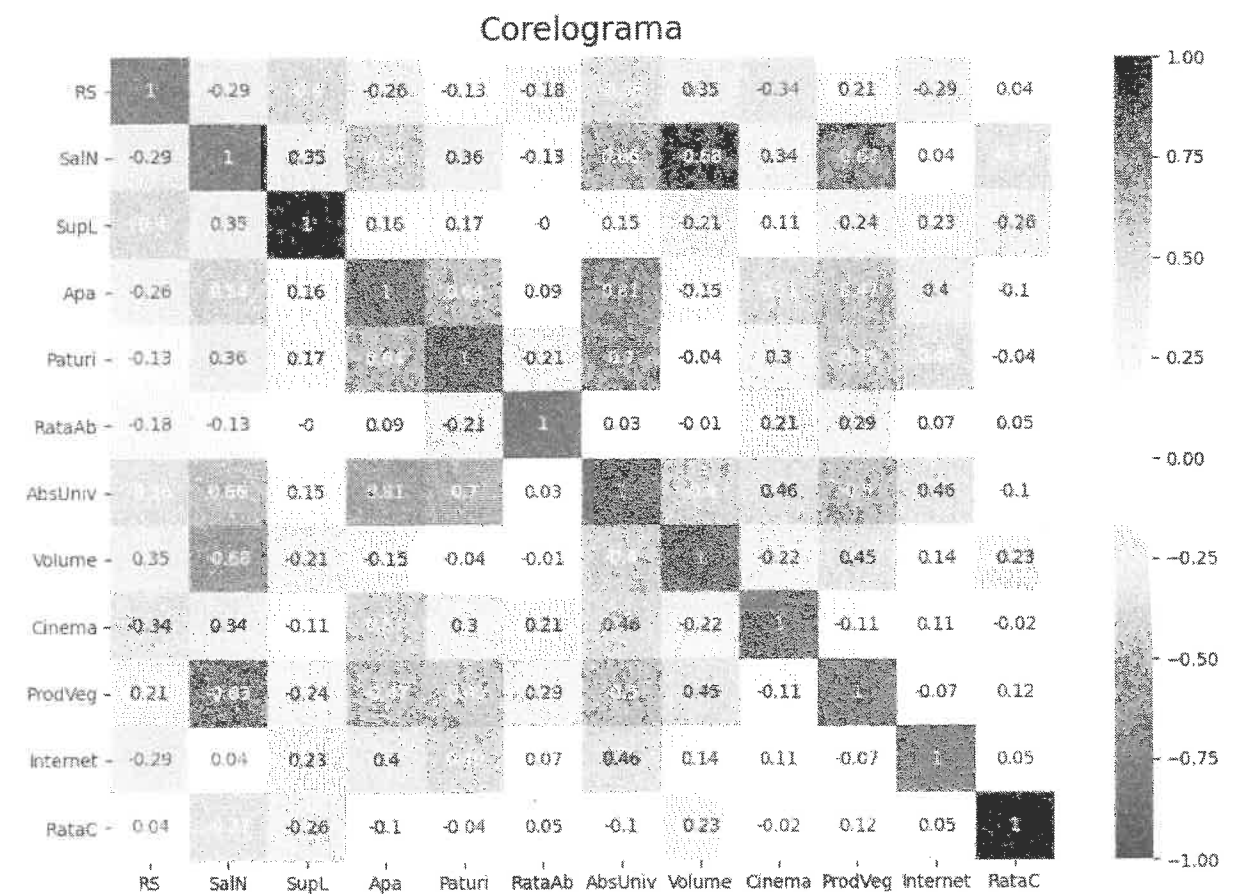


Figura 6.1 Corelograma matricei de corelație a variabilelor inițiale (observate)

Întrebarea este: câte tipuri de informații merită să fie investigate în mod amănunțit, exhaustiv? Geometric, este vorba de determinarea numărului de axe ce trebuie ales pentru o reprezentare multidimensională în așa fel, încât să obținem o acoperire informațională satisfăcătoare.

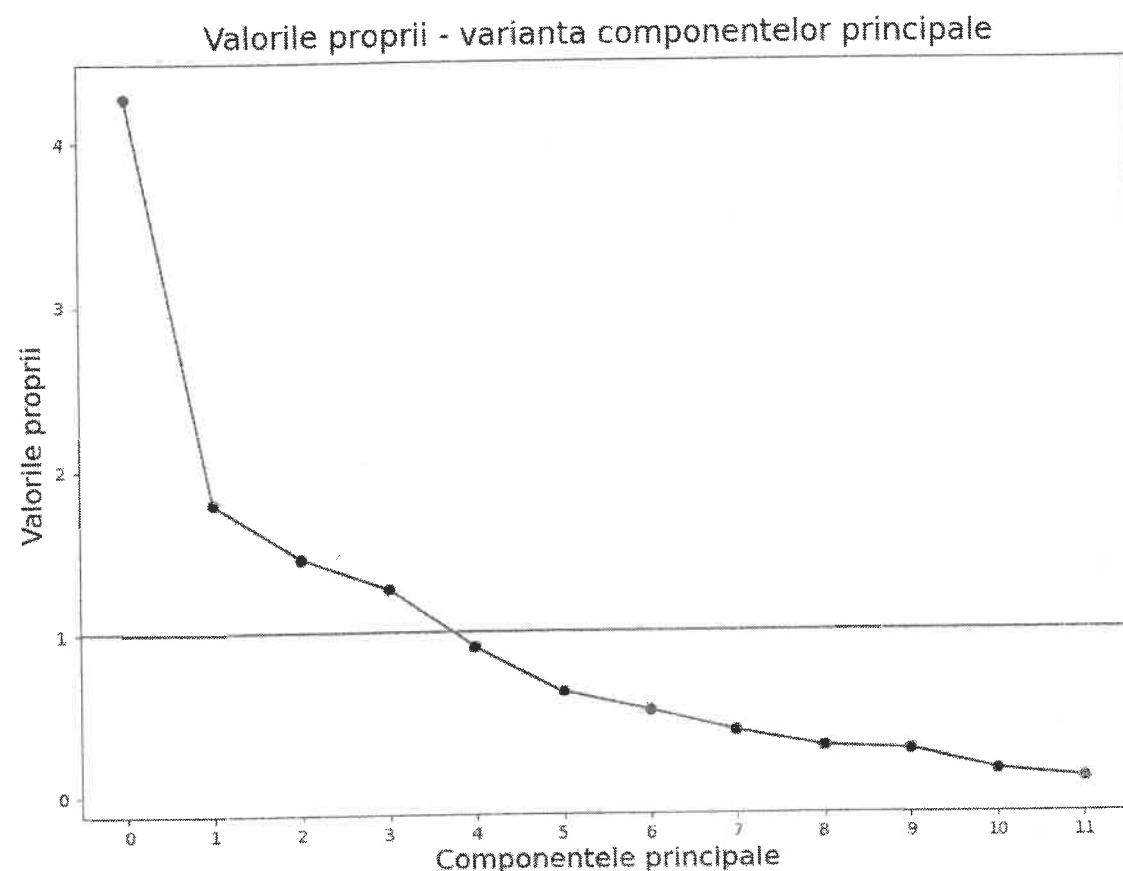


Figura 6.2 Graficul varianței explicate de componentele principale

Conform criteriilor lui Kaiser și Cattell, aplicate pe graficul varianței explicate de componentele principale (figura 6.2), putem reține 3 sau chiar 4 componente principale care au o varianță explicată mai mare decât 1, urmând a decela în continuare care este semnificația acestora.

Scorurile sunt standardizări ale componentelor principale:

$$C_{ik}^s = \frac{C_{ik}}{\sqrt{\alpha_k}}, i = \overline{1, n} \text{ și } k = \overline{1, s},$$

unde $\sqrt{\alpha_k}$ este abaterea standard a componentei C_k .

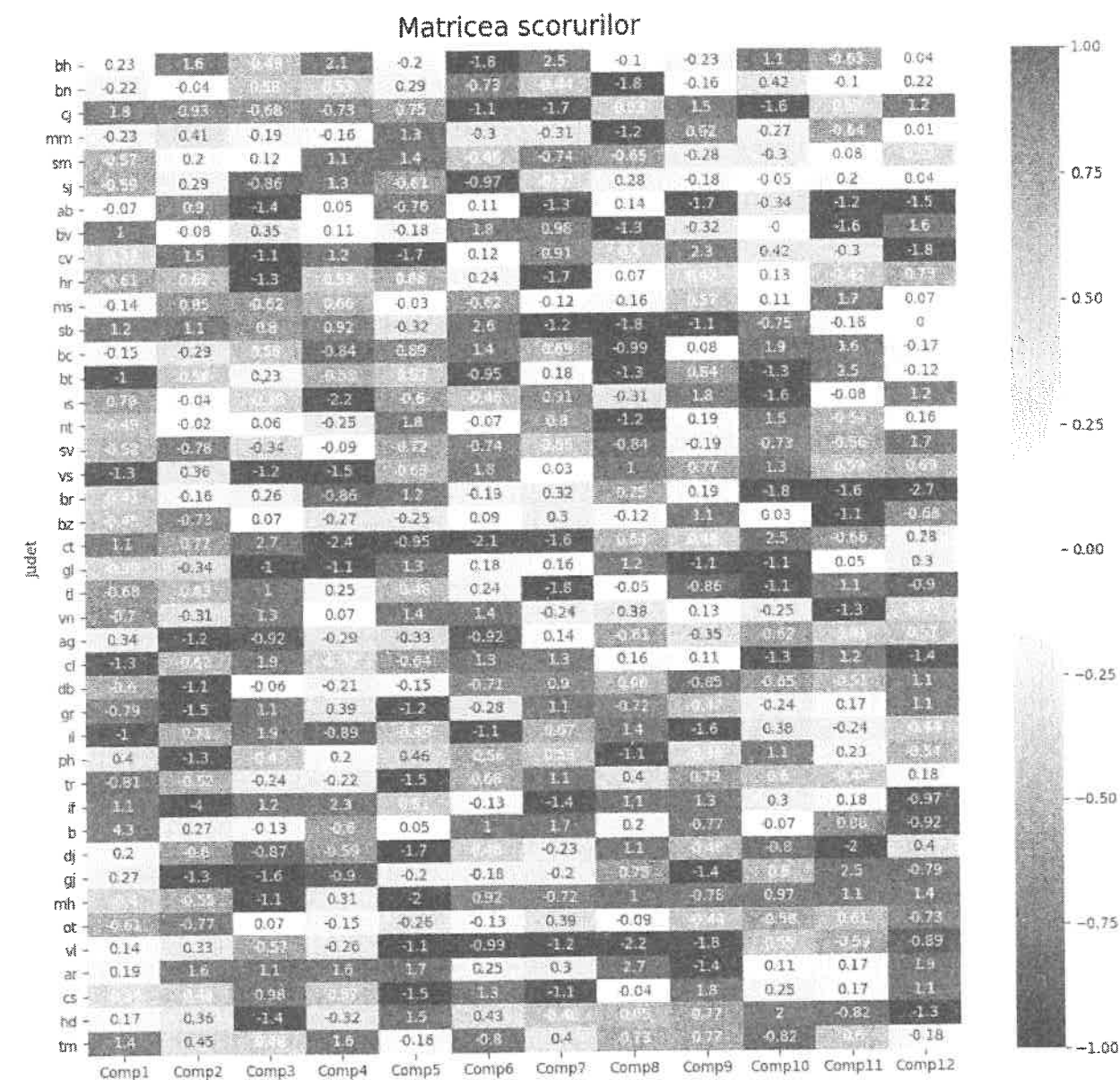


Figura 6.3 Graficul corelogramă a scorurilor - standardizările componentelor principale

Pe lângă corelograma scorurilor (figura 6.3), graficul histogramă a scorurilor primelor 5 componente principale standardizate este prezentat în figura 6.4.

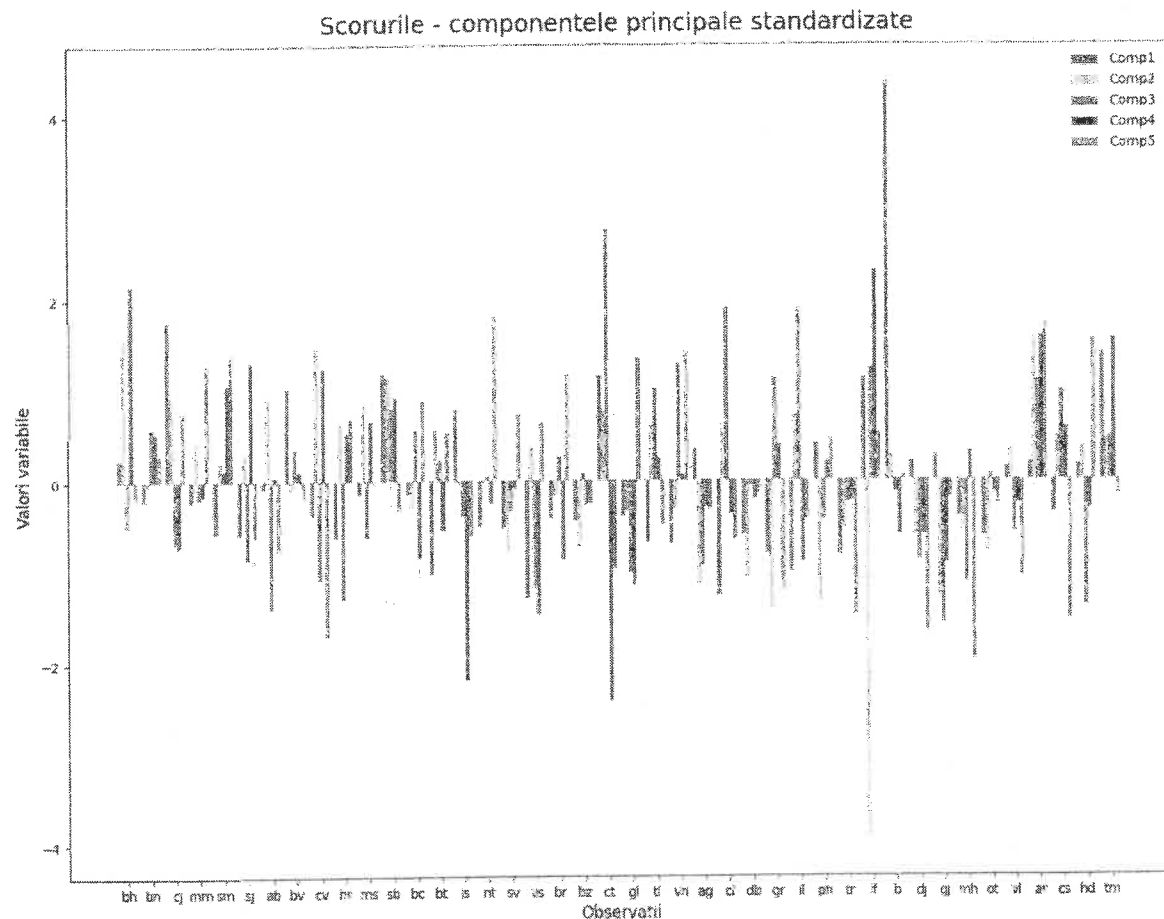


Figura 6.4 Graficul histogramă cu bare multiple, pentru scorurile asociate primelor 5 componente principale

Componentele principale constituie un nou spațiu de reprezentare a observațiilor numit *spațiul principal*. Baza acestui spațiu, vectorii unitari ai axelor, este constituită de vectorii proprii $a_k, k = \overline{1, m}$, iar coordonatele indivizilor în aceste noi axe sunt date de vectorii $C_k, k = \overline{1, m}$.

Așa cum am văzut un individ este reprezentat geometric de un punct într-un spațiu m -dimensional. Pătratul distanțelor de la un punct de index i spre centrul de greutate al norului de date este $\sum_{k=1}^m C_{ik}^2$. Un individ este cu atât mai bine reprezentat pe o axă oarecare a_j cu cât C_{ij}^2 are o valoare mai mare în raport cu $\sum_{k=1}^m C_{ik}^2$.

Calitatea reprezentării observației i pe axa a_j este dată de raportul $\frac{C_{ij}^2}{\sum_{k=1}^m C_{ik}^2}$. Valoarea raportului este egală cu pătratul cosinusului unghiului dintre vectorul punctului i și vectorul a_j .

Calitatea reprezentării observațiilor pe axele de coordonate ale componentelor principale este reliefată în graficele din figurile 6.5, 6.6 și 6.7. Contribuția observațiilor la axele componentelor principale este reliefată în graficul corelogramă din figura 6.8.

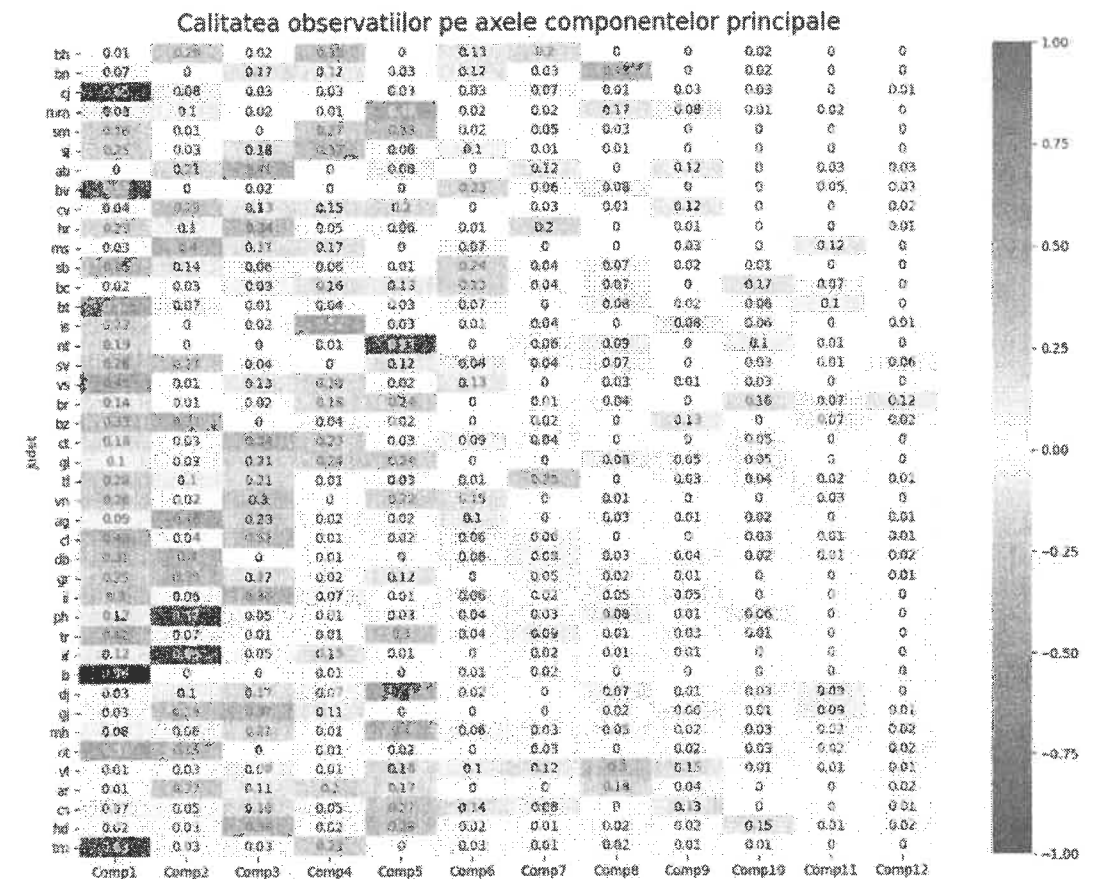


Figura 6.5 Graficul corelogramă a calității reprezentării observațiilor pe axele componentelor principale

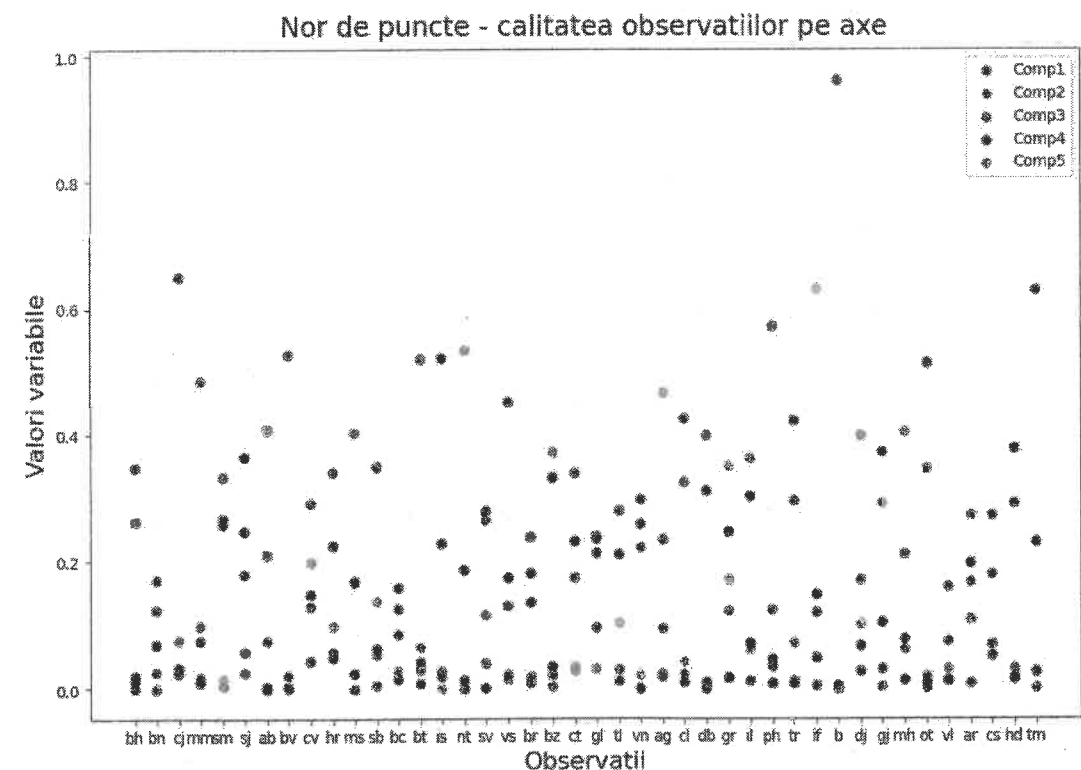


Figura 6.6 Graficul nor de puncte a calității reprezentării observațiilor pe axele componentelor principale

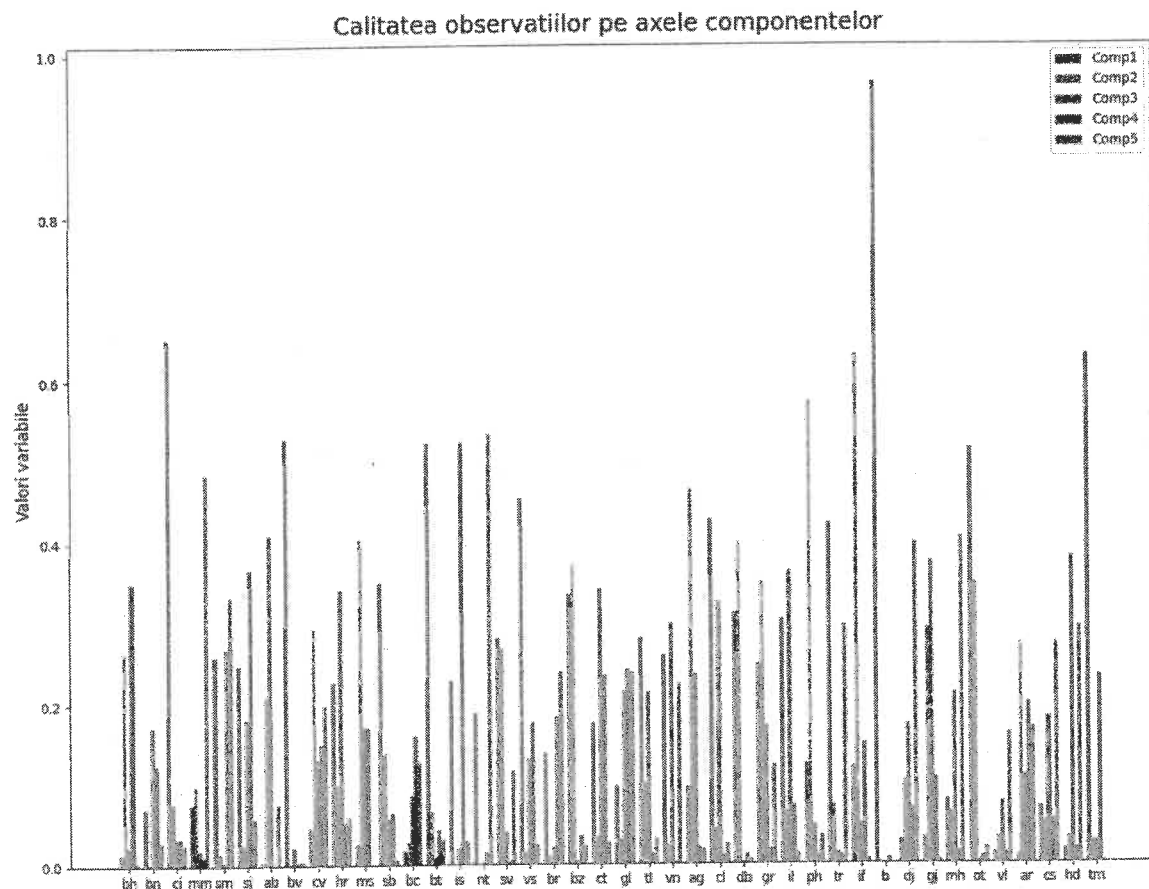


Figura 6.7 Graficul multi-histogramă a calității reprezentării observațiilor pe axele primelor 5 componente

Contribuția indivizilor la varianța axelor – Varianța explicată prin axa a_j este dată de relația:

$$\frac{1}{n} \sum_{i=1}^n C_{ij}^2 = \alpha_j$$

Partea din această varianță datorată individului i este $\frac{C_{ij}^2}{n}$. Contribuția individului i la varianța axei a_j este deci: $\frac{C_{ij}^2}{n \cdot \alpha_j}$.

Interpretarea componentelor principale și furnizarea unui nume, a unei semnificații pentru acestea este una din scopurile fundamentale ale analizei în componente principale. În contextul setului de date privitoare la dezvoltarea județelor României prin prizma indicatorilor macro-economici observați, se pot trage următoarele concluzii.

Prima componentă principală reprezintă nivelul de trai, sau calitatea vieții în marile metropole ale țării, cu preponderență în municipiul București, dar și în marile centre universitare: Cluj-Napoca, Iași, Timișoara.

Cea de a doua componentă principală caracterizează calitatea vieții din regiuni urbane care au fost cândva dezvoltate industrial, posibil în zone de extracție a materiilor prime sau de prelucrare a acestora și care în prezent trec printr-o perioadă de decădere economică, asociată cu fenomenul depopulării și cu degradarea condițiilor de viață.

Cea de a treia componentă privește nivel de trai în zona rurală, caracterizată printr-o rată ridicată a abandonului școlar și producții agricole vegetale și animale specifice, împreună cu o mai generoasă suprafață locuibilă *per capita*.

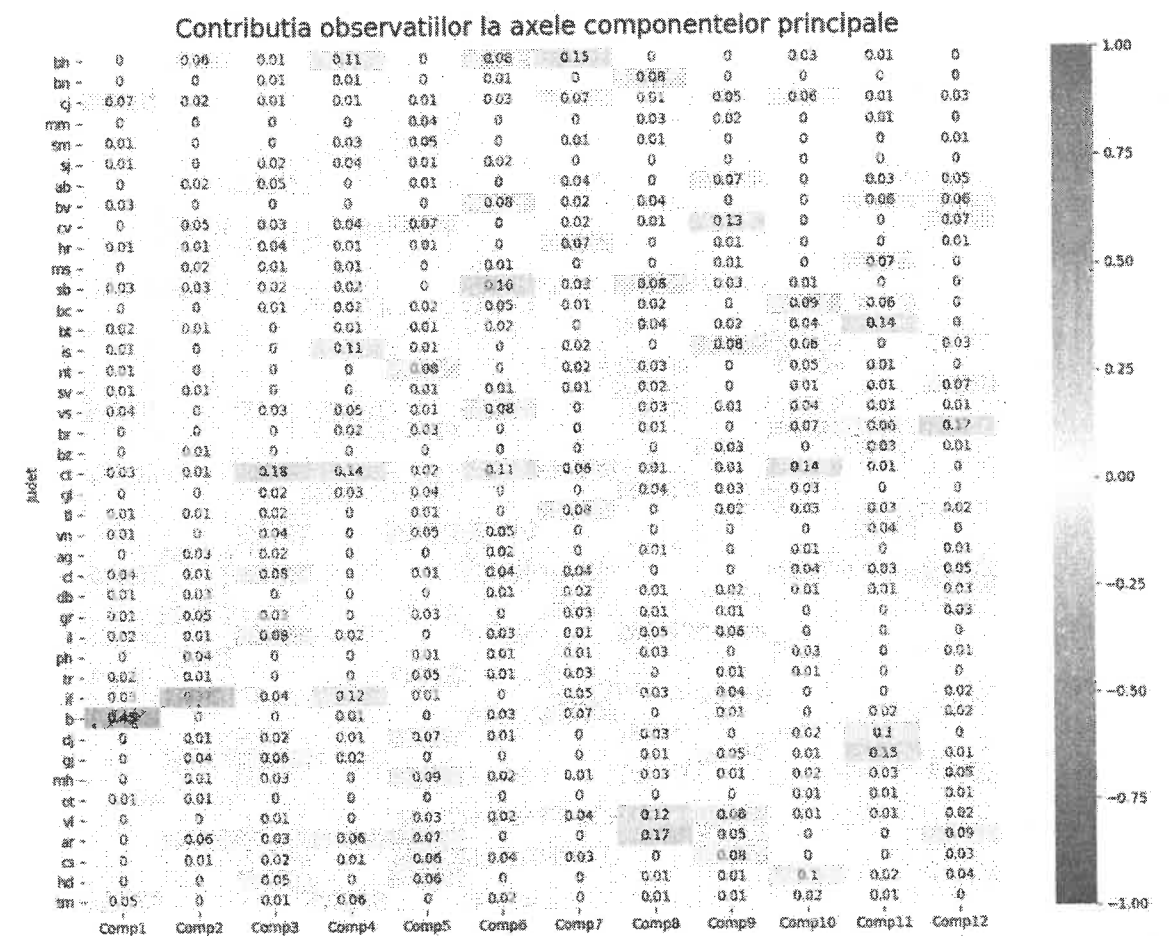


Figura 6.8 Graficul corelogramă a contribuției observațiilor la axele componentelor

Calculul coeficienților de corelație – coeficienții de determinare dintre o variabilă cauzală X_j și componenta principală C_k se calculează astfel:

$$R^2(C_r, X_j) = \frac{\text{Cov}(C_r, X_j)^2}{\text{Var}(C_r) \cdot \text{Var}(X_j)} = \frac{\text{Cov}(C_r, X_j)^2}{\alpha_r},$$

deoarece $\text{Var}(C_r) = \alpha_r$, iar $\text{Var}(X_j) = 1$.

Matriceal, obținem vectorul coeficienților de corelație dintre variabilele inițiale și C_r :

$$R_r = \frac{\frac{1}{n} X^t C_r}{\sqrt{\alpha_r}} = \frac{\frac{1}{n} X^t X a_r}{\sqrt{\alpha_r}} = \frac{\alpha_r a_r}{\sqrt{\alpha_r}} = a_r \sqrt{\alpha_r}$$

unde a_r este al r -lea vector propriu, corespunzător valorii proprii α_r , cu $r = \overline{1, s}$. Aceste corelații le vom numi în continuare corelații factoriale (*factor loadings*). Graficul corelațiilor factoriale, sau a factorilor de corelație, este prezentat în figura 6.9.

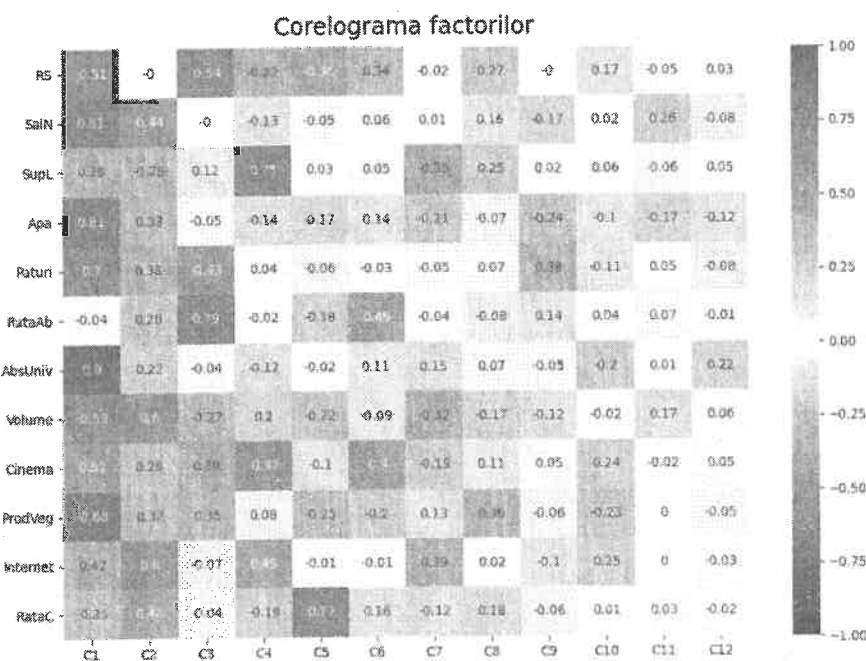


Figura 6.9 Graficul corelogramă a factorilor de corelație (*factor loadings*)

O altă fațetă ce privește legătura între componentele principale și variabilele cauzale este reliefată de conceptul de *comunalitate* (din termenul englez *communality*). *Comunalitatea* reprezintă cantitatea de varianță explicată în comun de către un grup de componentele principale (figurile 6.10 și 6.11). Așa cum menționat anterior, o componentă principală, C_k , preia sau explică o cantitate de varianță egală cu α_k , iar suma coeficienților de determinare dintre această componentă și variabilele cauzale este tot α_k . *Comunalitatea* unei variabile X_j în raport cu primele s componente principale este, deci, suma coeficienților de determinare dintre variabilă și aceste componente principale: $\sum_{k=1}^s R(X_j, C_k)^2$. Pentru $s = m$ această valoare este 1. Cele m componente principale preiau integral informația din X (figura 6.12).

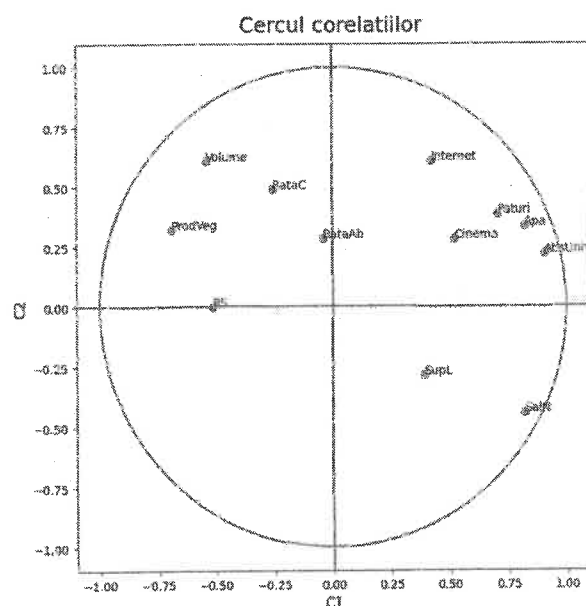


Figura 6.10 Variabilele inițiale în câmpul componentelor 1 și 2

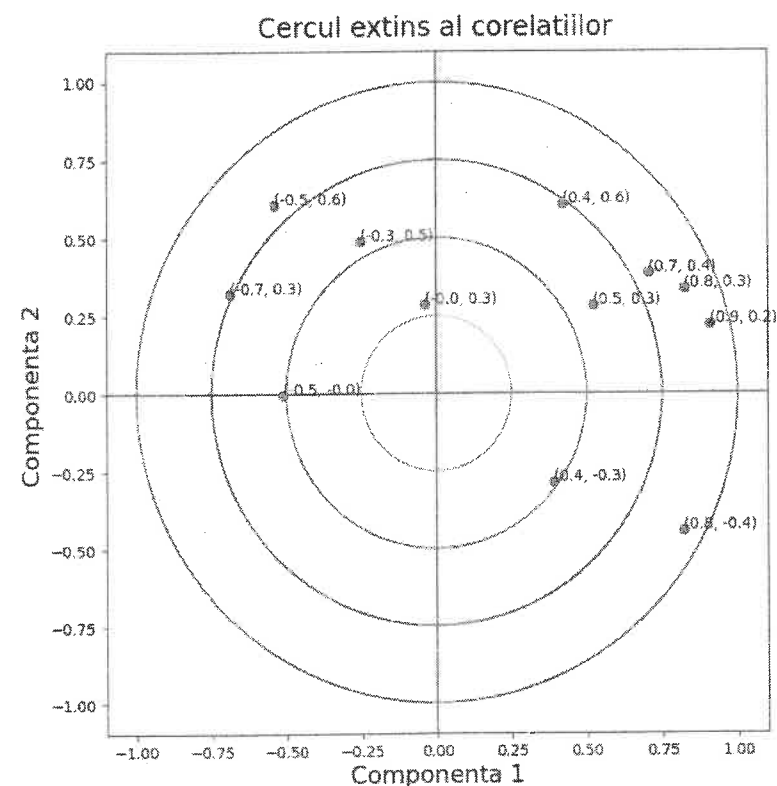


Figura 6.11 Corelația variabilelor inițiale cu componentele principale 1 și 2

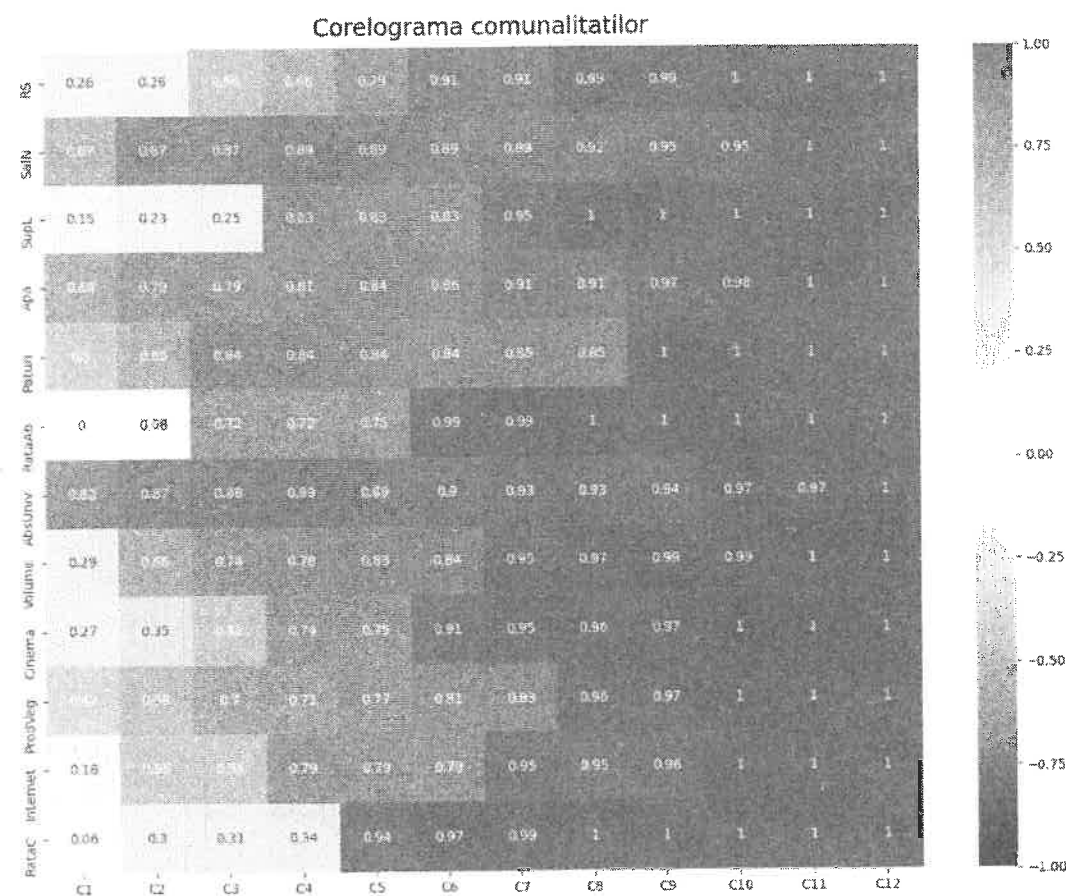


Figura 6.12 – Corelogramă a comunalităților componentelor principale regăsite în variabilele inițiale

6.2 Analiza exploratorie a factorilor (AEF)

Analiza exploratorie a factorilor (AEF) sau analiza factorială este o metodă statistică utilizată pentru a descrie variabilitatea între variabilele observate, posibil corelate, în termenii unui număr potențial mai mic de variabile neobservate numite factori.

Analiza factorială caută astfel de variații comune ca răspuns la variabile latente neobservate. Analiza factorială caută dimensiuni independente, ceea ce limitează aplicabilitatea sa în științele biologice.

Utilizatorii analizei factoriale consideră că ajută la gestionarea seturilor de date în care există un număr mare de variabile observate care se crede că reflectă un număr mai mic de variabile subiacente / latente.

Analiza factorială este legată de analiza componentelor principale (ACP), dar cele două nu sunt identice. De lungul timpului, au existat controverse semnificative în domeniu analizei datelor cu privire la diferențele dintre cele două tehnici. În analiza factorială, cercetătorul presupune că există un model cauzal subiacent, în timp ce ACP este pur și simplu o tehnică de reducere a numărului de variabile, sau a dimensiunii modelului. Analiza componentelor principale utilizează o transformare matematică aplicată datele originale, fără a se face vreo presupunere cu privire la forma matricei de covarianță. Scopul ACP este de a determina câteva combinații liniare ale variabilelor originale care pot fi utilizate pentru a rezuma setul de date, din perspectiva varianței, fără a pierde multe informații. În AEF, variabilele observate sunt modelate drept combinații liniare ale factorilor potențiali, plus termeni de „eroare”.

6.2.1 Datele de prelucrat

Analiza exploratorie a factorilor descrie fiecare variabilă cauzală ca o combinație liniară de factori comuni plus un factor unic sau specific, astfel:

$$\begin{cases} X_1 = u_{11}F_1 + u_{12}F_2 + \dots + u_{1q}F_q + e_1 \\ X_2 = u_{21}F_1 + u_{22}F_2 + \dots + u_{2q}F_q + e_2 \\ \dots \\ X_m = u_{m1}F_1 + u_{m2}F_2 + \dots + u_{mq}F_q + e_m \end{cases} \quad (1)$$

unde q este numărul factorilor ($q < m$), X_j , $j = \overline{1, m}$ sunt variabilele cauzale (variabilele observate) centrate, F_k , $k = \overline{1, q}$ sunt factorii comuni, e_j , $j = \overline{1, m}$ sunt factorii specifici iar u_{jk} , $j = \overline{1, m}$, $k = \overline{1, q}$ sunt coeficienții factoriali (factor loadings). Factorii comuni sunt ortogonali doi câte doi și de normă 1.

Matriceal, legătura dintre variabilele observate și factori se poate scrie astfel:

$$X = F \cdot u^t + e,$$

unde X este tabelul de observații, u^t este transpusa matricei coeficienților factoriali,

$$e = \begin{bmatrix} e_{11} & e_{12} & \dots & e_{1m} \\ e_{21} & e_{22} & \dots & e_{2m} \\ \dots & \dots & \dots & \dots \\ e_{n1} & e_{n2} & \dots & e_{nm} \end{bmatrix} \text{ este matricea factorilor specifici,}$$

$$\text{iar } F = \begin{bmatrix} f_{11} & f_{12} & \dots & f_{1q} \\ f_{21} & f_{22} & \dots & f_{2q} \\ \dots & \dots & \dots & \dots \\ f_{n1} & f_{n2} & \dots & f_{nq} \end{bmatrix} \text{ este matricea factorilor comuni așezați pe coloane.}$$

6.2.2 Ipoteze modelului

Factorii comuni sunt standardizați, prin urmare au varianță 1 și medie 0. Sunt construiți după principiul separației informaționale, deci sunt necorelați doi câte doi:

$$R(F_i, F_j) = 0, \text{Cov}(F_i, F_j) = 0, i = \overline{1, q}, j = \overline{1, q}, i \neq j.$$

Factorii specifici sunt de medie 0. Varianța factorilor specifici se notează cu

$$\psi_j = \text{var}(e_j), j = \overline{1, m}$$

și se numește varianța specifică.

Între factorii comuni și factorii specifici nu există suprapunere de informație, deci sunt absolut necorelați între ei:

$$R(F_i, e_j) = 0, \text{Cov}(F_i, e_j) = 0, i = \overline{1, q}, j = \overline{1, m}.$$

Aceste ipoteze sunt necesare pentru a estima în mod unic parametrii modelului.

Având în vedere aceste ipoteze, varianța unei variabile observate este:

$$\begin{aligned} \text{Var}(X_j) &= \text{Var}(u_{j1} \cdot F_1 + u_{j2} \cdot F_2 + \dots + u_{jq} \cdot F_q + e_j) = \\ &= u_{j1}^2 \cdot \text{Var}(F_1) + u_{j2}^2 \cdot \text{Var}(F_2) + \dots + u_{jq}^2 \cdot \text{Var}(F_q) + \text{Var}(e_j) = \\ &= \sum_{k=1}^q u_{jk}^2 + \psi_j. \end{aligned}$$

Covarianța dintre două variabile observate X_i și X_j este:

$$\begin{aligned} \text{Cov}(X_i, X_j) &= \text{Cov}(u_{i1}F_1 + \dots + u_{iq}F_q + e_i, u_{j1}F_1 + \dots + u_{jq}F_q + e_j) = \\ &= \sum_{k=1}^q u_{ik}u_{jk}. \end{aligned}$$

Covarianța dintre o variabilă observată și un factor comun este:

$$\text{Cov}(X_p, F_j) = \text{Cov}(u_{p1}F_1 + \dots + u_{pq}F_q + e_p, F_j) = u_{pj}$$

Ținând cont de aceste relații, matricea de covarianță a tabelului de observații se poate scrie:

$$V = u \cdot u^t + \psi, \text{ unde}$$

$$\psi = \begin{bmatrix} \psi_1 & 0 & \dots & 0 \\ 0 & \psi_2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & \psi_m \end{bmatrix}$$

este matricea diagonală a varianțelor factorilor specifici.

6.2.3 Estimarea existenței factorilor

Ipoteza existenței factorilor latenți este indusă de o corelație semnificativă între variabilele cauzale. Variabilitatea comună a variabilelor cauzale observate duce la ideea explicării acesteia prin existența unor factori ascunși. În consecință, pe baza matricei de corelație a variabilelor cauzale, s-ar putea estima existența factorilor comuni.

Există diferite teste concepute pentru a verifica ipoteza existenței factorilor latenți, pe baza matricei corelațiilor.

Estimarea existenței factorilor pe baza testul de sfericitate Bartlett – acesta este un test χ^2 care compară matricea de corelație a variabilelor observate cu matricea identității (unității). Dacă un nor multivariat de observații ar fi perfect sferic, atunci matricea sa de covarianță ar fi proporțională cu matricea identității. Statistica χ^2 testează următoarele ipotezele:

- H0: nu există factori
- H1: există cel puțin un factor comun

În cazul variabilelor cauzale absolut necorelate, determinantul matricelor de corelație este 1.

$$\chi^2_c = -\left(n - 1 - \frac{2 \cdot m + 5}{6}\right) \ln|R|,$$

unde R este matricea de corelație a variabilelor cauzale, n este numărul de instanțe sau observații și m este numărul de variabile inițiale sau observate. Aceste valori urmează o distribuție χ^2 (chi-pătrat) cu $m \cdot (m-1) / 2$ grade de libertate.

$$\text{Dacă } \chi^2_c > \chi^2(\alpha, m \cdot (m-1) / 2),$$

atunci ipoteza nulă este respinsă cu un nivel de încredere de $1-\alpha$.

Pentru estimarea existenței factorilor utilizăm indicele KMO (Kaiser-Meyer-Olkin). Acesta se bazează de asemenea pe matricea de corelație. Indicele global KMO este calculat după cum urmează:

$$KMO = \frac{\sum_i \sum_{j \neq i} r_{ij}^2}{\sum_i \sum_{j \neq i} r_{ij}^2 + \sum_i \sum_{j \neq i} a_{ij}^2}$$

unde, r_{ij} este coeficientul de corelație liniară dintre variabilele X_i și X_j , iar a_{ij} reprezintă coeficientul de corelație parțială dintre X_i și X_j .

Indexul KMO pentru fiecare variabilă observată este calculat după cum urmează:

$$KMO_i = \frac{\sum_{j \neq i} r_{ij}^2}{\sum_{j \neq i} r_{ij}^2 + \sum_{j \neq i} a_{ij}^2}, j = \overline{1, m}$$

pentru fiecare $i = \overline{1, n}$.

Indicii de mai sus dezvăluie care dintre variabile sunt mai puțin corelate cu celelalte, și prin urmare, oferă o variabilitate mai puțin comună.

Coeficientul de corelație parțială dintre două variabile X_i și X_j se calculează după cum urmează:

$$a_{ij} = \frac{t_{ij}}{\sqrt{t_{ii} - t_{jj}}}, \text{ unde } t_{ij} \text{ este termenul general al matricei } T = R^{-1}$$

Corelația parțială reprezintă legătura liniară dintre două variabile, în timp ce efectul celorlalte variabile din model asupra lor este neutralizat.

Interpretarea valorii indexului KMO:

- 0.90 - 1.00: foarte mare probabilitate de existență a factorilor
- 0.80 - 0.89: probabilitate bună de existență a factorilor
- 0.70 - 0.79: medie
- 0.60 - 0.69: mediocră
- 0.50 - 0.59: slabă
- 0.00 - 0.49: fără factori

6.2.4 Estimarea parametrilor modelului. Extragerea factorilor

Estimarea parametrilor modelului constă în determinarea coeficienților factoriali și, implicit, a factorilor comuni și a factorilor specifici.

Algoritmul de extragere a factorilor pornește de la ipoteza unui model cu un singur factor comun, după care se testează discrepanța dintre matricea de covarianță a variabilelor observate și cea produsă prin model:

$$V \approx \hat{V} = u \cdot u^t + \psi$$

Dacă testul este respins (discrepanța este prea mare din punct de vedere statistic), atunci se estimează un model cu doi factori comuni și așa mai departe, până când testul discrepanței este trecut.

Există mai multe metode de extragere a factorilor, în funcție de criteriile de testare a discrepanței dintre cele două matrice de covarianță:

- metoda probabilității maxime (*the maximum likelihood method*)
- metoda componentelor principale (*principal component analysis*)
- metoda celor mai mici pătrate (*the least squares method*)
- metoda factorilor principali (*principal axis factoring*)

În continuare, vom prezenta succint metoda probabilității maxime și metoda analizei componentelor principale.

6.2.5 Metoda probabilității maxime

Determinarea factorilor prin metoda probabilității maxime presupune ca datele să urmeze o distribuție normală multivariată și să fie prelevate în mod independent. Acest lucru reprezintă un dezavantaj pentru această metodă.

Procesul de extragere este unul iterativ și constă în maximizarea unei funcții de probabilitate calculată astfel:

$$l(\hat{V}) = -\frac{1}{2}n \cdot \log|2\pi \cdot \hat{V}| - \frac{1}{2}n \cdot \text{tr}(\hat{V}^{-1}V),$$

unde n este numărul de observații, V este matricea de covarianță a variabilelor observate, iar $\hat{V}$ este estimarea matricei de covarianță realizată prin model. Valoarea $\text{tr}(\hat{V}^{-1}V)$, reprezintă urma matricei (*trace*). Urma unei matrice există numai în cazul matricelor pătratice și este numărul obținut prin adunarea termenilor situați pe diagonala principală ai matricei (diagonala de la stânga sus la dreapta jos).

6.2.6 Metoda analizei componentelor principale

Analiza factorială și analiza în componente principale sunt deseori confundate. Unele aplicații specializate folosesc analiza în componente principale ca motor algoritmic pentru analiza factorială.

Notăm în continuare cu C matricea $n \times m$ a componentelor principale aranjate pe coloane: $C_1, C_2, \dots, C_m$. De asemenea, notăm cu A matricea $m \times m$ a coeficienților a_k aranjați, de asemenea, pe coloane: $a_1, a_2, \dots, a_m$.

Deoarece

$$C_k = X \cdot a_k, \text{ pentru } k = \overline{1, m},$$

atunci matriceal putem scrie $C = X \cdot A$.

Matricea A are coloanele ortogonale două câte două, deci $A^{-1} = A^t$. Rezultă că

$$X = C \cdot A^{-1} = C \cdot A^t,$$

sau altfel scris,

$$X = \sum_{k=1}^m C_k \cdot (a_k)^t.$$

Abaterea standard a unei componente principale este $\sigma(C_k)$ și este egală cu rădăcină pătrată din varianța componentei C_k , adică $\sqrt{\alpha_k}$. Putem așadar scrie:

$$X = \sum_{k=1}^m \sqrt{\alpha_k} \cdot \frac{C_k}{\sqrt{\alpha_k}} \cdot (a_k)^t = \sum_{k=1}^m S_k \cdot (R_k)^t,$$

unde $S_k = \frac{C_k}{\sqrt{\alpha_k}}$ reprezintă scorurile sau componentele standardizate, iar R_k este vectorul coloană al coeficienților de corelație dintre variabilele cauzale și componenta C_k , corelațiile factoriale sau *factor loadings* din ACP.

Această relație arată cum matricea X poate fi reconstituită pornind de la scoruri și corelațiile factoriale. După cum știm, corelațiile factoriale sunt din ce în ce mai mici de la o componentă la cealaltă. Dacă luăm în considerare numai primele q etape, tabelul X va fi aproximat prin cantitatea

$$\sum_{k=1}^q S_k \cdot (R_k)^t,$$

în timp ce cantitatea

$$\sum_{k=q+1}^m S_k \cdot (R_k)^t,$$

poate fi considerată ca o cantitate reziduală, neglijabilă, cu un aport nesemnificativ în reconstituirea tabelului X . Prin urmare:

$$X = \sum_{k=1}^q S_k \cdot (R_k)^t + \sum_{k=q+1}^m S_k \cdot (R_k)^t = S \cdot (R)^t + e,$$

unde S este matricea scorurilor, sau componentelor principale standardizate, R este matricea corelațiilor dintre componente principale și variabilele observate, iar e este aportul rezidual la reconstituirea matricei inițiale X .

Dezvoltând aceste sume și făcând comparație cu relațiile (1), se observă că factorii F_k din analiza factorială sunt, de fapt, scorurile (standardizări ale componentelor principale) din analiza în componente principale, iar coeficienții u din analiza factorială sunt, de fapt, corelațiile factoriale (*factor loadings*) din analiza în componente principale.

Prin rotația factorilor se încearcă obținerea unei matrice de coeficienți factoriali (u) cât mai ușor de interpretat, care să ușureze cât mai mult interpretarea factorilor. Sistemul de axe ortogonale reprezentat de factori este rotit în jurul originii într-o altă poziție. Prin rotația factorilor se minimizează numărul de variabile cu corelații factoriale mari pentru fiecare factor, simplificând astfel interpretarea factorilor.

6.2.7 Estimarea numărului de factori. Testul Bartlett

Analiza factorială pornește de la asumarea unui anumit număr de factori, q . Acest număr poate fi insuficient. Un model bun este un model în care matricea de covarianță estimată, produsă de model, se apropie cât mai mult de matricea de covarianță a variabilelor observate. Pentru testarea modelului se folosesc teste statistice.

O descriere perfectă a datelor observate printr-o serie de factori q implică faptul că

$$\hat{V} = V.$$

Multiplcând cu $\hat{V}^{-1}$ la stânga, obținem $\hat{V}^{-1} \cdot \hat{V} = \hat{V}^{-1} \cdot V$ și de aici

$$\hat{V}^{-1} \cdot V = (u \cdot u^t + \psi)^{-1} \cdot V = I,$$

unde I este matricea de identitate (unitate) de ordin m .

Statistica testului se calculează astfel:

$$\chi_C^2 = \left(n - 1 - \frac{2m+4q-5}{6} \right) \left(\left(\text{tr} \left((u \cdot u^t + \psi)^{-1} \cdot V \right) \right) - \log \left(\left| (u \cdot u^t + \psi)^{-1} \cdot V \right| \right) - m \right)$$

Trebuie remarcat faptul că statistica testului este 0 (zero) pentru o descriere perfectă a datelor observate prin q factori. Urma matricei

$$(u \cdot u^t + \psi)^{-1} \cdot V$$

este egală cu m , urma matricei identitate de ordinul m , iar logaritm din determinant este 0 (determinantul matricei $(u \cdot u^t + \psi)^{-1} \cdot V$ este 1, determinat de matrice unitate).

Prin urmare, cu cât valoarea statisticilor este mai mică, cu atât este mai mare șansa ca factorii să descrie în mod adecvat datele observate.

Și invers, cu cât valoarea statisticilor este mai mare, cu atât șansele ca factorii de descriere adecvată a datelor se diminuează.

Un astfel de test este testul Bartlett, care este un test χ^2 . Statistica testului se calculează astfel:

$$\chi_C^2 = \left(n - 1 - \frac{2m + 4q - 5}{6} \right) \log \frac{|\hat{V}|}{\left| \frac{n-1}{n} V \right|}$$

unde q este numărul de factori asumat, m este numărul variabilelor observate, n este numărul de observații, V este matricea de covarianță a variabilelor observate, iar $\hat{V}$ este estimarea matricei de covarianță produsă de model.

Ipoteze:

- H_0 : factorii modelului descriu adecvat datele;
- H_1 : factorii nu descriu adecvat datele.

Ipoteza H_0 este respinsă dacă

$$\chi_C^2 > \chi^2(r, \alpha),$$

unde $r = \frac{(m-q)^2 - m - q}{2}$ reprezintă numărul gradelor de libertate.

6.2.8 AEF - studiu de caz

Pentru studiul aplicat al analizei exploratorii a factorilor vom utiliza un set de date privitor la diferite cauze ale mortalității în țările Uniunii Europene.

Setul de date conține ca variabile observate la nivelul țărilor UE mai multe cauze ale mortalității și anume:

1. Consumul de Alcool
2. Accidente (în general)
3. Accidente de circulație
4. Afecțiuni ale inimii
5. Boli cronice
6. Afecțiuni ale ficatului
7. Cancer
8. Diabet
9. Consum de medicamente și narcotice
10. HIV SIDA
11. Omucideri
12. Pneumonie
13. Afecțiuni ale sistemului nervos
14. Sinucideri

Datele sunt obținute de la Eurostat și reprezintă incidența la 10,000 de locuitori la nivelul anului 2017. Nucleul analizei factoriale este încapsulat în clasa AEF din pachetul aef și are următoarea implementare.

```
'''
AEF - clasa ce încapsuleaza un model de prelucrare a datelor
pentru analiza exploratorie a factorilor (AEF)

X reprezenta matricea datelor de intrare (m variabile pe coloane
si n observatii pe randuri)
matricea X este de asteptat sa fie furnizata ca numpy ndarray
'''

import numpy as np
import acp.ACP as acp
import scipy.stats as sts

class AEF:

    def __init__(self, X):
        self.X = X

        # instatierea unui model de acp
        acpModel = acp.ACP(X)
        self.Xstd = acpModel.getXstd()
        self.R = acpModel.getCor()
        self.C = acpModel.getComp()
        self.alpha = acpModel.getValProp()
```

```

# extragerea contributiei observatiilor la varianta axelor
self.betha = acpModel.getContribObs()
# calitatea reprezentarii observatiilor pe axele factorilor
self.calObs = acpModel.getCalObs()

def getXstd(self):
    return self.Xstd

def getCor(self):
    return self.R

def getValProp(self):
    return self.alpha

def getComponente(self):
    return self.C

def getBetha(self):
    return self.betha

def getCalObs(self):
    return self.calObs

def explorare(self):
    n = np.shape(self.C)[0]

    # calculul scorurilor
    S = self.C / np.sqrt(self.alpha)

    # calculul cosinusurilor
    C2 = self.C * self.C
    sumObs = np.sum(C2, axis=1)
    q = np.transpose(np.transpose(C2) / sumObs)

    # calculul contributiilor
    beta = C2 / (self.alpha * n)

    # calculul comunalitatilor
    R2 = self.R * self.R
    common = np.cumsum(R2, axis=1)
    return S, q, beta, common

def calculTestBartlett(self, loads, epsilon):
    n = self.X.shape[0]
    # m = self.X.shape[1]
    m, q = np.shape(loadings)
    print(n, m, q)
    # matricea de corelatie a variabilelor observate
    V = self.R
    # factorii specifici
    psi = np.diag(epsilon)
    # estimarea matricei de corelatie a variabilelor
    # observate prin intermediul coeficientilor factoriali
    # si a factorilor specifici (reziduali)
    estimatedV = loads @ np.transpose(loadings) + psi
    # aproximarea matricei unitate (identitate)
    I = np.linalg.inv(estimatedV) @ V
    detI = np.linalg.det(I)

```

```

if detI > 0:
    traceI = np.trace(I)
    chi2 = (n - 1 - (2 * m + 4 * q - 5) / 6) * \
        (traceI - np.log(detI) - m)
    # calculul gradelor de libertate
    gradeLibertate = ((m - q) * (m - q) - m - q) / 2
    # chi patrat teroretic (tabelat)
    chi2Tab = 1 - sts.chi2.cdf(chi2, gradeLibertate)
else:
    chi2, chi2Tab = np.nan, np.nan
return chi2, chi2Tab

```

Programul apelator (mainAEF.py) utilizează clasa aef.AEF pentru crearea unei instanțe care să faciliteze prelucrarea datelor. Prezentăm în continuare codul Python pentru procesarea setului de date și generarea rezultatelor sub formă de tabele și grafice.

```

import factor_analyzer as fa
import numpy as np
import pandas as pd
import util.util as utl
import visual.visual as vi
import aef.AEF as aef
from sklearn.preprocessing import StandardScaler

# incarcare intr-un pandas DataFrame a datelor din fisierul CSV
fisier = "dataIN/MortalityEU.csv"
# specificam faptul ca avem in fisier celule populate cu valori nenumarice,
# respectiv avand valoarea ':'
tablou = pd.read_csv(fisier, index_col=0, na_values=':')
print(tablou)

numeObs = tablou.index[:]
print(numeObs)
numeVar = tablou.columns[:]
print(numeVar)

# numar coloane (variabile)
m = len(numeVar)
# numar randuri (observatii)
n = len(numeObs)

valori = tablou[numeVar].values
X = utl.inlocuireNAN(valori)

aefModel = aef.AEF(X)
Xstd = aefModel.getXstd()

scalare = StandardScaler()
t = pd.DataFrame(scalare.fit_transform(Xstd), index=numeObs, columns=numeVar)
# t = pd.DataFrame(Xstd, index=numeObs, columns=numeVar)
print(t)

# evaluare redundantei informatiei
# calculul testului Bartlett privind sfericitea
sfericitateBartlett = fa.calculate_bartlett_sphericity(t)
print("Testul Bartlett al sfericitatii: ", sfericitateBartlett)

# calculul indecsilor Kaiser, Meyer, Olkin (KMO)

```

```

# pentru a verifica daca esantionul este adecvat
kmo = fa.calculate_kmo(t)
print('Testul Kaiser-Meyer-Olkin privind adecvarea esantionului: ', kmo)

# crearea unei matrice cu o singura coloana pentru
# realizarea corelogramei
vector = kmo[0]
matrice = vector[:, np.newaxis]
print(matrice)
dataTab = pd.DataFrame(data=matrice, index=numeVar,
                       columns=('KMOs' for k in range(1)))

print("KMO total:", kmo[1])
# verificarea ipotezei H0 a testului statistic KMO
if kmo[1] < 0.5:
    print('Nu exista nici un factor semnificativ!')
    exit(1)
else:
    print('Poate fi identificat cel putin un factor semnificativ!')

alpha = aefModel.getValProp()
vi.valProp(alpha)

# crearea modelului AEF si aplicarea testului Bartlett
# pentru determinarea numarului de factori semnificativi
# utilizand FactorAnalyzer
chi2TabMin = 1
numarFactoriSemnificativi = 2
for k in range(2, m + 1):
    faModel = fa.FactorAnalyzer(n_factors=k)
    faModel.fit(t)

    chi2, chi2Tab = aefModel.calculTestBartlett(faModel.loadings_,
                                                faModel.get_uniquenesses())

    # print(faModel.loadings_, faModel.get_uniquenesses())
    print(chi2, chi2Tab)
    if np.isnan(chi2Tab) or np.isnan(chi2):
        break
    if chi2Tab < chi2TabMin:
        chi2TabMin = chi2Tab
        numarFactoriSemnificativi = k

print('Numarul de factori semnificativi extras este: ',
      numarFactoriSemnificativi)

# Crearea modelului cu numarul de factori semnificativi
numeFactori = ['F'+str(q+1) for q in range(numarFactoriSemnificativi)]
faFitModel = fa.FactorAnalyzer(n_factors=numarFactoriSemnificativi,
                               rotation=None)

faFitModel.fit(t)
# extragerea corelatiilor factoriale
# din obiectul FactorAnalyzer
loadsFitModel = faFitModel.loadings_
print(loadsFitModel)
loadsFitModelTab = pd.DataFrame(loadsFitModel, index=numeVar,
                                columns=numeFactori)
loadsFitModelTab.to_csv("dataOut/FA_FitModel_Loadings.csv")

```

```

vi.corelograma(dataTab, title='Indicii Kaiser-Meyer-Olkin (KMO)')
vi.corelograma(loadsFitModelTab, title="Corelograma corelatiilor factoriale")
vi.cercCorelExt(loadsFitModelTab, 0, 1, con=True,
                title="Coeficientii factoriali 1 si 2")
vi.cercCorel(loadsFitModelTab, 0, 2, title="Coeficientii factoriali 1 si 3")

betha = aefModel.getBetha()
bethaTab = pd.DataFrame(data=betha, index=numeObs,
                        columns=['F'+str(j+1) for j in range(m)])
vi.corelograma(bethaTab, title='Contributia observatiilor la varianta axelor')

calObs = aefModel.getCalObs()
calObsTab = pd.DataFrame(data=calObs, index=numeObs,
                        columns=['F'+str(j+1) for j in range(m)])
vi.corelograma(calObsTab, title='Calitatea reprezentarii observatiilor pe axele
factorilor')

alphaFA = faFitModel.get_eigenvalues()
print(type(alphaFA), alphaFA[1])
vi.valProp(alphaFA[0], titlu='Valorile proprii - FactorAnalyzer')

# Apply factorial rotation
faFitModel.fit_transform(t)
faFitModel.rotation
load_rot = faFitModel.loadings_
load_rotTab = pd.DataFrame(data=load_rot, index=numeVar,
                           columns=numeFactori)
load_rotTab.to_csv("Fa_loadings_varimax.csv")
vi.corelograma(load_rot, title='Coeficientii factoriali - Varimax')

vi.show()

```

Testul de sfericitate Bartlett indică faptul că există cel puțin un factor comun ascuns, neaparent care explice ce mai mare parte din varianța modelului.

Calcul indicilor Kaiser-Meyer-Olkin (KMO) ne confirmă prin valorile acestora pentru fiecare din variabilele inițiale că există potențial de factorizare (figura 6.13). Indicele KMO global are valoarea:

KMO total: 0.5941861226384726

Aceasta semnifică un potențial global mediocru de factorizare.

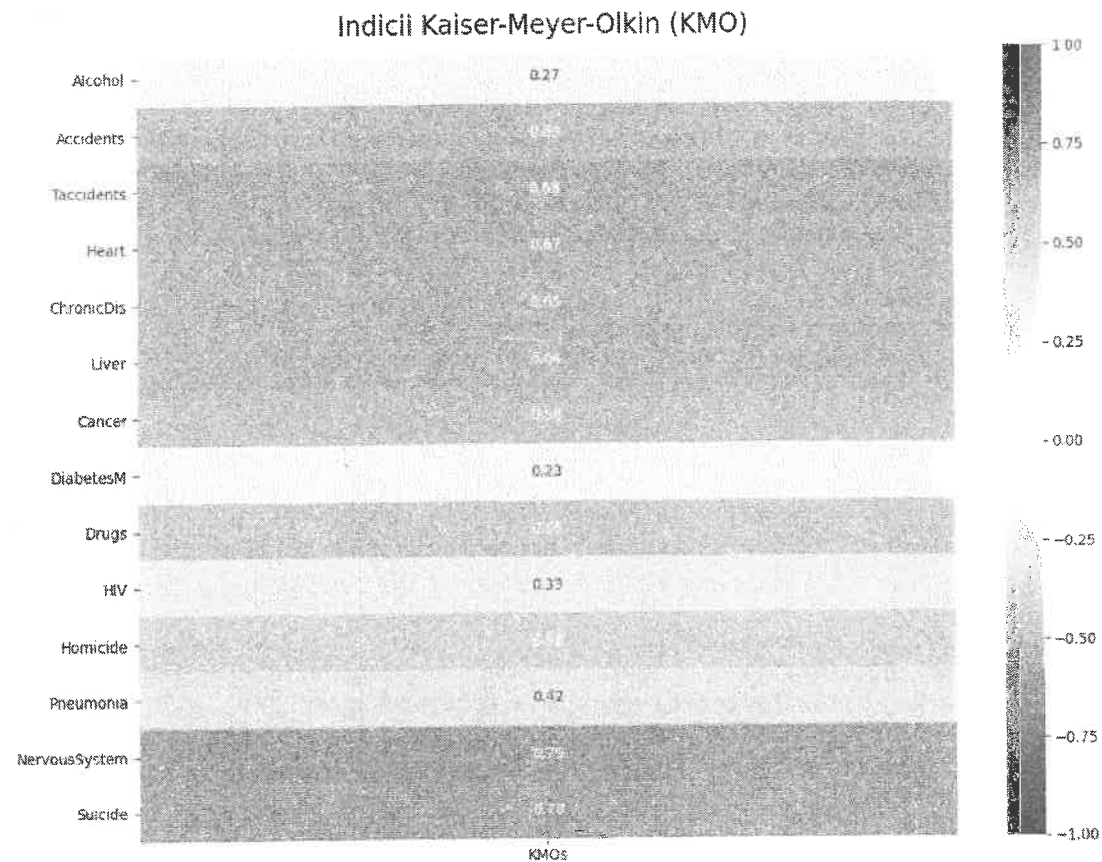


Figura 6.13 Valorile indicilor Kaiser-Meyer-Olkin, corespunzătoare variabilelor inițiale

În urma calculului succesiv a valorii statisticii Bartlett au fost identificați un număr de 5 factori semnificativi, față de cele 14 variabile observate.

Corelația dintre factorii semnificativi extrași și variabilele inițiale este prezentată prin intermediul graficului corelogramă a corelațiilor factoriale (figura 6.14).

De asemenea, prin cercul corelațiilor putem să plasăm variabilele inițiale în câmpul celor mai relevanți factori: Factorii 1-2 și respectiv factorii 1-3 (figura 6.15).

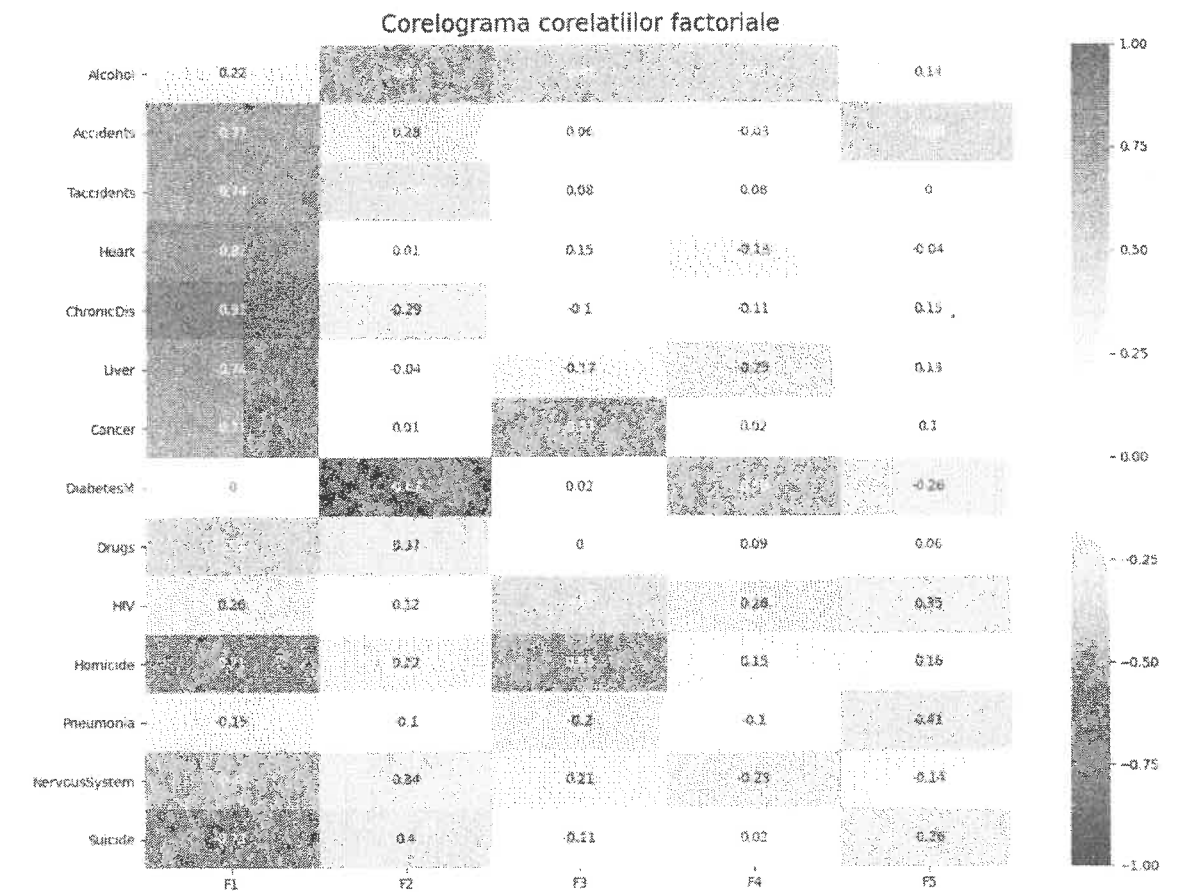


Figura 6.14 Graficul corelogramă corespunzător corelațiilor factoriale

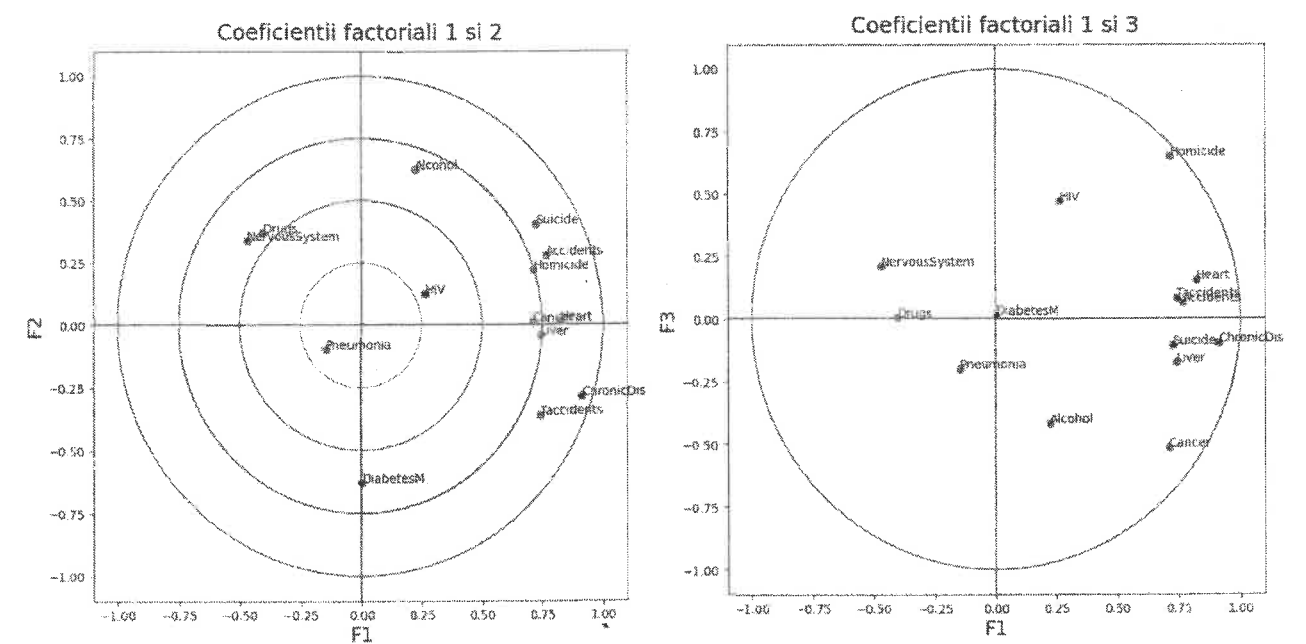


Figura 6.15 Cercul corelațiilor factoriale – variabilele inițiale în câmpurile factorilor F1-F2 și F1-F3

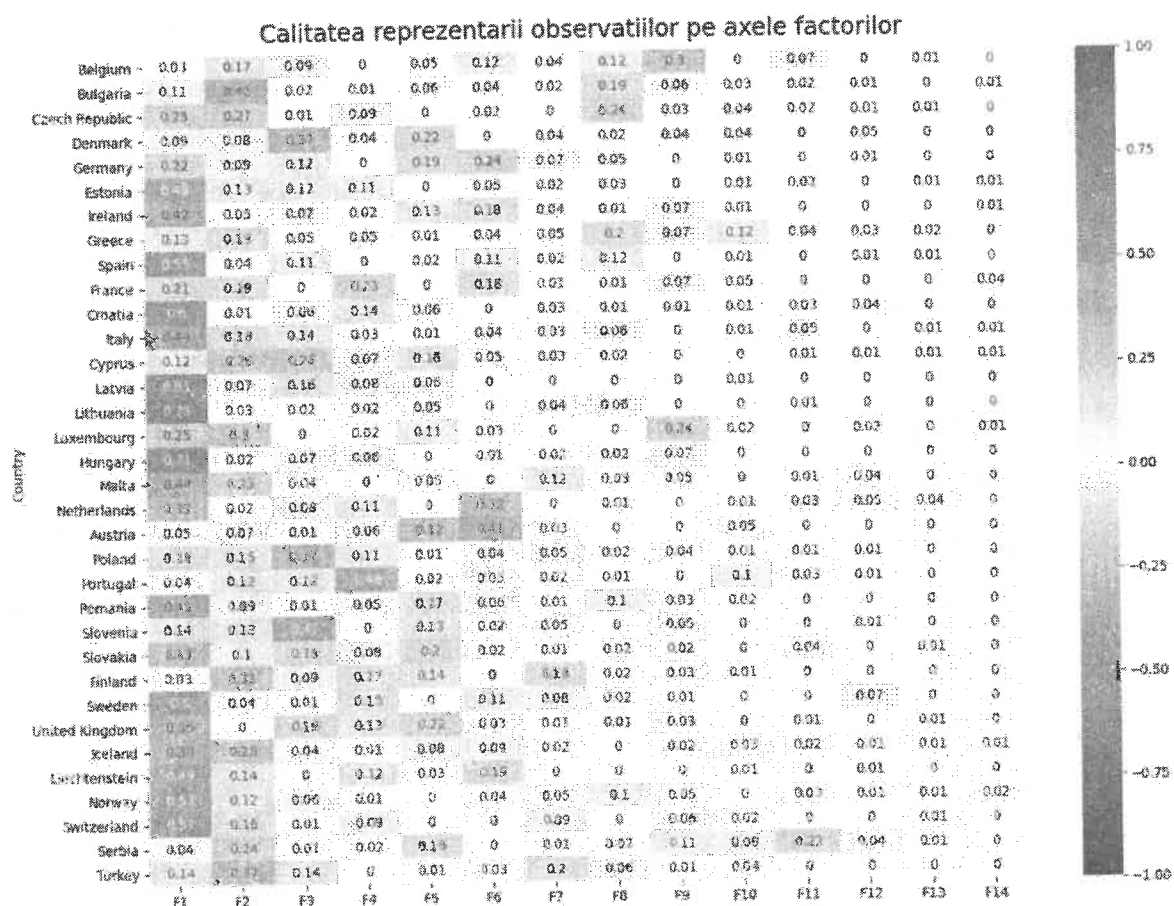


Figura 6.16 Corelograma calității reprezentării observațiilor pe axele factorilor

Graficul corelogramă a calității reprezentării observațiilor pe axele factorilor ne indică cât de bine se regăsesc observațiile, respectiv țările UE, pe axele factorilor comuni semnificativi (figura 6.16).

Corelograma contribuției observațiilor la varianța axelor indică o repartitie difuză și cumva uniformă a țărilor UE la varianța axelor factorilor (figura 6.17).

Primul factor comun semnificativ se identifică cu nivelul dezvoltare și calitatea serviciilor medicale asigurate prin sistemul național de sănătate. În marea majoritate a țărilor UE acesta este factorul determinant întrucât afecțiunile cronice și celelalte afecțiuni asimilabile afecțiunilor cronice (afecțiuni ale inimii, ficatului) au cea mai intensă corelație cu acest prim factor.

Al doilea factor pare să fie constituit de condițiile de mediu și de maniera în care indivizii se raportează la aceste condiții: presiune socială, stres, insecuritate etc.

Cel de al treilea factor identifică mai curând stilul de viață al indivizilor, maniera în care aleg să-și condică viața, împreună cu elemente de microclimat: familie, prieteni, vecinătate, regiune.

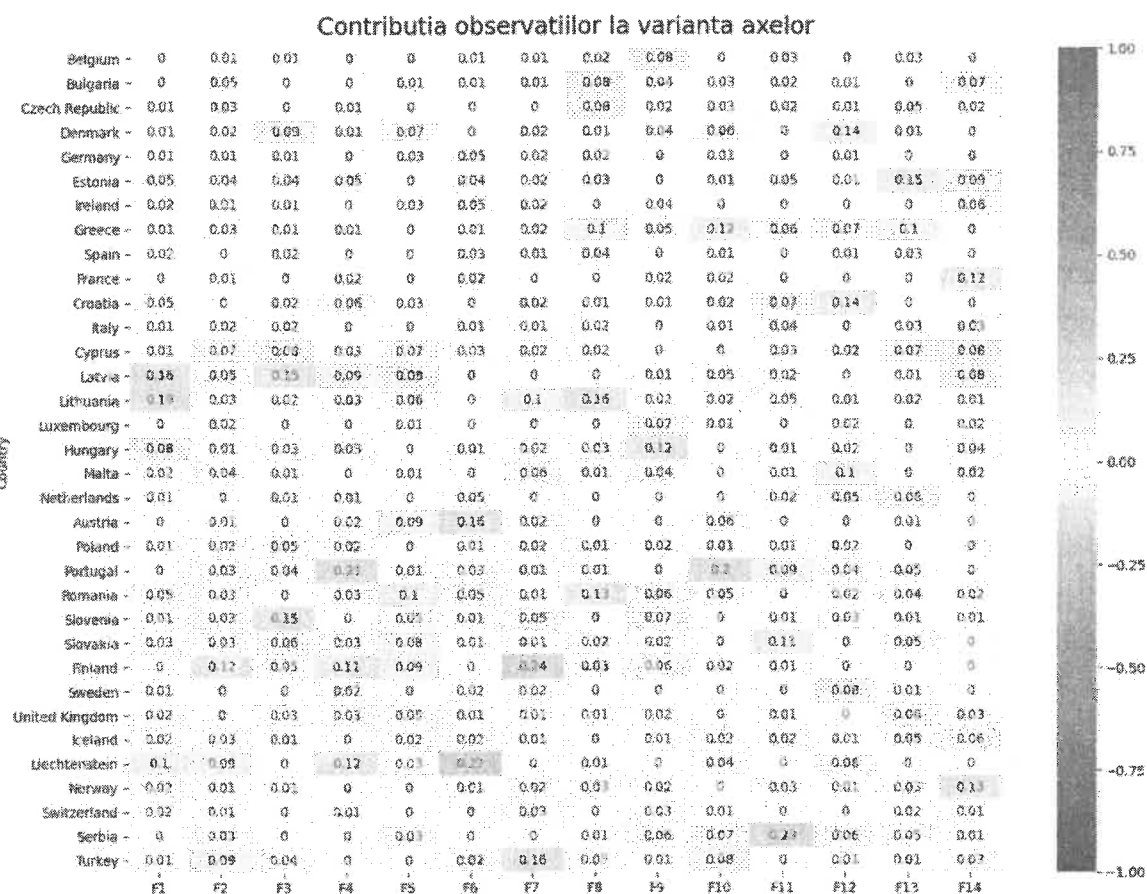


Figura 6.17 Graficul corelogramă a contribuției observațiilor la varianța axelor factorilor

6.3 Analiza corelațiilor canonice (ACC)

Analiza corelațiilor canonice (ACC) descrie relațiile liniare dintre 2 seturi de variabile inițiale sau cauzale, referitoare la același grup de indivizi (observații). A fost propusă și formalizată pentru prima dată de Harold Hotelling în 1936.

În cadrul metodelor statistice de analiză a datelor, analiza canonică (din greaca veche: bară $\kappa\alpha\nu\omega\nu$, tijă de măsurare, riglă) aparține familiei metodelor de regresie. Este o generalizare a analizei de regresie liniară multiplă.

În regresia liniară există o variabilă dependentă (explicată) și un set de variabile independente (explicative). Contrar regresiei liniare, în analiza corelației canonice ambele seturi de variabile joacă același rol. Analiza corelației canonice determină în ce măsură, 2 seturi de variabile reflectă sau nu aceeași realitate.

6.3.1 Datele de prelucrat

Datele sunt prezentate în două matrice X și Y cu n linii respectiv p și q coloane, având forma următoare:

$$X_{(n,p)} = \begin{pmatrix} x_{11} & \dots & x_{1j} & \dots & x_{1p} \\ \vdots & & \vdots & & \vdots \\ x_{i1} & \dots & x_{ij} & \dots & x_{ip} \\ \vdots & & \vdots & & \vdots \\ x_{n1} & \dots & x_{nj} & \dots & x_{np} \end{pmatrix}, \quad Y_{(n,q)} = \begin{pmatrix} y_{11} & \dots & y_{1j} & \dots & y_{1q} \\ \vdots & & \vdots & & \vdots \\ y_{i1} & \dots & y_{ij} & \dots & y_{iq} \\ \vdots & & \vdots & & \vdots \\ y_{n1} & \dots & y_{nj} & \dots & y_{nq} \end{pmatrix}.$$

Datele pot fi centrate sau standardizate. Vom prezenta în continuare varianta de analiză cu datele centrate.

Coloanele tabelului X definesc p variabile cantitative, iar coloanele tabelului Y definesc q variabile cantitative. Se presupune că matricea X este de rang p , iar matricea Y este de rang q .

ACC se desfășoară în k etape având ca rezultat extragerea a k perechi de variabile noi numite variabile canonice (z_i, u_i) , $i = \overline{1, k}$. Variabilele z_i fac parte din spațiul W_1 generat de coloanele matricei X iar variabilele u_i fac parte din spațiul W_2 generat de coloanele matricei Y . Variabilele dintr-o pereche canonică sunt maxim corelate între ele și complet necorelate față de celelalte variabile canonice din același spațiu.

6.3.2 Etapele analizei

Se determină un cuplu de variabile canonice (z_1, u_1) ca o combinație liniară de variabilele cauzale: z_1 este combinație liniară de variabilele $X_1, \dots, X_p$ iar u_1 este combinație liniară de variabilele $Y_1, \dots, Y_q$. Variabilele canonice sunt maxim corelate între ele.

$$z_1 = X \cdot a_1, \quad u_1 = Y \cdot b_1$$

$$z_1 = a_{11} \cdot X_1 + a_{21} \cdot X_2 + \dots + a_{p1} \cdot X_p = X \cdot a_1, \quad \text{unde } a_1 = \begin{bmatrix} a_{11} \\ \vdots \\ a_{p1} \end{bmatrix},$$

$$u_1 = b_{11} \cdot Y_1 + b_{21} \cdot Y_2 + \dots + b_{q1} \cdot Y_q = Y \cdot b_1, \quad \text{unde } b_1 = \begin{bmatrix} b_{11} \\ \vdots \\ b_{q1} \end{bmatrix}.$$

Variabilele canonice sunt maxim corelate între ele, deci înmulțindu-le cu scalarul corelația dintre ele se menține: $R(z_1, u_1) = R(\alpha \cdot z_1, \beta \cdot u_1)$. Pentru a asigura unicitatea lor, ele se determină sub restricția de normalitate:

$$(z_1)^t z_1 = 1 \text{ și } (u_1)^t u_1 = 1.$$

Soluția problemei puse la acest pas este următoarea: z_1 este primul vector propriu al matricei $P_1 \cdot P_2$ corespunzător celei mai mari valori proprii, iar u_1 este primul vector propriu al matricei

$P_2 \cdot P_1$ corespunzător aceleiași valori proprii. P_1 și P_2 sunt proiecțiile liniare ortogonale pe spațiile W_1 și W_2 , generate de coloanele matricelor X și Y . Valoarea proprie α_1 este coeficientul de corelație între variabilele canonice z_1 și u_1 .

Pentru un z_1 oarecare dat, $z_1 \in \mathbb{R}^n$, vectorul din spațiul W_2 care face un unghi minim cu z_1 este proiecția ortogonală a lui z_1 pe spațiul W_2 . Prin urmare, $R(z_1, u_1)$ este maximal dacă u_1 este coliniar cu proiecția ortogonală a lui z_1 pe spațiul W_2 (figura 6.18). Proiecția vectorului z_1 pe spațiul W_2 este în același timp și proiecția acestuia pe axa vectorului u_1 . Deoarece vectorii sunt normați, iar corelația dintre ei este cosinusul unghiului dintre ei, avem:

$$P_2 \cdot z_1 = R(z_1, u_1) \cdot u_1.$$

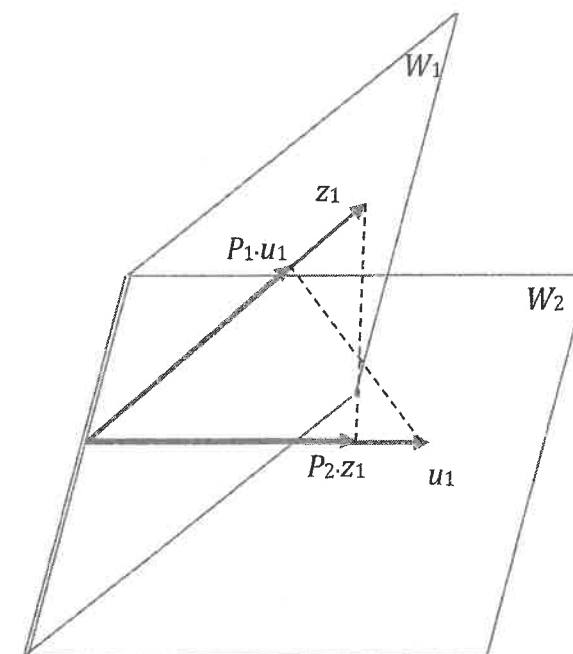


Figura 6.18 Vectorilor proprii z_1 și u_1 și proiecțiile lor în spațiile W_2 și W_1

În mod simetric, pentru u_1 dat, $R(z_1, u_1)$ este maximal dacă z_1 este coliniar cu proiecția ortogonală a lui u_1 pe spațiul W_1 , deci:

$$P_1 \cdot u_1 = R(z_1, u_1) \cdot z_1.$$

Înmulțind la stânga cu P_1 în relația pentru spațiul W_2 obținem:

$$P_1 P_2 \cdot z_1 = P_1 \cdot R(z_1, u_1) \cdot u_1 = R(z_1, u_1) \cdot P_1 \cdot u_1 = R(z_1, u_1) \cdot R(z_1, u_1) \cdot z_1 = R^2(z_1, u_1) \cdot z_1$$

rezultă că

$$P_1 P_2 \cdot z_1 = R^2(z_1, u_1) \cdot z_1,$$

unde $R^2(z_1, u_1)$ este maximal, z_1 este vector propriu al matricei $P_1 P_2$ corespunzător celei mai mari valori proprii $\alpha_1 = R^2(z_1, u_1)$.

Înmulțind la stânga cu P_2 în relația pentru spațiul W_1 obținem:

$$P_2 P_1 \cdot u_1 = P_2 \cdot R(z_1, u_1) \cdot z_1 = R(z_1, u_1) \cdot P_2 \cdot z_1 = R(z_1, u_1) \cdot R(z_1, u_1) \cdot u_1 = R^2(z_1, u_1) \cdot u_1$$

rezultă că

$$P_2 P_1 \cdot u_1 = R^2(z_1, u_1) \cdot u_1,$$

unde $R^2(z_1, u_1)$ este maximal, u_1 este vector propriu al matricei $P_2 P_1$ corespunzător celei mai mari valori proprii $\alpha_1 = R^2(z_1, u_1)$.

În mod iterativ, la pasul 2 este determinat al doilea cuplu de variabile canonice, z_2 și u_2 astfel încât $R^2(z_2, u_2)$ să fie maxim, sub restricțiile:

$$\begin{cases} (z_2)^t z_2 = 1, (u_2)^t u_2 = 1 \\ (z_2)^t z_1 = 0, (u_2)^t u_1 = 0 \end{cases}$$

Procedând ca la pasul 1, obținem că soluțiile problemei sunt vectorii proprii ai matricelor $P_1 P_2$ și $P_2 P_1$ corespunzător celei de-a doua valori proprii ca mărime. Valoarea proprie $\alpha_2 = R^2(z_2, u_2)$ este coeficientul de corelație între variabilele canonice z_2 și u_2 :

$$P_1 P_2 \cdot z_2 = \alpha_2 \cdot z_2,$$

$$P_2 P_1 \cdot u_2 = \alpha_2 \cdot u_2.$$

În plus față de pasul 1, trebuie demonstrat că z_2 și z_1 respectiv u_2 și u_1 nu sunt corelate, respectiv:

$$(z_2)^t z_1 = 0 \text{ și } (u_2)^t u_1 = 0.$$

Dezvoltăm produsul $\alpha_2 (z_2)^t z_1$ astfel:

$$\alpha_2 (z_2)^t z_1 = (z_2)^t P_2 P_1 z_1, \text{ deoarece } (P_1 P_2 \cdot z_2)^t = (\alpha_2 \cdot z_2)^t.$$

Rezultă că $\alpha_2 (z_2)^t z_1 = (z_2)^t P_1 P_2 P_1 z_1$, deoarece proiecția unui vector în propriul spațiu este vectorul însuși:

$$P_1 z_2 = z_2, \text{ sau } (z_2)^t P_1 = (z_2)^t$$

De asemenea,

$$\alpha_2 (z_2)^t z_1 = (z_2)^t P_1 P_2 z_1, \text{ deoarece } P_1 z_1 = z_1$$

și $\alpha_2 (z_2)^t z_1 = \alpha_1 (z_2)^t z_1$, deoarece $P_1 P_2 \cdot z_1 = \alpha_1 \cdot z_1$.

În concluzie $\alpha_2 (z_2)^t z_1 = \alpha_1 (z_2)^t z_1$. Cum însă $\alpha_1 \neq \alpha_2$, rezultă că $(z_2)^t z_1 = 0$.

În mod asemănător:

$$\alpha_2 (u_2)^t u_1 = (u_2)^t P_1 P_2 u_1 = (u_2)^t P_2 P_1 P_2 u_1 = (u_2)^t P_2 P_1 u_1 = \alpha_1 (u_2)^t u_1,$$

rezultând că $(u_2)^t u_1 = 0$.

Procedând în aceeași manieră, la pasul k este determinat de variabile canonice (z_k, u_k) , astfel încât acestea să fie maxim corelate, varianța lor să fie 1, iar coeficientul de corelație cu variabilele canonice determinate la pașii anteriori să fie 0.

$$\begin{cases} \underset{z_k, u_k}{\text{Max}}(R^2(z_k, u_k)) \\ (z_k)^t z_k = 1, (u_k)^t u_k = 1 \\ (z_k)^t z_i = 0, (u_k)^t u_i = 0, \quad i = \overline{1, k-1} \end{cases}$$

Soluțiile problemei sunt vectorii proprii ai matricelor $P_1 P_2$ și $P_2 P_1$ corespunzător valorii proprii de ordinul k .

Variabilele canonice ale aceleiași grupe sunt două câte două necorelate. Un cuplu de variabile canonice se mai numește și rădăcină canonică (*root*). Se poate arăta că și variabilele canonice de ordin diferit din grupe diferite sunt necorelate:

$$(z_r)^t u_k = (P_1 z_r)^t u_k = (z_r)^t P_1 u_k = (z_r)^t R(z_k, u_k) z_k = R(z_k, u_k) (z_r)^t z_k = 0,$$

$$\forall r \text{ în } \{1, 2, \dots, k-1\}$$

6.3.3 Factorii canonici

Variabilele canonice sunt combinații liniare de variabilele celor două grupe, astfel:

$$z_i = X a_i, u_i = Y b_i, \quad i = \overline{1, k-1}$$

unde a_i și b_i sunt factorii canonici corespunzători.

Se poate observa că matricele $P_1 P_2$ și $P_2 P_1$ sunt de ordinul n . În imensa majoritate a situațiilor numărul de indivizi este mult mai mare decât numărul de variabile. Prin urmare, operațiunile de determinare a vectorilor și valorilor proprii sunt foarte costisitoare pe astfel de matrice.

La etapa k se determină z_k , vector propriu al matricei $P_1 P_2$. Deci:

$$P_1 P_2 z_k = R^2(z_k, u_k) z_k.$$

Deoarece $P_1 = X(X^t X)^{-1} X^t$ și $P_2 = Y(Y^t Y)^{-1} Y^t$ (a se vedea regresia multiplă), iar $z_k = X a_k$, obținem:

$$X(X^t X)^{-1} X^t Y(Y^t Y)^{-1} Y^t X a_k = R^2(z_k, u_k) \cdot X a_k.$$

Înmulțind această relație la stânga cu $(X^t X)^{-1} X^t$ rezultă:

$$(X^t X)^{-1} X^t Y(Y^t Y)^{-1} Y^t X a_k = R^2(z_k, u_k) \cdot a_k.$$

Dacă efectuăm următoarele notații:

$$V_{11} = \frac{1}{n} X^t X,$$

$$V_{22} = \frac{1}{n} Y^t Y,$$

$$V_{12} = \frac{1}{n} X^t Y,$$

$$V_{21} = \frac{1}{n} Y^t X,$$

obținem:

$$V_{11}^{-1} V_{12} V_{22}^{-1} V_{21} a_k = R^2(z_k, u_k) \cdot a_k.$$

În mod asemănător se obține relația:

$$V_{22}^{-1} V_{21} V_{11}^{-1} V_{12} b_k = R^2(z_k, u_k) \cdot b_k.$$

Se poate observa că V_{11} este matricea de covarianță între variabilele din grupa X, V_{22} este matricea de covarianță între variabilele din grupa Y, V_{12} este matricea de covarianță între variabilele din grupa X și grupa Y, iar V_{21} este matricea de covarianță între variabilele din grupa Y și grupa X.

În concluzie a_k este vectorul propriu de ordinul k al matricei

$$V_{11}^{-1} V_{12} V_{22}^{-1} V_{21} \text{ corespunzător valorii proprii } \alpha_k = R^2(z_k, u_k),$$

iar b_k este vectorul propriu de ordinul k al matricei

$$V_{22}^{-1} V_{21} V_{11}^{-1} V_{12} \text{ corespunzător aceleași valori proprii.}$$

Deoarece aceste matrice au dimensiunile p, respectiv q, este mult mai convenabilă această modalitate de determinare a variabilelor canonice. Numărul de etape se determină după relația $m = \min(p, q)$.

6.3.4 Legăturile dintre factori

Știm că $P_2 z_k = R(z_k, u_k) u_k$. Înlocuind $P_2 = Y(Y^t Y)^{-1} Y^t$, $z_k = X a_k$ și $u_k = Y b_k$, rezultă:

$$Y(Y^t Y)^{-1} Y^t X a_k = R(z_k, u_k) Y b_k.$$

Dacă înmulțim această relație la stânga cu $(Y^t Y)^{-1} Y^t$, rezultă:

$$V_{22}^{-1} V_{21} a_k = R(z_k, u_k) b_k.$$

Relațiile între factori sunt deci următoarele:

$$b_k = \frac{1}{R(z_k, u_k)} V_{22}^{-1} V_{21} a_k$$

$$a_k = \frac{1}{R(z_k, u_k)} V_{11}^{-1} V_{12} b_k$$

6.3.5 Variația explicată și redundanța informațională

Cantitatea de variație explicată de fiecare pereche de variabile canonice, pentru fiecare din grupe, este dată de suma corelațiilor dintre variabilele canonice și variabilele grupelor:

$$VX_k = \sum_{j=1}^p R(z_k, X_j)^2, k = \overline{1, m}$$

$$VY_k = \sum_{j=1}^q R(u_k, Y_j)^2, k = \overline{1, m}$$

unde $R(z_k, X_j)^2$ este coeficientul de determinare (raportul de corelație) între variabila canonică z_k a cuplului k și variabila X_j din prima grupă (cu elementele în coloana j a matricei X). În mod similar, $R(u_k, Y_j)^2$ reprezintă coeficientul de determinare pentru grupa a doua și variabila u_k a cuplului canonic. Proporțional cu numărul de variabile inițiale din fiecare grupă, valorile sunt: $\frac{VX_k}{p}$ și $\frac{VY_k}{q}$.

Varianța totală explicată de cele m rădăcini canonice este:

$$VX = \sum_{k=1}^m VX_k$$

pentru prima grupă și

$$VY = \sum_{k=1}^m VY_k$$

pentru cea de a doua grupă.

Redundanța este dată de informația comună celor două grupe extrasă de rădăcinile (cuplurile) canonice. Informația comună este reflectată de corelațiile canonice. Dacă avem o anumită cantitate de informație extrasă de o variabilă canonică dintr-o grupă, partea din această informație care se găsește și în cealaltă grupă este aflată cu ajutorul corelației canonice astfel:

$$SX_k = VX_k \cdot \alpha_k, \quad k = \overline{1, m}$$

$$SY_k = VY_k \cdot \alpha_k, \quad k = \overline{1, m}$$

Redundanța la nivelul celor m rădăcini canonice este:

$$SX = \sum_{k=1}^m SX_k$$

$$SY = \sum_{k=1}^m SY_k$$

6.3.6 Standardizarea factorilor canonici

Interpretarea factorilor canonici este mai ușor de făcut dacă aceștia sunt standardizați. Factorii standardizați sunt calculați astfel:

$$as_{ik} = a_{ik} \cdot \frac{\sigma_{x_i}}{\sigma_{z_k}}$$

unde $i = \overline{1, p}, k = \overline{1, m}$, și

$$bs_{ik} = b_{ik} \cdot \frac{\sigma_{y_i}}{\sigma_{u_k}}$$

unde $i = \overline{1, q}, k = \overline{1, m}$.

Interpretarea coeficienților standardizați este asemănătoare celei de la regresia multiplă: creșterea cu o unitate a abaterii standard a variabilelor X_i sau Y_i generează o creștere cu as_{ik} sau bs_{ik} a abaterii standard a variabilelor z_k și u_k .

6.3.7 Relevanța rădăcinilor canonice. Testul de relevanță Bartlett χ^2

Testul Bartlett χ^2 este cel mai utilizat test statistic pentru evaluarea corelațiilor canonice. Rezultatul testului pentru o oarecare rădăcină canonică arată dacă rădăcina respectivă indică dependență între cele două grupe de variabile sau dimpotrivă, cele două grupe de variabile sunt independente.

Ipoteza H_0 : coeficientul de corelație $R(z_k, u_k)$ indică existența legăturii liniare dintre cele două grupe de variabile.

Ipoteza alternativă, H_1 : coeficientul de corelație $R(z_k, u_k)$ indică lipsa legăturii.

Pentru rădăcina canonică (z_k, u_k) testul se aplică astfel:

- a) Se calculează numărul gradelor de libertate aferent rădăcinii canonice de ordin k :

$$df_k = (p - i + 1)(q - i + 1)$$

unde p și q sunt numărul de variabile din prima și respectiv din a doua grupă.

- b) Se calculează statistica testului:

$$\chi_k^2 = \left(-n + 1 + \frac{p + q + 1}{2} \right) \log(1 - \lambda_k)$$

unde n este numărul de instanțe, iar λ_k este un indicator numit lambda Wilks care se calculează astfel:

$$\lambda_k = \prod_{i=k}^m (1 - R(z_i, u_i)^2)$$

unde m este numărul rădăcinilor canonice.

- c) Se determină valoarea critică a testului folosind distribuția χ^2 :

$$\chi_{c_k}^2(1 - \alpha, df_k), \text{ pentru un prag de semnificație } \alpha.$$

- d) Este aplicat testul astfel:

dacă $\chi_k^2 \geq \chi_{c_k}^2(1 - \alpha, df_k)$ atunci ipoteza H_0 este acceptată cu un nivel de încredere $1 - \alpha$, altfel este respinsă.

6.3.8 ACC - studiu de caz

Pentru studiul analizei corelațiilor canonice vom porni de la două seturi de date ce privesc țările Uniunii Europene: primul se referă la producția de energie electrică din diverse surse (arderea combustibililor fosili, din fisiune nucleară controlată, energie hidroelectrică și alte surse – tabelul 6.3), iar al doilea are în vedere consumul de energie electrică în sectoarele de bază ale economiei (industrie, transporturi, servicii, sectorul rezidențial – tabelul 6.4).

Tabelul 6.3 Producția de energie electrică după sursă în țările UE

Cod	Tara	Combustie	Nucleara	Hidro	Altele
AT	Austria	34.76	0	60.15	5.08
BE	Belgium	49.96	32.35	7.78	9.91
BG	Bulgaria	45.6	18.86	30.39	5.15
CH	Switzerland	5.24	18	75.91	0.85
CY	Cyprus	94.07	0	0	5.93
CZ	CzechRepublic	58.77	19.43	10.95	10.85
DE	Germany	49.18	13.04	7.02	30.76
DK	Denmark	72.14	0	0.07	27.79
EE	Estonia	95.86	0	0.22	3.93
EL	Greece	70.11	0	19.97	9.92
ES	Spain	49.58	7.29	18.21	24.92
FI	Finland	63.8	16.2	18.84	1.17
FR	France	22.99	50.71	20.46	5.84
HR	Croatia	46.13	0	51.95	1.92
HU	Hungary	73.95	22.19	0.59	3.27
IE	Ireland	74.85	0	6.22	18.93
IT	Italy	70.11	0	20.21	9.68
LT	Lithuania	71.04	0	24.54	4.43
LU	Luxembourg	29.95	0	65.82	4.24
LV	Latvia	37.19	0	61.63	1.17
MT	Malta	100	0	0	0
NL	Netherlands	88.97	1.91	0.14	8.98
NO	Norway	4.88	0	93.64	1.48
PL	Poland	89.66	0	7.02	3.32
PT	Portugal	52.18	0	26.92	20.9
RO	Romania	58.45	7.09	32.51	1.95
SE	Sweden	23.9	24.62	45.9	5.57

Cod	Tara	Combustie	Nucleara	Hidro	Altele
SI	Slovenia	39.49	20.86	39.27	0.38
SK	Slovakia	44.4	23.11	31.95	0.53
TR	Turkey	65.18	0	31.97	2.86
UK	UnitedKingdom	77.83	11.63	4.7	5.84

Tabelul 6.4 Consumul de energie electrică după sectoarele de activitate în țările UE

Cod	Tara	Industrie	Transporturi	Rezidential	Servicii
AT	Austria	32.31	32.16	25.22	10.31
BE	Belgium	31.51	29.03	25.28	14.18
BG	Bulgaria	29.33	33.26	25.98	11.43
CH	Switzerland	18.33	34.6	30.05	17.02
CY	Cyprus	13.19	57.14	16.48	13.19
CZ	CzechRepublic	35.32	25.36	26.69	12.62
DE	Germany	27.95	28.58	28.65	14.83
DK	Denmark	16.59	35.29	33.45	14.68
EE	Estonia	20.21	28.01	36.52	15.25
EL	Greece	19.03	44.87	25.4	10.7
ES	Spain	26.79	42.71	18.9	11.61
FI	Finland	47.72	20.44	23.82	8.02
FR	France	20.97	33.77	29.56	15.7
HR	Croatia	22.66	33.83	25.78	17.73
HU	Hungary	18	27.21	35.37	19.42
IE	Ireland	16.67	40.54	28.13	14.67
IT	Italy	25.59	34.57	25.86	13.97
LT	Lithuania	19.44	33.48	34.13	12.96
LU	Luxembourg	17.52	61.21	11.45	9.81
LV	Latvia	18.73	29.44	36.74	15.09
MT	Malta	11.36	63.64	13.64	11.36
NL	Netherlands	28.3	29.75	22.78	19.17
NO	Norway	32.61	28.24	23.07	16.09
PL	Poland	24.6	28.13	33.6	13.67
PT	Portugal	30.47	41.66	16.85	11.02
RO	Romania	31.47	22.87	37.05	8.6
SE	Sweden	37.33	25.63	22.4	14.64
SI	Slovenia	26.34	36.83	26.34	10.49
SK	Slovakia	37.99	23.14	20.17	18.69
TR	Turkey	35.31	23.3	33.02	8.37
UK	UnitedKingdom	20.12	37.44	31.79	10.66

Ne propunem să identificăm ce legătură există între cele două seturi de date care se referă la aceleași observații: țările Uniunii Europene.

Programul principal de prelucrare a setului de date (mainACC.py) conține următoarea implementare.

'''

Program principal pentru Analiza Corelatiilor Canonice (ACC)
Date de intrare: doua tabele de date referitoare la aceleasi instante
Jonctiunea intre tabele se face pe baza campurilor index

'''

```
import pandas as pd
import numpy as np
import util.util as utl
import visual.visual as vi
```

try:

```
    fisier_1 = 'dataIN/ProductieEnergie.csv'
    fisier_2 = 'dataIN/ConsumEnergie.csv'
    t1 = pd.read_csv(fisier_1, index_col=0)
    nume_variabile1 = [str(v) for v in t1.columns]
    t2 = pd.read_csv(fisier_2, index_col=0)
    nume_variabile2 = [str(v) for v in t2.columns]
```

jonctiune

```
t = t1.merge(t2, left_index=True, right_index=True)
```

crearea seturilor de date ca masive numpy ndarray

```
x = t.iloc[:, 1:5].values
y = t.iloc[:, 6:10].values
utl.inlocuireNaN(x)
utl.inlocuireNaN(y)
varX = t.columns[1:5]
varY = t.columns[6:10]
numeObs = t.index
```

Aplicare model

```
r, r2, z, u = utl.cca(x, y)
```

Construire/Aplicare model

```
n, p = np.shape(x)
q = np.shape(y)[1]
m = min(p, q)
pd.DataFrame(data=z, index=numeObs, columns=[str(i) for i in range(1,
m+1)]).to_csv("dataOUT/z.csv")
pd.DataFrame(data=u, index=numeObs, columns=[str(i) for i in range(1,
m+1)]).to_csv("dataOUT/u.csv")
p_value, chi2, dof = utl.bartlett_wilks_test(r, n, p, q, m)
# Salvare corelatii canonice si test de semnificatie
root_index = ['root' + str(i) for i in range(1, m + 1)]
chi2_tabel = pd.DataFrame(data={"r": r, "r2": r * r, "p_value": p_value, "chi2":
chi2, "dof": dof},
```

```
                        index=root_index)
```

```
chi2_tabel.to_csv("dataOUT/chi2.csv")
```

```
chi2_ = pd.DataFrame(chi2_tabel['p_value'])
```

```
vi.corelograma(chi2_, "Test de semnificatie Bartlett-Wilks")
```

Calcul corelatii dintre variabilele canonice seminficative

si variabilele observate

```
r_xz = np.corrcoef(x, z[:, :m_], rowvar=False)[p, p:]
r_yz = np.corrcoef(y, z[:, :m_], rowvar=False)[q, q:]
r_xu = np.corrcoef(x, u[:, :m_], rowvar=False)[p, p:]
r_yu = np.corrcoef(y, u[:, :m_], rowvar=False)[q, q:]
r_xz2 = r_xz*r_xz
r_yu2 = r_yu*r_yu
```

```

r_xz_tab = pd.DataFrame(r_xz, columns=['z' + str(i) for i in range(1, m_ + 1)],
index=varX)
r_yu_tab = pd.DataFrame(r_yu, columns=['u' + str(i) for i in range(1, m_ + 1)],
index=varY)
r_xz2_tab = pd.DataFrame(r_xz2, columns=['z' + str(i) for i in range(1, m_ + 1)],
index=varX)
r_yu2_tab = pd.DataFrame(r_yu2, columns=['u' + str(i) for i in range(1, m_ + 1)],
index=varY)
r_xz_tab.to_csv("dataOUT/r_xz.csv")
r_yu_tab.to_csv("dataOUT/r_yu.csv")
vi.corelograma(r_xz_tab, "Corelograma x-z")
vi.corelograma(r_yu_tab, "Corelograma y-u")
vi.corelograma(r_xz2_tab, "Varianta comuna x-z")
vi.corelograma(r_yu2_tab, "Varianta comuna y-u")
select_root = root_index[:m_]
i1 = root_index.index(select_root[0])
xlabel = select_root[0]
for i in range(1, len(select_root)):
    i2 = root_index.index(select_root[i])
    ylabel = select_root[i]
    vi.cercul_corelatiilor_2(r_xz_tab.iloc[:, [i1, i2]],
        r_yu_tab.iloc[:, [i1, i2]],
        xlabel, ylabel,
        "Corelatii cu variabilele observate")

# Calcul varianta comuna si redundanta
vx = np.sum(r_xz * r_xz, axis=0)
vy = np.sum(r_yu * r_yu, axis=0)
sx = vx * r2[:m_]
sy = vy * r2[:m_]

# Tabelare varianta/redundanta
t_vr = pd.DataFrame(data={
    'vx': vx, 'vy': vy, 'vx(%)': vx * 100 / p, 'vy(%)': vy * 100 / q,
    'sx': sx, 'sy': sy, 'sx(%)': sx * 100 / p, 'sy(%)': sy * 100 / q
}, index=root_index[:m_])
t_vr.to_csv("dataOUT/var_red.csv")
if n < 500:
    xlabel = 'z1/u1'
    i1 = root_index.index(select_root[0])
    for i in range(1, len(select_root)):
        i2 = root_index.index(select_root[i])
        ylabel = 'z' + str(i2 + 1) + '/u' + str(i2 + 1)
        vi.biplot(z[:, [i1, i2]], u[:, [i1, i2]],
            'Biplot instante in campul ' + xlabel + ' - ' + ylabel,
            list(numeObs), list(numeObs))

vi.show()

except Exception as ex:
    print ("Eroare!", ex.with_traceback(), sep="\n")

```

Funcțiile utilizate pentru calculul corelațiilor canonice și a statisticii Bartlett-Wilks sunt implementate în fișierul util.py prezentat în continuare.

```

import numpy as np
import scipy.stats as sts
import sklearn.preprocessing as pp

```

```

# Inlocuire celule NA(not available, not applicable) sau NaN (not-a-number)
# cu media pe coloana
def inlocuireNaN(X):
    avgs = np.nanmean(X, axis=0)
    pos = np.where(np.isnan(X))
    print(pos[:])
    X[pos] = avgs[pos[1]]

    return X

# Analiza corelatiilor canonice (ACC)
def cca(x, y):
    n, p = np.shape(x)
    q = np.shape(y)[1]
    x = pp.StandardScaler(with_std=False).fit_transform(x)
    y = pp.StandardScaler(with_std=False).fit_transform(y)
    vx = np.cov(x, rowvar=False)
    vy = np.cov(y, rowvar=False)
    cov = np.cov(x, y, rowvar=False)
    vxy = cov[:, p, :]
    vxy = np.transpose(vxy)
    vx_inv = np.linalg.inv(vx)
    vy_inv = np.linalg.inv(vy)
    h1 = vx_inv @ vxy
    h2 = vy_inv @ vxy
    m = min(p, q)
    if p == m:
        h = h1 @ h2
        valp, vecp = np.linalg.eig(h)
        k_inv = [k for k in reversed(np.argsort(valp))]
        r2 = valp[k_inv]
        a = vecp[:, k_inv]
        r = np.sqrt(r2)
        b = (h2 @ a) @ np.diag(1 / r)
        z = x @ a
        u = y @ b
    else:
        h = h2 @ h1
        valp, vecp = np.linalg.eig(h)
        k_inv = [k for k in reversed(np.argsort(valp))]
        r2 = valp[k_inv]
        b = vecp[:, k_inv]
        r = np.sqrt(r2)
        a = (h1 @ b) @ np.diag(1 / r)
        z = x @ a
        u = y @ b
    z = pp.normalize(z, axis=0)
    u = pp.normalize(u, axis=0)
    return r, r2, z, u

def bartlett_wilks_test(r, n, p, q, m):
    r_inv = np.flipud(r)
    l = np.flipud(np.cumprod(1 - r_inv * r_inv))
    dof = (p - np.arange(m)) * (q - np.arange(m))
    chi2 = (-n + 1 + (p + q + 1) / 2) * np.log(l)
    p_value = 1 - sts.chi2.cdf(chi2, dof)
    return p_value, chi2, dof

```

Rezultatele obținute sunt prezentate în continuare.

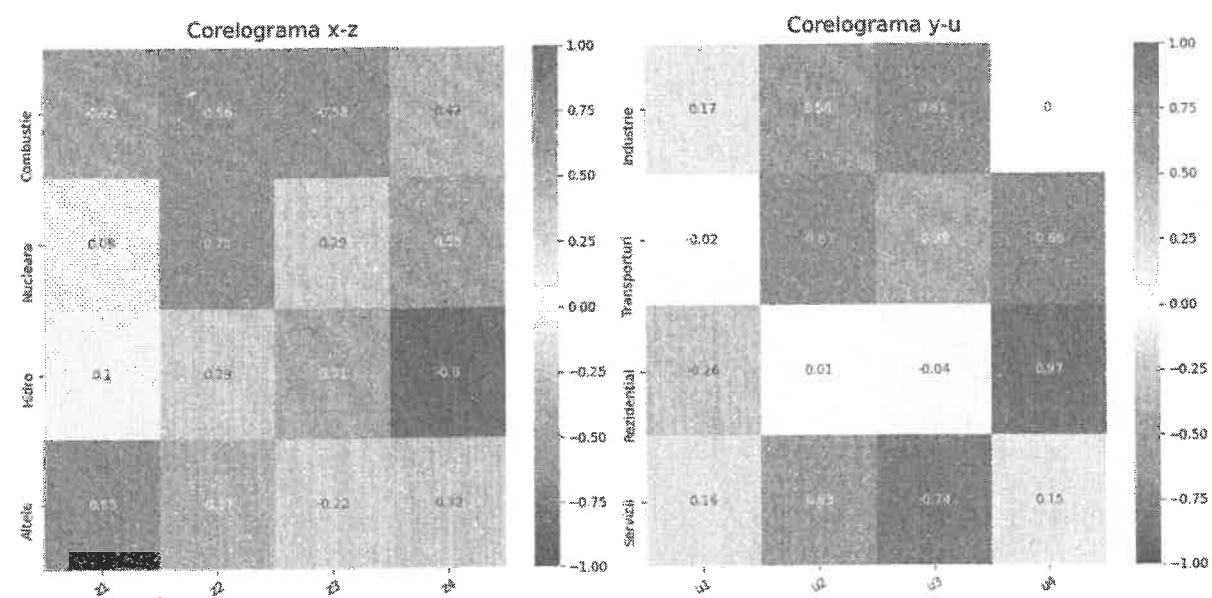


Figura 6.19 Corelogramele corelațiilor variabilelor inițiale cu rădăcinile canonice corespunzătoare

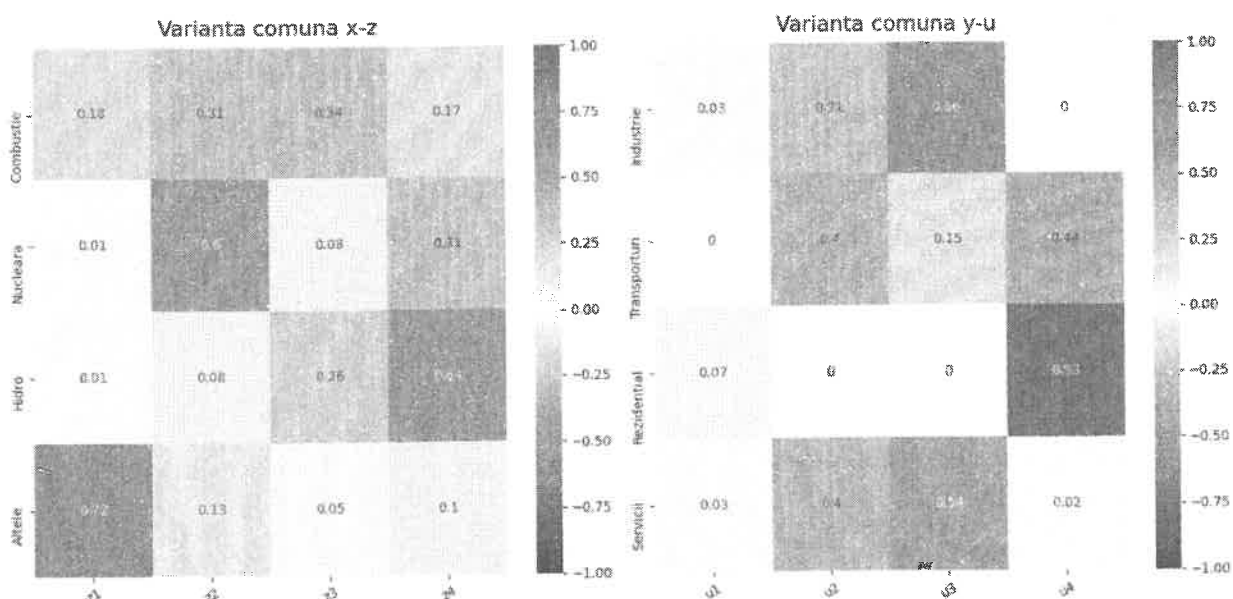


Figura 6.20 Corelogramele varianței comune a variabilelor inițiale și rădăcinilor canonice corespunzătoare

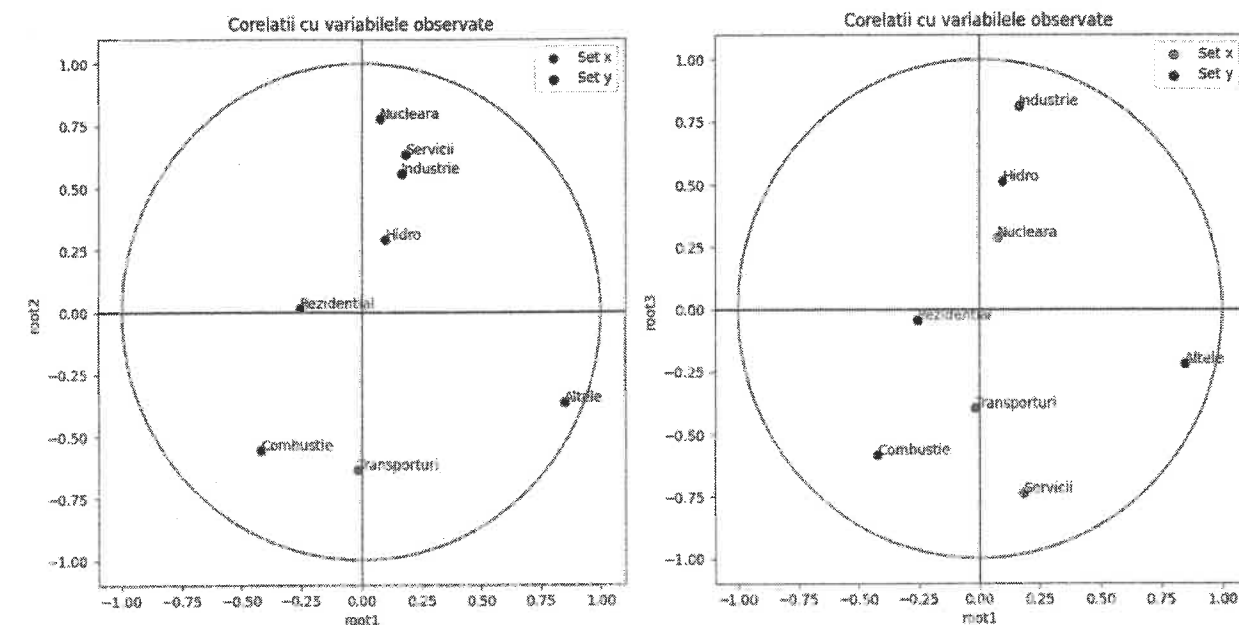


Figura 6.21 Cercul corelațiilor variabilelor inițiale din cele doua grupe în câmpul perechilor canonice root1 - root2 și respectiv root1 - root3

Din corelogramele corelațiilor variabilelor inițiale cu rădăcinile canonice corespunzătoare (figurile 6.19) reiese foarte clar că sectoarele industrie (coeficient de corelație 0.56 cu u_2) și servicii (coeficient de corelație 0.63 cu u_2) depind hotărâtor de sursa nucleară de energie (coeficient de corelație 0.78 cu z_2). De asemenea, din varianțele comune ale variabilelor inițiale și rădăcinilor canonice corespunzătoare (figurile 6.20) se poate concluziona că sectorul industrial (corelație 0.66 cu u_3) și cel al serviciilor (corelație 0.54 cu u_3) depind în continuare semnificativ de energia obținută din arderea combustibililor fosili (corelație 0.34 cu z_3). Cercul corelațiilor (figurile 6.21 și 6.22) susțin observațiile anterioare.

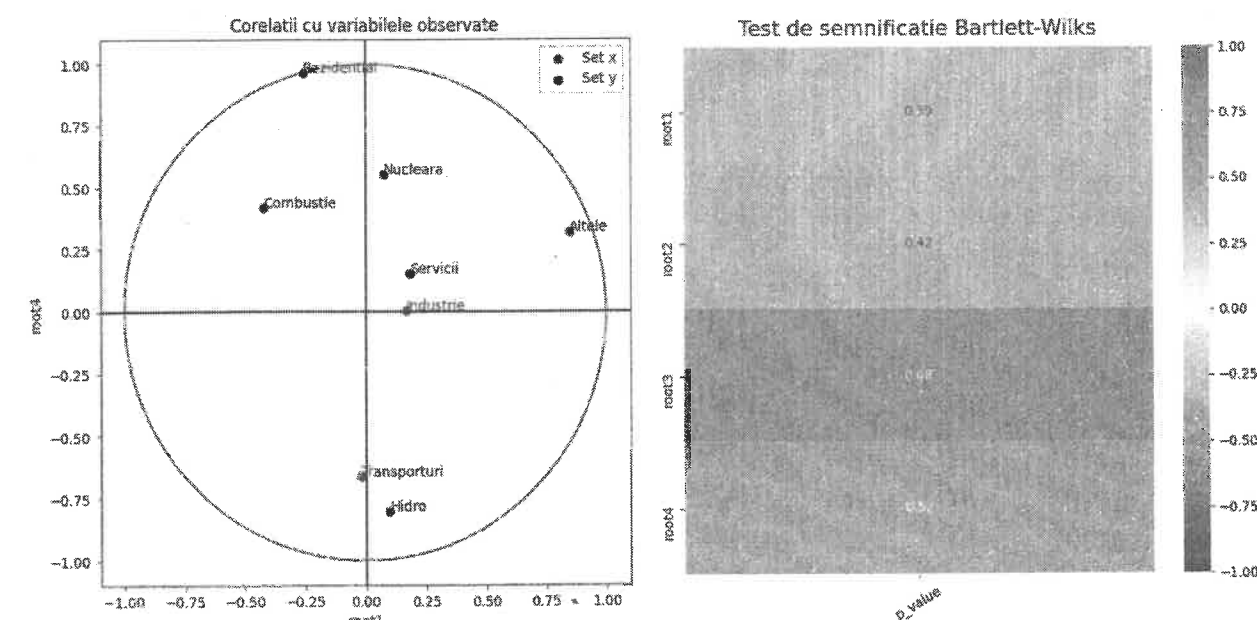


Figura 6.22 Cercul corelațiilor variabilelor inițiale din cele doua grupe în câmpul perechilor canonice root1 - root4

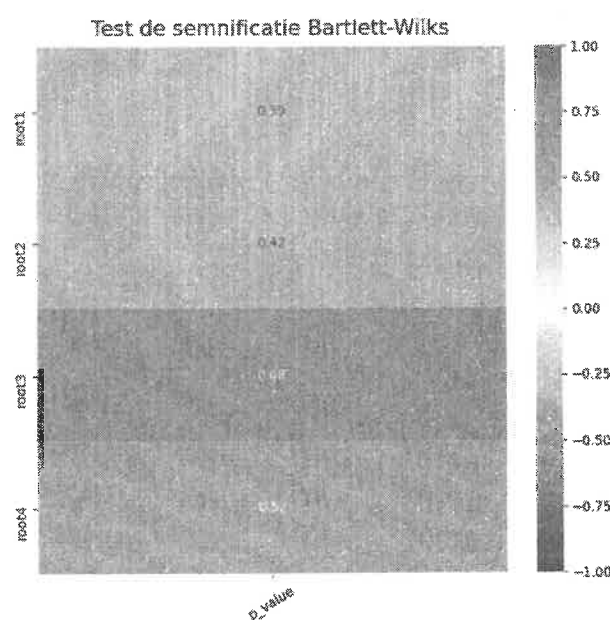


Figura 6.23 Graficul corelogramă a testului de semnificație Barlett-Wilks

Valorile statisticii Barlett-Wilks (figura 6.23) confirmă faptul că principalii consumatori al energiei obținute în urma arderii combustibililor fosili Europa sunt sectoarele industrial și cel al serviciilor. Sectorul rezidențial este totuși preponderent alimentat cu hidro-energie.

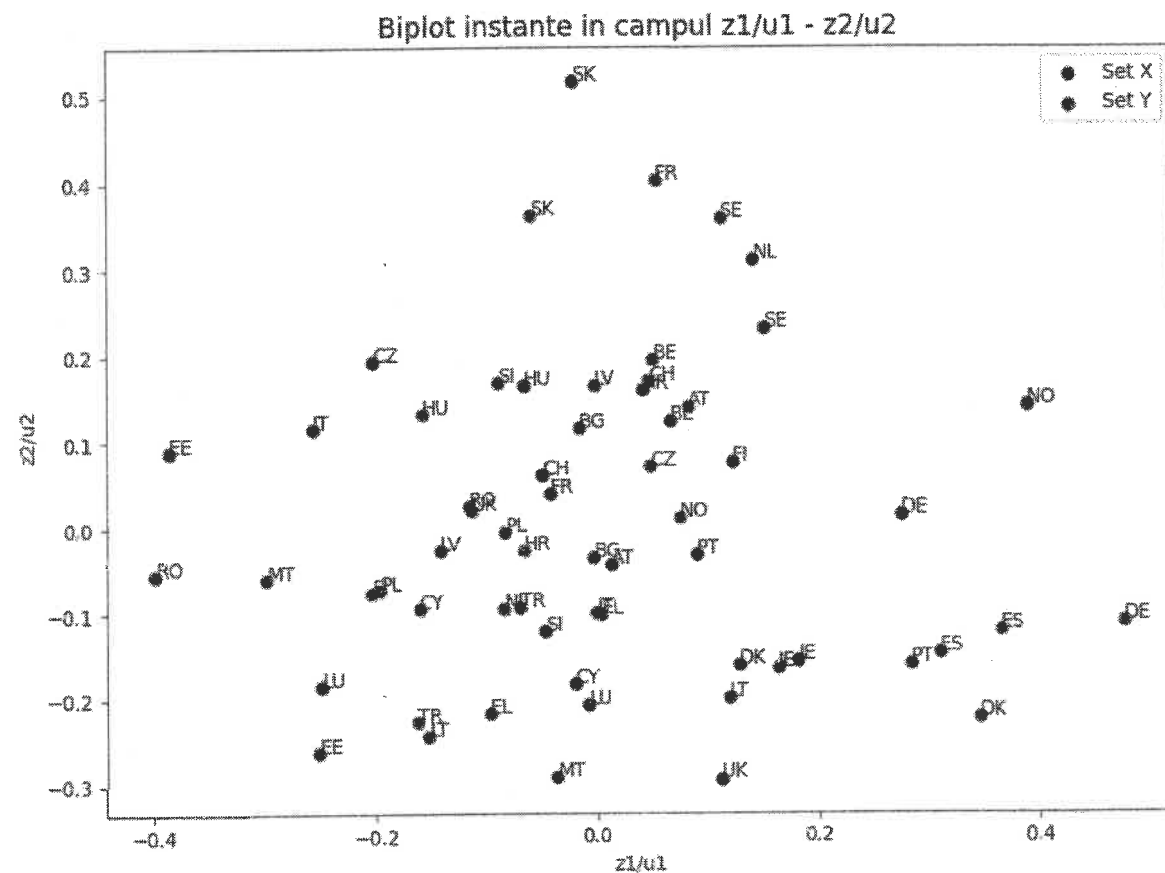


Figura 6.24 Graficul nor de puncte a instanțelor și variabilelor observate în câmpul perechilor canonice 1 și 2

Figurile 6.24, 6.25 și 6.26 prezintă distribuția țărilor UE în câmpul perechilor canonice. Se poate remarca o mișcare către producția de hidro-energie în rândul multor țări ce dispun de o astfel de resursă exploatabilă.

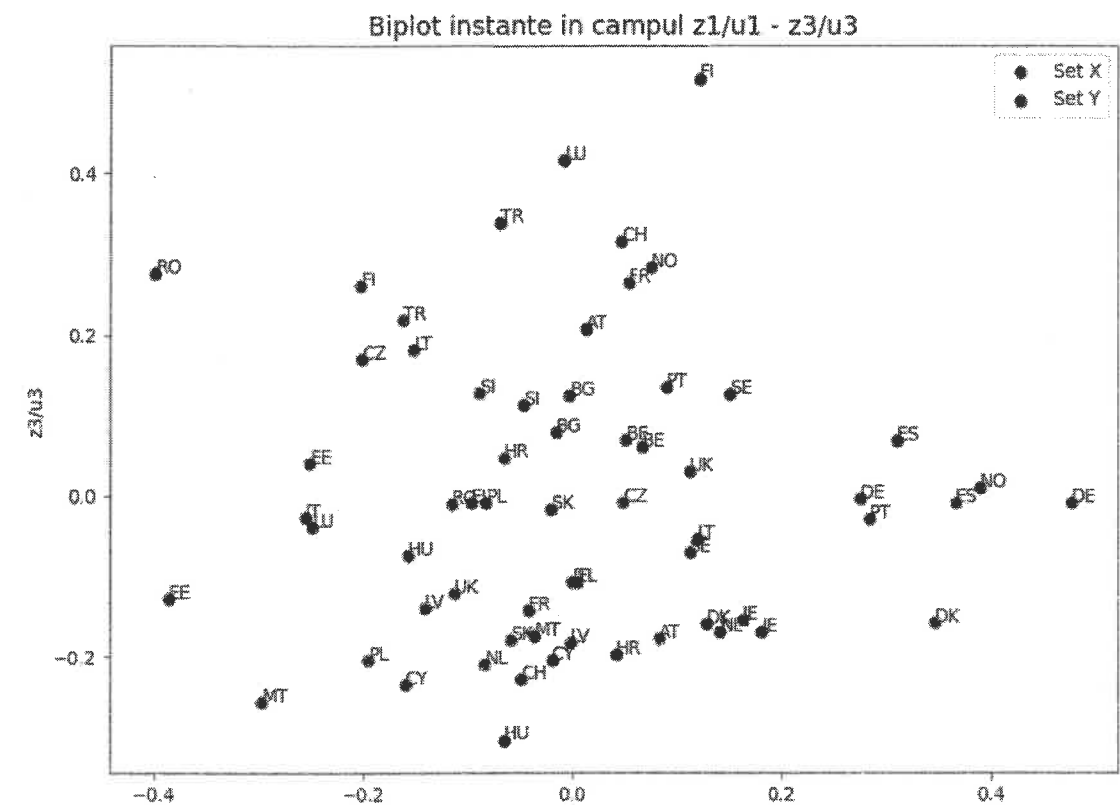


Figura 6.25 Graficul nor de puncte a instanțelor și variabilelor observate în câmpul perechilor canonice 1 și 3

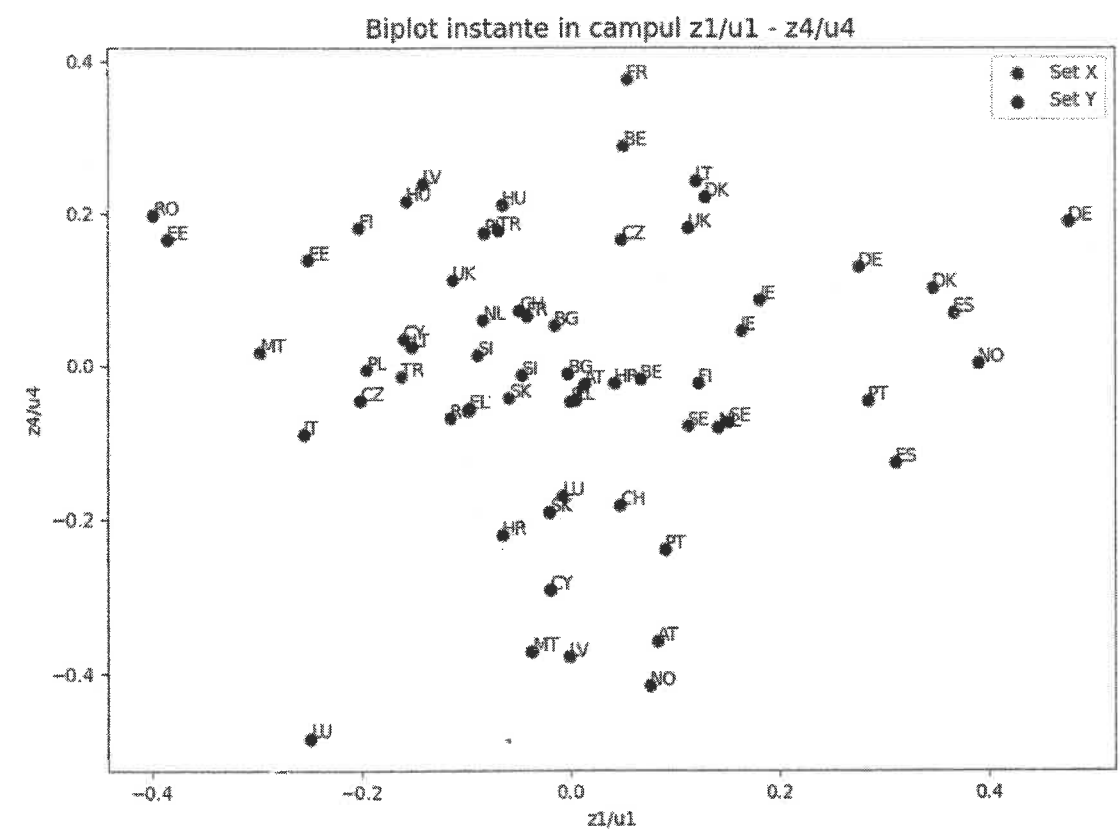


Figura 6.26 Graficul nor de puncte a instanțelor și variabilelor observate în câmpul perechilor canonice 1 și 4

6.4 Analiza discriminantă (AD)

Sub denumirea de analiză discriminantă (AD) sunt reunite o serie de metode explicative, descriptive și predictive, destinate studierii unei populații împărțite în clase. Fiecare individ este caracterizat printr-un ansamblu de variabile independente predictor și o variabilă calitativă identificând clasa din care face parte. Mulțimea de indivizi sau observații se împarte în două submulțimi:

- *eșantionul de bază*, pentru care se cunoaște valoarea variabilei calitative, deci pentru care se știe încadrarea pe clase a indivizilor;
- *eșantionul neinvestigat*, pentru care nu se cunoaște încadrarea indivizilor în clase, deci nici valoarea variabilei calitative.

Analiza discriminantă urmărește, pe de o parte, să identifice regulile pe baza cărora toți indivizii din mulțimea investigată să poată fi încadrați în clase, iar, pe de altă parte, să reducă numărul de variabile necesare împărțirii în clase sau de realizare a discriminării.

Primul aspect urmărit scoate în evidență caracterul predictiv, decizional al analizei discriminante, în timp ce cel de al doilea aspect reliefează caracterul ei descriptiv.

Analiza discriminantă se aplică în mod frecvent în recunoașterea formelor, în domeniul financiar (pentru previzionarea comportamentelor solicitanților de credit), în medicină (pe baza rezultatelor de laborator se caută acea funcție care estimează tipurile de afecțiuni ale unei maladii sau evoluția probabilă a acesteia), în meteorologie (previzionarea avalanșelor pornind de la analiza variabilelor legate de atmosferă, căderile de zăpadă etc.) și în multe alte domenii de activitate.

6.4.1 Organizarea datelor și notații

Matricea de observații:

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1m} \\ x_{21} & x_{22} & \dots & x_{2m} \\ \dots & \dots & \dots & \dots \\ x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix},$$

unde n este numărul de observații și m – numărul de variabile predictor (variabile independente).

Variabila discriminantă:

$$Y = \begin{bmatrix} y_1 \\ \dots \\ y_n \end{bmatrix}.$$

Este o variabilă calitativă. O valoare $y_i, i = \overline{1, n}$, reprezintă clasa din care face parte observația i .

Vectorii observațiilor sunt notați cu $X_i, i = \overline{1, n}$. Un vector X_i este linia i din matricea X .

Vectorii de variabile sunt notați cu X_j , și sunt constituiți din coloanele matricei X .

Matricea centrilor de grupă:

$$G = \begin{bmatrix} g_{11} & g_{12} & \dots & g_{1m} \\ g_{21} & g_{22} & \dots & g_{2m} \\ \dots & \dots & \dots & \dots \\ g_{q1} & g_{q2} & \dots & g_{qm} \end{bmatrix},$$

unde q este numărul de grupe. O valoare g_{kj} reprezintă media variabilei predictor j pentru grupa k .

Vectorii centrilor de grupă sunt notați cu $G_k, k = \overline{1, q}$, $G_k = \begin{bmatrix} g_{k1} \\ \dots \\ g_{km} \end{bmatrix}$.

Media generală: $\bar{X} = \begin{bmatrix} \bar{x}_1 \\ \dots \\ \bar{x}_m \end{bmatrix}$.

Matricea diagonală a frecvențelor grupelor: $D_G = \begin{bmatrix} n_1 & 0 & \dots & 0 \\ 0 & n_2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & n_q \end{bmatrix}$.

6.4.2 Indicatori de variabilitate și împrăștiere

Discriminarea între grupe se realizează cu ajutorul indicatorilor de variabilitate și împrăștiere.

1. **Matricele de împrăștiere** (*sum of square and cross product*) reflectă împrăștierea la nivelul întregii colectivități (*Sum of Square Total – SST*), în interiorul grupelor (*Sum of Square Within – SSW*) și împrăștierea grupelor între ele (*Sum of Square Between – SSB*).

SST este matricea de împrăștiere la nivelul întregii colectivități și arată gradul de împrăștiere în jurul mediei generale. Termenul general al matricei este:

$$\begin{aligned} SST_{jl} &= \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{il} - \bar{x}_l) = \sum_{k=1}^q \sum_{i \in k} (x_{ij} - \bar{x}_j)(x_{il} - \bar{x}_l) = \sum_{k=1}^q \sum_{i \in k} (x_{ij} - g_{kj} + g_{kj} - \bar{x}_j)(x_{il} - g_{kl} + g_{kl} - \bar{x}_l) = \\ &= \sum_{k=1}^q \sum_{i \in k} (x_{ij} - g_{kj})(x_{il} - g_{kl}) + \sum_{k=1}^q \sum_{i \in k} (g_{kj} - \bar{x}_j)(g_{kl} - \bar{x}_l) + \sum_{k=1}^q \sum_{i \in k} (x_{ij} - g_{kj})(g_{kl} - \bar{x}_l) + \sum_{k=1}^q \sum_{i \in k} (g_{kj} - \bar{x}_j)(x_{il} - g_{kl}) = \\ &= \sum_{k=1}^q \sum_{i \in k} (x_{ij} - g_{kj})(x_{il} - g_{kl}) + \sum_{k=1}^q n_k (g_{kj} - \bar{x}_j)(g_{kl} - \bar{x}_l) + \sum_{k=1}^q (g_{kl} - \bar{x}_l) \sum_{i \in k} (x_{ij} - g_{kj}) + \sum_{k=1}^q (x_{il} - g_{kl}) \sum_{i \in k} (g_{kj} - \bar{x}_j) \end{aligned}$$

Prima sumă rezultată în urma dezvoltării anterioare, respectiv:

$$\sum_{k=1}^q \sum_{i \in k} (x_{ij} - g_{kj})(x_{il} - g_{kl})$$

reprezintă termenul general al matricei de împrăștiere în interiorul grupelor, SSW .

A doua sumă:

$$\sum_{k=1}^q n_k (g_{kj} - \bar{x}_j)(g_{kl} - \bar{x}_l)$$

este termenul general al matricei împrăstierii între grupe, SSB .

Sumele trei și patru au valoarea 0, deoarece sumele simple ale abaterilor față de mediile grupelor sunt 0, respectiv:

$$\sum_{i \in k} (x_{ij} - g_{kj}) = 0, \sum_{i \in k} (x_{il} - g_{kl}) = 0.$$

Deci: $SST_{jl} = SSW_{jl} + SSB_{jl}, j = \overline{1, m}, l = \overline{1, m}$.

La nivel matriceal:

$$SST = SSW + SSB.$$

Dacă ținem cont de gradele de libertate, matricele de împrăștiere sunt calculate astfel:

$$MST = \frac{SST}{n-1}, MSW = \frac{SSW}{n-q}, MSB = \frac{SSB}{q-1}.$$

2. Matricele de covarianță – termenii generali ai matricelor de covarianță diferă de cei ai matricelor de împrăștiere prin faptul că se calculează ca valori medii.

$$T_{jl} = \frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{il} - \bar{x}_l), j = \overline{1, m}, l = \overline{1, m} - \text{covarianța totală}$$

$W_{jl} = \sum_{k=1}^q \frac{n_k}{n} \cdot \frac{1}{n_k} \sum_{i \in k} (x_{ij} - g_{kj})(x_{il} - g_{kl}), j = \overline{1, m}, l = \overline{1, m}$ - covarianța intra-grupă (în interiorul grupelor)

$$B_{jl} = \sum_{k=1}^q \frac{n_k}{n} (g_{kj} - \bar{x}_j)(g_{kl} - \bar{x}_l), j = \overline{1, m}, l = \overline{1, m} - \text{covarianța inter-grupă (între grupe)}.$$

Relația între termeni este aceeași ca și la matricele de împrăștiere:

$$T_{jl} = W_{jl} + B_{jl}, j = \overline{1, m}, l = \overline{1, m}.$$

La nivel matriceal:

$$T = W + B.$$

3. Varianța totală – este reflectată de valorile de pe diagonalele principale ale matricelor de covarianță și de împrăștiere:

$$VT = \text{Trace}(T) - \text{varianța totală generală},$$

$$VW = \text{Trace}(W) - \text{varianța totală intra-grupă},$$

$$VB = \text{Trace}(B) - \text{varianța totală inter-grupă},$$

sau

$$VT = \text{Trace}(SST), VW = \text{Trace}(SSW), VB = \text{Trace}(SSB).$$

4. Varianța generalizată – se calculează ca determinant al matricelor de covarianță și de împrăștiere:

$$VGT = |T|, VGW = |W|, VGB = |B|,$$

sau

$$VGT = |SST|, VGW = |SSW|, VGB = |SSB|.$$

6.4.3 Semnificația modelului. Teste statistice

Testarea modelului se face în două etape:

- un test F bazat pe statistica Wilks care arată dacă ansamblul de variabile predictor poate face discriminarea pe grupe a instanțelor;
- teste statistice individuale pentru fiecare variabilă predictor, prin care se decide dacă o variabilă poate fi sau nu un bun predictor.

a) Testul F global – are următoarele ipoteze statistice:

$$H_0: G_1 = G_2 = \dots = G_q$$

$$H_1: \exists \text{ două grupe } i, k \text{ astfel încât } G_i \neq G_k$$

Se calculează un indicator

$$\Lambda = \frac{|SSB|}{|SSB + SSW|}.$$

Cu cât valoarea lui Λ este mai mare, cu atât șansele ca ipoteza H_0 să fie respinsă sunt mai mari. Statistica testului este o valoare F calculată astfel:

$$F = \frac{1 - \Lambda^{1/b}}{\Lambda^{1/b}} \frac{ab - c}{m(q-1)},$$

unde a , b și c se calculează după cum urmează:

$$a = n - q - \frac{m - q + 2}{2}$$

$$b = \begin{cases} \sqrt{\frac{m^2(q-1) - 4}{m^2 + (q-1)^2 - 5}} & , \text{dacă } m^2 + (q-1)^2 - 5 > 0 \\ 1 & , \text{dacă } m^2 + (q-1)^2 - 5 \leq 0 \end{cases}$$

$$c = \frac{m(q-1) - 2}{2}$$

Dacă

$$F < F_{m(q-1), ab-c; \alpha}^{Critic}$$

ipoteza nulă H_0 este respinsă cu un nivel de încredere $1 - \alpha$.

b) Teste statistice la nivel de variabile predictor – o variabilă predictor poate fi considerată bun predictor dacă poate separa cât mai clar grupele. Deci raportul dintre varianța intergrupă și varianța intragrupă este cât mai mare. Fiind vorba de un raport între două varianțe, testul aplicat este testul F . Astfel pentru variabila predictor j , ipotezele nulă și alternativă sunt:

$$H_0: g_{1j} = g_{2j} = \dots = g_{qj}$$

$$H_1: \exists k, i \text{ două grupe astfel încât } g_{kj} \neq g_{ij}$$

$$\text{Statistica testului: } F_j = \frac{SSB_{jj}}{SSW_{jj}}.$$

Valoarea critică pentru $q-1$ și $n-q$ grade de libertate și un prag de semnificație α este

$$F_{q-1; n-q; \alpha}^{Critic}$$

Dacă

$$F_j > F_{q-1; n-q; \alpha}^{Critic}$$

ipoteza nulă este respinsă cu un nivel de încredere $1 - \alpha$.

6.4.4 Analiza discriminantă liniară (Linear Discriminant Analysis) – funcțiile Fisher

Mai este cunoscută și sub denumirea de analiză factorială discriminantă sau analiză canonică discriminantă. Perspectiva analizei este una asemănătoare celei de la analiza în componente principale. Analiza factorială discriminantă determină două variabile predictor, numite variabile discriminante, astfel încât instanțele să fie cât mai bine separate după aceste două variabile. Variabilele discriminante sunt, ca și în cazul analizei în componente principale, combinații liniare de variabilele inițiale (din matricea X) și necorelate două câte două.

Un criteriu natural de determinare a variabilelor discriminante este acela de a maximiza coeziunea claselor cu ajutorul varianței intraclase și a varianței interclase. Raportul dintre varianța interclase a variabilei și varianța totală (sau varianța intraclase) să fie cât mai mare.

Plecăm de la premisa că matricea de observații, X , este centrată. Prima variabilă discriminantă se determină astfel:

$$z_1 = \begin{bmatrix} z_{11} \\ z_{21} \\ \dots \\ z_{n1} \end{bmatrix} = X \cdot u_1,$$

unde u_1 este primul factor discriminant (coeficienții combinației liniare), $u_1 = \begin{bmatrix} u_{11} \\ u_{21} \\ \dots \\ u_{m1} \end{bmatrix}$.

Centrii de grupă pentru variabila z_1 sunt: $\bar{z}_1 = \begin{bmatrix} \bar{z}_{11} \\ \bar{z}_{21} \\ \dots \\ \bar{z}_{q1} \end{bmatrix} = G \cdot u_1$.

Dacă X nu este centrat, atunci $z_1 = u_{01} + X \cdot u_1$.

Variabilele discriminante pot fi văzute și ca funcții discriminante (figura 6.27), denumite și funcții Fisher, prin care se poate face o separare cât mai bună între instanțe.

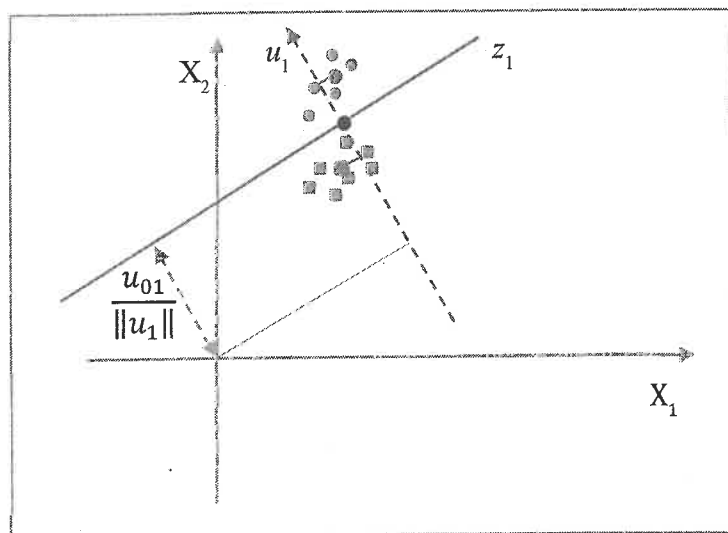


Figura 6.27 Funcțiile discriminante Fisher. Cazul bidimensional cu două clase

În această reprezentare grafică z_1 este un hiperplan de ecuație $u_{01} + X \cdot u_1$ care desparte instanțele (dreapta roșie continuă din figura 6.27). u_1 este normala la hiperplanul z_1 sau axa discriminantă (dreapta roșie întreruptă din figura 6.27) care trece prin origine, în cazul în care datele sunt centrate, sau prin centrul de greutate (punctul negru din figura 6.27) dacă datele nu sunt centrate.

Dacă datele sunt centrate, termenul liber u_{01} este 0. Axa u_1 este astfel aleasă încât să separe cât mai bine grupele, adică distanțele dintre proiecțiile centrilor de grupe pe axă să fie cât mai mari. Centrii de grupă sunt marcați cu roșu în figura 6.27.

Varianța totală a variabilei z_1 scrisă matriceal este:

$$VT_1 = \frac{1}{n} (z_1)^t z_1 = \frac{1}{n} (X \cdot u_1)^t X \cdot u_1 = \frac{1}{n} (u_1)^t X^t X \cdot u_1$$

unde simbolul t la exponent semnifică transpusa, respectiv vector linie $(u_1)^t$ este transpusa vectorului coloană u_1 .

Varianța interclasă a variabilei z_1 este:

$$VB_1 = \sum_{k=1}^q \frac{n_k}{n} (\bar{z}_{k1})^2,$$

unde $\bar{z}_{k1}$ este media variabilei pentru grupa k .

Matriceal varianța interclasă se poate scrie:

$$VB_1 = \frac{1}{n} (\bar{z}_1)^t D_G \cdot \bar{z}_1 = \frac{1}{n} (G \cdot u_1)^t D_G G \cdot u_1 = \frac{1}{n} (u_1)^t G^t D_G G \cdot u_1.$$

Varianța intraclasă este:

$$VW_1 = \sum_{k=1}^q \frac{n_k}{n} \cdot \frac{1}{n_k} \sum_{i \in k} (z_{i1} - \bar{z}_{k1})^2.$$

Relația dintre varianțe: $VT_1 = VB_1 + VW_1$.

Pentru o valoare a varianței totale dată, o variabilă va discrimina cu atât mai bine grupele cu cât sunt îndeplinite mai bine următoarele condiții:

- indivizii aparținând unei clase au valori cât mai apropiate cu putință, adică varianța intraclase este cât mai mică;
- mediile claselor sunt cât mai distanțate, adică varianța interclase este mai mare.

Puterea de discriminare a unei variabile este raportul dintre varianța interclase și varianța totală.

Maximizarea distanței dintre proiecțiile centrilor pe axa discriminantă u_1 este echivalentă cu maximizarea raportului $\frac{VB_1}{VT_1}$ pentru variabila discriminantă z_1 . Dacă notăm cu α_1 acest raport, obținem:

$$\alpha_1 = \frac{VB_1}{VT_1} = \frac{\frac{1}{n} (u_1)^t G^t D_G G \cdot u_1}{\frac{1}{n} (u_1)^t X^t X \cdot u_1}.$$

Problema de optim care se pune este:

$$\left\{ \begin{array}{l} \text{Maxim}_{u_1} \frac{\frac{1}{n} (u_1)^t G^t D_G G \cdot u_1}{\frac{1}{n} (u_1)^t X^t X \cdot u_1} \end{array} \right.,$$

unde u_1 este factorul discriminant, $B = \frac{1}{n} G^t D_G G$ este matricea de covarianță interclase iar

$T = \frac{1}{n} X^t X$ este matricea de covarianță totală.

Soluția problemei, u_1 , obținută prin rezolvarea problemei de optim, este vectorul propriu al matricei $T^{-1}B$, corespunzător celei mai mari valori proprii. Această valoare proprie este chiar α_1 , puterea de discriminare a variabilei z_1 .

Următoarele variabile discriminante se obțin la fel ca prima variabilă, având condiții suplimentare: să fie absolut necorelate cu celelalte variabile discriminante:

$$\frac{1}{n}(z_k)^t z_j = 0, j = \overline{1, k-1}.$$

Problema de optim devine:

$$\begin{cases} \text{Maxim}_{u^k} \frac{\frac{1}{n}(u_k)^t G^t D_G G \cdot u_k}{\frac{1}{n}(u_k)^t X^t X \cdot u_k} \\ \frac{1}{n}(u_k)^t X^t X \cdot u_j = 0, j = \overline{1, k-1} \end{cases}$$

Soluția problemei, factorul u_k , este vectorul propriu al matricei $T^{-1}B$ corespunzător valorii proprii de ordin k (în sensul descrescător al valorilor) α_k . În acest context α_k este puterea de discriminare a variabilei z_k .

Numărul de variabile discriminante este dat de numărul valorilor proprii nenule ale matricei $T^{-1}B$ (a se vedea în continuare analiza canonică discriminantă). Acest număr este $r = \min(m, q - 1)$.

Soluțiile modelului sunt:

$$T^{-1}B \cdot u_k = \alpha_k \cdot u_k, k = \overline{1, r},$$

$$z_k = X \cdot u_k, k = \overline{1, r}.$$

Puterea de discriminare a unei variabile discriminante se poate calcula și ca raport între varianța interclase și varianța intraclase. Principiul de optim în obținerea variabilelor discriminante este deci maximizarea raportului $\frac{VB_k}{VW_k}$. Calculată în această manieră, puterea de discriminare este mai mare:

$$\frac{VB_k}{VW_k} \geq \frac{VB_k}{VB_k + VW_k}.$$

Dacă notăm cu λ_k puterea de discriminare a variabilei discriminante z_k , astfel calculată, obținem:

$$\lambda_k = \frac{VB_k}{VW_k} = \frac{VB_k}{VT_k - VB_k} = \frac{VB_k}{VT_k \cdot \left(1 - \frac{VB_k}{VT_k}\right)} = \frac{\alpha_k}{1 - \alpha_k}.$$

Valorile $\lambda_k, k = \overline{1, r}$ sunt valorile proprii nenule ale matricei $W^{-1}B$ corespunzătoare aceluiași vectori proprii, u_k .

Soluțiile modelului modificat sunt:

$$W^{-1}B \cdot u_k = \lambda_k \cdot u_k, k = \overline{1, r},$$

$$z_k = X \cdot u_k, k = \overline{1, r}.$$

6.4.5 Analiza discriminantă: caz particular al analizei canonice

Analiza discriminantă este un caz particular al analizei canonice deoarece primul grup de variabile sunt cele cuprinse în tabelul de observații X , iar cel de-al doilea grup este dat de coloanele matricei disjunctive complete Y construite după variabila discriminantă.

Factorul canonic de ordin k din primul grup este vectorul propriu corespunzător valorii proprii de ordinul k al matricei $(V_{11})^{-1}V_{12}(V_{22})^{-1}V_{21}$, respectiv:

$$V_{11} = \frac{1}{n}X^t X$$

$$V_{22} = \frac{1}{n}Y^t Y \Rightarrow V_{22}^{-1} = nD_G^{-1}$$

$$V_{21} = \frac{1}{n}Y^t X = \frac{1}{n}D_G G$$

$$V_{12} = \frac{1}{n}G^t D_G$$

$$V_{11}^{-1}V_{12}V_{22}^{-1}V_{21} = n(X^t X)^{-1} \frac{1}{n}G^t D_G nD_G^{-1} \frac{1}{n}D_G G = (X^t X)G^t D_G G = T^{-1}B$$

Iar aceasta este matricea ale cărei valori proprii reprezintă puterea de discriminare a variabilelor discriminante și ai cărei vectori proprii sunt factorii discriminanți sau coeficienții funcțiilor discriminante.

În concluzie, factorii canonici ai primului grup de variabile, grupul variabilelor predictor X , sunt de fapt factorii discriminanți.

Analiza canonică ne permite să aplicăm testele de semnificație pentru fiecare variabilă discriminantă. Testul de semnificație al unei rădăcini canonice (a se vedea analiza canonică) ne arată în această situație dacă o variabilă discriminantă este sau nu este un bun predictor.

6.4.6 Funcții de clasificare

Axele discriminante sunt determinate prin calculul vectorilor proprii ai matricei $T^{-1}B$, unde $B = \frac{1}{n} G^t D_G G$, deci $T^{-1} \frac{1}{n} G^t D_G G$.

După cum am văzut în analiza în componente principale pe un tabel de observații X , axele principale (axele componentelor principale) se calculează ca vectori proprii normați ai matricei $\frac{1}{n} X^t X$. Dacă distanța dintre instanțele descrise în tabelul X nu ar fi una euclidiană, ci o distanță oarecare, de metrică M , atunci axele principale s-ar calcula ca vectori proprii normați ai matricei $\frac{1}{n} M X^t X$. Mai mult, dacă ponderea instanțelor nu ar fi egală, ci am folosi o matrice diagonală a ponderilor, D (matrice care ar avea pe diagonală ponderile indiviziilor – vezi analiza în componente principale ponderată), axele principale s-ar calcula ca vectori proprii ai matricei $\frac{1}{n} M X^t D X$. Prin urmare, analiza factorială discriminantă este o analiză în componente principale în care metrica M este inversa matricei de covarianță totală, T^{-1} , ponderile sunt date de matricea DG (matricea diagonală a frecvențelor grupelor), iar tabelul de observații este G , tabelul centrilor de grupă. Deci este vorba de o analiză în componente principale ponderată a centrilor de grupe, cu o metrică specială, metrica $M = T^{-1}$ (metrica Mahalanobis). Axele discriminante sunt axele principale ale acestui tip particular de analiză în componente principale.

Analiza factorială discriminantă se utilizează atunci când numărul de variabile predictor este foarte mare (sau suficient de mare), iar aplicarea modelului liniar de discriminare devine greoaie. În această situație se utilizează pentru clasificare variabilele discriminante, numite și funcțiile discriminante Fisher, care sunt semnificative ca putere de discriminare. Testul de semnificație este un test F , identic celui făcut variabilelor predictor, valorile calculate fiind valorile λ_k .

În situația în care modelul nu este suficient de complex, numărul de predictor este mic, clasificarea se face pe baza distanțelor Mahalanobis (distanța de metrica T^{-1}). Cu alte cuvinte, o instanță oarecare este clasificată în acea grupă de care este mai apropiată ca distanță. Acest principiu este standardizat sub forma unor **funcții de clasificare**, prin care se furnizează niște **scoruri**, scoruri folosite mai apoi pentru clasificare.

Funcțiile de clasificare sunt determinate deci, cu ajutorul distanțelor Mahalanobis. Distanța Mahalanobis dintre două instanțe a și b este:

$$d^2(a,b) = (a-b)^t T^{-1} (a-b).$$

Să luăm în considerare cazul simplificat în care avem două grupe, una cu centrul g_1 și una cu centrul g_2 .

O instanță oarecare $x = \begin{bmatrix} x_1 \\ \dots \\ x_m \end{bmatrix}$, care poate să fie atât o instanță din setul de bază cât și din setul de test, este clasificată în prima grupă dacă

$$d^2(x, g_1) \leq d^2(x, g_2),$$

sau în a doua grupă dacă

$$d^2(x, g_1) > d^2(x, g_2).$$

Înlocuind în prima inegalitate cu relația de calcul a distanței obținem:

$$(x - g_1)^t T^{-1} (x - g_1) \leq (x - g_2)^t T^{-1} (x - g_2)$$

$$(x^t - g_1^t) T^{-1} (x - g_1) \leq (x^t - g_2^t) T^{-1} (x - g_2), \text{ respectiv}$$

$$x^t T^{-1} x - g_1^t T^{-1} x - x^t T^{-1} g_1 + g_1^t T^{-1} g_1 \leq x^t T^{-1} x - g_2^t T^{-1} x - x^t T^{-1} g_2 + g_2^t T^{-1} g_2$$

Simplificând și ținând cont că

$$x^t T^{-1} g_1 = g_1^t T^{-1} x \text{ și } x^t T^{-1} g_2 = g_2^t T^{-1} x,$$

matricea T^{-1} fiind simetrică, obținem:

$$-2 \cdot g_1^t T^{-1} x + g_1^t T^{-1} g_1 < -2 \cdot g_2^t T^{-1} x + g_2^t T^{-1} g_2$$

$$\Rightarrow 2 \cdot g_1^t T^{-1} x - g_1^t T^{-1} g_1 > 2 \cdot g_2^t T^{-1} x - g_2^t T^{-1} g_2$$

Dacă notăm cu $F_1 = g_1^t T^{-1}$ și $F_2 = g_2^t T^{-1}$, rezultă că instanța x va fi clasificată în prima clasă dacă:

$$(F_1)^t \cdot x - \frac{1}{2} (F_1)^t \cdot g_1 \geq (F_2)^t \cdot x - \frac{1}{2} (F_2)^t \cdot g_2$$

și în cea de a doua clasă dacă:

$$(F_1)^t \cdot x - \frac{1}{2} (F_1)^t \cdot g_1 < (F_2)^t \cdot x - \frac{1}{2} (F_2)^t \cdot g_2.$$

Cu alte cuvinte instanța x va fi clasificată în acea clasă pentru care cantitatea

$$(F_k)^t \cdot x - \frac{1}{2}(F_k)^t \cdot g_k, k = \overline{1, 2} \text{ este mai mare.}$$

Pentru cazul general cu q clase, instanța x va fi clasificată în acea clasă pentru care cantitatea:

$$(F_k)^t \cdot x - \frac{1}{2}(F_k)^t \cdot g_k, k = \overline{1, q}$$

este maximă, $F_k = g_k^t T^{-1}$.

Vectorii F_k se numesc funcții de clasificare, iar valorile calculate cu ajutorul lor, scoruri. Deci o instanță se clasifică în acea clasă pentru care obține scorul maxim. Relațiile de calcul pentru scoruri, prin intermediul funcțiilor de clasificare sunt:

$$S_k(x) = F_{k0} + F_{k1} \cdot x_1 + F_{k2} \cdot x_2 + \dots + F_{km} \cdot x_m, k = \overline{1, q},$$

unde:

$$F_k = \begin{bmatrix} F_{k1} \\ \dots \\ F_{km} \end{bmatrix} = g_k^t T^{-1}, x = \begin{bmatrix} x_1 \\ \dots \\ x_m \end{bmatrix} \text{ este instanța clasificată, iar } F_{k0} = -\frac{1}{2}(F_k)^t \cdot g_k.$$

Instanța x va fi clasificată în acea grupă k , pentru care $S_k \geq S_i, i = \overline{1, q}, i \neq k$.

6.4.7 Discriminarea bayesiană

6.4.7.1 Probabilitate totală și probabilitate bayesiană.

Fie Ω un sistem complet de evenimente. Probabilitatea de apariție a unui eveniment A aparținând lui Ω este dată de relația probabilității totale:

$$P(A) = P(A_1)P(A/A_1) + P(A_2)P(A/A_2) + \dots + P(A_n)P(A/A_n),$$

unde $P(A_i), i = \overline{1, n}$, sunt probabilitățile de apariție a celorlalte evenimente, iar $P(A/A_i), i = \overline{1, n}$, sunt probabilitățile de apariție în simultaneitate a evenimentului A cu celelalte evenimente – probabilitățile condiționate.

$P(A)$ este deci o medie ponderată a probabilității de apariție a evenimentului A împreună cu fiecare din celelalte evenimente din Ω .

O importanță deosebită o are cunoașterea probabilității ca un eveniment să se producă pornind de la o anumită cauză din mulțimea de evenimente care-l determină, $P(A_k/A)$. Relația prin care se determină această probabilitate se numește relația lui Bayes:

$$P(A_k / A) = \frac{P(A_k)P(A / A_k)}{\sum_{i=1}^n P(A_i)P(A / A_i)}$$

Probabilitățile $P(A_i), i = \overline{1, n}$, se numesc probabilități apriorice, iar probabilitățile $P(A_i/A)$ se numesc probabilități aposteriorice.

6.4.7.2 Clasificatorul bayesian

Clasificatorul bayesian este de natură stohastică (probabilistică) și permite o clasificare cu bune rezultate la un cost scăzut. Notăm cu

$$g_k = G_k = \begin{bmatrix} g_{k1} \\ \dots \\ g_{km} \end{bmatrix} \text{ centrul grupei } k,$$

$k = \overline{1, q}$ reprezentând o linie în matricea G în conformitate cu notațiile precedente din cadrul analizei discriminante. Vom eticheta clasele cu $c_1, c_2, \dots, c_q$ (în sensul identificării lor).

Regula lui Bayes pentru asignarea unei instanțe $x = \begin{bmatrix} x_1 \\ \dots \\ x_m \end{bmatrix}$ la o clasă c_k mai degrabă decât la orice

altă clasă c_l este următoarea:

$$P(c_k/x) > P(c_l/x), l = \overline{1, q}, l \neq k \quad (1)$$

unde $P(c_k/x)$ reprezintă probabilitatea ca instanța x să aparțină clasei c_k , iar $P(c_l/x)$ reprezintă probabilitatea ca aceeași instanță să aparțină clasei c_l . Acestea sunt numite **probabilități aposteriorice**.

Vom numi **probabilități apriorice**, probabilitățile de apartenență la o anumită clasă în cazul în care nici o informație despre indivizii claselor nu este disponibilă. De obicei probabilitățile apriorice fie sunt determinate pe baza ponderii claselor, fie sunt egale cu $1/q$ sau au valori alese în mod subiectiv. Vom considera probabilitatea apriorică pentru o clasă oarecare c_l , ca fiind:

$$P(c_l) = \frac{n_l}{n}, l = \overline{1, q}.$$

Relația lui Bayes leagă probabilitățile apriorice de cele aposteriorice în felul următor:

$$P(c_k / x) = \frac{P(x / c_k) \cdot P(c_k)}{\sum_{l=1}^q P(x / c_l) P(c_l)} \quad (2)$$

În relația (2) $P(x/c_i)$ reprezintă probabilitatea ca în distribuția grupei c_i să avem o observație x . Aceste probabilități, probabilitățile condiționate din relația lui Bayes, se determină prin metode parametrice sau neparametrice.

6.4.7.3 Estimarea neparametrică a probabilităților condiționate. Metoda histogramelor

Pentru cazul unidimensional, estimarea pentru o valoare oarecare x este:

$$\hat{p}(x) = \frac{n_j}{\sum_{k=1}^q n_k \cdot dx},$$

unde q este numărul de intervale al distribuției, dx este mărimea intervalului (lățimea histogramei), j este numărul intervalului (histograma) în care se află x , iar n_k este numărul de valori din distribuție aflate în intervalul k .

Pentru cazul bidimensional:

$$\hat{p}(x) = \frac{n_j}{\sum_{k=1}^q n_k \cdot dx_1 \cdot dx_2},$$

unde dx_1 este mărimea intervalului pentru prima dimensiune iar dx_2 pentru a doua dimensiune (figura 6.28). Produsul $dx_1 \cdot dx_2$ este aria histogramelor.

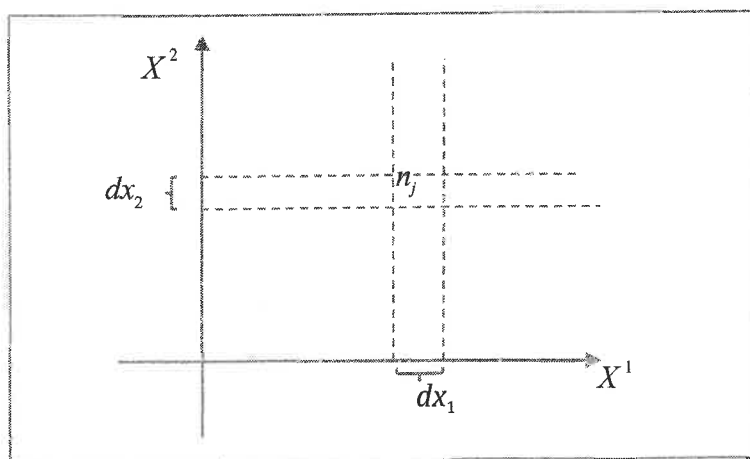


Figura 6.28 Estimare densitate de probabilitate

Pentru cazul m -dimensional:

$$\hat{p}(x) = \frac{n_j}{\sum_{k=1}^q n_k \cdot H},$$

unde $H = \prod_{j=1}^m dx_j$ este volumul histogramei (partea din spațiul R^m aferentă unui interval).

Există și alte metode neparametrice de estimare a probabilităților condiționate cum ar fi ferestre Parzen sau metode *kernel* de estimare a funcțiilor de densitate de probabilitate.

6.4.7.4 Metode parametrice

Aceste metode presupun asumarea unui model parametric pentru funcția de densitate și aplicarea ca atare a relației lui Bayes. Dacă folosim distribuția normală gaussiană pentru estimarea probabilităților condiționate, vom avea:

$$P(x/c_k) = (2\pi)^{-\frac{m}{2}} |T|^{-\frac{1}{2}} \cdot e^{\left(-\frac{1}{2}(x-g_k)^t T^{-1}(x-g_k)\right)},$$

unde T este matricea de covarianță, iar g_k este centrul clasei c_k .

Ținând cont de relația lui Bayes regula (1) devine: vom asigna individul x clasei c_l dacă

$$P(x/c_l)P(c_l) > P(x/c_k)P(c_k), \quad k=\overline{1, q}, l \neq k. \quad (3)$$

Înlocuind probabilitățile condiționate în regula lui Bayes (3), logaritmând și simplificând termenii comuni, obținem faptul că individul x este asignat clasei c_l dacă:

$$2 \cdot \ln P(c_l) - (x-g_l)^t T^{-1}(x-g_l) > 2 \cdot \ln P(c_k) - (x-g_k)^t T^{-1}(x-g_k) \quad (4)$$

pentru orice $k=\overline{1, q}, l \neq k$.

Rezultă că **este asignată instanța x clasei c_l dacă**

$$2 \cdot \ln P(c_l) - d_{T^{-1}}^2(x, g_l) > 2 \cdot \ln P(c_k) - d_{T^{-1}}^2(x, g_k), \quad \text{pentru orice } k=\overline{1, q}, l \neq k. \quad (5)$$

unde $d_{T^{-1}}^2(x, g_k) = (x-g_k)^t T^{-1}(x-g_k)$ $k=\overline{1, q}$, sunt distanțele Mahalanobis dintre instanța x și centrul celor q grupe.

Dacă vom considera cantitățile

$$S_k(x) = 2 \cdot \ln P(c_k) - d_{T^{-1}}^2(x, g_k), \quad k=\overline{1, q},$$

scoruri de clasificare ale instanței x în grupa c_k după modelul celor din funcțiile de clasificare de la clasificarea liniară, rezultă că regula devine următoarea:

instanța x este asignată clasei c_i dacă

$$S_i(x) > S_k(x), \text{ pentru orice } k = \overline{1, q}, k \neq i.$$

Se poate observa că diferența față de scorurile de la clasificarea liniară este dată de faptul că aceste scoruri depind și de ponderile grupelor.

6.4.8 Analiza discriminantă – studiu de caz

Studiul se referă la clasificarea mostrelor de sticlă¹ pornind de la caracteristici ale acestora în special compoziția în diverse elemente chimice precum sodiu, magneziu, fier etc. Aceste caracteristici vor furniza variabilele predictor. Clasificarea se face în șase grupe ținând cont de procedeul de fabricație și domeniul de utilizare.

6.4.8.1 Setul de date utilizat

Variabile predictor:

1. RI – index de refracție
2. Na – Compoziția în sodiu
3. Mg – Compoziția în magneziu
4. Al – Compoziția în aluminiu
5. Si – Compoziția în siliciu
6. K – Compoziția în potasiu
7. Ca – Compoziția în calciu
8. Ba – Compoziția în bariu
8. Fe – Compoziția în fier

Variabila de clasificare – *class* – tipurile de sticlă. Acestea pot fi:

wind_float – sticlă pentru ferestre obținută prin procedeul float²
wind_nfloat – sticlă obișnuită pentru ferestre
vwind_float – sticlă flotată pentru geamuri auto
container – sticlă pentru depozitare (borcane, sticle pentru băuturi etc)
tableware – sticlă pentru veselă
headlamps – sticlă pentru lămpi

Datele de intrare sunt grupate în două fișiere, *sticla_baza.csv*, eșantionul de bază (setul de antrenament) și *sticla_test.csv*, eșantionul neinvestigat (setul de test).

¹ Dua, D. and Karra Taniskidou, E. (2017). UCI Machine Learning Repository [http://archive.ics.uci.edu/ml]. Irvine, CA: University of California, School of Information and Computer Science.

² Geamul float - obținut prin procedeul de flotare a unei benzi de sticlă pe o baie metalică, se obține o sticlă cu fețe perfect plane cu transparența nedistorsionată de neregularitățile suprafeței.

Analiza va fi făcută prin două metode: analiza liniară discriminantă și clasificarea bayesiană. Pentru ambele metode se vor calcula și aplica funcțiile de clasificare pe variabilele discriminante.

6.4.8.2 Investigarea eșantionului de bază – construirea modelului

În această fază a analizei vor fi efectuate următoarele operațiuni:

- testarea variabilelor predictor prin determinarea puterii de discriminare a acestora;
- determinarea axelor discriminante (u), a variabilelor discriminante (z) și a funcțiilor de clasificare (F, F_0);
- calculul puterii de discriminare a variabilelor discriminante;
- reprezentarea grafică a instanțelor în primele două axe discriminante;
- evaluarea modelului prin matrice de acuratețe.

Puterea de discriminare a variabilelor predictor se calculează ca raport între varianțele medii interclasă și varianțele medii intraclasă ale acestor variabile. În tabelul 6.5 este prezentată puterea de discriminare a variabilelor predictor și rezultatul aplicării testului Fisher pentru semnificația acestor variabile ca variabile predictor.

Tabelul 6.5 Puterea de discriminare a variabilelor predictor

Variabila	Putere discriminare	p_value
RI	1.214458	0.3
Na	22.64015	0
Mg	54.64116	0
Al	29.2198	0
Si	2.339316	0.04
K	8.477439	0
Ca	1.60081	0.16
Ba	23.57839	0
Fe	3.014843	0.01

6.4.8.3 Implementarea Python a analizei discriminate

Programul ce conține fluxul coordonator de prelucrare este inclus în fișierul *mainAD.py*. Acesta invocă funcții din fișierele *util.py* și *visual.py* al căror conținut este furnizat în **Anexa 1** respectiv **Anexa 2**.

```
import pandas as pd
import numpy as np
import util.util as util
import visual.visual as vi
import sklearn.metrics as metrics

try:
    fisier1 = 'dataIN/sticla_baza.csv'
    t1 = pd.read_csv(fisier1, index_col=0)
    print(t1)
    # Inlocuire valori lipsa pentru un pandas DataFrame
    util.inlocuire_na_df(t1)
```

```

# Citire variabile categoriale
variabile = list(t1)
variabile_categoriale = t1.columns[9:10]
utl.codificare(t1, variabile_categoriale)

# Citire variabile predictor
variabile_predictor = t1.columns[0:9]
variabila_de_clasificare = t1.columns[9]
x = t1[variabile_predictor].values
y = t1[variabila_de_clasificare].values
print(y)
n, m = np.shape(x)

# Calcul centrui, frecvente, grupe si imprastiere
g, ng, xg, sst, ssb, ssw = utl.imprastiere(x, y)
q = len(g)

# Calcul putere de discriminare a variabilelor predictor
l_x, p_values = utl.putere_discriminare(ssb, ssw, n, q)
putere_discriminare_x = pd.DataFrame(
    data={
        'putere_discriminare': l_x, 'p_value': np.around(p_values, 2)
    }, index=variabile_predictor
)
putere_discriminare_x.to_csv('dataOUT/Putere_discriminare_x.csv')

# Aplicare analiza discriminanta liniara (ADL)
alpha, l, u = utl.adl(sst, ssb, n, q)
r = np.shape(u)[1]
z = x @ u
zg = xg @ u
sst_t = utl.tabelare(sst, tabel='dataOUT/sst.csv',
    nume_instante=variabile_predictor,
    nume_coloane=variabile_predictor)
ssb_t = utl.tabelare(ssb, tabel='dataOUT/ssb.csv',
    nume_instante=variabile_predictor,
    nume_coloane=variabile_predictor)
z_index = ['z' + str(i) for i in range(1, len(l) + 1)]
putere_discriminare = pd.DataFrame(
    data={
        'Putere_disc': l,
        'Putere_disc(%)': l * 100 / sum(l),
        'Putere_disc_cumulata(%)': np.cumsum(l) * 100 / sum(l),
        'alpha': alpha,
        'p_value': 1.0 - utl.sts.f.cdf(l, q - 1, n - q)
    }, index=z_index
)
putere_discriminare.to_csv("dataOUT/Putere_discriminare.csv")
utl.tabelare(u, z_index, variabile_predictor, 'dataOUT/u.csv')
utl.tabelare(z, z_index, t1.index, 'dataOUT/z.csv')

# Calcul functii de clasificare pe baza variabilelor discriminante
f, f0, f0_b = utl.functii_clasificare_z(z, zg, ng)
utl.tabelare(f, tabel='dataOUT/f.csv', nume_coloane=z_index, nume_instante=g)
pd.DataFrame(data={'adl': f0, 'bayes': f0_b}, index=g).to_csv('dataOUT/f0.csv')

# Predictie pe baza functiilor de clasificare ADL
clasificare_adl = utl.predict(z, f, f0, g)

```

```

# Predictie pe baza functiilor de clasificare Bayes
clasificare_bayes = utl.predict(z, f, f0_b, g)
cohen_kappa_adl = metrics.cohen_kappa_score(y, clasificare_adl)
cohen_kappa_bayes = metrics.cohen_kappa_score(y, clasificare_bayes)
clasifB_tabel = pd.DataFrame(data={
    'variabila_de_clasificare': y,
    'predictie_adl': clasificare_adl,
    'predictie_bayes': clasificare_bayes
}, index=t1.index)
clasifB_tabel.to_csv("dataOUT/ClasificareB.csv")
if r > 1:
    vi.scatter(z[:, 0], z[:, 1], y, np.array(t1.index),
        zg[:, 0], zg[:, 1], g, g,
        lx='z1 (' + str(l[0]) * 100 / sum(l)) + ')',
        ly='z2 (' + str(l[1]) * 100 / sum(l)) + ')')

err_bayes = clasifB_tabel[clasificare_bayes != y]
err_adl = clasifB_tabel[clasificare_adl != y]
acuratete_globala = pd.DataFrame(data={
    'acuratete': [100 - len(err_adl) * 100 / n, 100 - len(err_bayes) * 100 / n],
    'cohen_kappa': [cohen_kappa_adl, cohen_kappa_bayes]},
    index=['adl', 'bayes'])
acuratete_globala.to_csv('dataOUT/acuratete.csv')
err_bayes.to_csv('dataOUT/err_bayes.csv')
err_adl.to_csv("dataOUT/err_adl.csv")
mat_conf_adl = utl.discrim_acuratete(y, clasificare_adl, g)
mat_conf_bayes = utl.discrim_acuratete(y, clasificare_bayes, g)
mat_conf_adl.to_csv('dataOUT/mat_conf_adl.csv')
mat_conf_bayes.to_csv('dataOUT/mat_conf_bayes.csv')
for i in range(r):
    titlu = 'Distributie axa ' + str(i + 1) + '. Putere discriminare(%):' \
        + str(round(putere_discriminare.iloc[i, 1], 2))
    vi.distributie(z[:, i], y, g, titlu)

# Clasificare in setul test
fisier2 = 'dataIN/sticla_test.csv'
# fisier2 = gui.FileDialog("*.csv", "Set test")
if fisier2 != "":
    # Clasificare in noul set de test pe baza functiilor
    # de clasificare si a scorurilor discriminante
    t2 = pd.read_csv(fisier2, index_col=0)
    utl.inlocuire_na_df(t2)
    utl.codificare(t2, variabile_categoriale)
    z = t2[variabile_predictor].values @ u
    clasificare_adl_test = utl.predict(z, f, f0, g)
    clasificare_bayes_test = utl.predict(z, f, f0_b, g)
    tabel_clasificare_test = pd.DataFrame(data={
        'adl': clasificare_adl_test, 'bayes': clasificare_bayes_test
    }, index=t2.index)
    tabel_clasificare_test.to_csv('dataOUT/Clasificare_test.csv')

vi.show()
except Exception as ex:
    print("Eroare!", ex.with_traceback(), sep="\n")

```

6.4.8.4 Prezentarea și interpretarea rezultatelor

Dacă luăm în considerare un prag de semnificație limită de 0.05, variabilele marcate cu roșu în tabel sunt considerate variabile slab-predictor, care nu pot discrimina satisfăcător grupele. Variabila care reprezintă compoziția în magneziu (Mg) este cel mai bun predictor, având cea mai mare putere de discriminare.

Axele discriminante sunt ortogonale două câte două și sunt determinate astfel încât să maximizeze raportul dintre varianțele intergrupe și varianțele totale pe fiecare axă. Acestea sunt salvate în fișierul `u.csv` care este atașat proiectului. Numărul axelor discriminante este minimul dintre numărul de variabile predictor și numărul de grupe minus unu. În acest exemplu vor fi 5 axe discriminante.

Variabilele discriminante sunt proiecțiile instanțelor în axele discriminante. Acestea sunt salvate în fișierul `z.csv`, atașat proiectului. Interpretarea lor este condiționată de asocierea valorilor cu proiecțiile centrilor de grupă în aceleași axe discriminante.

Puterea de discriminare a variabilelor discriminante se calculează la fel ca în cazul variabilelor predictor. În tabelul 6.6 este prezentată puterea discriminatorilor `z`. Se poate observa că primele două variabile acoperă peste 90% din puterea de discriminare.

Tabelul 6.6 Putere discriminatori `z`

Variabile	Putere disc.	Putere disc. (%)	Putere disc. cumulată (%)	alpha	p_value
z1	134.5427	79.66374	79.66374	0.802078	1.11E-16
z2	21.37334	12.65531	92.31905	0.391644	2.22E-16
z3	8.339331	4.937779	97.25683	0.200757	4.69E-07
z4	2.896755	1.71519	98.97202	0.08025	0.015542
z5	1.736138	1.02798	100	0.049695	0.12901

În coloana *alpha* sunt prezentate valorile proprii ale matricei $T^{-1}B$ (T – matricea de covarianță a modelului, B – matricea de covarianță inter-grupe). Valorile proprii reprezintă raportul dintre varianța intergrupa și varianța totală a variabilelor discriminante. În coloana *p_value* sunt prezentate rezultatele aplicării testului Fisher pentru semnificația variabilelor discriminante `z`. Conform testului sunt semnificative primele 4 variabile discriminante (pentru un prag de semnificație de 0.05).

În figura 6.29 este prezentat graficul instanțelor și al centrilor de grupe în primele două axe discriminante.

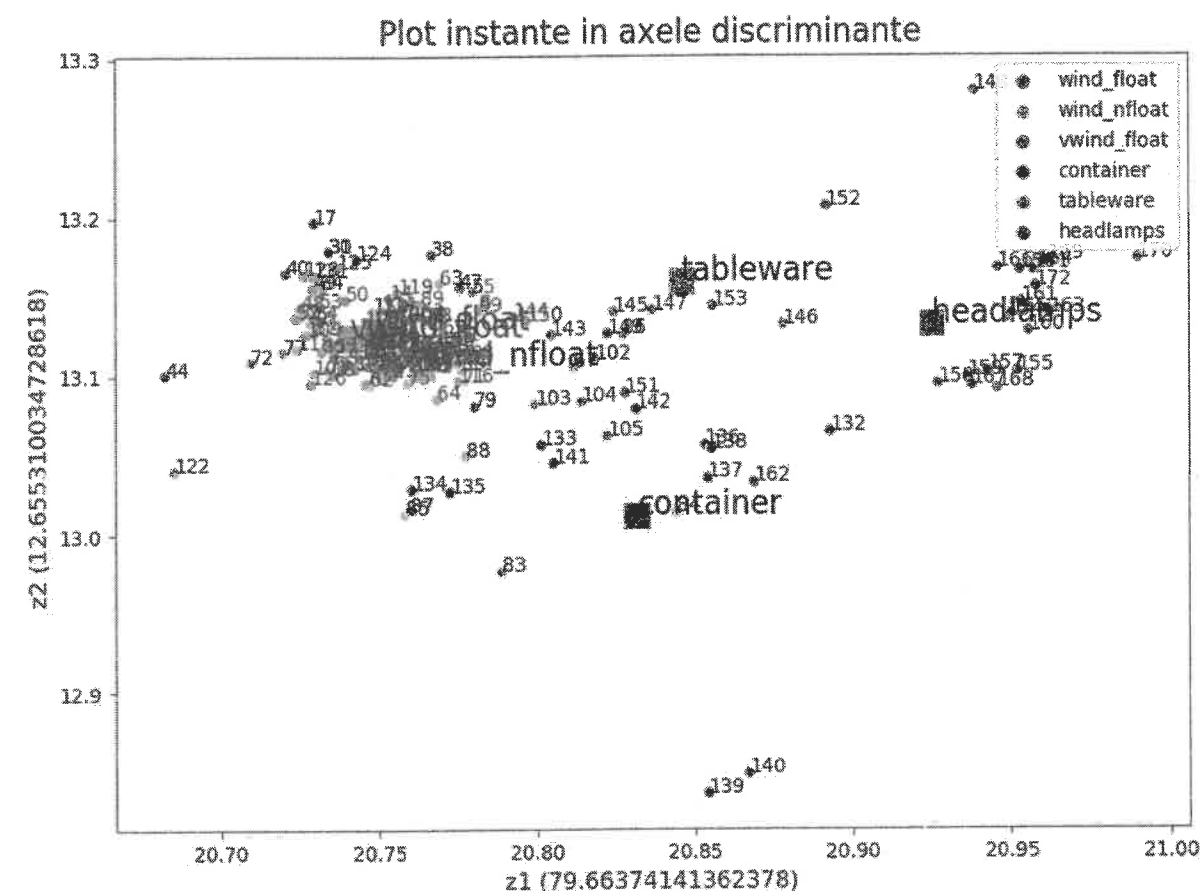


Figura 6.29 Grafic nor de puncte al instanțelor în prime 2 axe discriminante

Axele au menționată puterea de discriminare. Graficul ne permite o rapidă vizualizare a clasificării în eșantionul de bază. Grupele mai omogene (clar separate) sunt grupe *container* și *headlamps*. Centrii acestor grupe sunt mai distanțate față de ceilalți centrii iar instanțele grupelor sunt apropiate centrilor corespunzători. Grupa *tableware* deși are centrul bine evidențiat, instanțele sunt îndepărtate de centru și pot fi asociate altor grupe. În figura 6.30 este prezentat un detaliu pentru primele trei grupe (sticla pentru ferestre). Aceste grupe sunt mai puțin omogene.

O prezentare mai detaliată a distribuției instanțelor pe axa 1 este făcută în figura 6.31. Graficul din figura 6.31 prezintă curbele gaussiene ale distribuției pentru fiecare grupă, în axa u_1 , prima axa discriminată cu cea mai mare putere de discriminare. Graficul întărește concluziile trase anterior: grupa *headlamps* este cel mai bine separată. Omogenitatea grupelor poate fi estimată grafic (apreciată vizual) după coeficientul de aplatizare și suprapunerea curbelor. Curbele cu un coeficient al aplatizării mai mare indică o îndepărtare mai mare a instanțelor față de centrul (clasa *tableware*). Curbele cu suprapuneri indică o apropiere a centrilor. Grupele *container* și *tableware* au aplatizare mare. Grupele reprezentând sticla de geam au aplatizarea mai mică dar suprapunerile sunt mari.

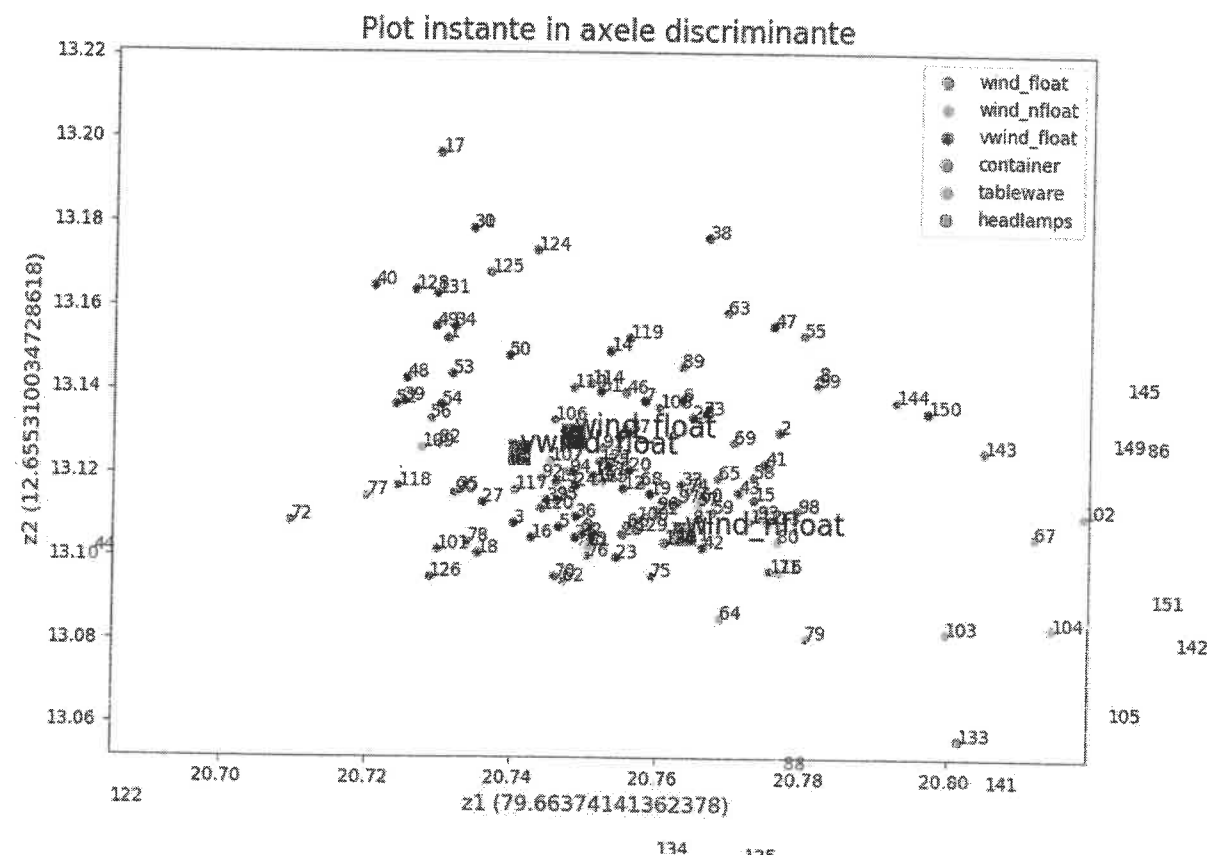


Figura 6.30 Plot detaliu instanțe

Funcțiile de clasificare sunt salvate în fișierele `f.csv` și `f0.csv` și sunt atașate studiului. În fișierul `f0.csv` sunt prezentați termenii liberi ai funcțiilor, atât pentru modelul liniar cât și pentru cel bayesian.

Matricele de acuratețe scot în evidență precizia modelelor de clasificare. În tabelul 6.7 este prezentată acuratețea globală. Aceasta exprimă procentual ponderea instanțelor corect clasificate. În tabel apare și un calcul pentru indicatorul de evaluare Cohen-Kappa. Acest indicator este calculat după relația:

$$K = 1 - \frac{1 - P_o}{1 - P_e},$$

unde P_o este acuratețea globală (efectivă) iar P_e este acuratețea estimată pentru un clasificator care clasifică aleatoriu instanțele.

Conform schemei lui Landis și Koch, o valoare mai decât 0 indică un clasificator care clasifică eronat (mai slab decât o clasificare la întâmplare), 0-0.20 un clasificator slab, 0.21-0.40 un clasificator mediu, 0.41-0.60 clasificator moderat, 0.61-0.80 clasificator bun și 0.81-1 clasificator foarte bun.

Conform indicelui Cohen, clasificatorii liniar și bayesian sunt considerați clasificatori moderați, pentru acest studiu de caz.

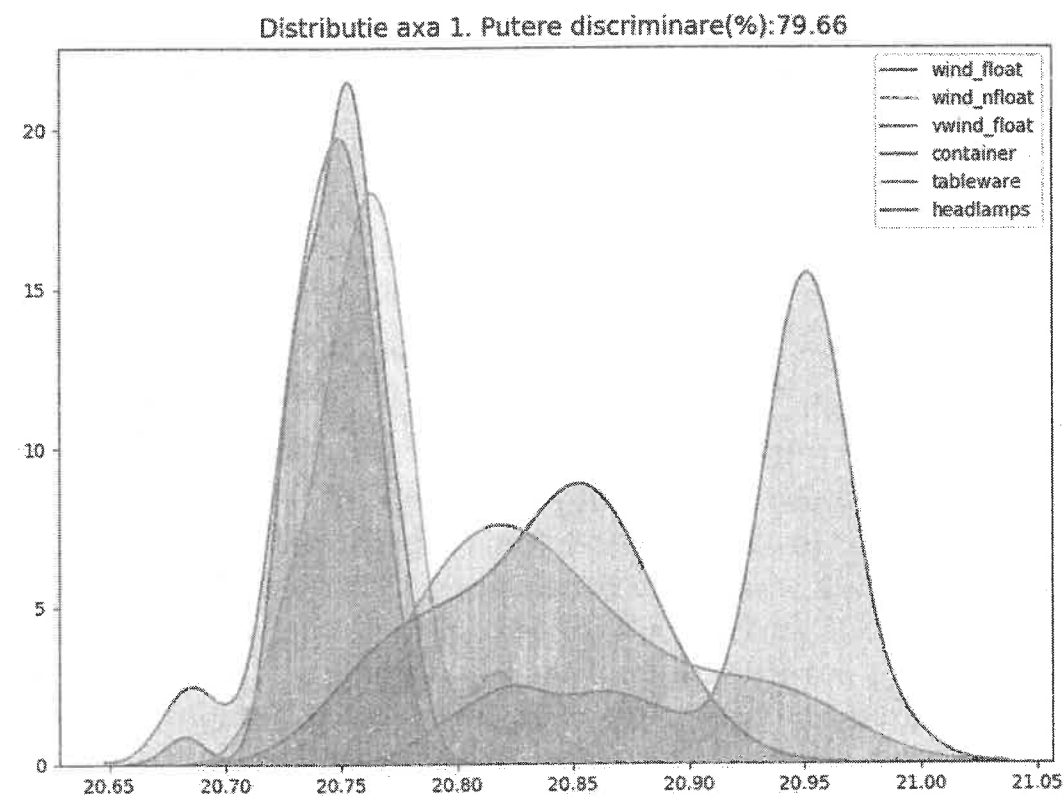


Figura 6.31 Distribuții pe grupe după axa 1

Tabelul 6.7 Acuratețea globală

Model	Acuratețe globală (%)	Cohen_kappa
lda	62.2093	0.500112
bayes	65.11628	0.498225

Diferența dintre cele două modele este mică. Modelul bayesian furnizează o precizie a clasificării ușor mai mare (65.11% din instanțe sunt bine clasificate).

Acuratețea clasificării pe grupe este prezentată în tabelele 6.8 și 6.9. Pe ultimele coloane apar procentele de acuratețe pe grupe. Celelalte valori reprezintă numărul de instanțe din grupa indicată de coloana 1, clasificate în grupa indicată de linia 1.

Tabelul 6.8 Acuratețea clasificării pe grupe – modelul liniar

Grupe	wind_float	wind_nfloat	vwind_float	container	tableware	headlamps	acuratețe
wind_float	33	12	9	0	0	0	61.11111
wind_nfloat	14	35	7	5	2	1	54.6875
vwind_float	2	3	8	0	0	0	61.53846
container	0	2	0	8	0	1	72.72727
tableware	1	0	2	0	3	0	50
headlamps	0	2	1	1	0	20	83.33333

Tabelul 6.9 Acuratețea clasificării pe grupe – modelul bayesian

Grupe	wind_float	wind_nfloat	vwind_float	container	tableware	headlamps	acuratete
wind_float	36	17	1	0	0	0	66.66667
wind_nfloat	12	50	0	1	1	0	78.125
vwind_float	8	5	0	0	0	0	0
container	0	6	0	4	0	1	36.36364
tableware	1	2	0	0	3	0	50
headlamps	2	2	0	1	0	19	79.16667

Modelul bayesian favorizează grupele mari. Termenii liberi din funcțiile de clasificare pentru modelul liniar sunt ajustați cu logaritmi probabilităților apriorice ale grupelor. Acest lucru face ca grupele mari să "atragă" instanțe din grupele învecinate. Prin urmare, crește acuratețea estimării în grupe precum *wind_float* și *wind_nfloat* și scade în grupe precum *container* și *vwind_float*. Din analiza matricelor se poate observa că erorile de clasificare mai mari sunt determinate de confuziile între primele trei grupe (sticla de geam). Sunt evidențiate în tabel pe fond albastru. Pe diagonalele principale avem numărul de instanțe bine clasificate.

Clasificarea instanțelor din eșantionul de bază și clasificările eronate sunt prezentate în fișierele *ClasificareB.csv*, *err_bayes.csv* și *err_lda.csv*, fișiere generate prin programul *mainAD.py*.

6.4.8.5 Clasificarea instanțelor din eșantionul de test – aplicarea modelului

Clasificarea instanțelor din eșantionul de test se face cu ajutorul funcțiilor de clasificare determinate în faza de construire a modelului. Rezultatul clasificării este prezentat în tabelul 6.10.

Tabelul 6.10 Rezultatul clasificării

Id	lda	bayes
1	wind_float	wind_float
2	wind_float	wind_float
3	wind_float	wind_float
4	wind_float	wind_float
5	vwind_float	wind_float
6	vwind_float	wind_float
7	wind_float	wind_nfloat
8	wind_nfloat	wind_nfloat
9	wind_float	wind_float
10	wind_float	wind_nfloat
11	vwind_float	wind_nfloat
12	wind_float	wind_float
13	wind_nfloat	wind_nfloat

Id	lda	bayes
14	wind_float	wind_float
15	wind_float	wind_float
16	wind_float	wind_float
17	wind_nfloat	wind_nfloat
18	vwind_float	wind_float
19	wind_nfloat	wind_nfloat
20	wind_nfloat	wind_nfloat
21	wind_nfloat	wind_nfloat
22	wind_float	wind_float
23	wind_float	wind_float
24	tableware	tableware
25	container	container
26	vwind_float	wind_nfloat
27	wind_nfloat	wind_nfloat
28	wind_nfloat	wind_nfloat
29	vwind_float	wind_nfloat
30	vwind_float	wind_float
31	vwind_float	wind_nfloat
32	wind_float	wind_float
33	container	wind_nfloat
34	wind_nfloat	wind_nfloat
35	wind_nfloat	wind_nfloat
36	tableware	wind_nfloat
37	tableware	wind_nfloat
38	headlamps	headlamps
39	headlamps	headlamps
40	headlamps	headlamps
41	headlamps	headlamps
42	headlamps	headlamps

6.5 Analiza de cluster (AC)

Algoritmii de clusterizare sau de clasificare nesupervizată sunt utilizați pentru a determina grupările naturale ale datelor sau pentru a furniza o împărțire convenabilă a datelor în grupuri.

Spre deosebire de clasificarea supervizată (analiza discriminantă), în clasificarea nesupervizată nu există o informație prealabilă despre grupele în care se va face clasificarea. Analiza de cluster este o metodă exploratorie. Numărul grupelor este subordonat scopului analizei.

Datele supuse analizei sunt valorile relațiilor dintre indivizii și variabilele aflate în studiu luate câte două – distanțe sau disimilarități. Înregistrarea acestor valori ia forma matricelor de distanță. Așadar, valorile supuse analizei reprezintă deja rezultatul unor calcule care au ca rezultat măsuri de disimilaritate dintre obiecte (instanțe sau variabile).

Grupele formate prin clasificare conțin indivizi asemănători între ei, cu disimilarități mici. Privită din această perspectivă, a formării de grupe omogene, analiza de cluster este o metodă de sinteză informațională, așa cum este și analiza în componente principale, doar că se referă, în principal, la instanțe, iar nu la variabile. O grupă omogenă poate fi reprezentată în diverse analize printr-un singur individ: centrul grupei.

Există mai multe motive pentru care o grupare a datelor este necesară:

- *identificarea trăsăturilor fundamentale ale datelor.* Se studiază relațiile semnificative existente între date;
- *obținerea unor reprezentări avantajoase în efectuarea analizelor;*
- *stocare și regăsire rapidă a informației.* În acest sens este preocupantă îmbunătățirea vitezei de acces prin furnizarea unor strategii de rutare pentru informația stocată. Eficiența grupării se măsoară prin eficiența timpului de regăsire a unei părți dorite din ansamblul datelor.

Algoritmii de clusterizare se împart în următoarele grupe:

- algoritmi ierarhici:
 - *Bottom-up*, aglomerativi – încep prin a considera toate punctele (observațiile, indivizii sau obiectele) ca clustere individuale și, la fiecare pas, îmbină cea mai apropiată pereche de clustere (*K-nearest neighbors* sau *K-nn*)
 - *Top-down*, divizivi – încep cu un singur *all-inclusive* cluster și, la fiecare pas, este împărțit un cluster până când rămân doar grupuri de puncte individuale.
- algoritmi de partiționare – de obicei, se începe cu o partiționare aleatorie (parțială), după care se rafinează iterativ:
 - Clusterizare bazată pe model (*K-means*)

O distincție între diferitele tipuri de clustere este dacă setul de clustere este imbricat sau neimbricat. O grupare partiționată este pur și simplu o împărțire a setului de obiecte în subseturi (clustere) care nu se suprapun, astfel încât fiecare obiect de date să fie exact într-un subset.

Pe de altă parte, un cluster ierarhic este un set de clustere imbricate care sunt organizate ca un copac. O grupare ierarhică nu își asumă o anumită valoare k , așa cum este necesar în gruparea *K-means*. Arborele generat poate corespunde unei taxonomii semnificative. Pentru a calcula gruparea ierarhică este necesară doar o matrice de distanță sau „proximitate”.

6.5.1 Algoritmi ierarhici

Fie $\Omega = \{w_1, w_2, \dots, w_n\}$ mulțimea obiectelor (indivizi/instanțe sau variabile) aflate în analiză. O *ierarhie*, notată cu H , este un ansamblu ordonat de mulțimi, formate din elemente ale mulțimii Ω , agregate la un anumit nivel, și care are proprietățile:

- $\Omega \in H$, adică submulțimea agregată la nivelul cel mai de sus conține toți indivizii;
- pentru orice $w_i, i = \overline{1, n}$ (n este numărul de obiecte), există $\{w_i\} \in H$ care formează submulțimile de bază, terminale;
- pentru orice $h, h' \in H$ există implicația: $h \cap h' \neq \emptyset \Rightarrow h \subset h' \text{ sau } h' \subset h$.

Reprezentarea grafică a rezultatului execuției algoritmilor ierarhici este **graficul dendrogramă**. Una dintre axele graficului este axa distanțelor, iar cealaltă axă este axa obiectelor. Graficul evidențiază distanțele de agregare din ierarhie.

În funcție de natura problemei, numărul de grupuri sau clase este decis de către specialistul din domeniu.

În ceea ce privește variația distanței de agregare, se alege partiția care corespunde diferenței maxime din punct de vedere al distanței dintre grupurile ierarhice.

Vom prezenta în continuare pașii prin care se construiește o ierarhie de obiecte:

Intrări: un set de $n(n-1)/2$ distanțe și un set de n mulțimi $h_i = \{w_i\}, i = \overline{1, n}$, aflate la nivelul cel mai de jos de agregare.

Pasul 1: se selectează mulțimile care vor fi agregate: h_i și h_k

Pasul 2: se adună obiectele mulțimilor h_i și h_k și se înlocuiesc cu un nouă mulțime:

$$h' = h_i \cup h_k$$

actualizându-se și distanțele între noua mulțime h' și celelalte mulțimi.

Pasul 3: Atât timp cât rămân cel puțin două mulțimi, se revine la pasul 1.

O ierarhie se construiește în $n-1$ etape/iterații. La fiecare iterație are loc o joncțiune, două mulțimi dispar în locul lor apărând una nouă formată prin reuniune. Se poate spune că, după fiecare joncțiune, se obține o nouă repartizare a obiectelor în mulțimi. O astfel de repartizare este numită **partiție**. Prima partiție corespunde nivelului cel mai de jos de repartizare:

$$P_1 = \{\{w_1\}, \{w_2\}, \dots, \{w_n\}\}.$$

Următoarele partiții vor avea $n-1, n-2, \dots, 1$ elemente. Ultima partiție este

$$P_n = \{\{w_1, w_2, \dots, w_n\}\}.$$

Partițiile pot fi evidențiate în graficul dendrogramă prin secționări paralele cu axa distanțelor. Numărul de mulțimi dintr-o partiție este egal cu numărul de brațe intersectate de secțiune.

În mod fundamental, un algoritm ierarhic de clusterizare aglomerativă funcționează după cum urmează:

- 1: Se calculează matricea de proximitate, dacă este necesar.
- 2: **Repetă:**
- 3: Sunt îmbinate cele mai apropiate două clustere.
- 4: Este actualizată matricea de proximitate pentru a reflecta proximitatea dintre noul cluster și clusterelor originale.
- 5: **Până când** rămâne un singur cluster.

Toate metodele de grupare ierarhică urmează acest algoritm. Diferențele sunt date de modul în care se aleg clusterelor care jonctonează la un moment dat și de modul în care sunt actualizate distanțele după jonctiune.

6.5.2 Metode de grupare ierarhică

Operațiunea-cheie a grupării pe baza principiului aglomerativ este calculul proximității dintre două clusterelor. În mod concret, definiția proximității clusterelor diferențiază diferitele metode sau tehnici ierarhice aglomerative.

- a) Gruparea prin legătură simplă (*single link* sau *MIN proximity*). Selecția clusterelor se face prin distanța minimă. Gruparea prin metoda legătură simplă este asemănătoare creării unei înlanțuiri și este potrivită pentru clusterelor de forme neeliptice (figura 6.32). Este sensibilă la valori anormale. Definește proximitatea clusterului ca cea mai mică distanță dintre două puncte, x și y , care se află în clusterelor diferite, A și B :

$$d(A, B) = \min \{d(x-y)\} \text{ pentru } x \in A, y \in B$$

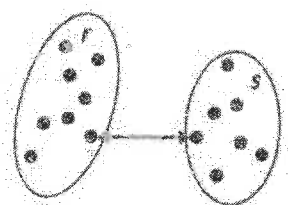


Figura 6.32 Gruparea prin legătură simplă

- b) Gruparea prin legătură completă (*complete link* sau *MAX proximity*). Selecția clusterelor se face prin distanța maximă. Legătura completă este mai puțin susceptibilă la zgomot și valori aberante, dar poate sparge grupuri mari și favorizează formele globulare (figura 6.33). Definește proximitatea clusterului ca cea mai îndepărtată distanță dintre două puncte, x și y , care se află în clusterelor diferite, A și B :

$$d(A, B) = \max \{d(x-y)\} \text{ pentru } x \in A, y \in B$$

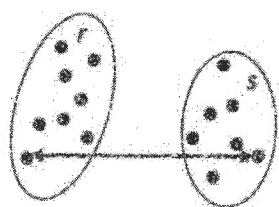


Figura 6.33 Gruparea prin legătură completă

- c) Gruparea prin legătură medie (*group average proximity*). Selecția clusterelor se face prin distanța medie. Este o abordare intermediară între abordările cu legătură simplă și completă (figura 6.34). Definește proximitatea clusterului ca distanța medie între două

puncte, x și y , care se află în clusterelor diferite, A și B (numărul de puncte din clusterul j este n_j):

$$d(A, B) = \sum \{d(x-y) / n_{A \cup B}\} \text{ pentru } x \in A, y \in B$$

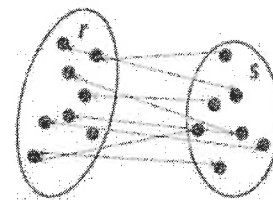


Figura 6.34 Gruparea prin legătură medie

- d) Metoda centroidă (figura 6.35). Selecția clusterelor se face prin distanța minimă. Definește proximitatea clusterului ca distanța medie între centrele de greutate g_A și g_B a două clusterelor diferite, A și B (numărul de puncte din clusterul j este n_j):

$$d(A, B) = \sum \{d(g_A - g_B) / n_{A \cup B}\} \text{ pentru } g_A \in A, g_B \in B$$

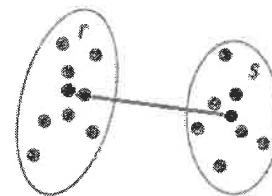


Figura 6.35 Gruparea prin metoda centroidă

- e) Metoda varianței minime sau Ward. Se bazează pe descompunerea varianței totale în varianță intraclase și varianță interclase. Presupune că un cluster este reprezentat de centroidul său și măsoară proximitatea dintre două clusterelor în ceea ce privește creșterea sumei pătratelor erorii (*sum of squared errors - SSE*) care rezultă din fuzionarea celor două clusterelor:

$$d(A, B) = SSE_{A \cup B} - SSE_A - SSE_B$$

unde A și B sunt grupuri. De reținut că pentru clusterizarea ierarhică, SSE începe de la 0.

6.5.3 Distanțe utilizate pentru calculul proximității

Asumând că x și y sunt puncte (indivizi, observații, obiecte) într-un spațiu multidimensional, având coordonatele de forma $x = \{x_1, x_2, \dots, x_n\}$ și respectiv $y = \{y_1, y_2, \dots, y_n\}$, fiecare coordonată x_j, y_j corespunzând valorii variabilei j pentru punctele x și y , cu $j = \overline{1, n}$, atunci vom descrie următoarele distanțe după cum urmează:

Distanța euclidiană:

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + \dots + (x_n - y_n)^2}$$

Distanța euclidiană standardizată:

$$d(x, y) = \sqrt{\left(\frac{x_1 - y_1}{\sigma_1}\right)^2 + \dots + \left(\frac{x_n - y_n}{\sigma_n}\right)^2}$$

unde $\sigma_1, \sigma_2, \dots, \sigma_n$ sunt abaterile standard corespunzătoare fiecărei variabile (dimensiuni).

Distanța Manhattan (*city-block* sau *taxicab*):

$$d(x, y) = |x_1 - y_1| + \dots + |x_n - y_n|$$

Distanța Mahalanobis:

$$d(x, y) = \sqrt{(x - y)^T C^{-1} (x - y)}$$

unde C^{-1} este inversa matricei de varianță-covarianță a setului de date. Dacă matricea de covarianță este matricea identitate, distanța Mahalanobis se reduce la distanța euclidiană. Dacă matricea de covarianță este diagonală, atunci măsurarea distanței rezultată se numește distanță euclidiană standardizată.

Distanța Chebyshev:

$$d(x, y) = \max\{|x_1 - y_1|, \dots, |x_n - y_n|\}$$

Corelația, ca distanță între variabile:

$$d(X, Y) = \text{corr}(X, Y) = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{(X - \bar{X})(Y - \bar{Y})}{\sigma_X \sigma_Y}$$

unde $\text{cov}(X, Y)$ este covarianța dintre variabilele X și Y , în timp ce σ_X și σ_Y sunt abaterile standard corespunzătoare.

6.5.4 Analiza de cluster – studiu de caz

Clasificarea țărilor din Europa după indicatorii de mortalitate³. Prezentul studiu se referă la gruparea țărilor din Europa prin algoritmi de clusterizare ierarhică după indicatorii de mortalitate pe cauze. Indicatorii utilizați în studiu sunt prezentați în tabelul 6.11, în conformitate cu clasificarea pe cauze de mortalitate Eurostat și reprezintă rate de mortalitate la 10,000 de locuitori. Datele se află în fișierul *MortalitateEU.csv* atașat studiului.

³ <https://ec.europa.eu/eurostat/web/health/causes-death/data/main-tables>

Tabelul 6.11 Semnificația variabilelor observate

Cod	Semnificație
<i>Alcohol</i>	Mortalitate determinată de abuzul de alcool
<i>Accidents</i>	Mortalitate determinată de accidente altele decât cele auto
<i>Taccidents</i>	Mortalitate determinată de accidente auto
<i>Heart</i>	Mortalitate determinată de afecțiuni cardiace
<i>ChronicDis</i>	Mortalitate determinată de boli cronice
<i>Liver</i>	Mortalitate determinată de boli hepatice
<i>Cancer</i>	Mortalitate determinată de cancer
<i>DiabetesM</i>	Mortalitate determinată de diabetul zaharat
<i>Drugs</i>	Mortalitate determinată de consumul de droguri
<i>HIV</i>	Mortalitate determinată de HIV
<i>Homicide</i>	Mortalitate determinată de omucideri
<i>Pneumonia</i>	Mortalitate determinată de pneumonii
<i>NervousSystem</i>	Mortalitate determinată de boli ale sistemului nervos
<i>Suicide</i>	Mortalitate determinată de suicid

Vom efectua două clasificări ale instanțelor, prin metoda Ward și prin legătură completă, și o clasificare a indicatorilor prin legătură completă.

6.5.4.1 Implementarea analizei de cluster în Python

Programul ce conține fluxul coordonator de prelucrare este inclus în fișierul *mainAC.py*. Acesta invocă funcții din fișierele *util.py* și *visual.py* al căror conținut este furnizat în **Anexa 1** respectiv **Anexa 2**.

```
import pandas as pd
import numpy as np
import visual.visual as vi
import util.util as utl
import scipy.cluster.hierarchy as hclust
import scipy.spatial.distance as hdist
import sklearn.decomposition as dec
import matplotlib as mpl
```

try:

```
    nume_fisier = 'dataIN/MortalitateEU.csv'
```

```
    mpl.rcParams['figure.max_open_warning'] = 200
    print(mpl.rcParams['figure.max_open_warning'])
```

```
    optiuni_trasare = ["Plot partitie in axe principale", "Trasare histograme", "Grupare variabile"]
```

```
    axe_discriminante = optiuni_trasare.__contains__("Plot partitie in axe principale")
```

```
    histograme = optiuni_trasare.__contains__("Trasare histograme")
```

```
    grupare_variabile = optiuni_trasare.__contains__("Grupare variabile")
```

```
    tabel = pd.read_csv(nume_fisier, na_values=":")
```

```
    utl.inlocuire_na_df(tabel)
```

```
    nume_variabile = list(tabel)
```

```
    coloana_index = 'Code'
```

```
    coloana_etichete = 'Country'
```

```
    print(tabel[coloana_etichete].values)
```

```

tabel.index = [str(v) for v in tabel[coloana_index]]
print(tabel.index)
tabel.index.name = coloana_index
print(tabel.index.name)
nume_instante = tabel[coloana_index].values
print(nume_instante)
variabile_prelucrate = nume_variabile[4:]
X = tabel[variabile_prelucrate].values
Xstd = utl.standardizare(X)
print(Xstd)

# Creare ierarhie instante
metode = list(hclust._LINKAGE_METHODS)
metrici = hdist._METRICS_NAMES

metoda = metode[5]
distanta = metrici[3]

if metoda == 'ward' or metoda == 'centroid' or metoda == 'weighted':
    distanta = 'euclidean'
else:
    distanta = metrici[2]

h = hclust.linkage(Xstd, method=metoda, metric=distanta)
# Identificare partiție de stabilitate maximă
m = np.shape(h)[0] # Numar maxim de jonctiuni
# Se scade din m indexul celei mai mari diferente dintre doua distante de
# jonctiune consecutive
# pentru a determina numarul de clustere din partiția cea mai stabila
k = m - np.argmax(h[1:m, 2] - h[:m-1, 2])
# Identificare clustere in partiția de maxima stabilitate
g_max, coduri = utl.clustere(h, k)
# Afisare cluster
utl.afisare_cluster(g_max, tabel.index, coloana_index, "dataOUT/P_max.csv")

# Determinare culori clustere
culori_clustere = utl.culori_clustere(h, k, coduri)
vi.dendrograma(h, etichete=tabel[coloana_index].values,
    titlu="Grupare instante. Partiția de maxima stabilitate. Metoda:" +
    metoda + ". Metrica:" + distanta, culori=culori_clustere)
if axe_discriminante:
    model_pca = dec.PCA(n_components=2)
    z = model_pca.fit_transform(Xstd)
    grupe = list(set(g_max))
    vi.plot_clustere(z[:, 0], z[:, 1], g_max, grupe, etichete=tabel.index,
        titlu='Partiția optimala')
if histograme:
    for v in variabile_prelucrate:
        vi.histograme(tabel[v].values, g_max, var=v)

# Ierarhie variabile
if grupare_variabile:
    metoda_v = metode[3]
    distanta_v = metrici[7]

h1 = hclust.linkage(X.transpose(), method=metoda_v, metric=distanta_v)
vi.dendrograma(h1, etichete=variabile_prelucrate,
    titlu="Grupare variabile. Metoda:" + metoda_v +
    ". Metrica:" + distanta_v)

```

```

n = np.shape(X)[0]

# Pregatire o lista de selectii pentru partiții incepand de la partiția cu doua
# clustere
lista_selectii = [str(i) + ' clustere' for i in range(2, n - 1)]
partiții = lista_selectii[0:5]

# Creare tabela cu partiția de maxima stabilitate si partițiile selectate
t_partiții = pd.DataFrame(index=tabel.index)
t_partiții['P_max'] = g_max
for v in partiții:
    k = lista_selectii.index(v) + 2 # Numar de clustere dorit
    g, coduri = utl.clustere(h, k)

# Salvare partiție
utl.afisare_cluster(g, tabel.index, coloana_index,
    "dataOUT/P_" + str(k) + ".csv")
culori_clustere = utl.culori_clustere(h, k, coduri)
vi.dendrograma(h, tabel[coloana_index].values,
    titlu='Partiția cu ' + v,
    culori=culori_clustere)
t_partiții["P_" + v] = g
if axe_discriminante:
    model_pca = dec.PCA(n_components=2)
    z = model_pca.fit_transform(Xstd)
    grupe = list(set(g))
    vi.plot_clustere(z[:, 0], z[:, 1], g, grupe, etichete=tabel.index,
        titlu='Partiția cu ' + v)

t_partiții.to_csv("dataOUT/Partiții.csv")

vi.show()
except Exception as ex:
    print("Eroare!", ex.with_traceback(), sep="\n")

```

6.5.4.2 Rezultatele obținute și interpretarea acestora

Clasificarea țărilor prin metoda Ward și metrica euclidiană. Mai întâi, vom determina și analiza partiția optimă, apoi o partiție aleasă după examinarea graficului dendrogramă.

Graficul dendrogramă este prezentat în figura 6.36, iar componența clusterelor în tabelul 6.12. După cum se poate observa, partiția optimă conține două clustere.

Analiza clusterelor se face urmărind distribuția fiecărui indicator pentru fiecare cluster. În acest fel se identifică particularitățile și diferențele dintre clustere. Vom prezenta ulterior câteva distribuții care scot în evidență diferențe clare între cele două clustere ale partiției optime.

Conform histogramelor, în țările din clusterul *c1* mortalitatea determinată de cancer, boli de inimă, boli de ficat, boli cronice și accidente auto este în mod clar mai mare decât la țările din clusterul *c0*. Este vorba de țări aflate în estul Europei.

Tabelul 6.12 Partiția optimală

Cluster	Cod țară
c1	BGR CZE EST HRV LVA LTU HUN ROU SVK
c0	BEL DNK DEU IRL GRC ESP FRA ITA CYP LUX MLT NLD AUT POL PRT SVN FIN SWE GBR ISL LIE NOR CHE SRB TUR

Partiția cu trei clustere este prezentată în tabelul 6.13. Graficul dendrogramă și graficul instanțelor în axele discriminante sunt prezentate în anexă. În această partiție apar două clustere pentru țările din estul Europei (c0 și c2), rezultate din scindarea clusterului c1 din partiția optimală. La finalul acestui studiu de caz sunt prezentate o serie de grafice histogramă pentru incidența cancerului și a bolilor cronice. Graficele indică o mortalitate mai mare în clusterul c2 (LVA LTU HUN SVK). Pentru celelalte cauze diferențele sunt mai mici.

Tabelul 6.13 Partiția cu 3 clustere

Clustere	Cod țară
c2	LVA LTU HUN SVK
c1	BEL DNK DEU IRL GRC ESP FRA ITA CYP LUX MLT NLD AUT POL PRT SVN FIN SWE GBR ISL LIE NOR CHE SRB TUR
c0	BGR CZE EST HRV ROU

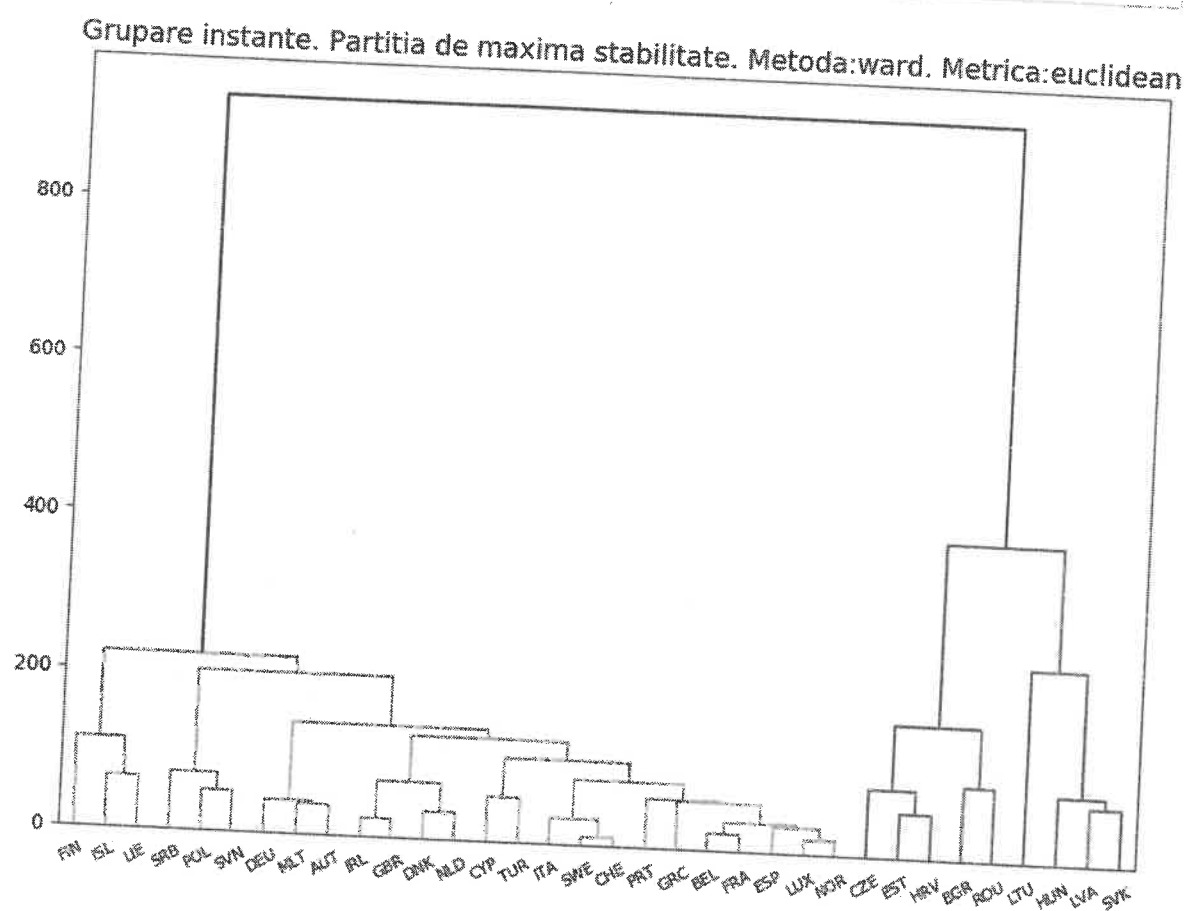


Figura 6.36 Graficul dendrogramă pentru partiția optimală

La finalul prezentării rezultatelor obținute sunt sintetizate grupările pe mai multe partiții și partiția optimală.

Clasificarea țărilor prin legătură completă și metrică Mahalanobis. În figura 6.36 este prezentat graficul dendrogramă cu evidențierea partiției optimale. Clasificarea în metrică Mahalanobis furnizează clustere coerente cu varianță intra-cluster mică și varianță inter-cluster mare (figura 6.37). Se observă existența multor clustere singleton (formate dintr-o singură instanță). Distanțele de jonctionare sunt mai apropiate între ele, prin urmare partițiile sunt uniform repartizate în graficul ierarhie. În tabelele 6.14 și 6.15 sunt prezentate clusterele pentru partiția optimală și cea cu 4 clustere. Se observă formarea unor clustere firești, dacă se ține cont de posibili factori care influențează mortalitatea pe cauze (dezvoltare economică, tradiții, cultură etc.). Tabelul clasificării cu mai multe partiții este prezentat la finalul acestui studiu.

Tabelul 6.14 Partiția optimală – gruparea prin metrică Mahalanobis

Cluster	Cod țară
c3	FIN
c1	LTU
c10	DEU MLT AUT SWE SRB
c6	CYP TUR
c8	CZE FRA HRV HUN NLD SVK GBR NOR CHE
c9	EST LVA
c7	BEL DNK IRL ESP ITA LUX POL SVN ISL
c0	GRC
c2	PRT
c5	BGR ROU
c4	LIE

Tabelul 6.15 Partiția cu 4 clustere – gruparea prin metrică Mahalanobis

Cluster	Cod țară
c1	EST LVA LTU
c0	PRT
c2	BGR CZE GRC FRA HRV CYP HUN NLD ROU SVK GBR LIE NOR CHE TUR
c3	BEL DNK DEU IRL ESP ITA LUX MLT AUT POL SVN FIN SWE ISL SRB

Grupare instanțe. Partitia de maxima stabilitate. Metoda:complete. Metrica:mahalanobis

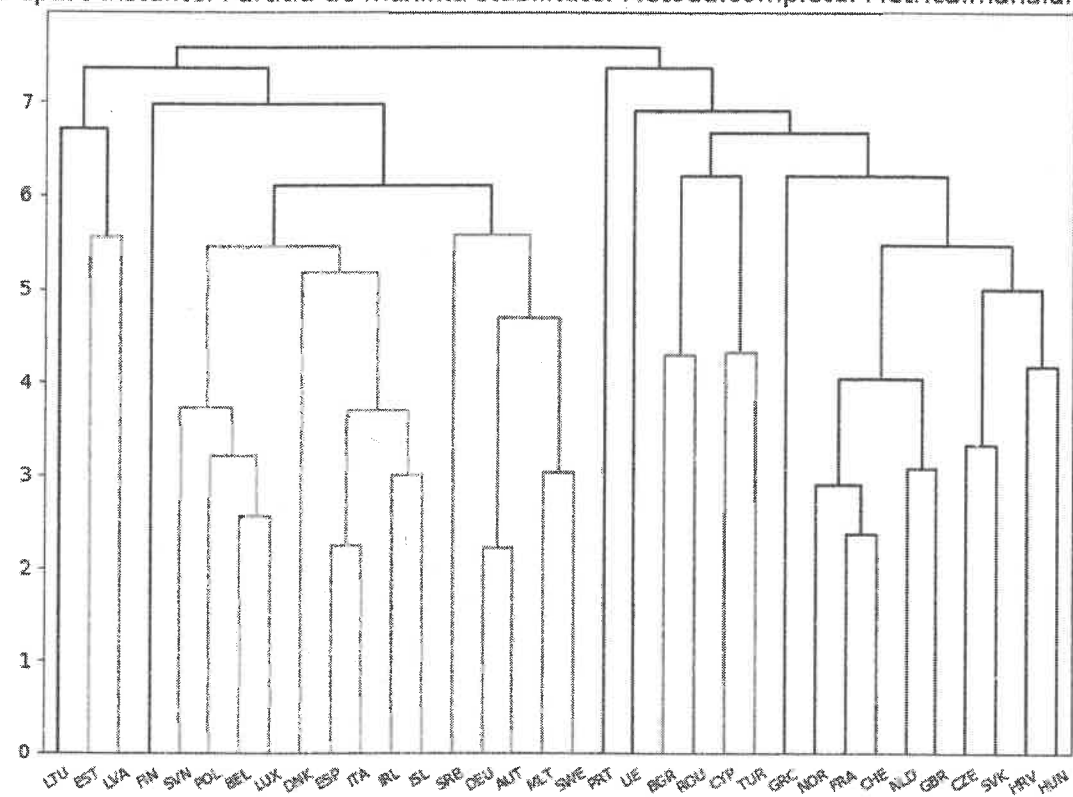


Figura 6.37 Gruparea prin legătură completă și metrică Mahalanobis

6.5.4.3 Clasificarea variabilelor

Deși se utilizează, în general, pentru clasificare de instanțe, analiza de clusteri poate fi utilizată și pentru grupare de variabile dacă sunt alese metrice potrivite. Pachetul Python *scipy* are implementate metrice pentru variabile, cum ar fi distanța bazată pe coeficienții de corelație. Distanța dintre două variabile este calculată astfel:

$$d_R^2(x, y) = 1 - R_{xy}^2.$$

În figura 6.38 este prezentat graficul dendrogramă cu evidențierea partiției optimale. Metoda de grupare este cea prin media legăturilor și ca metrică este utilizat coeficientul de corelație dintre variabile.

Grupare variabile. Metoda:average. Metrica:correlation

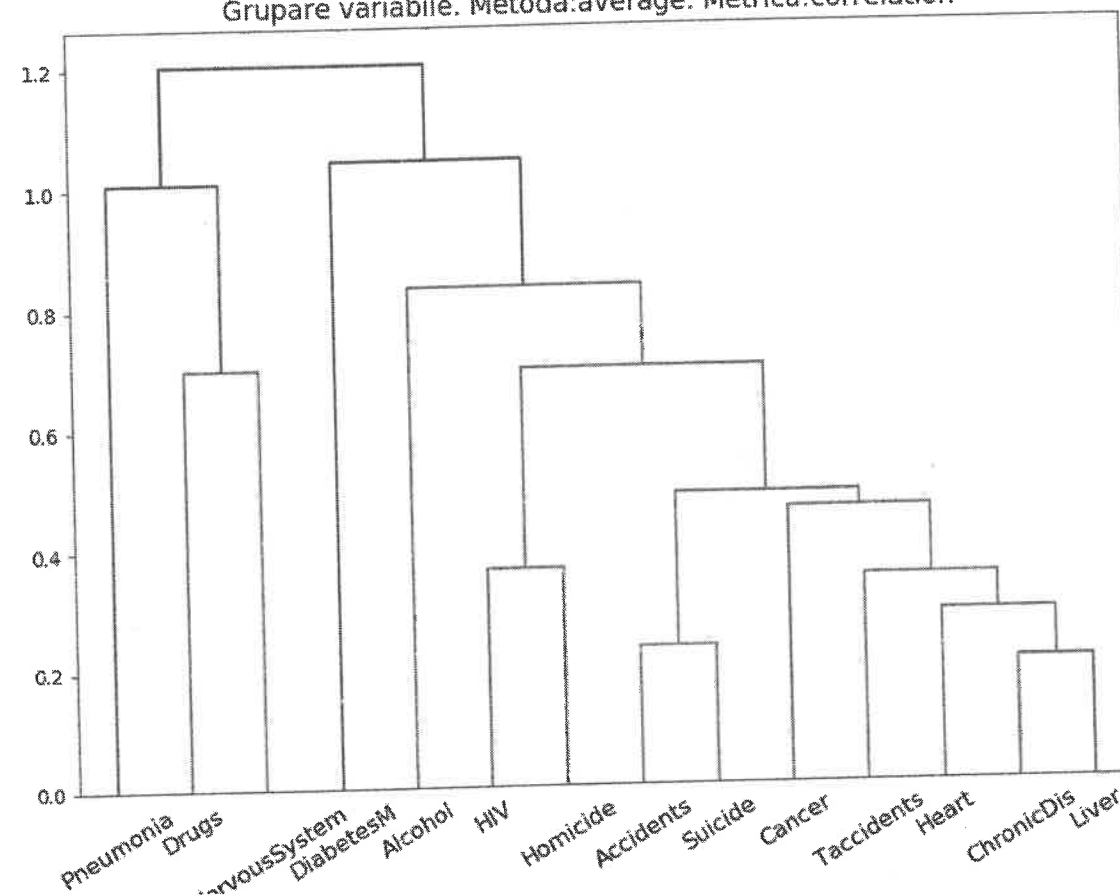


Figura 6.38 Gruparea variabilelor prin media legăturilor

Prezentăm în continuare o sinteză a graficelor și tabelor generate în urma rulării programului `mainAC.py`.

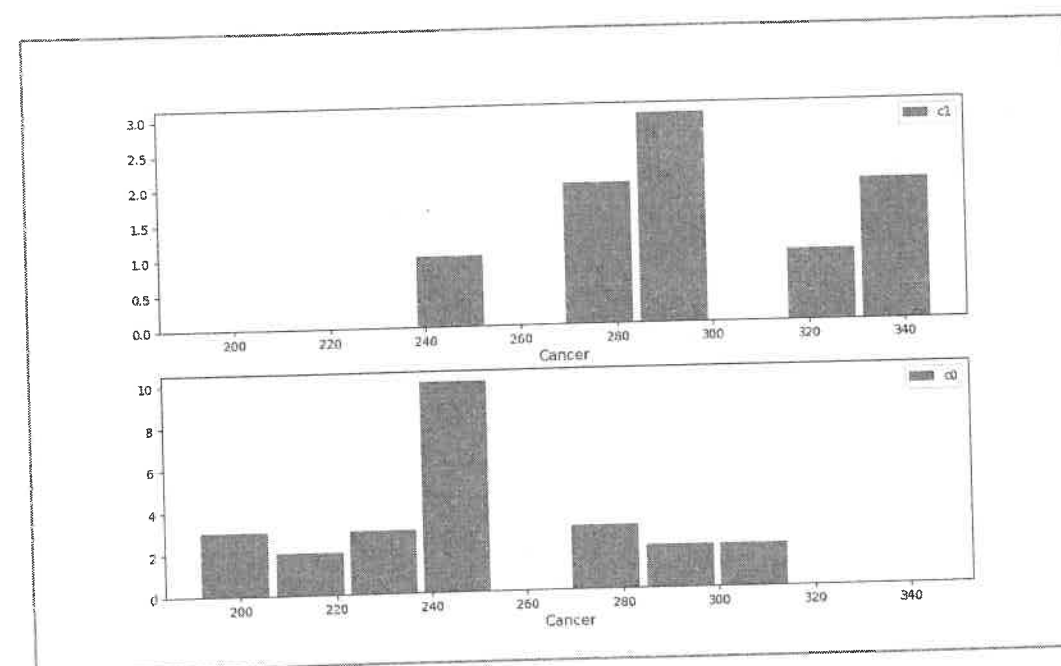


Figura 6.39 Partiția optimală. Histograma pentru cancer

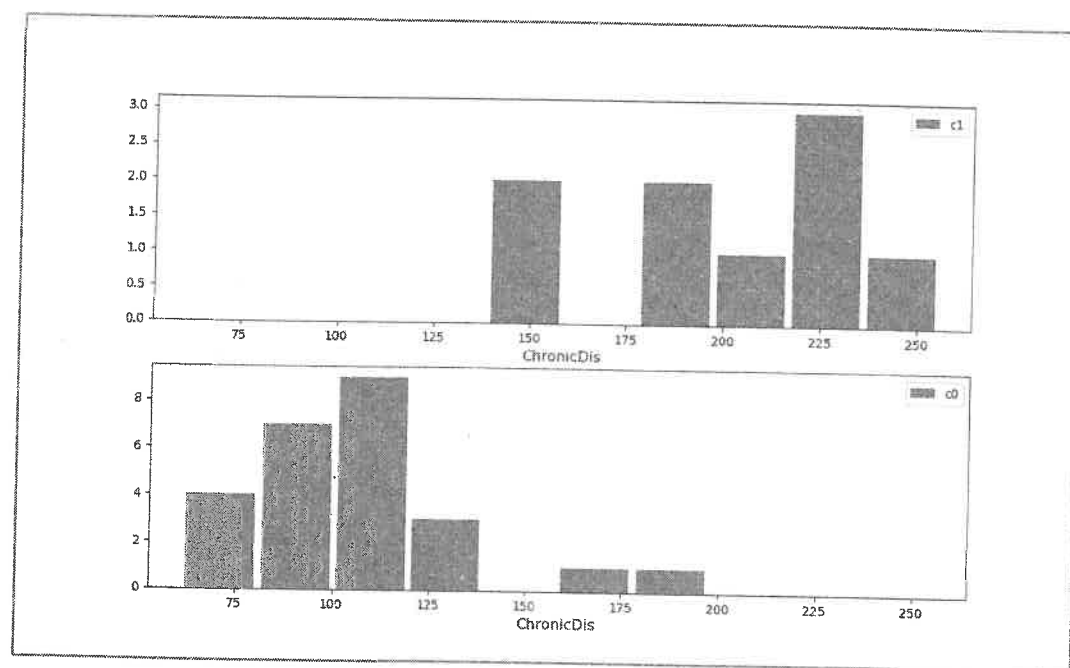


Figura 6.40 Partiția optimală. Histograma pentru boli cronice

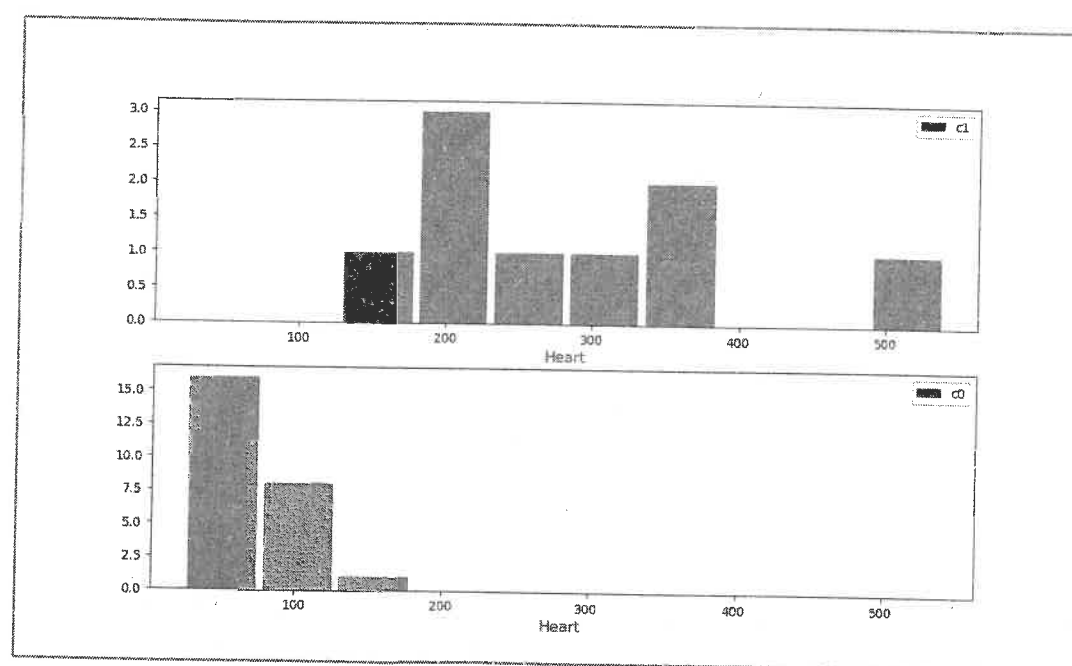


Figura 6.41 Partiția optimală. Histograma pentru boli inimă

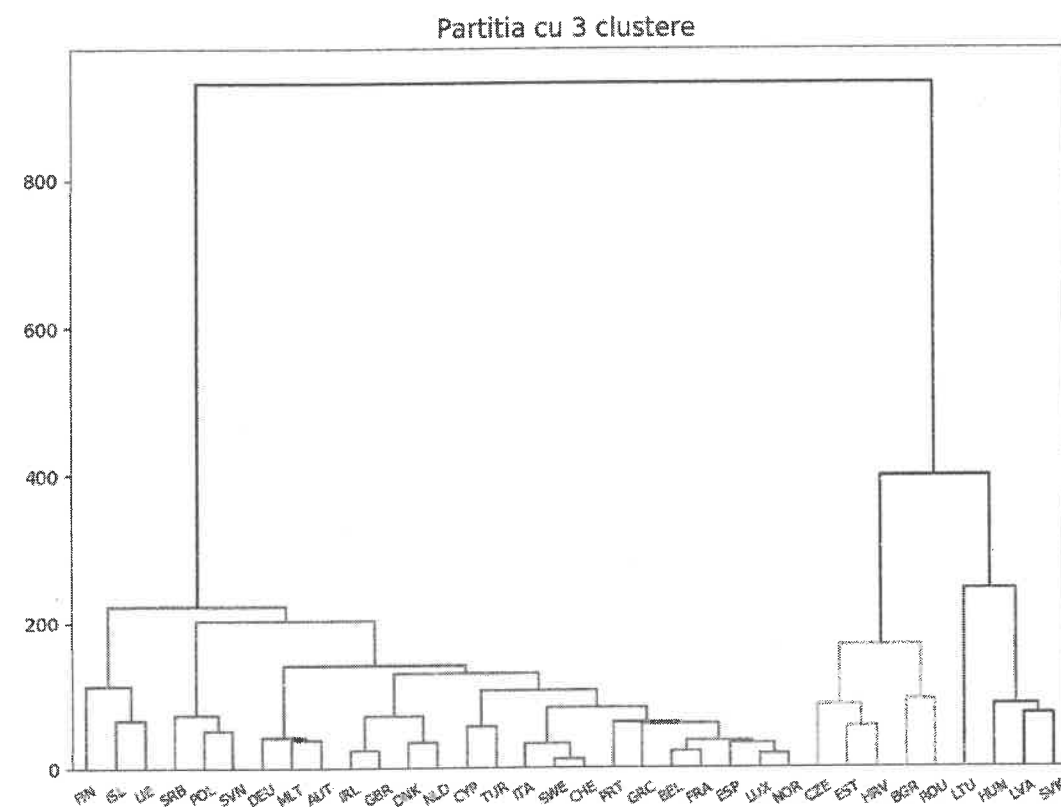


Figura 6.42 Dendrogram care evidențiază partiția cu 3 cluster

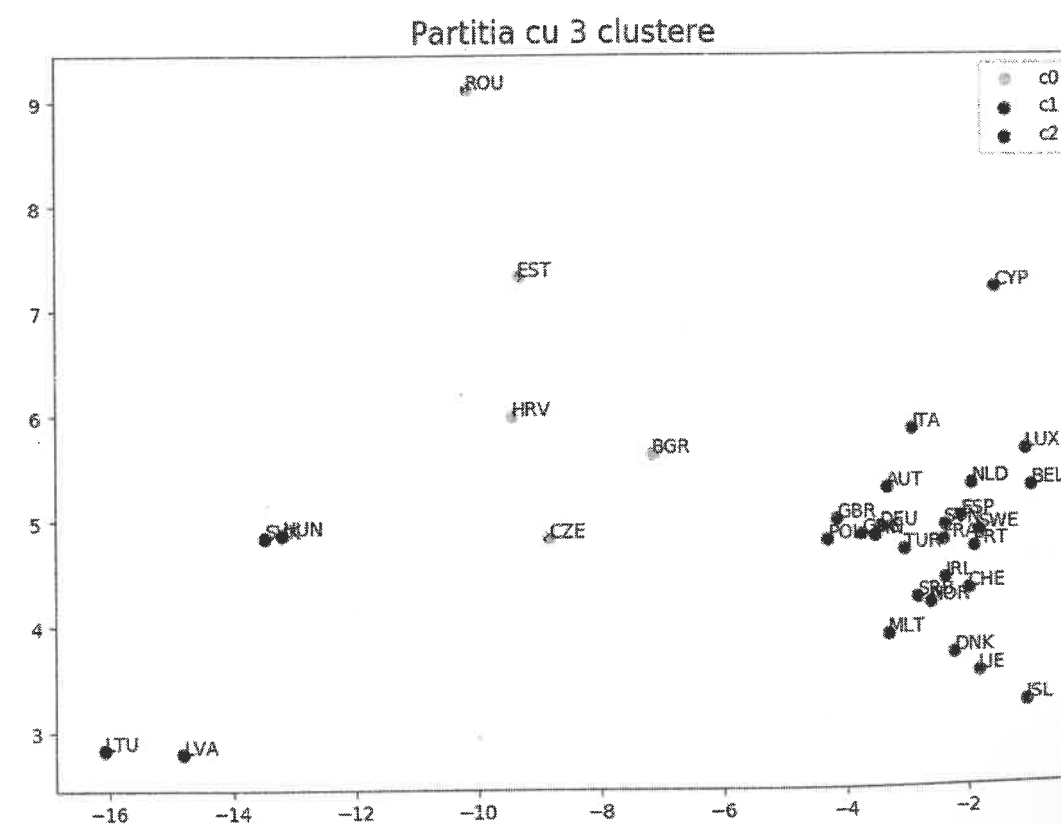


Figura 6.43 Reprezentarea în axe discriminante a partiției cu 3 cluster

Tabelul 6.16 Gruparea instanțelor prin metoda Ward și metrică euclidiană

Code	P_max	P_3 clustere	P_4 clustere	P_5 clustere	P_6 clustere
BEL	c0	c1	c3	c4	c4
BGR	c1	c0	c2	c3	c5
CZE	c1	c0	c2	c3	c5
DNK	c0	c1	c3	c4	c4
DEU	c0	c1	c3	c4	c4
EST	c1	c0	c2	c3	c5
IRL	c0	c1	c3	c4	c4
GRC	c0	c1	c3	c4	c4
ESP	c0	c1	c3	c4	c4
FRA	c0	c1	c3	c4	c4
HRV	c1	c0	c2	c3	c5
ITA	c0	c1	c3	c4	c4
CYP	c0	c1	c3	c4	c4
LVA	c1	c2	c1	c1	c2
LTU	c1	c2	c0	c0	c0
LUX	c0	c1	c3	c4	c4
HUN	c1	c2	c1	c1	c2
MLT	c0	c1	c3	c4	c4
NLD	c0	c1	c3	c4	c4
AUT	c0	c1	c3	c4	c4
POL	c0	c1	c3	c4	c1
PRT	c0	c1	c3	c4	c4
ROU	c1	c0	c2	c3	c5
SVN	c0	c1	c3	c4	c1
SVK	c1	c2	c1	c1	c2
FIN	c0	c1	c3	c2	c3
SWE	c0	c1	c3	c4	c4
GBR	c0	c1	c3	c4	c4
ISL	c0	c1	c3	c2	c3
LIE	c0	c1	c3	c2	c3
NOR	c0	c1	c3	c4	c4
CHE	c0	c1	c3	c4	c4
SRB	c0	c1	c3	c4	c1
TUR	c0	c1	c3	c4	c4

Tabelul 6.17 Gruparea instanțelor prin legătură completă și metrică Mahalanobis

Code	P_max	P_2 clustere	P_3 clustere	P_4 clustere	P_5 clustere	P_6 clustere
BEL	c7	c0	c2	c3	c2	c3
BGR	c5	c1	c1	c2	c4	c4
CZE	c8	c1	c1	c2	c4	c4
DNK	c7	c0	c2	c3	c2	c3
DEU	c10	c0	c2	c3	c2	c3
EST	c9	c0	c2	c1	c3	c5
IRL	c7	c0	c2	c3	c2	c3
GRC	c0	c1	c1	c2	c4	c4
ESP	c7	c0	c2	c3	c2	c3
FRA	c8	c1	c1	c2	c4	c4
HRV	c8	c1	c1	c2	c4	c4
ITA	c7	c0	c2	c3	c2	c3
CYP	c6	c1	c1	c2	c4	c4
LVA	c9	c0	c2	c1	c3	c5
LTU	c1	c0	c2	c1	c3	c5
LUX	c7	c0	c2	c3	c2	c3
HUN	c8	c1	c1	c2	c4	c4
MLT	c10	c0	c2	c3	c2	c3
NLD	c8	c1	c1	c2	c4	c4
AUT	c10	c0	c2	c3	c2	c3
POL	c7	c0	c2	c3	c2	c3
PRT	c2	c1	c0	c0	c0	c0
ROU	c5	c1	c1	c2	c4	c4
SVN	c7	c0	c2	c3	c2	c3
SVK	c8	c1	c1	c2	c4	c4
FIN	c3	c0	c2	c3	c1	c1
SWE	c10	c0	c2	c3	c2	c3
GBR	c8	c1	c1	c2	c4	c4
ISL	c7	c0	c2	c3	c2	c3
LIE	c4	c1	c1	c2	c4	c2
NOR	c8	c1	c1	c2	c4	c4
CHE	c8	c1	c1	c2	c4	c4
SRB	c10	c0	c2	c3	c2	c3
TUR	c6	c1	c1	c2	c4	c4

Bibliografie

1. Gheorghe Ruxanda, *Analiza datelor*, Editura ASE, Bucuresti, 2001.
2. Murtagh, F., Heck, A., *Multivariate data analysis*, Dordrecht, 1987.
3. Jiawei Han, Micheline Kamber, *Data Mining. Concepts and Techniques*, Morgan Kaufman Publishers, 2006.
4. Wes McKinney, *Python for Data Analysis – Data Wrangling with Pandas, Numpy, and Python*, Second Edition, O'Reilly Media, 2018.
5. Vergil Voineagu, Felix Furtună, Maria Elena Voineagu, Codrin Ștefănescu, *Analiza factorială a fenomenelor social-economice în profil regional*, Editura Aramis, București, 2002.
6. Diday, E., Pouget, J., Lemaire, J., Testu, F., *Elements d'analyse de donnee*, Dunod, Paris, 1985.
7. Brett Slatkin, *Effective Python - 59 Specific Ways to Write Better Python*, Addison-Wesley, 2015.
8. Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, Donald B. Rubin, *Bayesian Data Analysis*, Third Edition, CRC Press - Taylor & Francis Group, 2013.
9. ***, *The Python Standard Library*, <https://docs.python.org/3/library/index.html>, 2020.
10. ***, *The Python Language Reference*, <https://docs.python.org/3/reference/index.html>, 2020.
11. ***, *NumPy v1.19 Manual*, <https://numpy.org/doc/stable/index.html>
12. ***, *scikit-learn. Machine Learning in Python*, <https://scikit-learn.org/stable>, 2020.
13. Wes McKinney and the Pandas Development Team, *pandas: powerful Python data analysis.*
14. *toolkit. Release 1.1.4*, <https://pandas.pydata.org/docs/pandas.pdf>, 2020.
15. ***, *Scipy Reference*, <https://docs.scipy.org/doc/scipy/reference/index.html>, 2020.
16. ***, *Matplotlib: Visualization with Python*, <https://matplotlib.org/3.3.1/index.html>, 2020.
17. ***, *Seaborn: statistical data visualization*, <https://seaborn.pydata.org/>

Anexe

Anexa 1

Conținutul fișierului util.py

```
import numpy as np
import pandas as pd
import scipy.stats as sts
import pandas.api.types as pdt
import sklearn.preprocessing as pp
import collections as co
import scipy.linalg as lin
import sklearn.discriminant_analysis as disc
import visual.visual as vi
import scipy.stats as sstats

# Aplicare teste chi2 pentru un tabel t, la variabilele categoriale specificate
def chi2_tabel(t, variabile):
    m = len(variabile)
    p_value = np.zeros(shape=(m, m), dtype=np.float64)
    for i in range(m - 1):
        for j in range(i + 1, m):
            t_contingenta = pd.crosstab(t[variabile[i]], t[variabile[j]])
            test = sstats.chi2_contingency(t_contingenta)
            p_value[i, j] = test[1]
            p_value[j, i] = p_value[i, j]
    return p_value

# t - matricea frecventelor absolute (ndarray)
def corresp_benzecri(t):
    p, q = np.shape(t)
    n = np.sum(t)
    v_12 = t / n
    v_11inv = np.diag(1 / np.sum(v_12, axis=1))
    h1 = v_11inv @ v_12
    v_21 = np.transpose(v_12)
    v_22inv = np.diag(1 / np.sum(v_21, axis=1))
    h2 = v_22inv @ v_21
    if p <= q:
        h = h1 @ h2
    else:
        h = h2 @ h1
    valp, vecp = np.linalg.eig(h)
    k_inv = np.flipud(np.argsort(valp))
    alpha = np.real(valp[k_inv[1:]])
    if p <= q:
        a = np.real(vecp[:, k_inv[1:]])
        b = (h2 @ a) / np.sqrt(alpha)
    else:
        b = np.real(vecp[:, k_inv[1:]])
        a = (h1 @ b) / np.sqrt(alpha)
    return alpha, a, b
```

```
def corresp_greenacre(t):
    p, q = np.shape(t)
    m = min(p, q) - 1
    n = np.sum(t)
    f = t / n
    f_lin = np.sum(f, axis=1).reshape(p, 1)
    f_col = np.sum(f, axis=0).reshape(q, 1)
    f_ = f_lin @ np.transpose(f_col)
    r = (f - f_) / np.sqrt(f_)
    r2 = r * r
    chi2 = np.sum(r2) * n
    g_lib = (p - 1) * (q - 1)
    p_value = 1 - sts.chi2.cdf(chi2, g_lib)
    # print(np.round(r,4))
    u, s, ht = np.linalg.svd(r, full_matrices=False)
    # Coordonatele linie
    l = np.diag(1 / np.sqrt(f_lin[:, 0])) @ u[:, :m] @ np.diag(s[:m])
    c = np.diag(1 / np.sqrt(f_col[:, 0])) @ np.transpose(ht[:m, :]) @ np.diag(s[:m])
    return l, c, r, chi2, p_value
```

Calcul centrui, frecvente grupe, etichete grupe si matrice de imprastiere

x - tabel date

y - variabila de grupare

def imprastiere(x, y):

n, m = np.shape(x)

medii = np.mean(x, axis=0)

counter = co.Counter(y)

g = np.array([i for i in counter.keys()]) # preluare etichete

ng = np.array([i for i in counter.values()]) # preluare frecvente

q = len(g)

xg = np.ndarray(shape=(q, m))

for k, i in zip(g, range(len(g))):

 xg[i, :] = np.mean(x[y == k, :], axis=0)

xg_med = xg - medii

sst = n * np.cov(x, rowvar=False, bias=True)

ssb = np.transpose(xg_med) @ np.diag(ng) @ xg_med

ssw = sst - ssb

return g, ng, xg, sst, ssb, ssw

def standardizare(x):

medii = np.mean(x, axis=0)

abaterestd = np.std(x, axis=0)

Xstd = (x - medii) / abaterestd

return Xstd

def centrare(x):

medii = np.mean(x, axis=0)

return (x - medii)

Regularizare vectori proprii

def regularizare(t, y=None):

if type(t) is pd.DataFrame:

if isinstance(t, pd.DataFrame):

 for c in t.columns:

 minim = t[c].min()

 maxim = t[c].max()

 if abs(minim) > abs(maxim):

 t[c] = -t[c]

 if y is not None:

 k = t.columns.get_loc(c) # determina indexul coloanei

 y[:, k] = -y[:, k]

if isinstance(t, np.ndarray):

 for i in range(np.shape(t)[1]):

 minim = np.min(t[:, i])

 maxim = np.max(t[:, i])

 if np.abs(minim) > np.abs(maxim):

 t[:, i] = -t[:, i]

return None

Regularizare vectori proprii

def regularizare2(x, y):

 for i in range(np.shape(x)[1]):

 minim = np.min(x[:, i])

 maxim = np.max(x[:, i])

 if np.abs(minim) > np.abs(maxim):

 x[:, i] = -x[:, i]

 y[:, i] = -y[:, i]

return None

def acp(X):

 R = np.corrcoef(X, rowvar=False)

 # calcul vector si valori proprii

 valp, vecp = np.linalg.eig(R)

 # sortare valori proprii si vectori proprii

 k_inv = [k for k in reversed(np.argsort(valp))]

 alpha = valp[k_inv]

 a = vecp[:, k_inv]

 regularizare(a)

 # calcul corelatii factoriale

 Rxc = a * np.sqrt(alpha)

 # calcul componente

 # standardizare X

 medii = np.mean(X, axis=0)

 abaterestd = np.std(X, axis=0)

 Xstd = (X - medii) / abaterestd

 C = Xstd @ a

 return R, alpha, a, Rxc, C

Functie pentru inlocuirea valorilor lipsa

prin medie/modul pentru un papandas DataFrame

def inlocuire_na_df(t):

 for c in t.columns:

 if pdt.is_numeric_dtype(t[c]):

 if t[c].isna().any():

 medie = t[c].mean()

 t[c] = t[c].fillna(medie)

 else:

 if t[c].isna().any():

 modul = t[c].mode()

 t[c] = t[c].fillna(modul[0])

return None

Functie pentru inlocuirea valorilor lipsa

prin medie pentru un numpy ndarray

def inlocuire_na(X):

 medii = np.nanmean(X, axis=0)

 k_nan = np.where(np.isnan(X))

 X[k_nan] = medii[k_nan[1]]

return None

```
def tabelare_varianta(alpha):
    m = len(alpha)
    varianta_cumulata = np.cumsum(alpha)
    procent_varianta = alpha * 100 / m
    procent_cumulat = np.cumsum(procent_varianta)
    tabel_varianta = pd.DataFrame(data={"Varianta": alpha,
                                       "Varianta Cumulata": varianta_cumulata,
                                       "Procent varianta": procent_varianta,
                                       "Procent cumulat": procent_cumulat
                                      })

    return tabel_varianta
```

```
def tabelare(X, nume_coloane=None, nume_instante=None, tabel=None):
    X_tab = pd.DataFrame(X)
    if nume_coloane is not None:
        X_tab.columns = nume_coloane
    if nume_instante is not None:
        X_tab.index = nume_instante
    if tabel is None:
        X_tab.to_csv("tabel.csv")
    else:
        X_tab.to_csv(tabel)
    return X_tab
```

```
def evaluare(C, alpha, R):
    n = np.shape(C)[0]
    # Calcul scoruri
    S = C / np.sqrt(alpha)
    # Calcul cosinusuri
    C2 = C * C
    sum1 = np.sum(C2, axis=1)
    q = np.transpose(np.transpose(C2) / sum1)
    # Calcul contributii
    beta = C2 / (alpha * n)
    # Calcul comunalitati
    R2 = R * R
    Comun = np.cumsum(R2, axis=1)
    return S, q, beta, Comun
```

```
def bartlett_test(n, l, x, e):
    m, q = np.shape(l)
    v = np.corrcoef(x, rowvar=False)
    psi = np.diag(e)
    v_ = l @ np.transpose(l) + psi
    I_ = np.linalg.inv(v_) @ v
    det_v_ = np.linalg.det(I_)
    urma = np.trace(I_)
    chi2 = (n - 1 - (2 * m + 4 * q - 5) / 2) * (urma - np.log(det_v_) - m)
    g_lib = ((m - q) * (m - q) - m - q) / 2
    p_value = sts.chi2.cdf(chi2, g_lib)
    return chi2, p_value
```

```
def bartlett_factor(x):
    n, m = np.shape(x)
    r = np.corrcoef(x, rowvar=False)
    chi2 = -(n - 1 - (2 * m + 5) / 6) * np.log(np.linalg.det(r))
    g_lib = m * (m - 1) / 2
    p_value = 1 - sts.chi2.cdf(chi2, g_lib)
    return chi2, p_value
```

```
def bartlett_wilks_test(r, n, p, q, m):
    r_inv = np.flipud(r)
    l = np.flipud(np.cumprod(1 - r_inv * r_inv))
    dof = (p - np.arange(m)) * (q - np.arange(m))
    chi2 = (-n + 1 + (p + q + 1) / 2) * np.log(l)
    p_value = 1 - sts.chi2.cdf(chi2, dof)
    return p_value, chi2, dof
```

Analiza corelatiilor canonice

```
def cca(x, y):
    n, p = np.shape(x)
    q = np.shape(y)[1]
    x = pp.StandardScaler(with_std=False).fit_transform(x)
    y = pp.StandardScaler(with_std=False).fit_transform(y)
    vx = np.cov(x, rowvar=False)
    vy = np.cov(y, rowvar=False)
    cov = np.cov(x, y, rowvar=False)
    vxy = cov[:p, p:]
    vyx = np.transpose(vxy)
    vx_inv = np.linalg.inv(vx)
    vy_inv = np.linalg.inv(vy)
    h1 = vx_inv @ vxy
    h2 = vy_inv @ vyx
    m = min(p, q)
    if p == m:
        h = h1 @ h2
        valp, vecp = np.linalg.eig(h)
        k_inv = [k for k in reversed(np.argsort(valp))]
        r2 = valp[k_inv]
        a = vecp[:, k_inv]
        r = np.sqrt(r2)
        b = (h2 @ a) @ np.diag(1 / r)
        z = x @ a
        u = y @ b
    else:
        h = h2 @ h1
        valp, vecp = np.linalg.eig(h)
        k_inv = [k for k in reversed(np.argsort(valp))]
        r2 = valp[k_inv]
        b = vecp[:, k_inv]
        r = np.sqrt(r2)
        a = (h1 @ b) @ np.diag(1 / r)
        z = x @ a
        u = y @ b
    z = pp.normalize(z, axis=0)
    u = pp.normalize(u, axis=0)
    return r, r2, z, u
```

Codificarea variabilelor categoriale

```
def codificare(t, vars):
    for v in vars:
        t[v] = pd.Categorical(t[v]).codes
    return None
```

Analiza discriminanta liniara (ADL)

```
def adl(sst, ssb, n, q):
    m = len(sst)
    cov_inv = np.linalg.inv(sst)
    h = cov_inv @ ssb
```

```

if np.allclose(h, np.transpose(h)):
    valp, vecp = np.linalg.eig(h)
else:
    c = lin.sqrtrm(ssb)
    h = np.transpose(c) @ cov_inv @ c
    valp, vecp_ = np.linalg.eig(h)
    vecp = cov_inv @ c @ vecp_
    k_inv = np.flipud(np.argsort(valp))
    r = min(m, q - 1)
    alpha = np.real(valp[k_inv[r]])
    u = np.real(vecp[:, k_inv[r]])
    regularizare(u)
    l = alpha * (n - q) / ((1 - alpha) * (q - 1))
    return alpha, l, u

```

Calcul functii de clasificare pentru ADL si pentru Bayes

```

def functii_clasificare(x, xg, cov, ng):
    n = np.shape(x)[0]
    q = np.shape(xg)[0]
    cov_inv = np.linalg.inv(cov / n)
    f = xg @ cov_inv
    f0 = np.empty(shape=(q,))
    for i in range(q):
        f0[i] = -0.5 * f[i, :] @ xg[i, :]
    f0_b = f0 + np.log(ng / n) # Termenii Liberi pentru Bayes
    return f, f0, f0_b

```

Calcul functii de clasificare pe variabile discriminate

```

def functii_clasificare_z(z, zg, ng):
    n = np.shape(z)[0]
    q = np.shape(zg)[0]
    cov_inv = np.diag(1.0 / np.var(z, axis=0))
    f = zg @ cov_inv
    f0 = np.empty(shape=(q,))
    for i in range(q):
        f0[i] = -0.5 * f[i, :] @ zg[i, :]
    f0_b = f0 + np.log(ng / n) # Termenii Liberi pentru bayes
    return f, f0, f0_b

```

Predictie pe baza scorurilor de clasificare bayesiene

```

def predict_bayes(x, xg, cov, ng, g):
    n, m = np.shape(x)
    cov = cov / n
    q = len(g)
    dist = np.linalg.inv(cov)
    clasif = np.empty(shape=(n,), dtype=np.int64)
    s = np.empty(shape=(n, q))
    log_p_apriori = 2 * np.log(ng / n)
    for i in range(n):
        for k in range(q):
            d = (x[i, :] - xg[k, :]) @ dist @ (x[i, :] - xg[k, :])
            s[i, k] = log_p_apriori[k] - d
        clasif[i] = np.argmax(s[i, :])
    return g[clasif]

```

Predictie pe baza functiilor de clasificare

```

def predict(x, f, f0, g):
    n, m = np.shape(x)
    clasif = np.empty(shape=(n,), dtype=np.int64)

```

```

for i in range(n):
    rez = f @ x[i, :] + f0
    clasif[i] = np.argmax(rez)
return g[clasif]

```

```

def discrim_acuratete(y, clasif, g):
    q = len(g)
    n = len(y)
    mat_c = pd.DataFrame(data=np.zeros((q, q)), index=g, columns=g)
    for i in range(n):
        mat_c.loc[y[i], clasif[i]] += 1
    acuratete_grupe = np.diag(mat_c) * 100 / np.sum(mat_c, axis=1)
    mat_c['acuratete'] = acuratete_grupe
    return mat_c

```

```

def putere_discriminare(ssb, ssw, n, q):
    r = (n - q) / (q - 1)
    f = r * np.diag(ssb) / np.diag(ssw)
    p_value = 1 - sts.f.cdf(f, q - 1, n - q)
    return f, p_value

```

```

def adl_cluster(x, g):
    lda = disc.LinearDiscriminantAnalysis()
    lda.fit(x, g)
    u = lda.scalings_
    regularizare(u)
    zg = lda.means_ @ u
    z = x @ u
    grupe = lda.classes_
    return z, zg, grupe

```

```

def clustere(h, k):
    n = np.shape(h)[0] + 1
    g = np.arange(0, n)
    for i in range(n - k):
        k1 = h[i, 0]
        k2 = h[i, 1]
        g[g == k1] = n + i
        g[g == k2] = n + i
    g_ = pd.Categorical(g)
    return ['c' + str(i) for i in g_.codes], g_.codes

```

```

def afisare_cluster(g, etichete, nume_etichete, fisier):
    g_ = np.array(g)
    grupe = list(set(g))
    m = len(grupe)
    tabel = pd.DataFrame(index=grupe)
    clustere = np.full(shape=(m,), fill_value="", dtype=np.chararray)
    for i in range(m):
        cluster = etichete[g_ == grupe[i]]
        cluster_str = ""
        for v in cluster:
            cluster_str = cluster_str + (v + " ")
        clustere[i] = cluster_str
    tabel[nume_etichete] = clustere
    tabel.to_csv(fisier)

```

```

# h - ierarhia
# k - numar de culori
# coduri - codurile clusterelor
def culori_clustere(h, k, coduri):
    culori = np.array(vi._CULORI)
    nr_culori = len(culori)
    m = np.shape(h)[0]
    n = m + 1
    culori_clustere = np.full(shape=(2 * n * 1,), fill_value="", dtype=np.chararray)
    # stabilire culoare clustere singleton
    for i in range(n):
        culori_clustere[i] = culori[coduri[i] % nr_culori]
    # stabilire culoare jonctiuni
    for i in range(m):
        k1 = int(h[i, 0])
        k2 = int(h[i, 1])
        if culori_clustere[k1] == culori_clustere[k2]:
            culori_clustere[n + i] = culori_clustere[k1]
        else:
            culori_clustere[n + i] = 'k'
    return culori_clustere

```

Anexa 2

Conținutul fișierului visual.py

```

import seaborn as sb
import matplotlib.pyplot as plt
import matplotlib.colors as color
import numpy as np
import scipy.cluster.hierarchy as hclust
import statsmodels.graphics.mosaicplot as smosaic
import pandas as pd

_CULORI = ['y', 'r', 'b', 'g', 'c', 'm', 'sienna', 'coral', 'darkblue', 'lime', 'grey',
           'tomato', 'indigo', 'teal', 'orange', 'darkgreen']

# x sy y sunt masive numpy cu doua coloane
def biplot(x, y, xlabel="x", ylabel="y", title="Biplot", l1=None, l2=None):
    f = plt.figure(figsize=(10, 7))
    ax = f.add_subplot(1, 1, 1)
    assert isinstance(ax, plt.Axes)
    ax.set_title(title, fontsize=14)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.scatter(x[:, 0], x[:, 1], c='r', label='Set X')
    ax.scatter(y[:, 0], y[:, 1], c='b', label='Set Y')
    if l1 is not None:
        for i in range(len(l1)):
            ax.text(x[i, 0], x[i, 1], l1[i])
    if l2 is not None:
        for i in range(len(l2)):
            ax.text(y[i, 0], y[i, 1], l2[i])
    ax.legend()

def corelograma(t, title=None, valmin=-1, valmax=1):
    f = plt.figure(figsize=(8, 7))
    f1 = f.add_subplot(1, 1, 1)
    f1.set_title(title, fontsize=16, color='b', verticalalignment='bottom')
    f1.tick_params(axis='x', rotation=30)
    sb.heatmap(np.round(t, 2), cmap='bwr', vmin=valmin, vmax=valmax, annot=True, ax=f1)

def plot_varianta(alpha, titlu='Plot varianta'):
    n = len(alpha)
    f = plt.figure(titlu, figsize=(10, 7))
    f1 = f.add_subplot(1, 1, 1)
    f1.set_title(titlu, fontsize=16, color='b', verticalalignment='bottom')
    f1.set_xticks(np.arange(1, n + 1))
    f1.set_xlabel('Componenta', fontsize=12, color='r', verticalalignment='top')
    f1.set_ylabel('Varianta', fontsize=12, color='r', verticalalignment='bottom')
    f1.plot(np.arange(1, n + 1), alpha, 'ro-')
    f1.axhline(1, c='g')
    j_Kaiser = np.where(alpha < 1)[0][0]
    eps = alpha[:n - 1] - alpha[1:]
    d = eps[:n - 2] - eps[1:]
    conditie = d < 0
    if(conditie.any()):

```

```

        j_Cattel = np.where(d < 0)[0][0]
        f1.axhline(alpha[j_Cattel + 1], c='m')
    else:
        j_Cattel = j_Kaiser-2
    return j_Cattel + 2, j_Kaiser

def t_scatter(x, y, label=None, tx="", ty="", titlu='Scatterplot'):
    f = plt.figure(figsize=(10, 7))
    f1 = f.add_subplot(1, 1, 1)
    f1.set_title(titlu, fontsize=16, color='b', verticalalignment='bottom')
    f1.set_xlabel(tx, fontsize=12, color='r', verticalalignment='top')
    f1.set_ylabel(ty, fontsize=12, color='r', verticalalignment='bottom')
    f1.scatter(x=x, y=y, c='r')
    if label is not None:
        n = len(label)
        for i in range(n):
            f1.text(x[i], y[i], label[i])

def t_scatter_s(x, y, x1, y1, label=None, label1=None, tx="", ty="", titlu='Scatterplot
set suplimentar'):
    f = plt.figure(figsize=(10, 7))
    f1 = f.add_subplot(1, 1, 1)
    f1.set_title(titlu, fontsize=16, color='b', verticalalignment='bottom')
    f1.set_xlabel(tx, fontsize=12, color='r', verticalalignment='top')
    f1.set_ylabel(ty, fontsize=12, color='r', verticalalignment='bottom')
    f1.scatter(x=x, y=y, c='r')
    f1.scatter(x=x1, y=y1, c='b')
    if label is not None:
        n = len(label);
        p = len(label1)
        for i in range(n):
            f1.text(x[i], y[i], label[i], color='k')
        for i in range(p):
            f1.text(x1[i], y1[i], label1[i], color='k')

def cercul_corelatiilor_2(t1, t2, xlabel="x", ylabel="y", titlu="", s1="Set x", s2="Set
y"):
    f = plt.figure(figsize=(8, 7))
    ax = f.add_subplot(1, 1, 1)
    x = [v for v in np.arange(0, np.math.pi * 2, 0.01)]
    cosx = np.cos(x)
    sinx = np.sin(x)
    ax.plot(cosx, sinx)
    ax.axhline(0, color='k')
    ax.axvline(0, color='k')
    ax.set_title(titlu)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.scatter(t1.iloc[:, 0], t1.iloc[:, 1], c='r', label=s1)
    ax.scatter(t2.iloc[:, 0], t2.iloc[:, 1], c='b', label=s2)
    ax.legend()
    n = len(t1)
    m = len(t2)
    for i in range(n):
        ax.text(t1.iloc[i, 0], t1.iloc[i, 1], t1.index[i])
    for i in range(m):
        ax.text(t2.iloc[i, 0], t2.iloc[i, 1], t2.index[i])

```

```

def cercul_corelatiilor(R, k1, k2, titlu="Cercul corelatiilor"):
    plt.figure(figsize=(6, 6))
    plt.title(titlu, fontsize=16, color='b', verticalalignment='bottom')
    x = [v for v in np.arange(0, np.math.pi * 2, 0.01)]
    cosx = np.cos(x);
    sinx = np.sin(x)
    plt.plot(cosx, sinx)
    plt.axhline(0, color='g');
    plt.axvline(0, color='g')
    plt.scatter(R.iloc[:, k1], R.iloc[:, k2], c='r')
    plt.xlabel(R.columns[k1], fontsize=12, color='r', verticalalignment='top')
    plt.ylabel(R.columns[k2], fontsize=12, color='r', verticalalignment='bottom')
    for i in range(len(R)):
        plt.text(R.iloc[i, k1], R.iloc[i, k2], R.index[i])

def cercCorel(R, k1, k2, xLabel=None, yLabel=None,
              title="Cercul corelatiilor", valMin=-1, valMax=1):
    plt.figure(title, figsize=(8, 8))
    plt.title(title, fontsize=18, color='k', verticalalignment='bottom')
    if xLabel == None or yLabel == None:
        if isinstance(R, pd.DataFrame):
            plt.xlabel(xlabel=R.columns[k1], fontsize=16,
                      color='k', verticalalignment='top')
            plt.ylabel(ylabel=R.columns[k2], fontsize=16,
                      color='k', verticalalignment='bottom')
        else:
            # isinstance(R, np.ndarray):
            plt.xlabel(xlabel='Componenta ' + str(k1 + 1), fontsize=16,
                      color='k', verticalalignment='top')
            plt.ylabel(ylabel='Componenta ' + str(k2 + 1), fontsize=16,
                      color='k', verticalalignment='bottom')
    else:
        plt.xlabel(xlabel=xLabel, fontsize=16, color='k', verticalalignment='top')
        plt.ylabel(ylabel=yLabel, fontsize=16, color='k', verticalalignment='bottom')
    T = [t for t in np.arange(0, np.math.pi * 2, 0.01)]
    X = [np.cos(t) for t in T]
    Y = [np.sin(t) for t in T]
    plt.plot(X, Y)
    plt.axhline(0, color='g')
    plt.axvline(0, color='g')
    if isinstance(R, pd.DataFrame):
        plt.scatter(R.iloc[:, k1], R.iloc[:, k2], c='r', vmin=valMin, vmax=valMax)
        for i in range(len(R)):
            plt.text(R.iloc[i, k1], R.iloc[i, k2], R.index[i])
    else:
        # isinstance(R, np.ndarray):
        plt.scatter(R[:, k1], R[:, k2], c='r', vmin=valMin, vmax=valMax)
        for i in range(len(R)):
            # pozitia textului in grafic (x, y) si textul
            plt.text(R[i, k1], R[i, k2], '(' +
                    str(np.round(R[i, k1], 1)) +
                    ', ' +
                    str(np.round(R[i, k2], 1)) +
                    ')')
    return None

# construirea cercului corelatiilor
# avand incluse cercuri concentrice pentru cuartile
def cercCorelExt(R, k1, k2, con=False, xLabel=None, yLabel=None,
                 title='Cercul extins al corelatiilor', valMin=-1, valMax=1):
    plt.figure(title, figsize=(8, 8))

```



```

plt.title(title, fontsize=18, color='k', verticalalignment='bottom')
if xlabel == None or ylabel == None:
    if isinstance(R, pd.DataFrame):
        plt.xlabel(xlabel=R.columns[k1], fontsize=16,
                    color='k', verticalalignment='top')
        plt.ylabel(ylabel=R.columns[k2], fontsize=16,
                    color='k', verticalalignment='bottom')
    else:
        # isinstance(R, np.ndarray):
        plt.xlabel(xlabel='Componenta ' + str(k1 + 1), fontsize=16,
                    color='k', verticalalignment='top')
        plt.ylabel(ylabel='Componenta ' + str(k2 + 1), fontsize=16,
                    color='k', verticalalignment='bottom')
else:
    plt.xlabel(xlabel=xlabel, fontsize=16, color='k', verticalalignment='top')
    plt.ylabel(ylabel=ylabel, fontsize=16, color='k', verticalalignment='bottom')

# crearea cercului corelatiilor, de raza 1,
# cu centrul in originea axelor de coordonate
# avem nevoie de o lista de valori pentru unghi care sa descrie un cerc
# np.arange(i, f, pas) si furnizeaza valori reale
T = [t for t in np.arange(0, np.pi * 2, 0.01)]
X = [np.cos(t) * 1 for t in T] # f(t) = cos(t)
Y = [np.sin(t) * 1 for t in T] # f(t) = sin(t)
plt.plot(X, Y)

if con:
    for k in range(1, 3 + 1):
        X = [np.cos(t) * 0.25 * k for t in T]
        Y = [np.sin(t) * 0.25 * k for t in T]
        plt.plot(X, Y)

plt.axhline(0, color='g')
plt.axvline(0, color='g')
if isinstance(R, pd.DataFrame):
    plt.scatter(R.iloc[:, k1], R.iloc[:, k2], c='r', vmin=valMin, vmax=valMax)
    for i in range(len(R)):
        plt.text(R.iloc[i, k1], R.iloc[i, k2], R.index[i])
else:
    # isinstance(R, np.ndarray):
    plt.scatter(R[:, k1], R[:, k2], c='r', vmin=valMin, vmax=valMax)
    for i in range(len(R)):
        # pozitia textului in grafic (x, y) si continutul textul
        plt.text(R[i, k1], R[i, k2], '(' +
                  str(np.round(R[i, k1], 1)) +
                  ', ' +
                  str(np.round(R[i, k2], 1)) +
                  ')')

return None

def multiHist(X, width=0.8, title='Multi-histograma', xlabel='Observatii',
              ylabel='Valori variabile'):
    plt.figure(title, figsize=(15, 11))
    plt.title(title, fontsize=18, color='k', verticalalignment='bottom')
    plt.xlabel(xlabel=xlabel, fontsize=16, color='k', verticalalignment='top')
    plt.ylabel(ylabel=ylabel, fontsize=16, color='k', verticalalignment='bottom')

    if isinstance(X, pd.DataFrame):
        nrCol = X.values.shape[1]
        nrLin = X.values.shape[0]
        width /= nrCol

```

```

        ind = np.arange(nrLin)
        for j in range(nrCol):
            plt.bar(x=ind + (width * j), height=X.iloc[:, j].values,
                    width=width, label=X.columns[j], align='center')
            plt.xticks(ind + width * (nrCol - 1) / 2.0, X.index[:])
    if isinstance(X, np.ndarray):
        nrCol = X.shape[1]
        nrLin = X.shape[0]
        width /= nrCol

        ind = np.arange(nrLin)
        for j in range(nrCol):
            plt.bar(x=ind + (width * j), height=X[:, j],
                    width=width, label='Var' + str(j + 1), align='center')
            plt.xticks(ind + width * (nrCol - 1) / 2.0,
                       ('Set' + str(i + 1) for i in range(nrLin)))

plt.legend(loc='best')
return None

def obsNorPuncte(tabel, label=None, xlabel='Observatii', ylabel='Valori variabile',
                  title='Multi scatterplot'):
    f = plt.figure(title, figsize=(15, 11))
    f1 = f.add_subplot(1, 1, 1)
    f1.set_title(title, fontsize=18, color='k', verticalalignment='bottom')
    f1.set_xlabel(xlabel=xlabel, fontsize=16, color='k', verticalalignment='top')
    f1.set_ylabel(ylabel=ylabel, fontsize=16, color='k', verticalalignment='bottom')
    for j in range(len(tabel.columns)):
        f1.scatter(x=tabel.index[:, j].values,
                   y=tabel.iloc[:, j].values,
                   label=tabel.columns[j], cmap='viridis')
    plt.legend(loc='best')

def varNorPuncte(tabel, xlabel='Valori variabile', ylabel='Observatii',
                  title='Multi scatterplot'):
    f = plt.figure(title, figsize=(15, 11))
    f1 = f.add_subplot(1, 1, 1)
    f1.set_title(title, fontsize=18, color='k', verticalalignment='bottom')
    f1.set_xlabel(xlabel=xlabel, fontsize=16, color='k', verticalalignment='top')
    f1.set_ylabel(ylabel=ylabel, fontsize=16, color='k', verticalalignment='bottom')
    for j in range(len(tabel.columns)):
        f1.scatter(x=tabel.iloc[:, j].values, y=tabel.index[:, j],
                   label=tabel.columns[j], cmap='viridis')
    plt.legend(loc='best')

def norPuncte(x, y, label=None, xlabel="", ylabel="", title='Scatterplot'):
    plt.figure(title, figsize=(11, 8))
    plt.title(title, fontsize=18, color='k', verticalalignment='bottom')
    plt.xlabel(xlabel=xlabel, fontsize=16, color='k', verticalalignment='top')
    plt.ylabel(ylabel=ylabel, fontsize=16, color='k', verticalalignment='bottom')
    plt.scatter(x=x, y=y, cmap='viridis')
    if label is not None:
        n = len(label)
        for i in range(n):
            plt.text(x[i], y[i], label[i])

```

```

# Plot scoruri discriminante si centrii
def scatter(x, y, g, etichete, x1, y1, g1, etichete1,
            titlu='Plot instante in axele discriminante', lx='z1', ly='z2'):
    q = len(etichete1)
    # rampa = plt.get_cmap('rainbow',q)
    f = plt.figure(figsize=(10, 7))
    ax = f.add_subplot(1, 1, 1)
    ax.set_title(titlu, fontsize=16, color='b')
    ax.set_xlabel(lx, fontsize=12, color='b')
    ax.set_ylabel(ly, fontsize=12, color='b')
    sb.scatterplot(x=x, y=y, hue=g, ax=ax, hue_order=g1)
    sb.scatterplot(x=x1, y=y1, hue=g1, ax=ax, legend=False, marker='s',
                  s=200)
    for i in range(len(etichete)):
        ax.text(x[i], y[i], etichete[i])
    for i in range(len(etichete1)):
        ax.text(x1[i], y1[i], etichete1[i], fontsize=16)

def plot_clustere(x, y, g, grupe, etichete=None, titlu="Plot clustere"):
    g_ = np.array(g)
    f = plt.figure(figsize=(10, 7))
    ax = f.add_subplot(1, 1, 1)
    ax.set_title(titlu, fontsize=16, color='b')
    nr_grupe = len(_CULORI)
    for v in grupe:
        x_ = x[g_ == v]
        y_ = y[g_ == v]
        k = int(v[1:])
        if len(x_) == 1: # Cluster singleton
            ax.scatter(x_, y_, color='k', label=v)
        else:
            ax.scatter(x_, y_, color=_CULORI[k % nr_grupe], label=v)
    ax.legend()
    if etichete is not None:
        for i in range(len(etichete)):
            ax.text(x[i], y[i], etichete[i])

# Functie care traseaza distributia de probabilitate pe grupe
def distributie(z, y, g, titlu=""):
    f = plt.figure(figsize=(10, 7))
    assert isinstance(f, plt.Figure)
    ax = f.add_subplot(1, 1, 1)
    assert isinstance(ax, plt.Axes)
    ax.set_title(titlu, fontsize=14, color='b')
    for v in g:
        sb.kdeplot(data=z[y == v], shade=True, ax=ax, label=v)
    # plt.show()

```

```

# Grafic histograma
def histograme(x, g, var):
    grupe = set(g)
    g_ = np.array(g)
    m = len(grupe)
    l = np.trunc(np.sqrt(m))
    if l * l != m:
        l += 1
    c = m // l
    if c * l != m:
        c += 1

```

```

axe = []
f = plt.figure(figsize=(12, 7))
for i in range(1, m + 1):
    ax = f.add_subplot(1, c, i)
    axe.append(ax)
    ax.set_xlabel(var, fontsize=12, color='b')
for v, ax in zip(grupe, axe):
    y = x[g_ == v]
    ax.hist(y, bins=10, label=v, rwidth=0.9, range=(min(x), max(x)))
    ax.legend()

# Grafic dendrograma
def dendrograma(h, etichete=None, titlu='Grupare ierarhica', threshold=None,
               culori=None):
    f = plt.figure(figsize=(10, 7))
    ax = f.add_subplot(1, 1, 1)
    ax.set_title(titlu, fontsize=14, color='b')
    if culori is None:
        hclust.dendrogram(h, labels=etichete, leaf_rotation=30, ax=ax,
                        color_threshold=threshold)
    else:
        hclust.dendrogram(h, labels=etichete, leaf_rotation=30, ax=ax,
                        link_color_func=lambda k: culori[k])

# Grafic mozaic
def mozaic(t, varx, vary):
    f = plt.figure(figsize=(10, 7))
    ax = f.add_subplot(1, 1, 1)
    t_nou = pd.DataFrame()
    t_nou[varx] = t[varx]
    t_nou[vary] = t[vary]
    smosaic.mosaic(t_nou, [varx, vary], ax=ax, gap=0.01, label_rotation=30)

def show():
    plt.show()

```



Tipărit la Tipografia A.S.E.
Comanda nr. 1/2021

51,70 lei